Quantum-Grover 해설서

구성 문서 20개

생성일 2026-10-31

목차

	해설서 index — quantum computing 시작하기	3
1장	qubit와 state	10
2장	Dirac notation과 linear algebra	13
3장	single-qubit gate	16
4장	measurement	20
5장	multi-qubit와 entanglement	24
6장	quantum circuit model과 universality	28
7장	oracle과 phase kickback	32
8장	Deutsch–Jozsa algorithm	36
9장	Grover algorithm ★	40
10장	용어 정리와 cheat sheet	49
11장	state vector emulator — memory map과 resource 예산	54
12장	amplitude를 숫자로 — fixed-point와 Q1.22	63
13장	oracle과 diffusion operator — multiplier 없는 산술	72
14장	multiple solutions — Born-rule sampling과 enumeration	82
15장	논문 지도와 설계 결정표	93
16장	몇 번 돌릴지 모를 때 — BBHT shot loop와 checkpoint	104
17장	RVX SoC에 붙이기 — APB CSR · AHB DMA · UART	113
18장	무엇을 어떻게 증명하나 — 3단 reference model과 bit-exact	121
	부록 A. 수학 빠른 복습 — complex number와 linear algebra	130

해설서 index — quantum computing 시작하기

> **Part 0** — 이 글은 사전 지식이 필요 없습니다. 편하게 읽으세요.

quantum computer를 처음 배울 때 가장 먼저 드는 질문은 "이게 대체 왜 필요한가?"입니다. 이 글은 수식 없이, quantum computer가 어디에서 왔고 왜 만들어졌는지, 그리고 우리 project가 왜 하필 **Grover algorithm**을 다루는지를 이야기합니다. 이 글을 읽고 나면 1장부터 이어지는 수학이 "무엇을 향해 가는지" 감이 잡힐 것입니다.

---

1. computer는 어디서 왔는가

오늘날 우리가 쓰는 모든 computer의 기원은 1936년 영국 수학자 **Alan Turing**의 연구로 거슬러 올라갑니다. Turing은 애초에 편리한 기계를 만들려던 게 아니라, 독일 수학자 Hilbert가 던진 질문 — *"어떤 수학 명제가 증명 가능한지 판별하는 일반적인 algorithm이 존재하는가?"* — 에 답하려 했습니다.

이 질문에 답하려면 먼저 "**algorithm이란 무엇인가**"를 엄밀히 정의해야 했습니다. Turing은 사람이 종이와 연필로 계산할 때 하는 동작을 분석해, 그 모든 것을 흉내 낼 수 있는 하나의 가상 기계 — **Turing machine** — 을 제안했습니다. "algorithm이란 Turing machine이 수행할 수 있는 것"이라는 이 정의가 현대 computing의 토대가 되었고, 이후 수십억 대의 computer가 이 model 위에 만들어졌습니다.

---

2. Deutsch의 질문 — 물리학이 계산을 결정한다

Turing의 논증에는 미묘한 허점이 있었습니다. "사람이 종이에 하는 계산"이라는 비형식적 근거에 기대고 있었던 것이죠. 1985년 물리학자 **David Deutsch** 는 더 깊은 접근을 제안합니다.

> 모든 algorithm은 결국 **어떤 물리적 system**에서 수행된다 — 사람이든, 주판이든, electronic circuit든. 그렇다면 **다른 모든 물리 system을 효율적으로 simulation할 수 있는 단 하나의 범용 계산 장치**가 존재하는가?

이 질문이 중요한 이유는, 만약 그런 장치가 있다면 물리 법칙 자체로 그것이 진정한 universal computer임을 증명할 수 있기 때문입니다. 그런데 Deutsch가 이 질문을 파고들자 문제가 드러났습니다: **평범한 (고전) computer는 quantum mechanical system을 simulation하는 데 극도로 느리고 비효율적**이었습니다.

이 문제를 풀기 위해 Deutsch는 새로운 종류의 계산 장치, 곧 **quantum computer** 를 고안해야 했습니다. quantum computer는 classical computer가 하는 모든 것을 할 수 있으면서, quantum mechanical process도 효율적으로 simulation합니다.

💡 **비유**: classical computer로 quantum 세계를 simulation하는 것은, 물속 세계를 육지의 규칙만으로 설명하려는 것과 같습니다. 억지로는 되지만 터무니없이 느립니다. quantum computer는 아예 물속에서 헤엄치는 장치입니다.

---

3. classical computer가 quantum을 흉내 내기 힘든 이유

핵심만 짚어봅시다. 입자 여러 개로 이루어진 분자를 simulation한다고 합시다. 각 입자가 qubit인 것은 아니지만, 설명을 위해 그렇게 단순화하면, 이 system을 기술하는 데 $2^{\text{number of qubits}}$**개의 숫자(amplitude)** 가 필요합니다. 입자 하나가 늘 때마다 필요한 memory가 **두 배**로 뜁니다.

왜 하필 2의 거듭제곱일까요? 입자(qubit) 하나는 0과 1, 두 가지 state를 가집니다. 둘이 되면 가능한 조합은 00·01·10·11 — 네 가지이고, 이 **조합 하나하나마다 amplitude(숫자)가 한 개씩** 필요합니다. 셋이면 000…111로 여덟 가지죠. 입자를 하나 더 붙일 때마다 기존 조합이 각각 "새 입자가 0인 경우"와 "1인 경우"로 둘씩 쪼개지므로, 조합의 수가 정확히 두 배가 됩니다($2 \to 4 \to 8 \to \cdots \to 2^{\text{number of qubits}}$). 그래서 필요한 숫자의 개수가 곧 조합의 총 개수, 즉 $2^{\text{number of qubits}}$인 것입니다.

그런데 여기서 자연스러운 반문이 떠오릅니다. "입자마다 숫자 2개씩이면, 입자 수의 두 배만큼만 있으면 충분하지 않나?" 항상 그렇게 쓸 수는 없습니다. quantum의 중요한 성질이 여기서 드러납니다. state에는 "A는 이 state, B는 저 state"처럼 입자별로 따로 나눠 적을 수 **없는** 경우가 있습니다 — 예컨대 두 입자를 아무리 멀리 떨어뜨려 놓아도 측정하면 늘 똑같은 값이 나오도록 서로 묶인 state가 그렇습니다. 이 묶임을 **entanglement** 라 하는데, 이 상관관계를 온전히 담으려면 모든 조합의 숫자를 하나도 빠짐없이 들고 있어야 합니다. 그래서 압축이 원리적으로 불가능하고, 지수적 크기를 피할 수 없습니다. (entanglement는 5장에서 제대로 배웁니다.)

- 입자 10개 → $2^{10} \approx 1{,}000$개 숫자
- 입자 50개 → $2^{50} \approx 10^{15}$개 (천조 개)
- 입자 300개 → 2^{300} → 우주의 원자 수보다 많음

실제 원자는 qubit보다 복잡해서 더 많은 숫자가 필요합니다. 그래서 작은 분자를 빼면 classical computer로는 quantum simulation이 사실상 불가능합니다. 반면 **quantum computer는 입자가 하나 늘 때마다 qubit 몇 개만 추가하면 됩니다** — 필요한 자원이 지수적 증가 대신 선형 증가에 그칩니다.

> 🔑 **quantum computer의 근본 강점**: "$2^{\text{number of qubits}}$개의 amplitude"를 classical computer는 memory에 일일이 저장해야 하지만, quantum computer는 그만큼의 qubit만으로 그 state를 **자연스럽게 담습니다**. 이 "지수적으로 많은 amplitude를 다루는 능력"이 1장의 superposition에서 시작됩니다.

---

4. quantum computer는 무엇을 잘하는가

quantum computer가 **모든** 문제를 빠르게 푸는 마법 상자는 아닙니다. 대부분의 문제에서는 classical computer와 큰 차이가 없습니다. 그러나 몇몇 특정 문제군에서는 극적으로 앞섭니다.

algorithm	문제	classical	quantum	향상
Shor	큰 수의 소인수분해	exponential time	polynomial time	지수적 (RSA 암호 위협)
Grover	정렬 안 된 data에서 search	$O(\text{number of states})$	$O(\sqrt{\text{number of states}})$	quadratic
Deutsch-Jozsa	function이 constant/balanced인지 판별	$O(2^{\text{number of qubits}})$	$O(1)$	지수적 (교육적 예제)
quantum simulation	분자·물질 simulation	지수적으로 느림	효율적	지수적

이 표에서 **Grover와 Deutsch-Jozsa** 가 이 해설서의 종착지입니다. Shor algorithm은 더 복잡해서 다루지 않지만, Grover를 이해하면 quantum algorithm의 핵심 아이디어 — **interference로 solution의 probability를 키우는 것** — 을 익힐 수 있습니다.

---

5. 이 project가 Grover를 다루는 이유

우리 project의 목표는 **Grover algorithm을 FPGA hardware로 emulation**하는 것입니다. 조금 더 구체적으로 말하면, FPGA 안의 memory에 data array `value[]` 를 올려 두고 **predicate를 만족하는 원소가 몇 번 칸에 있는지를 Grover 방식으로 찾아 주는 accelerator IP** 입니다. predicate는 네 가지를 지원합니다 — threshold보다 작은가($<$), 큰가($>$), 같은가($=$), 그리고 두 threshold 사이인가($a < x < b$). 한 번 호출하면 predicate를 만족하는 index **하나**를 돌려주고, 호출을 되풀이하면 predicate를 만족하는 원소를 **중복 없이 전부 enumeration**할 수도 있습니다. board는 Digilent **Arty A7-100T**(Xilinx Artix-7)이고, IP는 그 위에 올린 **RVX** RISC-V SoC에 붙습니다. host는 **UART** 너머에서 명령을 보내고 결과를 받아올 뿐이며, amplitude 계산은 전부 FPGA 안에서 끝납니다. 다루는 규모는 **14 qubits**, 곧 $2^{14} = 16{,}384$ states를 담는 array입니다.

그러면 왜 하필 Grover일까요?

1. **algorithm이 단순하면서도 깊다.** Shor처럼 복잡한 number theory 없이, gate 몇 개를 되풀이하는 것만으로 동작합니다. 그런데도 quantum algorithm의 핵심 — amplitude amplification·interference — 을 온전히 담고 있어 hardware로 구현하기에 이상적입니다.
2. **적용 범위가 넓다.** "정렬되지 않은 data에서 predicate를 만족하는 항목 찾기"는 곧 **threshold 비교 질의**입니다. sensor log에서 기준치를 넘는 이상치를 골라내기, 어떤 interval에 드는 값들만 뽑아내는 range query, 조건에 걸리는 후보를 하나씩 전부 열거하기 — 위에 적은 predicate 네 가지가 정확히 이 질의들에 대응합니다. threshold를 찾을 때마다 더 낮춰 가면 같은 circuit으로 minimum 찾기도 됩니다. (project의 `4. Grover 응용 논문` directory가 이런 응용의 예입니다.)
3. **다만 "이차 향상"은 우리 것이 아니다.** 여기서 한 가지를 구분해야 합니다. 앞 절 표의 $O(\sqrt{\text{number of states}})$ 은 **진짜 quantum computer**가 oracle을 몇 번 부르는지를 센 값입니다. 우리처럼 classical circuit으로 흉내 내면 iteration 한 번마다 amplitude array 전체를 처음부터 끝까지 훑어야 하고 그 iteration을 $\sqrt{\text{number of states}}$ 번쯤 되풀이하므로, 총비용은 오히려 $\text{number of states} \times \sqrt{\text{number of states}}$ 만큼이 됩니다. 같은 문제를 array 한 번 훑어 푸는 평범한 classical program보다 **느립니다.** 이 해설서 어디에도 "우리 IP가 classical linear scan보다 빠르다"는 문장은 없습니다.

그렇다면 왜 만들까요? 비교 대상을 살펴보면 이 작업의 목적을 알 수 있습니다. 우리와 **같은 일**을 하는 것, 즉 amplitude array를 memory에 펼쳐 놓고 매 iteration 전부 훑는 것은 classical linear scan이 아니라 **state vector simulator**입니다. 구체적으로는 Qiskit의 `AerSimulator` 나 NumPy로 짠 state vector code가 우리의 비교 대상입니다. 그리고 이 IP의 값어치는 애초에 속도 우위가 아니라, quantum algorithm이 실제로 어떻게 동작하는지를 **결정적이고 재현 가능하게 재현·검증하는 전용 hardware**라는 데 있습니다. 같은 random number seed를 주면 언제나 같은 결과가 나오고, 계산 도중의 amplitude array를 통째로 꺼내 손계산이나 software model과 한 bit씩 대조할 수 있습니다. 진짜 quantum hardware에서는 둘 다 불가능한 일입니다 — 관측하는 순간 state가 바뀌기 때문입니다.

대신 대가가 있습니다. 모든 state의 amplitude를 memory에 하나하나 펼쳐 놓아야 하므로 **qubit를 하나 늘릴 때마다 필요한 memory가 두 배**로 뛰니다. 14 qubits는 amplitude array를 여러 벌 저장하며 계산하는 우리 구조를 이 board에 구현할 수 있는 상한입니다.

> ❓ **emulation? 실제 quantum computer가 아니고?**: 그렇습니다. 진짜 quantum hardware는 비싸고 불안정합니다. 대신 FPGA로 quantum state vector의 진화를 **classical 방식으로 계산**해 algorithm의 동작을 재현·검증합니다. 그러려면 state vector를 숫자 array로 펼쳐 놓고, gate가 그 array에 하는 일 — predicate를 만족하는 칸의 **sign 반전**과 모든 칸의 **inversion about the mean** — 과 measurement를 수치로 정확히 구현해야 하고, 그 수학이 바로 이 해설서의 내용입니다. (project의 `2. Grover 논문 (LPSoC)`, `3. QCE-FPGA 논문` 이 이 emulation 기법을 다룹니다.)

---

6. 앞으로의 여정 — 장 목록과 읽는 법

이 해설서는 algorithm의 수학적 model — qubit·superposition·gate·measurement·oracle·amplitude amplification — 을 기초부터 배운 뒤(0~10장), 같은 개념이 memory map·fixed-point data word·FSM·bus interface로 구현되는 과정을 따라갑니다(11~18장). 확정된 숫자(board, number of qubits, amplitude 형식, final configuration 등)의 source of truth는 15.4절 설계 결정표이며, 해설서 어느 장에 같은 값이 적혀 있든 어긋나면 15.4절을 기준으로 삼습니다.

장 목록

Part 0. 왜 quantum computer인가

장	file	무엇을 배우나
—	index — quantum computing 시작하기 (이 글)	computer의 기원, 왜 quantum computer인가, classical vs quantum, 이 project가 Grover를 다루는 이유

Part 1. quantum computing의 언어 (기초)

장	file	무엇을 배우나
1	qubit와 state	bit vs qubit, state vector, computational basis $\|0\rangle\|1\rangle$, amplitude·superposition, normalization
2	Dirac 표기법과 linear algebra	ket·bra, inner product $\langle\phi\|\psi\rangle$, conjugate transpose(dagger), $\|+\rangle\|-\rangle$ basis
3	단일 qubit gate	$X\|H\|Y\|Z$ gate, unitary matrix, circuit 표기, gate composition=matrix multiplication
4	measurement	computational basis measurement, Born rule($\|\alpha\|^2$), collapse, measurement 불가 정보, global phase

Part 2. 여러 qubit와 circuit

장	file	무엇을 배우나
5	다중 qubit와 entanglement	tensor product, multi-qubit state의 $2^{\text{number of qubits}}$ dimension, CNOT, Bell state·entanglement, product state
6	quantum circuit model과 universality	circuit 3단계, Hadamard로 uniform superposition 만들기, parallelism, universal gate·Toffoli, circuit의 순효과는 하나의 unitary

Part 3. quantum algorithm (project 핵심)

장	file	무엇을 배우나
7	oracle과 phase kickback	oracle U_f, black box, phase kickback, conditional sign 반전
8	Deutsch–Jozsa algorithm	Balanced vs Constant, $H^{\otimes \text{number of qubits}}$ 수학, interference, 한 번에 판별
9	**Grover algorithm**	amplitude amplification, inversion about the mean, diffusion operator, geometric rotation, number of iterations $\frac{\pi}{4}\sqrt{\text{number of states}}$, amplitude가 real number뿐인 이유, 어느 개념이 어느 장으로 가는가

Part 4. 참조

장	file	무엇을 배우나
10	용어정리와 cheat sheet	핵심 gate matrix·공식 요약, 용어집, 연습문제 답, 원문 대조표
부록 A	수학 빠른 복습	이 해설서가 쓰는 complex number(conjugate·absolute value·$e^{i\theta}$)·linear algebra(vector·matrix multiplication·dagger) 최소 정리

> 여기서부터(Part 5~8)가 이 project가 **실제로 만드는 물건**입니다. 여섯 편의 논문(`papers/` 2·3·4번 directory)을 설계 질문별로 **함께** 읽으면서, 확정된 설계 결정을 하나씩 세워 나갑니다. 04·05·06은 이 project 지도교수 연구실(중앙대) 논문입니다.

Part 5. 이론에서 hardware로 — 뼈대

장	file	무엇을 배우나
11	state vector emulator — memory map과 resource 예산	ket → 세 개의 array(`amp`·`data`·`mask`), qubit 하나 = memory 두 배, amplitude를 어디에 둘 것인가, memory-bound의 두 얼굴, 32 lanes 병렬과 banking, cycle 비용 model, emulator가 숨기는 것
12	amplitude를 숫자로 — fixed-point와 Q1.22	왜 floating-point를 안 쓰는가, Q format 명명 규약, 폭을 정할 때 재는 대상이 셋이라는 것, round-half-to-even, symmetric saturation·renormalization 없음, minimum fraction bits 공식과 그 전제, fraction bits를 공식이 아니라 재서 정한 과정

Part 6. datapath — 한 shot 안에서 벌어지는 일

장	file	무엇을 배우나
13	oracle과 diffusion operator — multiplier 없는 arithmetic	한 iteration 안의 FSM, square-root unit 없이 $1/\sqrt{\text{number of states}}$ 만들기, predicate 4종이 subtractor 하나를 공유하는 방법, two's complement negate, $2 \cdot \text{mean} - \text{amp}$ 가 circuit이 되는 순간, 2 passes의 필연성, 32 lanes accumulation 순서, multiplier가 있는 곳과 없는 곳
14	multiple solutions — Born-rule sampling과 enumeration	solution이 여럿이어도 oracle에는 공짜인 이유, oracle은 predicate를 판정하는 circuit라는 것, amplitude 동률로 argmax가 무너지는 지점, Born-rule sampling을 circuit으로, group·row·lane 순의 hierarchical measurement(Dual-Row Build · Two-Level Select), Found Mask로 중복 없이 enumeration, 역주행 등 두 함정

Part 7. 설계 좌표

장	file	무엇을 배우나
15	논문 지도와 설계 결정표	6편 계보도, **여덟 개의 대립축**, 확정 설계 결정표(**숫자의 source of truth**), platform과 resource 예산, **인용 시 주의할 오류 목록**, hardware 용어집

Part 8. shot 바깥 — 제어·system·verification

장	file	무엇을 배우나
16	몇 번 돌릴지 모를 때 — BBHT shot loop와 checkpoint	왜 solution이 몇 개인지 모르는 채 시작하는가(속도가 아니라 실기 충실도), BBHT(sampling range 1에서 시작 · growth factor 6/5), mean shot count가 스무 번을 넘는 이유, 바깥 FSM과 solution check, **a Checkpoint-Capacity of 3과 3-shot Lookahead policy**, 그 정당성, 무효화는 hardware가 한다
17	RVX SoC에 붙이기 — APB CSR · AHB DMA · UART	accelerator는 혼자 서지 않는다, RVX(`rvc_orca` · `munoc`)가 무엇인가, APB와 AHB를 같이 쓰는 이유, CSR map, AHB master로 data load, host UART protocol, hardware와 firmware의 경계
18	무엇을 어떻게 증명하나 — 3단 reference model과 bit-exact	왜 verification이 설계의 일부인가, float64 → bit-exact fixed-point model → RTL의 3단 verification ladder, bit-exact가 성립하기 위한 조건, 9.4절 손계산이 첫 test vector, Born sampler와 checkpoint를 어떻게 검증하나, 잰 수치와 그 조건, toolchain과 directory

읽는 순서

각 장은 바로 앞 장의 개념을 바탕으로 설명합니다. 순서대로 읽기를 권합니다. 특히 **superposition(1장) → gate(3장) → measurement(4장) → phase kickback(7장)** 은 Grover를 이해하는 데 필요한 4개의 기본 개념입니다. 0~10장이 quantum 이론이고, 11장부터는 그 이론을 실제 circuit으로 옮기는 이야기라 FPGA 기초(LUT·FF·BRAM·DSP)를 함께 가정합니다. 이론만 필요하다면 10장에서 멈춰도 됩니다.

```
처음 배우는 사람:                1장 → 2장 → ... → 9장  (순서대로)
linear algebra에 익숙한 사람:    1~2장 빠르게, 3장부터 정독
Grover만 급히 볼 사람:           1·3·4·5·6.2·7장 훑고 → 9장 (모르는 개념은 앞 장 참조)
hardware까지 갈 사람:            9장까지 마치고 → 11 → 12 → 13 → 14 → 16 → 17 → 18
                                   (FPGA 배경이 있으면 11.5·11.6절은 훑고 넘어가도 됩니다)
```

13장을 읽기 전에 12장의 내용이 필요합니다. 13장은 amplitude 한 칸이 Q1.22 fixed-point라는 것, round-half-to-even이 어느 지점에서 일어난다는 것, 넘치면 saturation이 걸린다는 것을 전부 12장의 결정으로 **전제하고** 씁니다. 12장을 건너뛰면 13장의 shift 위치와 data word 폭이 근거 없는 숫자로 보입니다.

15장은 순서에 넣지 않았습니다. 참조용이라 11~14장을 읽은 뒤 아무 때나 펼치면 되고, 확정된 수치가 필요할 때마다 15.4절로 돌아오게 됩니다. 다만 논문을 **인용하기 전에는** 반드시 15.6절 정오표를 먼저 보세요.

각 장은 시작 부분에 **"이 장을 읽기 위한 준비"** 로 필요한 배경 지식을 링크합니다. 앞 장에서 배운 개념을 뒤 장에서 쓰도록 배치했습니다.

독자에게 가정하는 것

- **알고 있다고 가정**: complex number(magnitude·conjugate·$e^{i\theta}$), linear algebra(vector·matrix·matrix multiplication·transpose), classical logic gate(AND·OR·NOT·XOR), 기초 programming, big-O notation($O(n)$, $O(\log n)$).
- **처음이라고 가정하고 설명**: qubit·state·superposition, Dirac(bra-ket) 표기법, quantum gate와 unitary matrix, measurement와 collapse, entanglement, oracle·phase kickback, amplitude amplification, Grover·Deutsch-Jozsa algorithm. → 이 개념들을 **1장부터 차근차근** 설명합니다.
- **Part 5~8(11~18장)에서 추가로 가정**: FPGA 기초(LUT·FF·BRAM·DSP, RTL, clock). 이 project 사용자는 FPGA 경험이 있습니다. 이 배경이 없어도 1~10장(quantum 이론)은 그대로 읽을 수 있고, hardware 용어는 15.7절 용어집에 모아 두었습니다.

> 이 정도 수학 배경이 없거나 오래 잊었다면, 먼저 부록 A. 수학 빠른 복습 을 보세요 — 이 해설서가 실제로 쓰는 complex number·linear algebra만 추려 두었습니다. 더 깊이 원하면 3Blue1Brown의 linear algebra 영상 시리즈와 complex number 기초를 권합니다. 원문 QCVC 에세이도 같은

> 배경을 가정합니다.

표기 약속

본 해설서는 이해를 돕기 위해 몇 가지 장치를 씁니다.

- 💡 **비유 박스**: 추상적인 quantum 개념을 일상적인 비유로 풀어줍니다.
- 🔍 **계산 따라가기**: 실제 vector·matrix 계산을 **한 줄 한 줄** 따라갑니다.
- ❓ **왜 이런가?**: 정의와 규칙의 이유·동기를 설명합니다.
- ⚠️ **흔한 오해 / 함정**: 처음 배울 때 헷갈리기 쉬운 지점을 짚습니다.
- 🔑 **핵심**: 꼭 기억할 것.
- ✏️ **연습**: 스스로 확인해 볼 문제 (답은 10장 cheat sheet 또는 본문에).
- 수식은 LaTeX(`$...$`, `$$...$$`)로, state는 $|0\rangle$, $|\psi\rangle$ 처럼 ket으로 표기합니다.

다루지 않는 것

classical linear scan과의 속도 경쟁은 이 해설서의 주제가 아닙니다(5절). 그 밖에 두 가지 범위를 더 정해 둡니다. **minimum search는 주제가 아니라 확장입니다.** threshold를 계단처럼 낮춰 가며 minimum에 수렴시키는 Dürr–Høyer loop는 hardware에 넣지 않고 host firmware의 `while` 한 줄로 처리하기로 했습니다. 그래서 해설서에서도 14.9절 한 절 분량입니다. superconducting·trapped-ion 같은 **quantum hardware의 물리**도 범위 밖입니다 — amplitude라는 숫자만 다루면 충분하다는 것이 1.7절의 결론입니다.

함께 보는 자료와 원문 출처

문서	위치	성격
학습 해설서 (본 문서)	`study_references/`	배경지식 + algorithm 해설 + 확정 설계, 읽고 배우는 **주교재**
논문 해설본	../papers/papers_ko/	논문 6편의 한국어 해설, 원문 대조용 보조 자료
원문 논문·강의자료	../papers/	원본 PDF (기초이론 3편 + FPGA·응용 논문 6편)
기술문서	../design_references/	확정된 설계의 source of truth (RTL · port · CSR · 운용)

해설서가 주교재이고 논문 해설본은 원문 대조용 보조 자료입니다. 그래서 각 장은 **자체 완결**이어야 하며, 해설본을 읽어야만 이해되는 서술을 남기지 않습니다.

이 해설서는 project `documents/study_references/papers/1. 양자컴퓨터 기초이론/` 의 세 자료를 학부생용으로 재구성·통합한 것입니다.

1. **Quantum Computing for the Very Curious** — Andy Matuschak & Michael Nielsen (quantum.country). qubit·gate·measurement·CNOT·entanglement·범용성의 기초. → 주로 1~6장.
2. **[양자컴퓨터 기초 이론] 5. 그로버 알고리즘과 도이치 알고리즘** — hgmin1159.github.io. oracle·amplitude amplification·Grover·Deutsch-Jozsa 개요. → 7~9장.
3. **양자 컴퓨팅 입문 (2) - Grover's Algorithm** — infossm.github.io (Samsung SW Membership). Grover의 엄밀한 유도(phase kickback·diffusion operator·geometric rotation·number of iterations). → 9장.

추가 참고 문헌: Sutor, R. (2019). *Dancing With Qubits*; Bernhardt, C. (2019). *Quantum Computing for Everyone*; Nielsen & Chuang, *Quantum Computation and Quantum Information*; Grover, L. K. (1996), *A fast quantum mechanical algorithm for database search*.

Part 5~8의 원문 — 논문 6편 (`papers/` 2·3·4번 directory)

Part 5~8은 아래 여섯 편을 설계 질문별로 가로질러 다룹니다. 각 편의 한국어 해설본과 원문 PDF 목록은 논문 목록의 「보유 원문과 한국어 해설본」에 있습니다(원문 대조용).

- **[04] Developing a Grover's Emulator on Standalone FPGAs** — Choi & W. Lee (중앙대), AIMS Math. 9(11), 2024. Grover 전용 최적화 4종 + RISC-V SoC. → 11·13장(설계도).

- **[05] Precision-aware Fixed-point Emulation of Grover's** — Choi 외, W. Lee 교신 (중앙대), Quantum Inf. Process. 25:214, 2026. truncation error 이론·minimum fraction bits 공식. → 12장.
- **[06] Standalone FPGA-Based QAOA Emulator** — Choi 외, W. Lee (중앙대), arXiv:2502.11316, 2025. 같은 골격의 QAOA emulator. → 11·13장(multiplier 대비).
- **[07] Towards Complete and Scalable Emulation (HPRC)** — El-Araby 외 (U. Kansas), IEEE TC 72(8), 2023. 외부 DDR로 32 qubits, C2Q initialization. → 11·12장(대조군).
- **[08] Generalized Grover's Algorithm for Multiple Phase Inversion States** — Byrnes, Forster, Tessler, PRL 120, 060501, 2018. diffusion operator 일반화. → 11.10절(실기에서는 이득이지만 emulator에서는 손해가 되는 일반화) · 14.0절(기호 충돌 주의).
- **[09] Quantum Minimum Searching for AUD in Wireless IoT** — Habibie 외 (INSA Lyon), IEEE IoT J. 11(12), 2024. Dürr–Høyer minimum search의 응용 사례. → 14.9절 · 15장(**확장 참고 사례**로 강등 — 남는 것은 "oracle은 predicate를 판정한다"는 관점과 응용 사례 하나입니다).

여섯 편의 계보 바깥에서 설계 결정 하나의 근거가 되는 논문이 한 편 더 있습니다.

- **[BBHT] Tight bounds on quantum searching** — Boyer, Brassard, Høyer, Tapp (1998). solution이 몇 개인지 모르는 채 Grover를 돌리는 표준 절차(sampling range를 1에서 시작해 실패할 때마다 growth factor 6/5 = 1.2×로 키움)와 success probability lower bound의 **원전**. → 16.2절. 09번 논문의 Algorithm 1·2도 이 절차를 옮겨 온 것이므로, **iteration schedule을 인용할 때는 09번이 아니라 1998년 원전을 인용하십시오.** 원문 PDF는 `papers/` 에 없습니다.

---

7. 학습 태도에 관한 한마디

원문 QCVC 에세이의 저자는 이렇게 말합니다:

> quantum computing을 이해하려면 반드시 **수학적 model에 익숙해져야 한다.** 화려한 비유 영상만으로는 몇 년을 봐도 진짜로 이해하지 못한다. 농구 이야기를 듣는 것과 실제로 농구를 하는 것이 다르듯이.

겁먹을 필요는 없습니다. 필요한 수학은 **complex number와 linear algebra(vector·matrix)** 뿐이고, 이 해설서는 계산을 한 줄씩 따라갑니다. 다만 눈으로만 읽지 말고, 종이에 직접 $H|0\rangle$ 을 계산해 보는 습관이 중요합니다.

💡 **비유**: 질문(연습문제)을 틀리는 것은 실패가 아니라 **가장 빠르게 배우는 순간**입니다. 어디를 모르는지 정확히 짚어주기 때문입니다. 틀린 부분을 찾아 다시 계산해 보세요.

---

8. 요약

- **computer의 기원**: Turing(1936)이 "algorithm=Turing machine이 하는 것"으로 정의 → 현대 computing의 토대.
- **Deutsch의 질문**: "모든 물리 system을 효율적으로 simulation하는 범용 장치가 있는가?" → classical computer는 quantum을 못 흉내 냄 → quantum computer의 탄생.
- **근본 강점**: 입자가 하나 늘 때마다 amplitude는 두 배. classical은 지수적으로 증가, quantum은 입자 수만큼의 qubit로 자연스럽게 담음.
- **잘하는 문제**: Shor(소인수분해, 지수적)·Grover(search, 이차)·quantum simulation. 만능은 아님.
- **이 project**: BRAM에 올린 data array에서 네 가지 predicate($<\cdot>\cdot=\cdot$interval)를 만족하는 index를 찾아 주는 **Grover emulator IP**(Arty A7-100T + RVX SoC, 14 qubits). 하나를 돌려주거나 호출을 되풀이해 전부 enumeration합니다.
- **속도가 값어치가 아니다**: $O(\sqrt{\text{number of states}})$ 은 실제 quantum computer의 이야기이고, classical circuit으로 흉내 내면 classical linear scan보다 **느립니다.** 상대는 Qiskit `AerSimulator`·NumPy state vector code며, 이 IP의 값어치는 **결정적·재현 가능한 재현과 verification**에 있습니다.

다음 장에서는 이 모든 것의 출발점, **qubit** 가 정확히 무엇인지 — 왜 그 state가 "숫자"가 아니라 "vector"인지 — 부터 배웁니다.

1장. qubit와 state

> **이 장을 읽기 위한 준비**: complex number(magnitude $|\alpha|$, conjugate α^*)와 2-dimensional vector. 이 기호들이 낯설면 부록 A 수학 빠른 복습을 먼저 보세요. 0장을 먼저 읽으면 좋습니다.

classical computer의 정보 단위가 **bit** 이듯, quantum computer의 정보 단위는 **qubit** 입니다. 이 장에서는 qubit의 state가 왜 "0 또는 1"이라는 숫자가 아니라 **vector**인지, 그 vector의 component(amplitude)가 무엇을 뜻하는지, 그리고 "superposition"이 정확히 무엇인지를 배웁니다. 이 장은 이 해설서의 출발점입니다. 여기서 익힌 개념을 바탕으로 3장의 gate와 9장의 Grover를 배웁니다.

1.1 bit 복습 — state가 "숫자"인 세계

classical bit의 state는 **0 또는 1**, 둘 중 하나의 숫자입니다. 물리적으로는 capacitor의 전하, 자기 방향 등으로 저장되지만, programmer는 그런 세부를 잊고 "이 bit는 0이다 / 1이다"라는 **추상적인 숫자**로만 다룹니다.

bit의 state는 **하나의 값**이고, 언제든 들여다보면 그 값을 정확히 알 수 있습니다.

1.2 qubit — state가 "vector"인 세계

qubit에도 state가 있습니다. 그런데 qubit의 state는 숫자가 아니라 **2-dimensional vector**입니다. 이 vector가 속한 공간을 **state space**라고 부릅니다.

가장 먼저 알아야 할 두 개의 특별한 state가 있습니다. classical bit의 0, 1에 대응하는 state입니다.

$$|0\rangle := \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad |1\rangle := \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

이 둘을 **computational basis state**라고 합니다. $|0\rangle$ 은 classical bit의 0, $|1\rangle$ 은 1 역할을 합니다.

> ⚠️ **함정 — $|0\rangle$ 은 "zero vector"가 아니다**: $|0\rangle$ 은 component가 $(1, 0)$ 인 unit vector입니다. 숫자 0이나 zero vector $(0, 0)$ 과 전혀 다릅니다. $|0\rangle$ 은 그냥 "이 qubit는 0 state"라는 뜻의 **하나의 기호**로 보세요.

ket notation

$|0\rangle$ 에서 세로줄 $|$ 과 꺾쇠 $\rangle$ 는 각각 따로 의미가 있는 게 아닙니다. **"이것은 state(vector)다"** 라고 표시하는 하나의 묶음 기호이고, 이를 **ket**이라 부릅니다. ket은 그냥 vector입니다. (notation 자체는 2장에서 더 다룹니다.)

> 💡 **비유**: $|0\rangle$ 은 숫자 0 처럼 "하나의 대상을 가리키는 이름표"입니다. 숫자 0 이 실은 "additive identity"를 가리키듯, $|0\rangle$ 은 vector $(1, 0)$ 을 가리킵니다.

1.3 superposition — qubit의 진짜 힘

qubit가 흥미로운 이유는 state가 $|0\rangle$ 이나 $|1\rangle$ **만** 될 수 있는 게 아니기 때문입니다. 이 둘의 **linear combination**도 가능합니다.

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}$$

여기서 α, β 는 **complex number**이고, **amplitude**라 부릅니다. 이렇게 여러 computational basis state가 동시에 섞인 state를 **superposition** 이라고 합니다.

예를 들어 다음도 가능한 qubit state입니다:

$$|\psi\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\1\end{bmatrix}$$

이 state는 "$|0\rangle$ 이기도 하고 $|1\rangle$ 이기도 한" state입니다. 다만 이 표현은 오해하기 쉽습니다.

> ⚠️ **흔한 오해 — superposition은 "0.5는 0, 0.5는 1"이 아니다**: superposition은 "qubit가 이미 0 또는 1로 정해져 있는데 우리가 그 값을 모르는 상태"를 뜻하지 않습니다. qubit는 **정말로** $|0\rangle$ 과 $|1\rangle$ 이 동시에 존재하는 state에 있습니다. 이 차이는 4장 measurement와 8장 interference에서 드러납니다. amplitude α, β 는 probability가 아니라 **complex number**라는 점도 중요합니다 (negative·imaginary 값도 가능 → interference의 근원).

1.4 normalization — amplitude가 지켜야 할 규칙

아무 α, β 나 되는 것은 아닙니다. amplitude는 다음 **normalization condition**을 만족해야 합니다.

$$|\alpha|^2 + |\beta|^2 = 1$$

여기서 $|\alpha|^2$ 은 complex number α 의 magnitude의 square입니다 ($\alpha = a + bi$ 이면 $|\alpha|^2 = a^2 + b^2$). 기하학적으로 이 조건은 **state vector의 길이가 1** 이라는 뜻입니다.

왜 이런 조건이 필요한지는 4장 measurement에서 설명하겠지만, 미리 살펴보면 $|\alpha|^2$ 은 measurement에서 0이 나올 **probability**이고 $|\beta|^2$ 은 1이 나올 probability입니다. probability의 sum은 1이어야 하므로 $|\alpha|^2 + |\beta|^2 = 1$ 입니다.

🔍 **계산 따라가기 — 위 uniform superposition은 normalize되어 있는가?**

$$\left|\frac{1}{\sqrt{2}}\right|^2 + \left|\frac{1}{\sqrt{2}}\right|^2 = \frac{1}{2} + \frac{1}{2} = 1 \quad \checkmark$$

한 가지만 미리 새겨 두세요. normalization은 gate를 **끝낸 뒤**의 state가 만족해야 하는 조건이며, gate를 계산하는 **도중**의 중간값에까지 적용되는 조건은 아닙니다. 이 차이가 나중에 amplitude를 fixed-point로 적을 때 integer bits를 한 bit 더 쓰게 만듭니다(12.3절).

✏️ **연습 1-1**: $|\psi\rangle = \frac{1}{2}|0\rangle + c\,|1\rangle$ 이 normalize되려면 $|c|$ 는? *(answer: $\frac{\sqrt{3}}{2}$. $\frac{1}{4} + |c|^2 = 1$.)*

1.5 자주 등장하는 state들

앞으로 계속 나올 두 superposition state에 이름이 붙어 있습니다.

$$|+\rangle := \frac{|0\rangle + |1\rangle}{\sqrt{2}} = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\1\end{bmatrix}, \qquad |-\rangle := \frac{|0\rangle - |1\rangle}{\sqrt{2}} = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\-1\end{bmatrix}$$

둘 다 $|0\rangle$ 과 $|1\rangle$ 의 uniform superposition이지만, $|1\rangle$ 앞 sign이 각각 $+$ 와 $-$ 입니다. 이 **sign(phase)의 차이** 가 quantum algorithm의 핵심 재료입니다.

> 🔑 **핵심**: $|+\rangle$ 과 $|-\rangle$ 은 measurement하면 둘 다 0과 1을 50:50으로 주지만, **서로 다른 state**입니다. 이 둘을 구별하는 능력이 4장에서 Hadamard gate로 드러나고, 7장 phase kickback을 이해하는 바탕이 됩니다.

1.6 measurement로 읽을 수 없는 정보 — amplitude는 숨어 있다

Alice가 qubit를 $\alpha|0\rangle + \beta|1\rangle$ 로 준비해 Bob에게 주되 α, β 값은 말해주지 않았다고 합시다. Bob이 어떤 실험을 하면 α, β 를 알아낼 수 있을까요?

불가능합니다. state는 직접 관측되지 않습니다. amplitude α, β 는 일종의 **숨은 정보(hidden information)** 입니다.

이것이 고전적 직관과 크게 다른 점입니다. 자동차 engine은 정밀 장비로 내부 state를 알아낼 수 있지만, qubit의 amplitude는 물리 법칙이 관측을 금지합니다. Bob이 할 수 있는 것은 4장에서 배울 **measurement** 뿐이고, measurement는 확률적 결과 하나(0 또는 1)만 주고 그 순간 α, β 를 파괴합니다.

> ❓ **그럼 qubit에 무한한 정보를 저장할 수 있나?**: α 가 complex number이니 그 decimal expansion에 무한한 bit를 넣고 싶겠지만, measurement로 α 를 정확히 읽을 방법이 없으므로 불가능합니다. qubit 하나에서 꺼낼 수 있는 classical information은 결국 1 bit뿐입니다.

FPGA emulator에서는 이 제약이 없습니다 — amplitude가 BRAM에 숫자로 저장되어 있어 값을 직접 읽을 수 있기 때문입니다. 그렇다고 그중 **가장 큰 값을 그냥 집어내는 것이 measurement가 되지는 않습니다.** predicate를 만족하는 element가 둘 이상이면 그 amplitude들은 서로 정확히 같은 값이라, maximum comparator는 언제 돌려도 같은 index 하나만 내놓습니다(14.3절).

1.7 물리적 실체는 잊어도 된다

qubit가 물리적으로 무엇인지 — electron의 spin, photon의 polarization, atom의 energy level — 는 이 해설서에서 중요하지 않습니다. classical programmer가 bit가 어떤 capacitor에 저장되는지 신경 쓰지 않듯이 말입니다.

다음 정의를 기억해 두세요:

> 🔑 **qubit의 정의 (4줄 요약)**
>
> - (a) qubit는 state를 가진다.
> - (b) 그 state는 추상적인 수학적 대상이다.
> - (c) bit의 state가 숫자(0/1)라면, qubit의 state는 **2-dimensional complex vector**다.
> - (d) 그 vector가 속한 곳이 **state space**이며, vector의 길이는 항상 1이다.

우리 project의 FPGA emulator도 정확히 이 관점을 따릅니다. qubit의 물리를 흉내 내는 것이 아니라 amplitude $2^{\text{number of qubits}}$ 칸을 **23-bit fixed-point 숫자로 BRAM에 펼쳐 놓습니다.** 그리고 gate를 matrix multiplication으로 계산하지 않습니다 — array를 훑는 두 가지 일로 계산합니다. 하나는 predicate를 만족하는 칸의 **sign을 뒤집는** pass, 다른 하나는 모든 칸을 `2 × mean − 자기 자신` 으로 바꾸는 pass입니다. sign 반전은 subtraction 하나, 2×는 left shift, 칸 수가 power of 2라서 mean도 arithmetic shift로 구할 수 있으므로 이 두 pass에는 **multiplier가 한 개도 들어가지 않습니다.**

이 장의 개념이 hardware에서 각각 무엇이 되는지만 미리 적어 둡니다. **amplitude**는 BRAM 한 칸에 들어가는 23-bit fixed-point integer가 되고, 그 형식을 정하는 일이 12장입니다. **normalization**은 circuit이 매 iteration 강제하는 조건이 아니라 algorithm이 저절로 지켜 주는 성질이어서, datapath에 renormalization 단계가 아예 없습니다(12.5절). **superposition**을 만드는 일은 array 전체를 같은 constant 하나로 채우는 initialization 한 번으로 끝납니다(13.2절). amplitude가 일반적으로 complex number라는 이 장의 서술은 그대로지만, Grover가 계산하는 state에서는 amplitude가 real number를 벗어나지 않는다는 것이 9.8절에서 밝혀집니다 — 그래서 우리 array에는 imaginary part 칸이 없습니다.

1.8 이 장의 요약

- **bit vs qubit**: bit의 state는 숫자(0/1), qubit의 state는 2-dimensional complex vector.
- **computational basis**: $|0\rangle = (1,0)^T$, $|1\rangle = (0,1)^T$. classical 0, 1에 대응하는 두 특별한 state.
- **superposition**: $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, amplitude α, β 는 **complex number**. "동시에 존재"이지 "모르는 것"이 아님.
- **normalization**: $|\alpha|^2 + |\beta|^2 = 1$ (state vector 길이 1). measurement probability의 sum이 1이기 때문.
- $|+\rangle, |-\rangle$: uniform superposition이지만 $|1\rangle$ sign이 다른 두 state. phase 차이가 algorithm의 재료.
- **숨은 정보**: amplitude는 직접 measurement 불가. qubit에서 꺼내는 classical information은 1 bit.

다음 장에서는 이 vector들을 능숙하게 다루기 위한 도구 — **Dirac notation과 inner product** — 을 배웁니다. 두 state가 얼마나 "닮았는지"를 재는 방법입니다.

2장. Dirac notation과 linear algebra

> **이 장을 읽기 위한 준비**: 1장의 ket·superposition·normalization. complex conjugate와 vector transpose(→ 필요하면 부록 A).

1장에서 qubit state를 ket $|\psi\rangle$ 으로 썼습니다. 이 장에서는 그 짝인 **bra** $\langle\psi|$ 를 도입하고, 둘을 합친 **inner product**를 배웁니다. inner product는 "두 state가 얼마나 닮았는가"를 재는 도구이며, measurement probability·normalization·unitary condition을 모두 이 도구로 표현할 수 있습니다. 이 notation(Dirac notation)에 익숙해지면 이후 모든 계산이 쉬워집니다.

2.1 ket과 bra

1장에서 본 ket은 **column vector** 입니다.

$$|\psi\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}$$

여기에 짝을 이루는 대상을 정의합니다. 같은 ψ 를 쓰되 꺾쇠 방향이 반대인 **bra**:

$$\langle\psi| := \begin{bmatrix} \alpha^* & \beta^* \end{bmatrix}$$

즉 bra $\langle\psi|$ 는 ket의 component를 **complex conjugate**하고 **가로로 눕힌(transpose한)** row vector입니다. ket이 "ket", bra가 "bra"인데, 둘을 합치면 "bra-ket = bracket(괄호)"이 됩니다. 물리학자 Paul Dirac(1939)의 말장난입니다.

💡 **비유**: ket $|\psi\rangle$ 이 세로로 선 화살표라면, bra $\langle\psi|$ 는 그 화살표를 inner product 계산에 쓰는 가로 slot으로 바꾼 것입니다.

2.2 conjugate transpose(dagger, †) operation

ket을 bra로 바꾸는 operation에는 이름이 있습니다. **conjugate transpose(adjoint)** 또는 **dagger operation**, 기호로 † 입니다.

$$|\psi\rangle^\dagger = \langle\psi|, \qquad \langle\psi|^\dagger = |\psi\rangle$$

matrix에 대해서도 정의됩니다. **transpose한 뒤 모든 entry를 complex conjugate**하면 됩니다:

$$U^\dagger := (U^T)^* \qquad \Longrightarrow \qquad \begin{bmatrix} a & b \\ c & d \end{bmatrix}^\dagger = \begin{bmatrix} a^* & c^* \\ b^* & d^* \end{bmatrix}$$

이 † 는 "Hermitian conjugate"라고도 부릅니다. 3장에서 unitary condition $U^\dagger U = I$ 를 정의할 때 핵심 도구가 됩니다.

> 🔑 **유용한 규칙**: product의 dagger는 순서를 뒤집습니다. $(A|\psi\rangle)^\dagger = \langle\psi|A^\dagger$. 신발을 신고 양말을 신었다면, 되돌릴 때는 양말을 먼저 벗고 신발을 벗는 것과 같습니다.

2.3 inner product — 두 state가 얼마나 닮았나

이제 bra와 ket을 곱합니다. bra $\langle\phi|$ (row vector, 1×2) 와 ket $|\psi\rangle$ (column vector, 2×1) 를 곱하면 **scalar(숫자)** 하나가 나옵니다. 이것이 **inner product** 입니다.

$$\langle\phi|\psi\rangle = \begin{bmatrix} \gamma^* & \delta^* \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \gamma^*\alpha + \delta^*\beta$$

(여기서 $|\phi\rangle = \gamma|0\rangle + \delta|1\rangle$.) 표기할 때 가운데 세로줄 두 개 $\langle\phi||\psi\rangle$ 를 하나로 줄여 $\langle\phi|\psi\rangle$ 로 씁니다.

inner product의 의미:

- $\langle\phi|\psi\rangle = 0$ 이면 두 state는 **orthogonal**, 즉 완전히 구별 가능.
- $\langle\phi|\psi\rangle$ 의 magnitude가 1에 가까우면 두 state가 거의 같음.

🔍 **계산 따라가기 — $|0\rangle$ 과 $|1\rangle$ 은 orthogonal한가?**

$$\langle 0|1\rangle = \begin{bmatrix}1 & 0\end{bmatrix}\begin{bmatrix}0\\1\end{bmatrix} = 1\cdot 0 + 0\cdot 1 = 0 \quad \checkmark$$

orthogonal합니다. 그래서 measurement로 $|0\rangle$ 과 $|1\rangle$ 을 완벽히 구별할 수 있습니다. 또한 각각의 자기 자신과의 inner product:

$$\langle 0|0\rangle = 1, \qquad \langle 1|1\rangle = 1$$

즉 $\{|0\rangle, |1\rangle\}$ 은 **orthonormal basis**입니다.

✏️ **연습 2-1**: $|+\rangle$ 과 $|-\rangle$ 은 orthogonal할까요? $\langle +|-\rangle$ 을 계산해 보세요. *(answer: 0. orthogonal합니다 — 10장에 풀이.)*

2.4 inner product로 표현한 길이(normalization)

1장의 normalization condition은 inner product로 간단히 표현됩니다. state $|\psi\rangle$ 의 **길이의 square**는 자기 자신과의 inner product입니다.

$$\||\psi\rangle\|^2 = \langle\psi|\psi\rangle = \alpha^*\alpha + \beta^*\beta = |\alpha|^2 + |\beta|^2$$

따라서 normalization condition은 한 줄로:

$$\langle\psi|\psi\rangle = 1$$

> 🔑 **핵심**: "state는 normalize되어 있다"는 곧 $\langle\psi|\psi\rangle = 1$. 이 조건이 3장 gate가 unitary여야 하는 이유, 4장 measurement probability의 sum이 1인 이유와 직접 연결됩니다.

2.5 unit vector $|e_i\rangle$ 와 matrix entry 뽑기

계산에 유용한 도구 하나. $|e_i\rangle$ 를 "i 번째 component만 1이고 나머지는 0인 vector"라고 하면, qubit에서는 $|e_0\rangle = |0\rangle$, $|e_1\rangle = |1\rangle$ 입니다.

linear algebra의 기본 사실 두 가지:

- $A|e_k\rangle$ 는 matrix A 의 k **번째 column**.
- $\langle e_i|A|e_k\rangle$ 는 matrix A 의 (i,k) **entry**.

이 "샌드위치" $\langle\cdot|A|\cdot\rangle$ 형태는 앞으로 자주 나옵니다. 예를 들어 3장에서 gate의 특정 entry를 확인하거나, 9장에서 diffusion operator를 분석할 때 씁니다.

2.6 outer product — ket 곱하기 bra

inner product가 "bra × ket = scalar"였다면, 순서를 뒤집은 **outer product** "ket × bra"는 **matrix**를 만듭니다.

$$|\psi\rangle\langle\phi| = \begin{bmatrix}\alpha\\\beta\end{bmatrix}\begin{bmatrix}\gamma^* & \delta^*\end{bmatrix} = \begin{bmatrix}\alpha\gamma^* & \alpha\delta^*\\ \beta\gamma^* & \beta\delta^*\end{bmatrix}$$

특히 $|\psi\rangle\langle\psi|$ 는 $|\psi\rangle$ 방향으로 projection하는 operator입니다. 이 형태는 9장 Grover diffusion operator $U_\psi = 2|\psi\rangle\langle\psi| - I$ 에 쓰이므로 지금 눈에 익혀 두세요.

🔍 **계산 따라가기 — $|0\rangle\langle 0|$ 은?**

$$|0\rangle\langle 0| = \begin{bmatrix}1\\0\end{bmatrix}\begin{bmatrix}1 & 0\end{bmatrix} = \begin{bmatrix}1 & 0\\ 0 & 0\end{bmatrix}$$

이 matrix를 임의 state에 곱하면 $|0\rangle$ component만 남습니다 (projection). 확인: $|0\rangle\langle 0|(\alpha|0\rangle + \beta|1\rangle) = \alpha|0\rangle$.

다만 hardware는 이 matrix를 **만들지 않습니다.** 14 qubits면 $|\psi\rangle\langle\psi|$ 는 $16{,}384 \times 16{,}384$, 곧 2억 칸이 넘는 matrix라 FPGA에 올릴 방법이 없습니다. 그런데 $|\psi\rangle$ 가 uniform superposition일 때 $2|\psi\rangle\langle\psi| - I$ 를 amplitude array에 적용한 결과를 component별로 적어 보면, 각 칸이 `2 × mean − 자기 자신` 으로 바뀔 뿐입니다. matrix를 저장하고 곱하는 대신, array를 훑으며 sum을 구하고 빼면 됩니다(13.5절).

2.7 notation 정리 — 헷갈리지 않기

표현	이름	모양	결과
$\|\psi\rangle$	ket	column vector 2×1	state
$\langle\psi\|$	bra	row vector 1×2	ket의 conjugate transpose
$\langle\phi\|\psi\rangle$	inner product	scalar	닮은 정도 (0=orthogonal)
$\|\psi\rangle\langle\phi\|$	outer product	matrix 2×2	operator (projection 등)
$\langle\phi\|A\|\psi\rangle$	expectation value 형태	scalar	matrix entry·mean

⚠️ **함정 — 순서가 전부다**: $\langle\phi|\psi\rangle$ (scalar)와 $|\psi\rangle\langle\phi|$ (matrix)는 완전히 다른 대상입니다. bra가 앞이면 숫자, ket이 앞이면 matrix. 항상 "row vector × column vector = 숫자, column vector × row vector = matrix"로 크기를 따져보세요.

2.8 이 장의 요약

- **ket·bra**: $|\psi\rangle$ 은 column vector, $\langle\psi| = |\psi\rangle^\dagger$ 는 conjugate transpose한 row vector. 합쳐서 bra-ket(Dirac notation).
- **dagger** †: transpose + complex conjugate. matrix는 $U^\dagger = (U^T)^*$. 곱은 순서 뒤집힘 $(A|\psi\rangle)^\dagger = \langle\psi|A^\dagger$.
- **inner product** $\langle\phi|\psi\rangle$: scalar. 0이면 orthogonal(구별 가능). $\{|0\rangle, |1\rangle\}$ 은 orthonormal basis.
- **normalization**: $\langle\psi|\psi\rangle = 1$ = 길이 1. 이후 unitary condition·measurement probability를 설명하는 조건.
- **outer product** $|\psi\rangle\langle\phi|$: matrix(operator). $|0\rangle\langle 0|$ 은 projection. Grover diffusion operator $2|\psi\rangle\langle\psi| - I$ 에 쓰이는 형태.

다음 장에서는 이 state들을 **바꾸는** 방법 — quantum gate — 를 배웁니다. gate는 unitary matrix이고, dagger와 inner product가 그 정당성을 뒷받침합니다.

3장. single-qubit gate

> **이 장을 읽기 위한 준비**: 1장 qubit·superposition, 2장 inner product·dagger. matrix × vector multiplication.

qubit state를 알았으니 이제 그 state를 **바꾸는** 방법을 배웁니다. classical computer가 AND·OR·NOT gate로 bit를 조작하듯, quantum computer는 **quantum gate**로 qubit를 조작합니다. quantum gate는 모두 **unitary matrix**이며, state vector에 matrix를 곱해 gate를 적용합니다. 이 장에서 배울 $X \cdot H \cdot Z$ gate는 7장·8장·9장 algorithm의 부품이 됩니다. 특히 **Hadamard(H)** 는 superposition을 만드는 데 쓰입니다.

3.1 gate = matrix, 적용 = matrix multiplication

single-qubit gate는 2×2 **matrix** U 로 표현됩니다. 입력 state가 $|\psi\rangle$ 이면 출력은 그냥 matrix multiplication:

$$|\psi_{\text{out}}\rangle = U|\psi\rangle$$

quantum gate는 state vector에 **linear**하게 작용합니다. 즉 superposition의 각 항에 따로 작용한 뒤 더하면 됩니다:

$$U(\alpha|0\rangle + \beta|1\rangle) = \alpha\, U|0\rangle + \beta\, U|1\rangle$$

이 linearity 덕분에 gate가 $|0\rangle$ 과 $|1\rangle$ 에 무엇을 하는지만 알면 모든 state에 대한 작용을 알 수 있습니다.

3.2 X gate (NOT) — 0과 1 뒤집기

가장 간단한 gate. classical NOT처럼 $|0\rangle \leftrightarrow |1\rangle$ 을 뒤집습니다.

$$X|0\rangle = |1\rangle, \qquad X|1\rangle = |0\rangle, \qquad X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

superposition에는 linear하게 작용해 amplitude를 맞바꿉니다:

$$X(\alpha|0\rangle + \beta|1\rangle) = \alpha|1\rangle + \beta|0\rangle = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \beta \\ \alpha \end{bmatrix}$$

> ❓ **왜 이름이 X 인가?**: 물리학자 Pauli가 1927년 x 축 회전을 기술하려 도입한 matrix σ_x 에서 왔습니다. quantum computing에서는 회전 맥락은 사라지고 이름만 남아 X 가 되었습니다. NOT gate와 X gate는 같은 말입니다.

circuit notation과 시간의 흐름

quantum circuit에서 gate는 상자로, qubit는 왼쪽에서 오른쪽으로 흐르는 **quantum wire**로 그립니다. **왼쪽→오른쪽이 시간의 흐름**입니다.

```
|ψ⟩ ──[ X ]── X|ψ⟩
```

> ⚠️ **함정 — 아무것도 안 하는 circuit이 물리적으로 가장 어렵다**: gate 없는 wire(identity operation)는 수학적으로 사소하지만, 실제 hardware에서는 qubit state를 **깨지지 않게 유지**하는 것이 가장 어려운 축에 듭니다. state는 외부 영향에 매우 민감하기 때문입니다.

3.3 X를 두 번 = identity

X 를 연달아 두 번 적용하면 원래로 돌아옵니다. matrix로 확인:

$$XX = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I$$

$XX = I$ 이므로 두 개의 X gate는 identity(quantum wire)와 같습니다. 이렇게 **matrix multiplication으로 circuit을 분석**하는 것이 표준 기법입니다.

> ⚠️ **함정 — circuit 순서 vs matrix 순서는 반대**: circuit에서 먼저 H, 다음 X 를 적용하면 출력은 $XH|\psi\rangle$ 입니다 (HX 가 아님!). circuit은 왼→오로 읽지만 matrix multiplication은 오른쪽부터 작용하기 때문입니다. **"나중 gate를 왼쪽에"** 쓴다고 기억하세요.

3.4 Hadamard gate (H) — superposition을 만드는 열쇠

이 해설서에서 가장 중요한 gate입니다. H 는 computational basis를 **uniform superposition**으로 보냅니다.

$$H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} = |+\rangle, \qquad H|1\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}} = |-\rangle$$

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

🔍 **계산 따라가기 — $H|0\rangle$**

$$H|0\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \frac{|0\rangle + |1\rangle}{\sqrt{2}} \quad \checkmark$$

H 는 "$|0\rangle$ 이라는 확실한 state"를 "$|0\rangle$ 과 $|1\rangle$ 이 반반인 superposition"으로 펼칩니다.

💡 **비유 (원문의 배 이야기)**: 고대에 북아프리카에서 이베리아 반도로 가려면 아프리카·유럽을 빙 둘러 육로로 가야 했습니다. 그런데 **배**를 발명하면 바다를 가로질러 지름길로 갑니다. Hadamard 같은 gate는 classical computer가 갈 수 없는 state들("바다")에서 계산하도록 해 지름길을 마련합니다.

H를 두 번 = identity (interference의 첫 맛)

H 를 $|0\rangle$ 에 두 번 적용하면 어떻게 될까요? 직관적으로는 "더 골고루 섞이겠지" 싶지만, 실제로는 **원래 $|0\rangle$ 로 돌아옵니다.**

🔍 **계산 따라가기**: $H|0\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}}$. 여기에 다시 H:

$$H\left(\frac{|0\rangle + |1\rangle}{\sqrt{2}}\right) = \frac{1}{\sqrt{2}}\left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} + \frac{|0\rangle - |1\rangle}{\sqrt{2}}\right) = \frac{1}{2}(2|0\rangle) = |0\rangle$$

$|1\rangle$ 항은 **서로 상쇄(cancel)** 되고 $|0\rangle$ 항은 **보강(reinforce)** 됩니다. 이것이 바로 **interference** 이며, $HH = I$ 입니다.

> 🔑 **핵심 — interference가 quantum algorithm의 심장**: 많은 quantum algorithm은 (1) Hadamard로 state를 넓게 펼치고, (2) oracle로 solution에 표시한 뒤, (3) 상쇄·보강 패턴으로 solution state만 남기고 나머지를 지웁니다. 8장 Deutsch-Jozsa와 9장 Grover가 정확히 이 구조입니다.

qubit마다 H 를 하나씩 걸면($H^{\otimes \text{number of qubits}}$, 6.2절) $\text{number of states} = 2^{\text{number of qubits}}$ 개의 computational basis state가 모두 amplitude $1/\sqrt{\text{number of states}}$ 를 나눠 갖는 uniform superposition이 됩니다. 우리 project의 14 qubits라면 16,384 states이고 $1/\sqrt{16{,}384} = 1/128 = 0.0078125$ 입니다. 이 값을 23-bit fixed-point(Q1.22 — 표기의 뜻은 12.2절)에 담으면 **integer 32,768 하나**이고 rounding 없이 정확합니다. 그래서 emulator의 initialization은 $2^{14} \times 2^{14}$ 짜리 matrix를 곱하는 일이 아니라 **constant 32,768을 16,384칸에 채워 넣는 일**이 됩니다(13.2절). 수식과 circuit이 이렇게 달라 보이는 것은 우리가 matrix를 만드는 대신 그 matrix가 만들어 낼 **결과**를 곧장 적어 넣기 때문입니다.

3.5 Y·Z gate — 나머지 Pauli matrix

X 와 함께 Pauli가 도입한 두 matrix Y, Z 도 유용한 gate입니다.

$$Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, \qquad Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Z gate가 특히 중요합니다. $|0\rangle$ 은 그대로 두고 $|1\rangle$ 의 **sign만 뒤집습니다**:

$$Z|0\rangle = |0\rangle, \qquad Z|1\rangle = -|1\rangle$$

이 "sign 뒤집기 = phase 반전"이 7장 phase kickback과 9장 Grover oracle의 핵심 동작입니다. $\{X, Y, Z\}$ 를 합쳐 **Pauli matrix**라 부릅니다.

✏️ **연습 3-1**: $Z|+\rangle$ 은? *(answer: $Z\frac{|0\rangle+|1\rangle}{\sqrt{2}} = \frac{|0\rangle-|1\rangle}{\sqrt{2}} = |-\rangle$. Z 가 $|+\rangle \leftrightarrow |-\rangle$ 을 맞바꿈.)*

3.6 unitary matrix — 왜 이것만 gate가 되나

임의의 single-qubit gate는 **unitary matrix** U 입니다. unitary의 정의(대수적):

$$U^\dagger U = I$$

즉 U 의 conjugate transpose(2장)를 곱하면 identity matrix가 됩니다. 이게 왜 중요할까요?

> 🔑 **unitary = 길이 보존**: unitary matrix는 vector의 **길이를 바꾸지 않습니다**. $\|U|\psi\rangle\| = \||\psi\rangle\|$. real vector space의 회전·반사가 길이를 안 바꾸는 것의 complex number version입니다.

gate에서 이 성질이 필요한 까닭은 다음과 같습니다. 입력 state는 normalize(길이 1)되어 있고, 출력도 정당한 state이려면 길이 1이어야 합니다(1장, 4장). unitary만이 이를 보장합니다. 사실 **길이를 보존하는 matrix는 unitary뿐**이라는 정리도 성립합니다.

다만 이 보장은 amplitude를 정확한 real number로 다룰 때의 이야기입니다. amplitude를 유한한 bit 수로 적는 순간 operation마다 rounding이 끼어들어, 실제 hardware에서 norm은 정확히 1로 유지되지 않습니다. 우리 설계는 그 어긋남을 rounding error 규모로 보고 **감수하며, 매 iteration 끝에 길이를 1로 되돌리는 renormalization을 하지 않습니다**(12.5절).

🔍 **계산 따라가기 — H 는 unitary인가?**: H 는 real symmetric matrix라 $H^\dagger = H$. 따라서 $H^\dagger H = HH = I$ (3.4절에서 확인). ✓ unitary.

⚠️ **함정 — 아무 matrix나 gate가 아니다**: 예컨대 $J = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$ 은 $J|0\rangle, J|1\rangle$ 은 normalize된 state를 주지만 unitary가 아닙니다($J^\dagger J \neq I$). $J|-\rangle$ 을 계산하면 길이가 0이 되어 정당한 state가 아닙니다. 그래서 gate로 쓸 수 없습니다.

3.7 unitary의 또 다른 성질 — reversibility

$U^\dagger U = I$ 는 $U^\dagger = U^{-1}$ 을 뜻합니다. 즉 **모든 quantum gate는 inverse가 존재**하며, 그 역은 $U^\dagger$ 입니다. quantum computation은 (measurement 전까지) 항상 **되돌릴 수 있습니다(reversible)**.

이는 classical gate와 큰 차이입니다. classical AND gate는 출력 0을 보고 입력을 복원할 수 없지만(정보 손실), quantum gate는 정보를 잃지 않습니다. 6장에서 이 reversibility 때문에 classical circuit을 quantum circuit으로 옮길 때 Toffoli gate가 필요한 이유를 봅니다.

3.8 주요 single-qubit gate 요약표

gate	matrix	작용	쓰임
I (identity)	$\begin{bmatrix}1 & 0\\0 & 1\end{bmatrix}$	아무것도 안 함	quantum wire
X (NOT)	$\begin{bmatrix}0 & 1\\1 & 0\end{bmatrix}$	$\|0\rangle \leftrightarrow \|1\rangle$	bit 반전
Z	$\begin{bmatrix}1 & 0\\0 & -1\end{bmatrix}$	$\|1\rangle$ sign 반전	**phase 반전** — oracle의 sign 반전(circuit에서는 two's complement negate)
Y	$\begin{bmatrix}0 & -i\\i & 0\end{bmatrix}$	bit+phase 반전	error correction 등
H (Hadamard)	$\frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\1 & -1\end{bmatrix}$	computational basis ↔ superposition	**superposition 생성** (모든 algorithm)

이 표는 10장 cheat sheet에 다시 정리되어 있습니다. X, H, Z 셋은 반드시 손으로 계산할 수 있어야 합니다.

이 장의 matrix들이 우리 FPGA emulator에서 어떤 circuit이 되는지 한 문단에 모아 둡니다. amplitude를 memory에 펼쳐 두는 구조에서 gate 적용은 원리적으로 $2^{\text{number of qubits}} \times 2^{\text{number of qubits}}$ matrix multiplication이지만, 3.4절에서 본 것처럼 Grover가 쓰는 gate는 matrix를 만들지 않고 **작용만** 구현하면 됩니다. Z 류의 phase 반전이 특히 그렇습니다 — 표에 "-1 을 곱한다"고 적힌 연산이 circuit에서는 multiplier가 아니라 **two's complement negate**(bit 반전 후 $+1$) 한 번입니다. predicate에 걸린 칸에 대해 oracle이 amplitude에 하는 일은 이것이 전부입니다(13.4절). 덧붙이면 이 표의 다섯 gate 가운데 우리 datapath가 실제로 circuit으로 만드는 것은 H 의 **결과**와 Z 의 **sign 반전** 둘뿐입니다. $X \cdot Y$ 는 이론을 세우는 데 꼭 필요하지만, diffusion operator를 gate sequence로 분해하지 않고 inversion about the mean 산술로 곧장 계산하기 때문에 circuit에는 등장하지 않습니다(13.5절).

3.9 이 장의 요약

- **gate = unitary 2×2 matrix**, 적용 = matrix multiplication $U|\psi\rangle$. linear라서 superposition 각 항에 따로 작용.
- X **(NOT)**: $|0\rangle \leftrightarrow |1\rangle$. $XX = I$.
- H **(Hadamard)**: $|0\rangle \to |+\rangle$, $|1\rangle \to |-\rangle$. superposition을 만드는 핵심. $HH = I$ — $|1\rangle$ 상쇄·$|0\rangle$ 보강의 **interference**.
- Z: $|1\rangle$ sign 반전 = phase 반전. oracle의 부품.
- **unitary** $U^\dagger U = I$: 길이 보존 ⇒ normalization 유지 ⇒ 정당한 gate. 길이 보존 matrix는 unitary뿐.
- **reversibility**: 모든 gate는 inverse $U^\dagger = U^{-1}$ 가 존재. quantum computation은 measurement 전까지 되돌릴 수 있음.
- **circuit vs matrix 순서 반대**: circuit "먼저 H 다음 X" = matrix XH.

다음 장에서는 계산의 마지막 단계 — **measurement** — 를 배웁니다. 직접 읽을 수 없던 amplitude가 어떻게 probability로 바뀌고 state가 어떻게 collapse하는지를 봅니다.

4장. measurement

> **이 장을 읽기 위한 준비**: 1장 amplitude·normalization, 3장 gate(H).

지금까지 qubit state를 만들고(1장) gate로 변형했습니다(3장). 하지만 state vector 자체는 숨은 정보라 직접 볼 수 없습니다(1.6절). 계산 결과를 꺼내려면 **measurement**를 해야 합니다. 이 장에서는 measurement가 amplitude를 어떻게 probability로 바꾸는지(Born rule), measurement 후 state가 어떻게 collapse하는지, 그리고 왜 normalization condition이 여기서 나오는지를 배웁니다. measurement는 모든 quantum algorithm의 **마지막 단계**이며, Grover가 "solution을 높은 probability로" 준다는 말의 의미가 여기서 정확해집니다.

4.1 computational basis measurement와 Born rule

qubit가 $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ state에 있습니다. 이를 **computational basis measurement**를 하면:

- probability $|\alpha|^2$ 로 결과 **0** 을 얻고,
- probability $|\beta|^2$ 로 결과 **1** 을 얻습니다.

이 규칙 — amplitude의 absolute value square가 probability — 을 **Born rule**이라 부릅니다. measurement 결과는 평범한 **classical bit** 하나입니다. 이 classical information을 가지고 이후 다른 처리를 할 수 있습니다.

🔍 **계산 따라가기 — $|+\rangle$ 을 measurement하면?**

$|+\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$ 이므로:

$$\text{probability}(0) = \left|\tfrac{1}{\sqrt{2}}\right|^2 = \tfrac{1}{2}, \qquad \text{probability}(1) = \left|\tfrac{1}{\sqrt{2}}\right|^2 = \tfrac{1}{2}$$

50:50. 동전 던지기와 같습니다.

✏️ **연습 4-1**: state $|\psi\rangle = \frac{1}{2}|0\rangle + \frac{\sqrt{3}}{2}|1\rangle$ 를 measurement하면 결과 1이 나올 probability는? *(answer: $\frac{3}{4}$. 10장에 풀이.)*

4.2 argmax는 왜 measurement가 아닌가

Born rule은 amplitude 하나하나에 **probability**를 배정합니다. 그런데 state vector를 memory array에 펼쳐 놓고 계산하는 emulator에서는 다른 방법을 생각해 볼 수 있습니다. amplitude를 전부 읽을 수 있으니 random number 같은 것 없이 **가장 큰 amplitude를 가진 칸**을 골라 답으로 내놓으면 되지 않을까요? algorithm이 solution 쪽 amplitude를 키워 놓았을 테니 제일 높은 막대가 solution일 것이고, probability에 따라 sample을 뽑는 과정을 circuit으로 만들 필요도 없어 보입니다. maximum의 **위치**를 고르는 이 연산을 **argmax**라고 부릅니다.

argmax는 measurement가 아닙니다. Born rule이 정의하는 것은 $|a_i|^2$ 이라는 **distribution**이고 measurement는 그 distribution에서 sample을 하나 뽑는 일인데, argmax는 distribution을 받아 순위 1등만 남기고 나머지를 통째로 버립니다. probability 0.9로 나올 답과 0.5로 나올 답이 argmax에게는 똑같이 1등이고, probability 0.49짜리 답은 아예 없는 것과 같아집니다. 이렇게 하면 "높은 probability로 solution을 준다"는 quantum algorithm의 동작을 재현하지 못합니다.

더 결정적인 문제는 **tie**입니다. predicate를 만족하는 답이 여럿인 문제 — 예컨대 "threshold보다 작은 값이 든 칸을 전부 찾아라" — 를 생각해 보면, algorithm은 그 답들을 완전히 대칭으로 다룹니다. 출발 amplitude가 서로 같고 이후의 연산이 index를 구별하지 않으므로, solution 여러 개의 amplitude는 매 순간 **정확히 같은 값**을 유지하며 커집니다. 그러면 argmax circuit이 내놓는 것은 comparator의 우선순위가 정한 index **하나**이고, circuit이 결정론적이니 몇 번을 다시 돌려도 같은 하나입니다. 나머지 solution들은 probability가 낮아서 못 나오는 것이 아니라 **원리적으로 나올 수 없습니다.** predicate를 만족하는 element를 전부 enumeration해야 하는 우리 accelerator에서는 이 방식으로 목적을 달성할 수 없습니다. 그래서 Measurement Unit을 comparator가 아니라 probability sampler로 만듭니다. 이 논증을 숫자로 확인하는 자리는 14.3절입니다.

4.3 classical hardware로 Born rule 흉내내기

진짜 quantum 장치에서는 물리적 과정으로 이 sampling이 이루어지지만, emulator에서는 우리가 circuit으로 만들어야 합니다. 방법은 통계학에서 **inverse transform sampling**이라 부르는 고전적인 기법이고, 필요한 부품은 random number generator 하나와 accumulator 하나입니다.

amplitude $a_0, a_1, \ldots, a_{\text{number of states}-1}$ 의 absolute value square를 앞에서부터 차례로 더한 **prefix sum**을 생각합니다.

$$\text{prefix sum}(i) = \sum_{k \le i} |a_k|^2$$

prefix sum은 0에서 출발해 1까지 올라가는 계단이고, i 번째 계단의 높이 차가 정확히 $|a_i|^2$ 입니다. $[0, 1)$ 에서 random number r 을 균등하게 하나 뽑은 뒤 $\text{prefix sum}(i) > r$ 이 되는 **첫 번째** i 를 답으로 씁니다. i 가 뽑히려면 r 이 interval $[\text{prefix sum}(i-1),\ \text{prefix sum}(i))$ 안에 떨어져야 하는데 그 interval의 길이가 바로 $|a_i|^2$ 이므로 이 절차는 Born rule을 정확히 재현합니다. 4.1절의 동전 던지기를 state 하나가 면 하나인 주사위로 늘린 것뿐입니다.

이 방법에는 hardware 설계에 유용한 성질이 있습니다. **probability mass는 index에 붙어 있지, 훑는 순서에 붙어 있지 않습니다.** 0번부터 차례로 훑든, array를 여러 조각으로 나눠 조각을 먼저 고르고 그 안에서 다시 고르든, 각 index가 뽑힐 probability는 $|a_i|^2$ 그대로입니다. 훑는 순서를 바꿀 수 있으므로 amplitude를 한 칸씩 직렬로 훑는 대신 여러 칸을 동시에 읽는 병렬 구조를 쓸 수 있고, 그것이 row를 먼저 고르고 그 안에서 칸을 고르는 우리 Measurement Unit의 계층 구조입니다(14.5절).

이렇게 뽑아낸 measurement 결과는 결국 **index 번호 하나**입니다. qubit 하나에 bit 하나씩이라, 우리 설계의 14 qubits라면 16,384칸 중 몇 번째인지를 가리키는 14-bit integer 하나이고, 이 값이 status register를 거쳐 host로 올라갑니다(17.4절). amplitude array에는 real number가 16,384개나 들어 있지만 밖으로 나오는 것은 번호 하나뿐입니다. array를 통째로 읽어 낼 수도 있는 emulator가 굳이 그러지 않는 이유는, 실기 quantum에서 가능한 동작만 circuit으로 재현하려는 것입니다(1.6절).

4.4 measurement는 state를 collapse시킨다

measurement의 가장 중요한 성질: **measurement는 state를 바꿉니다.** measurement 후 state는 관측된 결과에 해당하는 computational basis state로 **collapse**합니다.

- 결과가 0이면 → measurement 후 state는 $|0\rangle$.
- 결과가 1이면 → measurement 후 state는 $|1\rangle$.

collapse 후에는 α, β **가 완전히 사라집니다.** measurement 후 state $|0\rangle$ 이나 $|1\rangle$ 에는 원래 amplitude의 흔적이 없습니다.

> ⚠️ **함정 — 한 번 measurement하면 되돌릴 수 없다**: gate는 reversible하지만(3.7절), **measurement는 irreversible**입니다. measurement 하는 순간 superposition이 파괴됩니다. 그래서 algorithm은 계산을 다 끝낸 뒤에 measurement를 하며, **한 번의 실행에서 measurement는 한 번뿐**입니다.

💡 **비유**: superposition state는 빠르게 돌고 있는 동전입니다. "measurement"는 손으로 탁 눌러 앞/뒤를 확정하는 것입니다. 누른 뒤에는 그냥 앞면(또는 뒷면)일 뿐, "얼마나 빠르게 돌고 있었는지(α, β)"는 알 수 없습니다.

measurement가 확률적이면서 irreversible하다는 두 성질을 합치면 quantum computation의 실행 단위가 정해집니다. state를 준비하고, gate를 걸고, 마지막에 measurement로 classical bit를 꺼내면 한 번의 실행이 끝납니다. 이 한 번을 **shot**이라고 부릅니다. 한 shot이 주는 것은 probability distribution에서 뽑은 sample 하나뿐이므로, 원하는 답을 얻으려면 대개 **실행 자체를 여러 번 반복**합니다. Grover도 예외가 아닙니다 — Grover iteration을 몇 번 돌릴지 정확히 맞추면 한 shot으로 거의 확실하게 답이 나오지만, solution이 몇 개인지 몰라 그 횟수를 맞출 수 없으면 shot을 여러 번 실행하며 답을 찾아야 합니다. 우리 설계의 조건(14 qubits, 1 solution, board benchmark와 같은 100 workloads)에서 그 mean이 **22.7 shots**입니다. 흔히 짐작하는 "네댓 번"이 아니고, shot을 몇 번 어떻게 실행할지 정하는 바깥 loop가 16장의 주제입니다.

그런데 실패한 shot이 무엇을 남기는가에서 물리 장치와 emulator가 갈라집니다. 진짜 quantum 장치에서는 measurement가 state를 collapse시키므로, 방금까지 키워 놓은 amplitude가 사라지고 다음 shot은 반드시 처음부터 다시 준비해야 합니다. 반면 emulator의 measurement는 4.3절의 절차 그대로 **array를 읽어 square의 prefix sum을 쌓는 일**이고, 읽기는 array를 바꾸지 않습니다. 강제되는 collapse가 없으니 shot이 실패한 뒤에도 amplitude array에는 직전까지 계산해 둔 state가 고스란히 남아 있습니다. 이 남은 state를 버리지 않고 몇 벌 남겨 두었다가 다음 shot의 출발점으로 재활용하는 것이 우리 설계의 **checkpoint**이고(16.5절), 이는 "emulator의 measurement는 읽기 전용"이라는 성질을 이용한 것입니다. 실기 quantum에서는 할 수 없는 일이므로, 이런 재활용으로 계산 결과가 달라지지는 않는지 따로 확인해야 합니다(16.6절).

4.5 normalization condition의 진짜 이유

이제 1장에서 미뤄둔 질문에 답할 수 있습니다. 왜 $|\alpha|^2 + |\beta|^2 = 1$ 이어야 할까요?

measurement 결과는 0 아니면 1, 둘 중 하나입니다. 두 probability의 sum은 반드시 1이어야 합니다:

$$\text{probability}(0) + \text{probability}(1) = |\alpha|^2 + |\beta|^2 = 1$$

normalization condition은 곧 "measurement probability의 sum이 1"이라는 조건입니다. 그리고 gate가 unitary(길이 보존)여야 하는 이유도 여기 있습니다(3.6절): gate가 길이를 바꾸면 measurement probability의 sum이 1이 아니게 되어 버립니다.

> 🔑 **핵심 — 모든 게 맞물린다**: state는 normalization(길이 1) → measurement probability의 sum이 1이라서. gate는 unitary(길이 보존) → normalization을 유지해야 하니까. measurement는 Born rule($|\text{amplitude}|^2$) → 이 셋이 하나의 일관된 체계입니다.

4.6 circuit에서의 measurement notation

circuit에서 measurement는 계량기(반원) 기호로 그리고, 나온 classical bit는 **이중선**으로 표시합니다.

```
|ψ⟩ ─[ H ]─| D |═══ m  (classical bit 0 또는 1)
```

measurement 후 qubit는 보통 버려지므로(discard) 오른쪽에 quantum 선을 그리지 않는 것이 관례입니다.

4.7 왜 computational basis measurement만 배우면 되는가

measurement에는 다른 종류(다른 basis에서의 measurement)도 있지만, **computational basis measurement + gate** 조합으로 임의의 measurement를 흉내 낼 수 있습니다. 그래서 원리적으로는 computational basis measurement만 알면 충분합니다.

예를 들어 "$|+\rangle$ basis로 measurement"하고 싶으면, 먼저 H gate를 적용한 뒤 computational basis로 measurement하면 됩니다. 이것이 다음 절의 핵심 예제입니다.

4.8 결정적 예제 — $|+\rangle$ 과 $|-\rangle$ 구별하기

1장에서 $|+\rangle$ 과 $|-\rangle$ 은 "measurement하면 둘 다 50:50이라 그냥은 구별 못 한다"고 했습니다. 그런데 gate를 쓰면 **완벽히** 구별할 수 있습니다.

핵심은 H 가 자기 자신의 inverse라는 것($HH = I$, 3.4절):

$$H|+\rangle = |0\rangle, \qquad H|-\rangle = |1\rangle$$

🔍 **계산 따라가기**: $|+\rangle = H|0\rangle$ 이므로 $H|+\rangle = HH|0\rangle = |0\rangle$. 마찬가지로 $|-\rangle = H|1\rangle$ 이므로 $H|-\rangle = |1\rangle$.

따라서 circuit "**H 적용 → measurement**" 를 쓰면:

- 입력이 $|+\rangle$ 이면 → 항상 0 (probability 1)
- 입력이 $|-\rangle$ 이면 → 항상 1 (probability 1)

computational basis에서는 구별 불가능하던 두 state가, H 하나로 **결정적으로** 구별됩니다.

> 🔑 **핵심 — 이것이 quantum algorithm의 축소판**: "measurement 직전에 적절한 gate를 걸어, 숨은 phase 정보(+/− sign)를 measurement 가능한 형태(0/1)로 바꾼다." 8장 Deutsch-Jozsa와 9장 Grover의 마지막 Hadamard가 정확히 이 일을 합니다.

4.9 global phase는 무의미하다

measurement probability가 $|\text{amplitude}|^2$ 이라는 사실에서 중요한 corollary가 나옵니다. state 전체에 magnitude 1인 complex number $e^{i\theta}$ 를 곱한 것 — **global phase** — 는 아무 관측 결과에도 영향을 주지 않습니다.

$$|\psi\rangle \quad \text{와} \quad e^{i\theta}|\psi\rangle \quad \text{는 물리적으로 같은 state}$$

왜냐하면 $|e^{i\theta}\alpha|^2 = |\alpha|^2$ 이라 모든 measurement probability가 동일하기 때문입니다.

> ⚠️ **global phase vs relative phase**: global phase(**모든** 항에 같은 $e^{i\theta}$)는 무의미하지만, 항들 **사이의** relative phase(예: $|+\rangle$ 의 $+$ 와 $|-\rangle$ 의 $-$)는 **매우 중요**합니다. 4.8절에서 봤듯 relative phase는 gate로 관측 가능해집니다. Grover에서 oracle이 만드는 sign -1 도 relative phase라 의미가 있습니다.

이 때문에 quantum computing에서는 X 와 $-X$ 를 "global phase만큼 같다"며 구별하지 않기도 합니다. 9장 Grover에서 state 전체에 -1 을 곱하는 자리가 나오는데, 이는 결과에 영향이 없어 무시할 수 있습니다.

4.10 이 장의 요약

- **Born rule**: $\alpha|0\rangle + \beta|1\rangle$ measurement 시 probability $|\alpha|^2$ 로 0, $|\beta|^2$ 로 1. 결과는 classical bit.
- **argmax는 measurement가 아니다**: maximum의 위치를 고르는 것은 distribution을 버리는 일. solution이 여럿이면 amplitude가 정확히 같아 늘 같은 하나만 나오고, 나머지는 **원리적으로** 나올 수 없음.
- **inverse transform sampling**: $\text{prefix sum}(i) = \sum_{k \le i} |a_k|^2$ 에 uniform random number를 떨어뜨려 Born rule을 circuit으로 재현. 훑는 순서를 바꿔도 distribution은 같으므로 병렬화가 가능.
- **collapse**: measurement 후 state는 $|0\rangle$ 또는 $|1\rangle$ 로 collapse, amplitude α, β 는 소멸. **irreversible**.
- **shot**: 준비 → gate → measurement 한 번이 한 번의 실행. sample 하나만 주므로 실행을 여러 번 반복함.
- **emulator의 measurement는 읽기 전용**: 강제되는 collapse가 없어 실패한 shot 뒤에도 state가 남음 — checkpoint의 뿌리.
- **normalization의 이유**: $|\alpha|^2 + |\beta|^2 = 1$ = measurement probability의 sum. gate가 unitary인 이유와 한 체계.
- **computational basis measurement + gate**로 임의 measurement 흉내 가능.
- H**→measurement로** $|+\rangle, |-\rangle$ **결정적 구별**: 숨은 phase → measurement 가능한 0/1. algorithm의 축소판.
- **global phase 무의미**: $e^{i\theta}|\psi\rangle \equiv |\psi\rangle$. 단, 항들 사이 **relative phase**는 결정적으로 중요.

다음 장에서는 qubit를 여러 개로 늘립니다. tensor product로 dimension이 $2^{\text{number of qubits}}$ 인 state space가 열리고, CNOT gate와 **entanglement**라는 순수 quantum 현상을 만납니다.

5장. multi-qubit와 entanglement

> **이 장을 읽기 위한 준비**: 1장 state·superposition, 3장 gate, 4장 measurement.

지금까지는 qubit 하나만 다뤘습니다. 하지만 유용한 계산을 하려면 여러 qubit가 필요하고, qubit들이 **서로 상호작용**해야 합니다. 이 장에서는 여러 qubit의 state가 어떻게 표현되는지(qubit가 하나 늘 때마다 dimension이 두 배가 됩니다), qubit를 상호작용시키는 **CNOT gate**, 그리고 순수 quantum 현상인 **entanglement**를 배웁니다. 이렇게 지수적으로 커지는 dimension이 바로 0장에서 말한 quantum computer의 힘의 원천이며, 9장 Grover가 qubit 몇 개로 $2^{\text{number of qubits}}$ 개 항목을 한꺼번에 다루는 근거입니다.

5.1 두 qubit의 state — 네 개의 computational basis

qubit 하나의 computational basis는 $|0\rangle, |1\rangle$ 둘이었습니다. qubit **둘**이면 classical bit 2개처럼 네 가지 조합이 가능합니다:

$$|00\rangle, \quad |01\rangle, \quad |10\rangle, \quad |11\rangle$$

그리고 이들의 임의의 superposition도 가능합니다:

$$|\psi\rangle = \alpha|00\rangle + \beta|01\rangle + \gamma|10\rangle + \delta|11\rangle$$

amplitude $\alpha, \beta, \gamma, \delta$ 는 complex number이고, normalization condition은 마찬가지로:

$$|\alpha|^2 + |\beta|^2 + |\gamma|^2 + |\delta|^2 = 1$$

measurement하면 네 결과 00, 01, 10, 11 중 하나가 각각 $|\alpha|^2, |\beta|^2, |\gamma|^2, |\delta|^2$ 의 probability로 나옵니다(4장 Born rule의 자연스러운 확장).

5.2 $2^{\text{number of qubits}}$ — 지수적으로 커지는 state space

일반적으로 qubit가 여러 개면 computational basis state는 $\text{number of states} = 2^{\text{number of qubits}}$ 개입니다.

number of qubits	number of states	예
1	2	$\|0\rangle, \|1\rangle$
2	4	$\|00\rangle, \ldots, \|11\rangle$
3	8	$\|000\rangle, \ldots, \|111\rangle$
10	1,024	
300	2^{300}	우주의 원자 수보다 많음

qubit 여러 개의 state를 완전히 기술하려면 $2^{\text{number of qubits}}$ 개의 complex amplitude가 필요합니다. 이것이 0장에서 말한 핵심입니다: **300 qubits면 classical computer가 도저히 저장할 수 없는 수의 amplitude를 자연스럽게 담습니다.**

> 🔑 **핵심 — 이것이 quantum parallelism의 근원**: 모든 qubit에 각각 H gate를 걸면(6장) $2^{\text{number of qubits}}$ 개 computational basis state가 **전부 동시에** 담긴 uniform superposition이 만들어집니다. oracle 한 번으로 이 state 전부를 한꺼번에 처리하는 것이 9장 Grover의 출발점입니다. 3 qubits = 8 states, ..., search space $\text{number of states} = 2^{\text{number of qubits}}$.

3-qubit 예제 (원문 notation)

원문에서 3 qubits uniform superposition을 이렇게 씁니다:

$$|\psi\rangle = \sum_{x=0}^{7} \frac{1}{\sqrt{8}}|x\rangle = \frac{1}{\sqrt{8}}\big(|000\rangle + |001\rangle + \cdots + |111\rangle\big)$$

여덟 state가 모두 amplitude $\frac{1}{\sqrt{8}}$ 로 동등하게 담깁니다. ($\frac{1}{\sqrt{8}} = \frac{1}{\sqrt{2^3}}$, normalization: $8 \times \frac{1}{8} = 1$.) 9장에서 이 state가 Grover의 시작점이 됩니다.

amplitude가 complex number라는 것은 **일반적인** state의 이야기입니다. 우리가 만들 emulator는 표준 Grover 하나만 돌리는데, 그 algorithm의 연산 과정 — uniform superposition에서 출발해 sign을 뒤집고 inversion about the mean을 하는 것 — 에서는 amplitude가 항상 real number로 유지됩니다. 그래서 amplitude 하나당 real part·imaginary part 두 칸을 잡을 필요 없이 **real-valued fixed-point 한 벌**만 저장합니다. 여기서는 결론만 알아두고, 이 성질이 성립하는 세 조건은 9.8절에서 설명합니다. 다만 이 저장 방식에는 제약이 있어서, amplitude에 complex phase를 직접 곱하는 기법을 얹는 순간 datapath를 complex number에 맞게 다시 설계해야 합니다 — solution이 몇 개인지 세는 표준 기법인 quantum counting(phase estimation)을 설계에서 제외한 이유가 그것입니다(16.1절).

5.3 tensor product — 두 qubit를 합치기

qubit 두 개를 각각 $|a\rangle = \alpha|0\rangle + \beta|1\rangle$, $|b\rangle = \gamma|0\rangle + \delta|1\rangle$ 로 준비한 뒤 나란히 놓으면, 합쳐진 state는 **tensor product**로 계산됩니다. 그냥 distributive law처럼 전개하면 됩니다:

$$|a\rangle \otimes |b\rangle = (\alpha|0\rangle + \beta|1\rangle)(\gamma|0\rangle + \delta|1\rangle) = \alpha\gamma|00\rangle + \alpha\delta|01\rangle + \beta\gamma|10\rangle + \beta\delta|11\rangle$$

notation: $|0\rangle|0\rangle$, $|0\rangle \otimes |0\rangle$, $|00\rangle$ 은 모두 같은 뜻입니다. 여러 gate를 여러 qubit에 동시에 적용할 때도 tensor product를 씁니다. 예를 들어 $H^{\otimes \text{number of qubits}}$ 는 "모든 qubit 각각에 H" 를 뜻하며(6장), Grover와 Deutsch-Jozsa의 첫 단계입니다.

💡 **비유**: tensor product는 "각 qubit가 독립적으로 가질 수 있는 state를 Cartesian product처럼 전부 나열"하는 것입니다. qubit가 늘수록 조합이 곱으로 불어나 $2^{\text{number of qubits}}$ 이 됩니다.

5.4 CNOT gate — qubit를 상호작용시키기

single-qubit gate만으로는 qubit들이 서로 영향을 못 줍니다. 계산하려면 **two-qubit gate**가 필요합니다. 대표가 **CNOT(controlled-NOT)** 입니다.

두 개의 wire(qubit)가 있고, 하나는 **control qubit**, 다른 하나는 **target qubit**입니다. 규칙은 간단합니다:

> **control qubit가 1이면 target qubit를 뒤집는다(NOT). 아니면 아무것도 안 한다.**

네 computational basis state에 대한 작용:

$$|00\rangle \to |00\rangle, \quad |01\rangle \to |01\rangle, \quad |10\rangle \to |11\rangle, \quad |11\rangle \to |10\rangle$$

앞 qubit(control qubit)가 1인 마지막 두 경우에만 뒤 qubit가 뒤집혔습니다. 한 줄 공식으로:

$$|x, y\rangle \to |x,\ y \oplus x\rangle$$

여기서 $\oplus$ 는 **modulo-2 addition(XOR)**입니다 ($1 \oplus 1 = 0$). classical programming의 `if (control) target = NOT target;` 와 같습니다.

> ❓ **왜 XOR인가?**: quantum computing의 CNOT이 classical computing의 XOR에 대응하기 때문입니다. 이 $\oplus$ notation은 7장 oracle $U_f|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle$ 에서 다시 핵심으로 등장합니다.

computational basis $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$ 순서로 matrix로 쓰면 이렇습니다 — 앞 두 computational basis state($|00\rangle, |01\rangle$)는 그대로 두고, 뒤 두 computational basis state($|10\rangle, |11\rangle$)의 자리만 맞바꾸는 형태입니다:

$$\text{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

CNOT도 물론 unitary이고($\mathrm{CNOT}^\dagger\,\mathrm{CNOT} = I$), superposition에는 linear하게 작용합니다.

5.5 entanglement — 나눌 수 없는 state

다음 계산을 살펴봅시다. $|00\rangle$ 에서 시작해 **첫 qubit에 H, 그다음 CNOT** 을 적용해 봅시다.

🔍 계산 따라가기

1단계 — 첫 qubit에 H: $|0\rangle \to \frac{|0\rangle+|1\rangle}{\sqrt{2}}$ 이므로 두 qubit state는

$$|00\rangle \xrightarrow{H\otimes I} \frac{|00\rangle + |10\rangle}{\sqrt{2}}$$

2단계 — CNOT (첫=control qubit, 둘째=target qubit): $|00\rangle$ 은 control qubit가 0이라 그대로, $|10\rangle$ 은 control qubit가 1이라 target qubit를 뒤집어 $|11\rangle$:

$$\frac{|00\rangle + |10\rangle}{\sqrt{2}} \xrightarrow{\text{CNOT}} \frac{|00\rangle + |11\rangle}{\sqrt{2}}$$

이 출력 state $\frac{|00\rangle+|11\rangle}{\sqrt{2}}$ 를 **Bell state**라 하고, **entangled state**의 대표 예입니다.

왜 entanglement가 특별한가

이 state는 **두 개의 독립된 single-qubit state의 tensor product로 쓸 수 없습니다.** 즉 $|a\rangle \otimes |b\rangle$ 형태로 분해가 안 됩니다 (product state가 아님).

measurement 결과는 다음과 같습니다:

- 첫 qubit의 measurement에서 0이 나오면, 둘째 qubit는 **반드시** 0.
- 첫 qubit가 1이면, 둘째는 **반드시** 1.

두 qubit의 measurement 결과가 완벽히 **correlated**되어 있습니다. 아무리 멀리 떨어뜨려도 그렇습니다. 이것이 Einstein이 "spooky action at a distance"라 부른 entanglement입니다.

> **🔑 product state vs entangled state**
>
> - **product state**: $|a\rangle \otimes |b\rangle$ 로 분해됨. 각 qubit를 독립적으로 기술 가능. 예: $|00\rangle$, $|+\rangle|-\rangle$.
> - **entangled state**: 분해 불가능. 예: $\frac{|00\rangle+|11\rangle}{\sqrt{2}}$. 부분들이 전체보다 정보가 적음.

entanglement는 quantum teleportation, quantum error correction, 그리고 빠른 quantum algorithm의 핵심 resource입니다.

⚠️ 함정 — CNOT은 target qubit만 바꾸는 게 아니다: "control qubit는 그대로, target qubit만 바뀐다"는 computational basis에서만 맞습니다. 예컨대 $|+\rangle|-\rangle$ 을 CNOT에 넣으면 $|-\rangle|-\rangle$ 이 나와 **control qubit가 바뀝니다**. computational basis 직관을 다른 basis에 그대로 옮기면 틀립니다.

✏️ 연습 5-1: $\frac{|00\rangle+|01\rangle}{\sqrt{2}}$ 는 product state일까요, entangled state일까요? *(answer: product state. 첫 qubit로 묶어내 보세요 — 10장에 풀이.)*

이처럼 분해할 수 없다는 성질이 hardware budget을 결정합니다. state가 늘 product state로 유지된다면 qubit 하나를 amplitude 두 개로 적어 $2 \times \text{number of qubits}$ 개의 수만 들고 있으면 됩니다. entangled state는 그렇게 쪼갤 수 없으니 $2^{\text{number of qubits}}$ 개의 amplitude를 **전부** 들고 있어야 하고, qubit당 몇 개의 수로 압축할 방법이 없습니다. Grover의 시작점인 uniform superposition은 아직 product state($|+\rangle^{\otimes \text{number of qubits}}$)이지만, oracle이 한 칸의 sign을 뒤집는 순간 product state로 분해되지 않는 state가 됩니다 — 2 qubits에서 $\frac{1}{2}(|00\rangle + |01\rangle + |10\rangle - |11\rangle)$ 을 연습 5-1과 같은 방법으로 묶어내 보면 실패합니다. 즉 **entanglement = 압축 불가 = 지수 memory**이고, algorithm이 도는 내내 amplitude array를 통째로 들고 있어야 한다는 뜻입니다. state vector emulator의 속도가 산술 능력보다 memory의 제약에 더 크게 좌우되는(11.6절) 근본 이유가 여기 있습니다.

5.6 qubit가 많을 때의 CNOT

CNOT은 qubit가 더 많은 계산에도 등장합니다. 예를 들어 3 qubits에서 "둘째=control qubit, 셋째=target qubit"인 CNOT은:

$$|x, y, z\rangle \to |x, y, z \oplus y\rangle$$

첫 qubit x 는 관여 안 하니 그대로, control qubit y 도 그대로, target qubit z 만 y 가 1일 때 뒤집힙니다. 이런 식으로 여러 qubit 사이에 conditional logic를 쌓아 복잡한 계산을 만듭니다.

이 장의 부품 가운데 CNOT은 우리 RTL에 module로 등장하지 않습니다. emulator가 gate를 하나씩 state vector에 적용하는 기계가 아니라, oracle과 diffusion operator라는 **operator 단위**로 amplitude array 전체를 한 번에 훑는 기계이기 때문입니다(그 차이는 바로 다음 6.1절에서 정확히 짚습니다). 그렇다고 CNOT을 흘려보내도 되는 것은 아닙니다 — 논문의 circuit diagram과 Qiskit code는 여전히 gate로 적혀 있어서, 그 그림을 읽고 우리 operator에 대응시키려면 CNOT과 $\oplus$ 표기가 필요합니다. 이 장에서 CNOT은 직접 만들 circuit이 아니라 **논문을 읽기 위한 표기**로 이해해 두면 됩니다.

5.7 이 장의 요약

- $\text{number of states} = 2^{\text{number of qubits}}$: computational basis state마다 complex amplitude 하나. 지수적 state space가 quantum parallelism의 근원.
- **tensor product**: qubit를 합칠 때 distributive law처럼 전개. $|a\rangle \otimes |b\rangle$. $H^{\otimes \text{number of qubits}}$ = 모든 qubit에 H.
- **CNOT**: control qubit가 1이면 target qubit 뒤집기. $|x, y\rangle \to |x, y \oplus x\rangle$ ($\oplus$ = XOR). qubit 상호작용의 기본.
- **entanglement**: $\frac{|00\rangle + |11\rangle}{\sqrt{2}}$ (Bell state)처럼 product state로 분해 안 되는 state. measurement 결과가 완벽히 correlated.
- **product state vs entangled state**: 분해 가능 vs 불가능. entanglement는 빠른 algorithm·teleportation의 resource.
- CNOT 직관("target qubit만 변함")은 computational basis에서만 유효.

다음 장에서는 이 부품들 — single-qubit gate + CNOT + measurement — 을 합쳐 **완전한 quantum computation model**을 정의하고, $H^{\otimes \text{number of qubits}}$ 로 uniform superposition을 만드는 법과 universality를 봅니다. algorithm 직전의 마지막 준비입니다.

6장. quantum circuit model과 universality

> **이 장을 읽기 위한 준비**: 3장 gate, 4장 measurement, 5장 multi-qubit·$2^{\text{number of qubits}}$·CNOT.

부품이 다 모였습니다: qubit(1장), single-qubit gate(3장), measurement(4장), CNOT(5장). 이 장에서는 이들을 조립해 **quantum computation이 무엇인지**를 딱 세 단계로 정의하고, algorithm에 꼭 필요한 기법 — $H^{\otimes \text{number of qubits}}$ **로 uniform superposition 만들기** — 를 정리합니다. 또 classical computation을 quantum으로 옮기는 데 필요한 Toffoli gate와 universality도 짚습니다. 이 장이 끝나면 7장부터 실제 algorithm으로 들어갑니다.

6.1 quantum computation — 딱 세 단계

일반적인 quantum computation은 다음 세 단계로 이루어집니다.

> 🔑 **quantum computation의 3단계**
>
> 1. **initialization**: 모든 qubit를 computational basis state(보통 $|00\cdots0\rangle$)로 준비한다.
> 2. **gate 적용**: single-qubit gate와 CNOT을 순서대로 적용한다.
> 3. **measurement**: computational basis로 measurement한다. 결과 $00\cdots0$ 이 나올 probability는 해당 amplitude의 absolute value square.

이 세 단계를 circuit diagram으로 그리면 다음과 같습니다:

```
|0) ─[H]──────●──────[?]──| D |= m0
              |
|0) ─[H]──[X]──[?]──| D |= m1
|0) ─[H]──────────[?]──| D |= m2
      ↑        ↑         ↑
superposition  CNOT 등  measurement
```

우리 project의 FPGA emulator도 이 3단계를 그대로 밟습니다. 다만 2단계를 **gate 하나하나의 matrix multiplication으로 처리하지 않는다**는 점이 중요합니다. ① initialization은 $H^{\otimes \text{number of qubits}}$ 라는 gate를 실제로 돌리는 대신, 그 결과 모든 칸이 $\frac{1}{\sqrt{\text{number of states}}}$ 임을 미리 알고 amplitude array 전 칸에 그 constant를 써 넣는 한 번의 쓰기 pass입니다. ② 그다음은 gate가 아니라 **oracle과 diffusion operator라는 두 operator**를 통째로 정해진 횟수만큼 반복 적용합니다. 한 번의 iteration은 32 lanes가 amplitude array를 두 번 훑는 2 passes이고, 이 경로에는 multiplier가 한 개도 없습니다(13장). ③ measurement는 amplitude의 square를 probability로 삼아 random number로 하나를 뽑는 **Born-rule sampling**입니다 — 가장 큰 amplitude를 고르는 것이 아닙니다(14.4절). 임의의 gate matrix를 받아 곱해 주는 general-purpose engine을 만들지 않는 이유는 간단합니다: Grover가 쓰는 unitary는 oracle과 diffusion operator **둘로 고정**되어 있어서, 그 둘을 전용 circuit으로 구현하는 편이 훨씬 적은 자원을 쓰기 때문입니다. 설계값의 source of truth는 15.4절입니다.

6.2 uniform superposition 만들기 — $H^{\otimes \text{number of qubits}}$

거의 모든 quantum algorithm의 **첫 단계**는 "모든 가능성을 동시에 담기" 위해 모든 qubit에 Hadamard를 거는 것입니다.

$$H^{\otimes \text{number of qubits}}|0\rangle^{\otimes \text{number of qubits}} = \frac{1}{\sqrt{2^{\text{number of qubits}}}} \sum_{x \in \{0,1\}^{\text{number of qubits}}} |x\rangle$$

즉 $|00\cdots0\rangle$ 에서 시작해 각 qubit에 H 를 걸면, $2^{\text{number of qubits}}$ 개의 모든 computational basis state가 **똑같은 amplitude** $\frac{1}{\sqrt{2^{\text{number of qubits}}}}$ 로 담긴 uniform superposition이 됩니다.

🔍 **계산 따라가기 — 2 qubits**

$$H|0\rangle \otimes H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} \otimes \frac{|0\rangle + |1\rangle}{\sqrt{2}} = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$$

네 state가 각각 amplitude $\frac{1}{2} = \frac{1}{\sqrt{4}}$. normalization: $4 \times \frac{1}{4} = 1$. ✓

> 🔑 **핵심 — "$2^{\text{number of qubits}}$ 개를 한꺼번에"**: qubit마다 H gate 하나씩이라는 아주 적은 resource로 $2^{\text{number of qubits}}$ 개 state를 동시에 담습니다. 이 uniform superposition이 8장 Deutsch-Jozsa와 9장 Grover의 출발선입니다. Grover에서는 이 state를 $|\psi\rangle$ 라 부릅니다.

Hadamard 수학 — sign까지 담은 일반식

한 qubit에서 $u \in \{0, 1\}$ 에 대해:

$$H|u\rangle = \frac{1}{\sqrt{2}}\big(|0\rangle + (-1)^u|1\rangle\big) = \frac{1}{\sqrt{2}} \sum_{v\in\{0,1\}} (-1)^{uv}|v\rangle$$

$(-1)^u$ 인수는 probability(absolute value)에는 영향이 없지만 **phase(sign)** 에는 영향을 줍니다 — 물리학자들이 **phase change**라 부르는 것. qubit 여러 개로 확장하면:

$$H^{\otimes \text{number of qubits}}|u\rangle = \frac{1}{\sqrt{2^{\text{number of qubits}}}} \sum_{v\in\{0,1\}^{\text{number of qubits}}} (-1)^{u\cdot v}|v\rangle$$

여기서 $u \cdot v$ 는 bit별 inner product입니다. 이 일반식이 8장에서 interference를 계산하는 핵심 도구가 되니, 지금은 "H 는 $(-1)^{u\cdot v}$ sign을 부여한다"만 기억해 두세요.

✏️ **연습 6-1**: $H^{\otimes 3}|000\rangle$ 에서 computational basis state $|101\rangle$ 의 amplitude는 얼마일까요? *(answer: $\frac{1}{\sqrt{8}}$. 10장에 풀이.)*

6.3 quantum parallelism과 그 한계

uniform superposition $\frac{1}{\sqrt{2^{\text{number of qubits}}}} \sum_x |x\rangle$ 에 function f 를 계산하는 oracle을 한 번 걸면, 원리적으로 $2^{\text{number of qubits}}$ 개 입력에 대한 f 값이 **한 번에** 계산됩니다. 이를 **quantum parallelism**이라 합니다.

하지만 여기 결정적인 함정이 있습니다.

> ⚠️ **함정 — parallelism만으로는 공짜 점심이 없다**: $2^{\text{number of qubits}}$ 개 답을 동시에 "계산"해도, 4장에서 배웠듯 **measurement하면 그중 딱 하나만** 무작위로 나옵니다. 나머지는 collapse로 사라집니다. 그래서 단순히 병렬 계산만으로는 이득이 없습니다.

이때 필요한 것이 **interference** 입니다: 원하는 답의 amplitude는 보강하고 원치 않는 답의 amplitude는 상쇄시켜, **measurement에서 solution이 높은 probability로 나오게** 만드는 것. 3.4절 $HH = I$에서 본 상쇄·보강이 바로 이 도구입니다. Deutsch-Jozsa와 Grover는 서로 다른 방식으로 이 interference를 설계합니다.

💡 **비유**: quantum parallelism은 "모든 복권을 동시에 사는 것"입니다. 하지만 긁을 수 있는 건 한 장뿐. interference는 "당첨 복권만 두껍게, 꽝 복권은 얇게" 조작해, 아무거나 한 장 긁어도 당첨될 probability를 높이는 기술입니다.

"measurement하면 collapse한다"는 것은 실기 quantum 장치의 이야기입니다. 우리 emulator의 amplitude는 BRAM에 적힌 숫자일 뿐이고, Measurement Unit은 그 array를 읽어 square를 probability로 삼아 하나를 뽑을 뿐 amplitude를 수정하지 않습니다 — **읽기 전용**입니다(16.5절). 그렇다고 위 함정을 피해 가는 것은 아닙니다. measurement 한 번이 답 후보 하나이고, 이 한 번을 **shot**이라 부릅니다. solution이 몇 개인지 모르는 채로 도는 우리 설계는 답 하나를 얻기까지 mean 22.7 shots가 듭니다(14 qubits·1 solution, board benchmark와 같은 100 workloads, 16.3절). collapse가 없다고 답의 개수가 늘지는 않습니다. 대신 실패한 shot이 남긴 amplitude array를 다음 shot에서 이어 쓸 수 있습니다.

6.4 classical computation을 quantum으로 — reversibility 문제

quantum computer가 classical computer보다 최소한 못하지 않으려면, 임의의 classical circuit을 quantum circuit으로 옮길 수 있어야 합니다. NOT은 쉽습니다 — 그냥 X gate. 문제는 **AND** 입니다.

AND gate는 입력 2-bit x, y → 출력 1-bit $x \wedge y$. 그런데 3.7절에서 봤듯 quantum gate는 **reversible**해야 하고, unitary gate는 들어간 만큼의 qubit가 그대로 나옵니다. 입력 2 → 출력 1인 AND는 **irreversible**이라 그대로는 quantum gate가 될 수 없습니다.

> ❓ **왜 AND는 irreversible인가?**: 출력이 0이라는 것만 보고는 입력이 $00, 01, 10$ 중 무엇이었는지 복원할 수 없습니다(정보 손실). 반면 quantum gate는 정보를 잃지 않습니다.

6.5 Toffoli gate — reversible AND

해결책은 **Toffoli gate**(controlled-controlled-NOT, CCNOT)입니다. CNOT과 비슷하지만 **control qubit가 두 개**(x, y)이고 target qubit가 하나(z)입니다. 두 control qubit가 **모두** 1일 때만 target qubit를 뒤집습니다:

$$|x, y, z\rangle \to |x, y, z \oplus (x \wedge y)\rangle$$

target qubit를 $z = 0$ 으로 시작하면 출력의 셋째 qubit가 $x \wedge y$ — 바로 AND입니다! 여분의 qubit를 하나 쓰는 대가로 reversibility를 얻습니다. Toffoli + X 로 임의의 classical circuit(AND·NOT은 universal set)을 quantum circuit으로 옮길 수 있습니다.

Toffoli 자체는 기본 gate set에 없지만, **CNOT과 single-qubit gate들로 조립**할 수 있습니다. 그 구체적 circuit은 외워둘 필요 없이 "필요하면 찾아 쓰면 되는 것"입니다.

> 🔑 **universal gate set**: single-qubit gate + CNOT 만으로 임의의 unitary 연산을 (원하는 precision으로) 근사할 수 있습니다. 이를 **universal**이라 합니다. classical computing의 "NAND만으로 모든 circuit"에 대응합니다. Toffoli는 여기에 classical logic를 얹는 다리 역할입니다.

이 oracle을 reversible하게 구현하는 아이디어 — "여분 qubit에 $\oplus f(x)$ 로 결과를 쓴다" — 가 7장 oracle $U_f|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle$ 의 정확한 형태입니다.

6.6 circuit의 순효과는 하나의 unitary

measurement 전까지 여러 gate를 순서대로 적용한 circuit 전체의 효과는 **하나의 큰 unitary matrix** $U = U_k \cdots U_2 U_1$ 입니다 (unitary의 product는 unitary). 그래서 quantum circuit = state space 위의 하나의 unitary transformation, 그리고 마지막에 measurement. 이 관점이 9장 Grover에서 "oracle과 diffusion operator의 product를 여러 번 적용"을 분석할 때 유용합니다.

이 성질은 hardware에서 계산 결과를 재사용하는 근거가 됩니다. Grover iteration 한 번을 $G = D\,O$(oracle O 뒤에 diffusion operator D)라 쓰면 이를 반복한 circuit 전체가 $G^{\text{number of iterations}}$ 라는 **하나의** unitary이고, matrix multiplication은 associative하므로 다음이 성립합니다:

$$(DO)^{\text{target iterations}} = (DO)^{\text{target iterations} - \text{completed iterations}} \cdot (DO)^{\text{completed iterations}}$$

왼쪽은 "처음부터 target iterations만큼 돌린다"이고, 오른쪽은 "이미 completed iterations만큼 돌아 있는 state에서 차이만큼만 더 돌린다"입니다. 둘은 같은 operation이므로 도달한 state도 같습니다. 이 한 줄이 뒤에서 실패한 shot의 amplitude array를 버리지 않고 이어 쓰는 **checkpoint**의 정당성 근거가 됩니다(16.6절).

6.4~6.5에서 본 reversibility 제약은 state를 물리적으로 유지하는 실기 quantum에만 적용됩니다. 우리 emulator는 unitary의 **효과**를 숫자로 재현할 뿐이라, oracle 안의 Predicate Unit을 reversible logic로 짤 이유가 없습니다 — `value < thr` 같은 비교는 subtractor 하나와 그 sign bit로 끝나고, Toffoli 분해도 여분의 ancilla qubit도 등장하지 않습니다(13.3절). universal gate set도 같은 맥락에서 이해하면 됩니다. 그것은 "이 부품들로 어떤 quantum algorithm이든 만들 수 있다"는 가능성의 정리이지, 우리가 만들 기계가 universal해야 한다는 요구가 아닙니다. Toffoli와 CNOT은 우리가 synthesis할 circuit에 쓰이지 않고, 논문의 circuit diagram을 읽는 데 쓰입니다.

6.7 이 장의 요약

- **quantum computation 3단계**: ① $|0\cdots0\rangle$ initialization → ② single-qubit gate + CNOT 적용 → ③ computational basis measurement($\text{probability} = |\text{amplitude}|^2$).
- $H^{\otimes \text{number of qubits}}|0\rangle^{\otimes \text{number of qubits}} = \frac{1}{\sqrt{2^{\text{number of qubits}}}}\sum_x |x\rangle$: 모든 algorithm의 첫 단계. qubit마다 gate 하나로 $2^{\text{number of qubits}}$ states의 uniform superposition.
- **Hadamard 일반식**: $H^{\otimes \text{number of qubits}}|u\rangle = \frac{1}{\sqrt{2^{\text{number of qubits}}}}\sum_v (-1)^{u\cdot v}|v\rangle$. sign(phase)를 뿌림 → interference의 도구.
- **quantum parallelism의 한계**: $2^{\text{number of qubits}}$ 개를 계산해도 measurement는 하나만. 진짜 비결은 **interference**(solution 보강·non-solution 상쇄).
- **reversibility**: AND는 irreversible → 그대로는 quantum gate 불가.
- **Toffoli(CCNOT)**: reversible AND. $|x,y,z\rangle \to |x,y,z\oplus(x\wedge y)\rangle$. + X 로 classical circuit을 quantum circuit으로.
- **universality**: single-qubit gate + CNOT = universal. circuit 전체 = 하나의 unitary.

여기까지가 quantum computing의 **model 전체**입니다. 다음 장(7장)부터는 이 model로 실제 algorithm을 만듭니다. 첫 주제는 algorithm이 "solution이 무엇인지"를 아는 통로 — **oracle** — 과 그 과정에 쓰이는 **phase kickback** 입니다.

7장. oracle과 phase kickback

> **이 장을 읽기 위한 준비**: 1.5절 $\|-\rangle$, 3장 $X\|Z\|H$, 4장 phase, 5장 CNOT·$\oplus$, 6장 Toffoli.

이제 algorithm을 살펴봅니다. quantum search·판별 algorithm은 모두 "우리가 찾는 solution이 무엇인지"를 알려주는 통로가 필요합니다. 그 통로가 **oracle** 입니다. 그리고 oracle을 유용하게 만드는 결정적 trick이 **phase kickback** — 함숫값을 amplitude의 **sign**에 반영하는 방법입니다. 이 장의 두 개념은 8장 Deutsch–Jozsa와 9장 Grover를 이해하는 바탕입니다. 특히 phase kickback을 이해하면 "Grover oracle이 solution state의 sign을 뒤집는다"는 말의 의미를 알 수 있습니다.

7.1 oracle이란 무엇인가

oracle은 고대 그리스의 신탁에서 따온 이름입니다. 입력을 넣으면 **1 또는 0** (참/거짓)만 돌려주는 function입니다.

$$f : \{0,1\}^{\text{number of qubits}} \to \{0,1\}$$

우리는 function의 **내부 동작은 모른다**고 가정합니다. 입력에 대해 0/1을 돌려주는 것만 알 뿐, 어떻게 그 값을 계산하는지는 관심 밖입니다. 그래서 oracle을 **black box**라고도 부릅니다.

💡 **비유**: oracle은 "solution 판별기"입니다. 어떤 상자를 열어 "이게 내가 찾던 민트 초콜릿이냐?"고 물으면 예(1)/아니오(0)만 답하는 기계. 어떻게 판별하는지는 몰라도, 물어볼 수는 있습니다.

우리 project에서 oracle은 "찾고자 하는 항목이면 1, 아니면 0"을 판별합니다. Grover는 이 oracle을 이용해 solution 항목을 찾아냅니다.

7.2 oracle을 quantum으로 — 두 가지 형태

quantum computer의 모든 연산은 unitary여야 합니다(3장). oracle function도 unitary matrix로 표현해야 하는데, 두 가지 표준 형태가 있습니다.

형태 A — bit flip oracle (여분 qubit 사용)

6장 Toffoli에서 본 가역적 implementation 방식입니다. 입력 register $|x\rangle$ 와 별도의 **출력 qubit** $|y\rangle$ 를 두고, 함숫값을 y 에 XOR로 씁니다:

$$U_f\,|x\rangle|y\rangle = |x\rangle\,|y \oplus f(x)\rangle$$

$f(x) = 1$ 이면 y 가 뒤집히고, $f(x) = 0$ 이면 그대로. 이는 가역적이라 정당한 unitary입니다. ($\oplus$ 는 5장의 XOR.)

형태 B — phase oracle (sign 뒤집기)

Grover가 실제로 쓰는 형태입니다. 여분 qubit 없이, solution state의 **amplitude sign만** 뒤집습니다:

$$U_f\,|x\rangle = \begin{cases} -|x\rangle & \text{if } f(x) = 1 \\ +|x\rangle & \text{if } f(x) = 0 \end{cases}$$

한 줄로 쓰면:

$$U_f\,|x\rangle = (-1)^{f(x)}|x\rangle$$

solution($f(x) = 1$)인 state에만 -1 이 붙습니다. **probability(absolute value)는 그대로**이고 phase(sign)만 뒤집힌다는 게 핵심입니다.

🔍 **예제 (원문) — 3 qubits에서 $|010\rangle$ 만 solution이라면**

uniform superposition에 phase oracle을 걸면, $|010\rangle$ 앞에만 minus sign이 붙습니다:

$$U_f|\psi\rangle = \tfrac{1}{\sqrt{8}}\big(|000\rangle + |001\rangle - |010\rangle + |011\rangle + |100\rangle + |101\rangle + |110\rangle + |111\rangle\big)$$

다만 이 sign은 **measurement만으로는 드러나지 않습니다.** 이 minus sign은 전체에 곱해진 global phase가 아니라 항들 **사이의** relative phase라 물리적으로 분명히 의미가 있습니다(4.9절). 하지만 지금은 여덟 amplitude의 **크기가 모두 같아서** measurement probability($|\text{amplitude}|^2$)가 여전히 균등합니다 — 표식이 붙어도 measurement 결과에는 아직 차이가 없는 것이죠. 그래서 9장 Grover의 diffusion operator가 이 "숨은 sign 표시"를 큰 amplitude로 바꿔줘야 합니다.

> 🔑 **핵심 — oracle은 "solution에 표시"를 한다**: phase oracle은 solution state에 minus sign이라는 **보이지 않는 표식**을 답니다. 이 표식만으로는 아직 measurement에서 이득이 없습니다. 표식을 amplitude로 증폭하는 것이 algorithm의 나머지 절반입니다.

7.3 phase kickback — 형태 A가 형태 B로 변신하는 마법

oracle을 **형태 A($U_f|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle$)로 만들었는데**, 출력 qubit $|y\rangle$ 를 특별한 state $|-\rangle$ 로 두면 **형태 B(phase oracle)** 와 같은 동작을 얻습니다. 이것이 **phase kickback** 입니다.

출력 qubit를 $|y\rangle = |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$ 로 두고 계산해 봅시다.

🔍 **계산 따라가기**

$$U_f\,|x\rangle|-\rangle = U_f\,|x\rangle\frac{|0\rangle - |1\rangle}{\sqrt{2}} = \frac{1}{\sqrt{2}}\big(|x\rangle|0 \oplus f(x)\rangle - |x\rangle|1 \oplus f(x)\rangle\big)$$

이제 $f(x)$ 값으로 경우를 나눕니다.

경우 1 — $f(x) = 0$: $0 \oplus 0 = 0,\ 1 \oplus 0 = 1$ 이므로

$$\frac{1}{\sqrt{2}}(|x\rangle|0\rangle - |x\rangle|1\rangle) = |x\rangle\frac{|0\rangle - |1\rangle}{\sqrt{2}} = |x\rangle|-\rangle$$

그대로입니다.

경우 2 — $f(x) = 1$: $0 \oplus 1 = 1,\ 1 \oplus 1 = 0$ 이므로

$$\frac{1}{\sqrt{2}}(|x\rangle|1\rangle - |x\rangle|0\rangle) = |x\rangle\frac{|1\rangle - |0\rangle}{\sqrt{2}} = -|x\rangle\frac{|0\rangle - |1\rangle}{\sqrt{2}} = -|x\rangle|-\rangle$$

$|0\rangle, |1\rangle$ 이 뒤바뀌어 **전체에** -1 이 붙었습니다!

두 경우를 합치면:

$$U_f\,|x\rangle|-\rangle = (-1)^{f(x)}\,|x\rangle|-\rangle$$

> 🔑 **phase kickback의 정체**: 분명히 U_f 는 출력 qubit $|y\rangle$ 를 건드리는 연산인데, $|y\rangle = |-\rangle$ 로 두면 y **는 그대로이고 대신 입력** $|x\rangle$ **앞에 sign** $(-1)^{f(x)}$ **가 "튀어(kick back)"** 나옵니다. 출력 qubit에 일어나야 할 변화가 입력 qubit의 phase 변화로 나타난 것입니다.

출력 qubit $|-\rangle$ 은 계산 내내 변하지 않고 변하지 않은 상태(constant)로 남으므로 실질적으로는 입력 register에 phase oracle(형태 B)을 건 것과 같습니다:

$$\underbrace{U_f|x\rangle|-\rangle}_{\text{형태 A} + |-\rangle} = \underbrace{(-1)^{f(x)}|x\rangle}_{\text{형태 B}}\ |-\rangle$$

💡 **비유**: $|-\rangle$ 은 "sign을 되받아치는 거울"입니다. oracle이 출력 qubit를 뒤집으려 하면, $|-\rangle$ 거울은 자기가 뒤집히는 대신 그 뒤집힘을 입력 qubit의 sign으로 반사해 보냅니다.

왜 하필 $|-\rangle$ 인가

$|-\rangle$ 은 X gate의 eigenstate입니다: $X|-\rangle = -|-\rangle$. 즉 "뒤집혀도 sign만 바뀌고 형태는 그대로"인 state라, 뒤집힘이 sign으로 전환됩니다. 그리고 $|-\rangle = HX|0\rangle$ 이므로 $|0\rangle$ 에 X 다음 H 를 걸어 쉽게 만들 수 있습니다.

✏️ **연습 7-1**: solution이 아닌 입력($f(x) = 0$)에 대해 $U_f|x\rangle|-\rangle$ 은 무엇이 될까요? *(답: $|x\rangle|-\rangle$ — 그대로. $(-1)^0 = 1$ 이라 sign이 안 붙습니다. 10장에 풀이.)*

7.4 conditional sign flip — operator로 쓰기

phase oracle을 operator 형태로 정리해 둡시다. 조건을 만족하는 입력들의 집합을 $A = \{x : f(x) = 1\}$, 그 크기를 $\text{number of solutions} = |A|$ 라 하면, phase oracle은 다음 unitary로 쓸 수 있습니다:

$$U_f = I - 2\sum_{\omega \in A} |\omega\rangle\langle\omega|$$

🔍 **확인**: 2.6절 outer product에서 $|\omega\rangle\langle\omega|$ 는 $|\omega\rangle$ 방향 projection이었습니다. 서로 다른 computational basis state끼리는 orthogonal이므로 sum에서 항 하나만 살아남고, 검산은 항이 하나였을 때와 똑같은 두 줄로 끝납니다.

- $|\omega'\rangle \in A$ 에 작용: $\omega = \omega'$ 항만 남아 $|\omega'\rangle - 2|\omega'\rangle = -|\omega'\rangle$. ✓ (sign 뒤집힘)
- A 의 모든 원소와 orthogonal인 $|x\rangle$ 에 작용: 항마다 $\langle\omega|x\rangle = 0$ 이므로 $|x\rangle - 0 = |x\rangle$. ✓ (그대로)

solution이 하나뿐인 경우는 sum이 한 항으로 줄어든 **special case** $U_f = I - 2|\omega\rangle\langle\omega|$ 입니다. 교과서가 Grover를 소개할 때 이 형태를 쓰기 때문에 solution은 늘 하나라는 인상을 받기 쉬운데, f 가 조건을 판정하는 function인 이상 number of solutions는 우리가 고르는 값이 아니라 data가 정하는 값입니다. solution이 하나도 없을 수도 있고 모든 state가 solution일 수도 있습니다.

여기서 성질 하나를 미리 살펴두겠습니다. 위 식에서 A 의 원소들은 서로 **완전히 대칭**입니다 — 어느 항도 따로 구별하지 않고 전부 같은 계수의 projection일 뿐이고, 뒤에 올 diffusion operator 역시 index를 구별하지 않습니다. 그래서 증폭이 끝난 뒤 모든 solution의 amplitude는 **정확히 같아집니다**(9.7절 · 14.3절). "가장 큰 amplitude를 고르면 solution이 나온다"는 measurement 방식이 solution이 여럿일 때 실패하는 이유가 바로 이 대칭성입니다.

이렇게 "특정 computational basis의 amplitude sign만 뒤집는" 연산을 **conditional sign flip**이라 부릅니다. 이 $I - 2\sum_{\omega\in A}|\omega\rangle\langle\omega|$ 형태가 9장 Grover의 oracle 단계이고, 이와 짝을 이루는 $2|\psi\rangle\langle\psi| - I$ 가 diffusion operator입니다. 두 개의 "reflection"이 Grover의 전부입니다.

7.5 oracle을 실제로 만들기 — Z gate trick

"모든 qubit가 $|1\rangle$ 일 때만 sign을 뒤집는" oracle은 실제로 간단히 구현됩니다. 마지막 qubit 하나만 빼고 나머지를 모두 control qubit로, 마지막을 target qubit로 하는 **multi-controlled Z gate**를 쓰면 됩니다. $Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$ 는 target qubit가 $|1\rangle$ 일 때만 sign을 뒤집으므로(3.5절), 모든 control qubit가 $|1\rangle$ 일 때만 $|11\cdots1\rangle$ 의 sign이 반전됩니다.

특정 bit string ω 에 대한 oracle도, ω 의 0인 자리에 X gate를 앞뒤로 감싸 그 bit string을 $|11\cdots1\rangle$ 자리로 옮긴 뒤 이 trick을 쓰면 됩니다. 다만 이 조립법이 무엇을 해 주는 것인지는 정확히 새겨 두어야 합니다. 이는 **판정 function f 를 이미 circuit으로 갖고 있을 때 그것을 phase oracle(형태 B)로 바꿔 놓는 방법**이지, 어느 입력이 조건을 만족하는지 알아내는 방법이 아닙니다. X 를 어느 자리에 감을지 정하려면 ω 를 이미 알고 있어야 하므로, 위 배치를 그대로 쓴 circuit은 search를 하는 것이 아니라 답을 아는 채로 algorithm 골격이 도는지 확인하는 실습용 circuit입니다. 실제 search에서 f 는 조건을 계산하는 classical circuit이고, 그것을 6.4절의 방식으로 가역화해 형태 A로 만든 다음 출력 qubit를 $|-\rangle$ 로 두면 7.3절의 kickback이 형태 B를 만들어 냅니다. 그동안 circuit은 어느 입력이 조건에 걸리는지 모르고, 알 필요도 없습니다. diffusion operator와 짝을 맞춘 전체 implementation은 9장에서 정리합니다.

> ❓ **emulator에서는?**: FPGA emulator는 이 sign flip을 physical gate가 아니라 **state vector의 해당 성분에 -1 을 곱하는 연산**으로 구현합니다. 대상은 미리 정해 둔 solution index가 아니라 **predicate를 만족하는 모든 칸**이고, 판정하는 것도 index가 아니라 그 칸에 담긴 값 `value[i]` 입니다(13.3절). 이것이 oracle의 수치 implementation 전부입니다. 그래서 oracle을 정확히 이해하는 것이 hardware 설계의 출발점입니다.

우리 accelerator의 oracle도 이 장의 정의를 글자 그대로 지킵니다. 다른 점은 판정 대상이 index가 아니라 data라는 것뿐입니다 — `data_mem` 의 i 번 값을 host가 넘긴 threshold와 견주어 `<` · `>` · `=` · 범위 네 predicate 중 하나를 계산하고, 참인 칸에만 sign을 뒤집습니다(13.3절). sign flip 자체는 two's complement integer의 bit를 뒤집고 1을 더하는 것이라 multiplier가 필요 없습니다(13.4절). 이 장의 정의에 따른 제약도 있습니다 — circuit은 실행 전에 어느 칸이 뒤집힐지 모르므로 predicate를 만족하는 칸이 몇 개인지(number of solutions)도 미리 알 수 없고, Grover iteration을 몇 번 돌려야 하는지 모르는 채로 시작해야 합니다(16장).

7.6 이 장의 요약

- **oracle** $f : \{0,1\}^{\text{number of qubits}} \to \{0,1\}$: solution이면 1, 아니면 0을 주는 black box. 내부는 모름.
- **형태 A (bit flip)**: $U_f|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle$. 여분 qubit에 XOR. 가역.
- **형태 B (phase oracle)**: $U_f|x\rangle = (-1)^{f(x)}|x\rangle$. 조건을 만족하는($f(x) = 1$) 항의 amplitude **sign만** 뒤집음. Grover가 사용.
- **phase kickback**: 형태 A의 출력 qubit를 $|-\rangle$ 로 두면 $U_f|x\rangle|-\rangle = (-1)^{f(x)}|x\rangle|-\rangle$. 출력의 뒤집힘이 입력의 phase로 되차임 ⇒ 형태 A + $|-\rangle$ = 형태 B.
- **왜** $|-\rangle$: $X|-\rangle = -|-\rangle$ (eigenstate)라 뒤집힘이 sign으로 전환. $|-\rangle = HX|0\rangle$.
- **conditional sign flip**: $U_f = I - 2\sum_{\omega \in A} |\omega\rangle\langle\omega|$ (A = 조건을 만족하는 입력 집합, $\text{number of solutions} = |A|$). solution이 하나뿐인 경우는 special case. A 의 원소들이 완전 대칭이라 증폭 뒤 amplitude가 정확히 같아집니다. diffusion operator와 짝을 이루어 두 개의 reflection.
- 실제 implementation: multi-controlled Z + X 감싸기는 f 를 circuit으로 갖고 있을 때 **phase oracle로 바꾸는** 방법이지 solution을 알아내는 방법이 아닙니다. emulator는 **predicate를 만족하는 모든 칸**의 성분에 -1 을 곱합니다.

다음 장에서는 이 oracle과 interference를 결합한 **첫 실제 algorithm — Deutsch–Jozsa** 를 봅니다. phase kickback과 $H^{\otimes \text{number of qubits}}$ 를 이용해 classical computer보다 적은 질의로 답을 얻는 과정을 전체 계산으로 확인합니다.

8장. Deutsch–Jozsa algorithm

> **이 장을 읽기 위한 준비**: 6장 $H^{\otimes \text{number of qubits}}$·Hadamard 일반식, 7장 oracle·phase kickback.

이제 quantum algorithm 하나를 처음부터 끝까지 살펴봅니다. **Deutsch–Jozsa algorithm**은 실용성은 떨어지지만, quantum computer가 classical computer를 **큰 차이로** 앞설 수 있음을 보여준 최초의 사례입니다. 무엇보다 7장 phase kickback과 6장 Hadamard interference가 어떻게 작용하는지를 계산으로 보여주므로, 9장 Grover를 이해하는 데 도움이 됩니다.

8.1 풀려는 문제 — balanced인가 constant인가

function $f : \{0,1\}^{\text{number of qubits}} \to \{0,1\}$ 이 주어졌는데, 이 function이 다음 둘 중 하나임이 **보장**되어 있습니다.

- **constant function**: 모든 입력에 대해 항상 같은 값 (전부 0 또는 전부 1).
- **balanced function**: 입력의 정확히 절반은 0, 나머지 절반은 1로 매핑.

질문: f 가 constant인가 balanced인가?

8.2 classical computer로는 얼마나 걸리나

classical computer로 확실히 판별하려면 최악의 경우 얼마나 많이 물어봐야 할까요? 입력은 총 $2^{\text{number of qubits}}$ 개입니다.

- **운이 좋으면**: 두 번 물어 서로 다른 값이 나오면 즉시 "balanced"로 확정.
- **최악의 경우**: 입력을 계속 물었는데 절반+1번($2^{\text{number of qubits}-1}+1$)까지 전부 같은 값이 나와야 비로소 "constant"라고 확정할 수 있음. (절반까지 모두 0이어도 나머지 절반이 1일 수 있으니, 절반을 넘겨봐야 안다.)

즉 classical computer는 최악의 경우 $2^{\text{number of qubits}-1}+1$ **번**의 oracle 호출이 필요합니다. qubit가 늘어나면 지수적으로 늘어납니다.

이 비교에서 세는 단위는 처음부터 끝까지 **oracle을 몇 번 호출하는가**이고, 그 이득은 진짜 quantum hardware에서 성립하는 이야기입니다. $2^{\text{number of qubits}}$ 개의 amplitude를 memory에 펼쳐 놓고 계산하는 emulator는 oracle 한 번을 처리하는 데도 array 전체를 훑어야 하므로 호출 횟수의 이득을 그대로 얻지는 못합니다. emulator에서는 속도의 이득 대신 interference가 답을 만드는 과정을 한 칸씩 재현하고 검증할 수 있습니다.

> 🔑 **Deutsch–Jozsa의 충격**: quantum computer는 oracle을 **단 한 번** 호출해 100% 확실하게 답합니다. query complexity $O(1)$. classical computer의 $O(2^{\text{number of qubits}})$ 와 비교하면 exponential speedup입니다.

8.3 algorithm circuit

circuit 구조는 이렇습니다 (입력 qubit 여러 개 + 출력 qubit 1개):

```
|0⟩ ─[H]─┬─────────┬─[H]─| D |= m0
|0⟩ ─[H]─┤         ├─[H]─| D |= m1
  ⋮      |   Uf    |       ⋮
|0⟩ ─[H]─┤         ├─[H]─| D |= m(number of qubits - 1)
|1⟩ ─[H]─┴─────────┴──────  (출력 qubit, |−⟩로 준비)
```

단계:

1. 입력 qubit는 모두 $|0\rangle$, 출력 qubit는 $|1\rangle$ 로 시작.
2. **모든** qubit에 H (출력 qubit는 $H|1\rangle = |-\rangle$ 가 됨 → 7장 phase kickback 준비).
3. oracle U_f 한 번 적용.
4. 입력 qubit에 다시 $H^{\otimes \text{number of qubits}}$.

5. 입력 qubit measurement.

8.4 단계별 계산 — interference가 답을 만든다

1~2단계: uniform superposition + kickback 준비

입력 qubit에 $H^{\otimes \text{number of qubits}}$ 를 걸면(6.2절):

$$|x\rangle^{\otimes \text{number of qubits}} = H^{\otimes \text{number of qubits}}|0\rangle^{\otimes \text{number of qubits}} = \frac{1}{\sqrt{2^{\text{number of qubits}}}} \sum_{x \in \{0,1\}^{\text{number of qubits}}} |x\rangle$$

출력 qubit는 $H|1\rangle = |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$.

3단계: oracle + phase kickback

출력 qubit가 $|-\rangle$ 이므로 7.3절 phase kickback이 일어납니다. oracle이 입력 각 항에 $(-1)^{f(x)}$ 를 붙입니다:

$$U_f \left(\frac{1}{\sqrt{2^{\text{number of qubits}}}} \sum_x |x\rangle \right) |-\rangle = \left(\frac{1}{\sqrt{2^{\text{number of qubits}}}} \sum_x (-1)^{f(x)} |x\rangle \right) |-\rangle$$

출력 qubit $|-\rangle$ 은 이후에도 그대로인 constant이므로 따로 두고, 입력 register만 봅니다:

$$|\phi\rangle = \frac{1}{\sqrt{2^{\text{number of qubits}}}} \sum_x (-1)^{f(x)} |x\rangle$$

함숫값 $f(x)$ 가 amplitude의 **sign**으로 표현되었습니다.

4단계: 다시 $H^{\otimes \text{number of qubits}}$ — interference

여기에 6.2절 Hadamard 일반식 $H^{\otimes \text{number of qubits}}|x\rangle = \frac{1}{\sqrt{2^{\text{number of qubits}}}} \sum_v (-1)^{x \cdot v} |v\rangle$ 을 적용:

$$H^{\otimes \text{number of qubits}}|\phi\rangle = \frac{1}{2^{\text{number of qubits}}} \sum_x (-1)^{f(x)} \sum_v (-1)^{x \cdot v} |v\rangle = \frac{1}{2^{\text{number of qubits}}} \sum_v \left[\sum_x (-1)^{f(x) + x \cdot v} \right] |v\rangle$$

우리는 특히 $|v\rangle = |0 \cdots 0\rangle$ **state의 amplitude**에 주목합니다. $v = 0$ 이면 $x \cdot v = 0$ 이므로 그 amplitude는:

$$(|0 \cdots 0\rangle \text{ 의 amplitude}) = \frac{1}{2^{\text{number of qubits}}} \sum_x (-1)^{f(x)}$$

5단계: 두 경우 분석

f 가 constant면 — 모든 $f(x)$ 가 같은 값 c:

$$\frac{1}{2^{\text{number of qubits}}} \sum_x (-1)^c = \frac{1}{2^{\text{number of qubits}}} \cdot (-1)^c \cdot 2^{\text{number of qubits}} = (-1)^c = \pm 1$$

$|0 \cdots 0\rangle$ 의 amplitude가 ± 1! normalization 때문에 다른 모든 amplitude는 0이어야 합니다. 따라서 measurement를 하면 **100% probability로** $|0 \cdots 0\rangle$.

f 가 balanced면 — 절반은 $(-1)^0 = +1$, 절반은 $(-1)^1 = -1$:

$$\frac{1}{2^{\text{number of qubits}}} \sum_x (-1)^{f(x)} = \frac{1}{2^{\text{number of qubits}}} \Big(\underbrace{+1 \cdots +1}_{2^{\text{number of qubits}-1}\text{개}} \underbrace{-1 \cdots -1}_{2^{\text{number of qubits}-1}\text{개}} \Big) = \frac{1}{2^{\text{number of qubits}}} \cdot 0 = 0$$

$+1$ 들과 -1 들이 **완벽히 상쇄**됩니다. $|0 \cdots 0\rangle$ 의 amplitude가 0이므로, measurement를 하면 **절대로** $|0 \cdots 0\rangle$ **이 안 나옵니다**.

8.5 결론 — measurement 한 번으로 판별

> 🔑 **판별 규칙**
>
> - measurement 결과가 $|0\cdots0\rangle$ (전부 0) → f 는 **constant**. (probability 100%)
> - measurement 결과에 1이 하나라도 있음 → f 는 **balanced**. (probability 100%)

oracle 단 한 번으로 확실하게 판별했습니다. 이 결과는 **interference**로 설명됩니다:

- constant일 때: 모든 항의 phase가 같아 $|0\cdots0\rangle$ 로 **constructive interference**.
- balanced일 때: phase가 반반이라 $|0\cdots0\rangle$ 에서 **destructive interference**.

💡 **비유**: $2^{\text{number of qubits}}$ 개의 파도(amplitude)를 만들어 한 지점($|0\cdots0\rangle$)에 모읍니다. constant function은 모든 파도가 같은 phase(마루끼리)라 크게 겹쳐 솟고, balanced function은 마루와 골이 반반이라 그 지점에서 잔잔해집니다. 파도의 높이를 재면(measurement) 어느 쪽인지 압니다.

✏️ **연습 8-1**: 3 qubits에서 f 가 balanced function일 때, 마지막 measurement에서 $|000\rangle$ 이 나올 probability는? *(답: 0. balanced면 $|0\cdots0\rangle$ amplitude가 상쇄돼 0. 10장에 풀이.)*

8.6 Deutsch–Jozsa에서 배울 것 — Grover로 가는 다리

이 algorithm의 실용성은 낮습니다(constant/balanced 판별은 현실 문제가 아님). 하지만 **quantum algorithm의 3박자 구조**를 보여줍니다:

단계	Deutsch–Jozsa	다음 장 Grover
① 펼치기	$H^{\otimes \text{number of qubits}}$ 로 uniform superposition	$H^{\otimes \text{number of qubits}}$ 로 uniform superposition
② 표시하기	oracle + phase kickback → $(-1)^{f(x)}$	oracle → 조건을 만족하는 항의 sign flip
③ interference	$H^{\otimes \text{number of qubits}}$ 한 번 → destructive/constructive interference	**iteration마다** diffusion operator → 점진 증폭

가장 큰 차이는 ③입니다. Deutsch–Jozsa는 interference **한 번**으로 답이 딱 떨어지지만(구조가 특수해서), Grover는 solution이 여럿 섞인 일반 search라 **Grover iteration을 여러 번** 돌려 amplitude를 조금씩 키워야 합니다. 그 number of iterations는 조건을 만족하는 항이 하나뿐일 때 대략 $\frac{\pi}{4}\sqrt{\text{number of states}}$ 번이고, 그런 항이 여러 개면 $\frac{\pi}{4}\sqrt{\text{number of states}/\text{number of solutions}} - \frac{1}{2}$ 로 줄어듭니다(9.5절). 이것이 $O(\sqrt{\text{number of states}})$ 의 근원인데, 여기서도 세는 단위는 oracle 호출 횟수입니다.

> ❓ **원문의 Deutsch algorithm과 Deutsch–Jozsa?**: 원래 Deutsch algorithm은 입력 qubit가 하나뿐인 초기 version이고, Deutsch–Jozsa는 이를 여러 qubit로 확장한 것입니다. 원리는 위 계산과 동일합니다.

우리가 만드는 accelerator는 Deutsch–Jozsa를 돌리지 않습니다. accelerator가 아는 연산은 predicate 판정과 inversion about the mean뿐이고 이 장의 circuit은 그 목록에 없습니다. 이 장에서는 **interference가 답을 만들어 내는 과정을 처음부터 끝까지 수식으로 계산**합니다. 해설서에서 이 과정을 모두 계산하는 곳은 이 장뿐입니다. 8.4절에서 $\frac{1}{2^{\text{number of qubits}}}\sum_x (-1)^{f(x)}$ 가 phase가 같으면 보강해 ±1 이 되고 반반이면 상쇄해 0 이 되는 것을 손으로 확인했는데, 다음 장에서 solution amplitude는 키우고 non-solution amplitude는 눌러 놓는 diffusion operator가 하는 일이 바로 그 constructive interference와 destructive interference입니다(9.3절). circuit에서는 그 interference가 inversion about the mean이라는 subtraction 한 줄로 나타납니다(13.5절).

8.7 이 장의 요약

- **문제**: f 가 constant(전부 같은 값)인가 balanced(절반씩 0/1)인가?
- **classical computer**: 최악 $2^{\text{number of qubits}-1}+1$ 회 oracle 호출 ($O(2^{\text{number of qubits}})$).
- **quantum**: oracle **1회** — query complexity $O(1)$. exponential speedup(세는 단위는 oracle 호출 횟수이고, amplitude를 펼쳐 계산하는 emulator는 이 이득을 얻지 못합니다).
- **circuit**: $H^{\otimes \text{number of qubits}}$ → U_f(phase kickback, 출력 qubit $|-\rangle$) → $H^{\otimes \text{number of qubits}}$ → measurement.
- **핵심 계산**: $|0\cdots0\rangle$ amplitude $= \frac{1}{2^{\text{number of qubits}}}\sum_x (-1)^{f(x)}$. constant면 ±1(constructive interference), balanced면 0(destructive interference).

- **판별**: measurement가 전부 0 → constant, 1이 있으면 → balanced. probability 100%.
- **Grover로의 다리**: 펼치기-표시-interference의 3박자. Grover는 interference를 **Grover iteration마다** 되풀이해 점진 증폭 → $O(\sqrt{\text{number of states}})$ (조건을 만족하는 항이 여러 개면 $\frac{\pi}{4}\sqrt{\text{number of states}/\text{number of solutions}} - \frac{1}{2}$ 회).

다음 장에서는 **Grover algorithm**을 다룹니다. amplitude amplification·inversion about the mean·diffusion operator·geometric rotation을 모두 동원해, 정렬 안 된 항목들 가운데서 solution을 $O(\sqrt{\text{number of states}})$ 에 찾아내는 과정을 살펴봅니다.

9장. Grover algorithm ★

> **이 장을 읽기 위한 준비**: 1장 amplitude·superposition, 4장 measurement, 6.2절 $H^{\otimes \text{number of qubits}}$, 7장 oracle. 뒷부분 수식은 2.6절 outer product도 씁니다.

이 project의 목적지입니다. **Grover algorithm**은 정렬되지 않은 항목들 가운데서 조건에 맞는 항목을 **약 $\sqrt{\text{number of states}}$ 번** 만에 찾습니다.

이 장은 **그림 한 장**(amplitude bar chart)에서 출발합니다. 먼저 Grover가 하는 일을 말로 이해하고, 실제로 손으로 한 번 돌려서 solution을 얻는 과정을 확인한 뒤, "왜 $\sqrt{\text{number of states}}$ 번인지 · 왜 π 가 나오는지"를 rotation 그림으로 풀고, 마지막에 수식으로 정리합니다. 수식이 어렵게 느껴지면 **9.7절**은 건너뛰고 직관만 챙겨도 됩니다 — 다만 9.7절의 결론 하나, **solution이 여럿이면 그 amplitude가 항상 정확히 같다**는 사실만은 반드시 챙기셔야 합니다. 그 결론은 손계산으로도 볼 수 있게 **9.4절**에 함께 넣어 두었으니, 9.7절을 건너뛰더라도 9.4절의 뒷부분은 건너뛰지 마세요. 이 사실에 따라 나중에 measurement circuit 전체의 설계가 달라집니다.

9.1 무슨 문제를 푸는가

100개의 상자 중 하나에 민트 초콜릿이 들어 있습니다. 상자를 하나씩 열어봅니다.

- **운이 좋으면**: 첫 상자에 있음 → 1번.
- **운이 나쁘면**: 99개를 열어도 없어 100번째에서 발견 → 100번.

mean으로 봐도 최악으로 봐도 상자 수에 비례합니다. search는 $O(\text{number of states})$. 상자를 열어 "이게 내가 찾는 거냐"를 확인하는 게 곧 oracle 호출이에요(7장).

data가 **정렬**돼 있으면 binary search로 $O(\log \text{number of states})$ 에 찾지만, 정렬은 강한 전제이고 정렬이 불가능한 경우도 많습니다. Grover는 **정렬 안 된** data에서 $O(\sqrt{\text{number of states}})$ 를 달성합니다 — binary search만큼 극적이진 않아도(quadratic speedup), 정렬 없이 얻는 확실한 이득이고, 이 $O(\sqrt{\text{number of states}})$ 가 **최적**임이 증명돼 있습니다.

> 🔑 숫자로 느껴봅시다: 항목이 100만 개면 classical search는 최악 100만 번, Grover는 약 $\sqrt{10^6} = 1{,}000$ 번. 항목이 많을수록 격차가 벌어집니다.

여기서 비교의 전제를 짚고 넘어가겠습니다. 방금의 $O(\text{number of states})$ 대 $O(\sqrt{\text{number of states}})$ 비교는 **진짜 quantum hardware**에서 성립하는 이야기입니다. 이 project가 만드는 것은 quantum computer가 아니라 Grover를 classical circuit으로 흉내 내는 **emulator**이고, emulator는 iteration 한 번마다 모든 state의 amplitude를 전부 훑어야 하므로 총비용이 $O\left(\text{number of states} \cdot \sqrt{\text{number of states}/\text{number of solutions}}\right)$ 입니다. 즉 우리 circuit은 **classical linear scan보다 오히려 느립니다.** 그래서 이 해설서는 우리 accelerator를 classical search와 비교하지 않습니다. 비교 대상은 같은 일을 하는 software state vector simulator — Qiskit AerSimulator나 NumPy로 짠 것 — 이고, 이 accelerator의 값어치는 속도 우위가 아니라 **quantum algorithm의 동작을 결정적이고 재현 가능하게 재현·검증하는 전용 hardware**라는 데 있습니다. algorithm의 $\sqrt{\text{number of states}}$ 는 우리가 흉내 내는 **대상의 성질**이지 우리 circuit의 성능 지표가 아닙니다.

9.2 문제를 기호로

- qubit의 개수를 number of qubits, search space의 크기를 $\text{number of states} = 2^{\text{number of qubits}}$ 라 씁니다.
- oracle: 조건을 만족하면 $f(x) = 1$, 아니면 0 (7장).
- 조건을 만족하는 index 전체를 **solution 집합** $A = \{\, x : f(x) = 1 \,\}$ 이라 하고, 그 크기를 $\text{number of solutions} = |A|$ 라 씁니다.

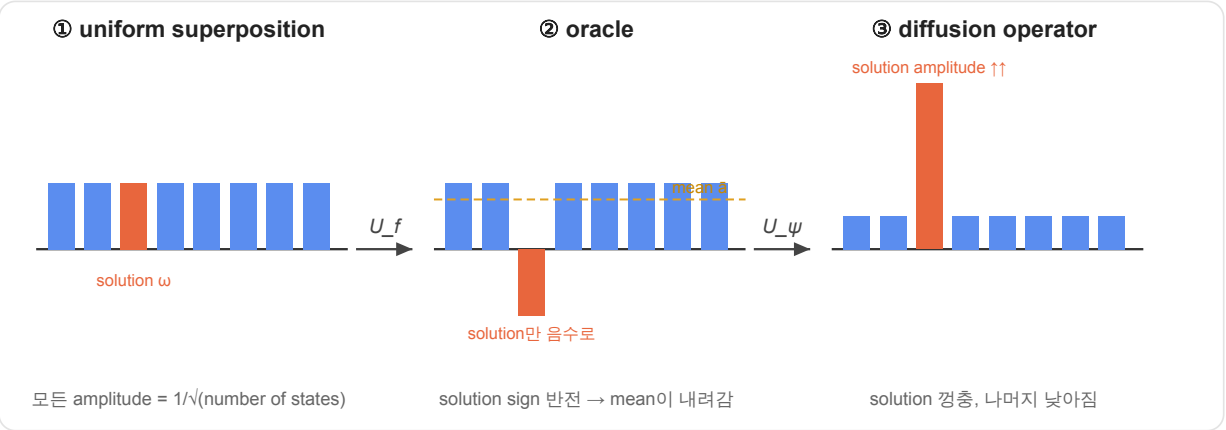
이 장은 처음부터 solution이 하나 이상인 경우($\text{number of solutions} \geq 1$)를 정식 설정으로 둡니다. solution이 하나뿐인 경우는 그중 **가장 단순한 special case**이고, 그때만 유일한 solution을 ω 라 부르겠습니다. 교과서가 흔히 solution 하나짜리부터 시작하는 것은 기호가 짧아지기 때문이지 그게 표준 문제라서가 아닙니다. 실제 search에서 oracle은 "이 index가 solution이냐"가 아니라 **predicate를 만족하느냐**만 판정하고, 그 predicate를 몇 칸이 만족하는지는 data에 따라 정해집니다(14.2절). solution을 하나로 못 박는 것은 "solution이 하나뿐임을 이미 안다"는 꽤 강한 가정이에요.

Grover는 oracle을 약 $\frac{\pi}{4}\sqrt{\text{number of states}/\text{number of solutions}}$ 번 호출한 뒤 높은 probability로 solution 하나를 내놓습니다. 이 횟수가 어디서 오는지는 **9.5절**에서 풀립니다. (0 solutions, 즉 조건을 만족하는 칸이 하나도 없는 경우는 rotation angle 자체가 정의되지 않아 algorithm 바깥에

서 따로 판정해야 합니다 — hardware에서 이를 어떻게 끝낼지는 아직 확정하지 않았습니다(16.9절).)

9.3 핵심 장면 — 막대 두 번 흔들기

Grover가 실제로 무엇을 하는지, **amplitude bar chart** 한 장이면 다 담깁니다. 막대 하나가 state 하나의 amplitude(measurement probability의 뿌리)라고 보세요.



시작. 모든 qubit에 H 를 걸어 **uniform superposition**을 만듭니다(6.2절). 막대가 전부 똑같은 높이 $\frac{1}{\sqrt{\text{number of states}}}$ 예요. 지금 measurement를 하면 solution이 나올 probability는 $\frac{\text{number of solutions}}{\text{number of states}}$ — 그냥 찍는 거랑 같습니다.

이제 Grover **한 번**은 딱 두 동작입니다.

동작 1 — oracle: 조건에 걸리는 막대를 아래로 뒤집기. oracle U_f 는 **조건을 만족하는 모든 칸**의 amplitude sign을 뒤집어 **negative**로 만듭니다(7.2절). 걸린 칸은 $-\frac{1}{\sqrt{\text{number of states}}}$, 나머지는 그대로 $+\frac{1}{\sqrt{\text{number of states}}}$. circuit은 어느 칸이 걸릴지 미리 알지 못하고, 알 필요도 없습니다 — 판정만 할 뿐입니다.

그런데 이 state에서 measurement를 하면 소용없어요. $\left|-\frac{1}{\sqrt{\text{number of states}}}\right|^2 = \left|\frac{1}{\sqrt{\text{number of states}}}\right|^2$ 라 probability는 여전히 다 똑같거든요. sign은 **아직 안 보이는 표식**일 뿐입니다. 그래서 두 번째 동작이 필요합니다.

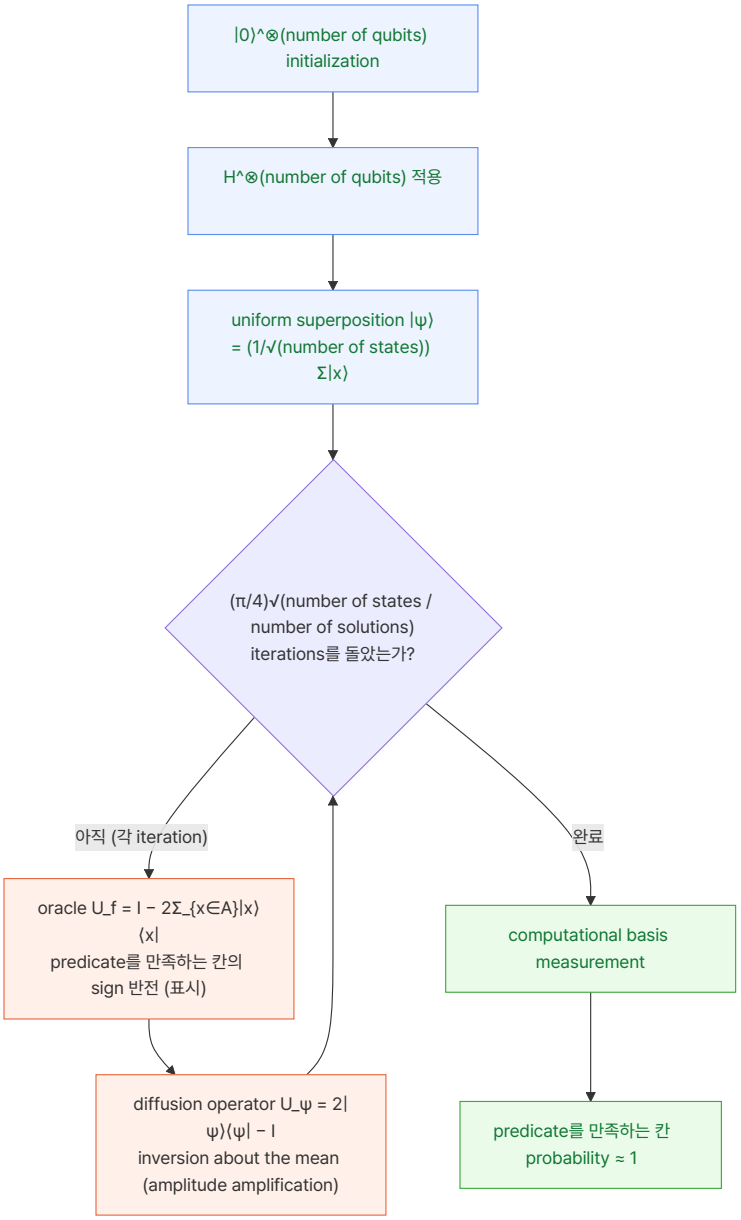
동작 2 — diffusion operator: mean을 기준으로 모두 뒤집기. 이게 Grover의 심장인 **"inversion about the mean"** 예요. 말은 거창하지만 규칙은 한 줄입니다:

> 모든 막대를 **mean line을 기준으로** 위아래로 뒤집는다. (mean보다 위에 있던 건 그만큼 아래로, 아래 있던 건 그만큼 위로.)

두 동작을 거치면 다음과 같이 달라집니다. 동작 1에서 solution 막대는 mean line **한참 아래**(negative)로 내려가 있었죠. 그러니 mean 기준으로 뒤집으면 solution은 mean **한참 위**로 솟구칩니다. 반면 나머지 막대들은 원래 mean 근처였으니 살짝만 내려가요. 결과적으로 **solution 막대만 껑충 커지고 나머지는 낮아집니다.**

💡 **시소 비유**: oracle은 solution을 시소 바닥까지 꾹 눌러 놓습니다. diffusion operator(inversion about the mean)는 시소를 통째로 뒤집어, 바닥에 있던 solution을 꼭대기로 올려요. 나머지는 원래 가운데쯤이라 크게 안 움직입니다.

이 두 동작(눌렀다 → 뒤집기)을 반복할 때마다 solution 막대가 조금씩 더 커집니다. 충분히 커지면 measurement를 했을 때 solution이 높은 probability로 나오죠. Grover 전체 흐름을 그림으로 보면:



9.4 진짜로 한 번 돌려보기 — 4 states

작은 예제인 4 states(2 qubits)를 손으로 끝까지 계산해 봅시다. 먼저 solution이 하나인 경우이고, 그 solution을 ω 라 하겠습니다. 네 state의 amplitude를 순서대로 추적할게요.

시작 — uniform superposition. 네 state 모두 amplitude $\frac{1}{2}$:

$$\text{amplitude} = \left(\tfrac{1}{2},\ \tfrac{1}{2},\ \tfrac{1}{2},\ \tfrac{1}{2}\right)$$

solution을 뽑을 probability는 $\left|\frac{1}{2}\right|^2 = \frac{1}{4}$. (넷 중 하나 찍기와 같음.)

동작 1 — oracle. solution(세 번째라 하죠)의 amplitude만 negative로:

$$\left(\tfrac{1}{2},\ \tfrac{1}{2},\ -\tfrac{1}{2},\ \tfrac{1}{2}\right)$$

동작 2 — diffusion operator(inversion about the mean). 먼저 mean을 구합니다:

$$\bar{a} = \frac{1}{4}\left(\tfrac{1}{2} + \tfrac{1}{2} - \tfrac{1}{2} + \tfrac{1}{2}\right) = \frac{1}{4}$$

각 막대를 mean 기준으로 뒤집는 것은 $a \mapsto 2\bar{a} - a = \frac{1}{2} - a$ 로 계산됩니다(왜 이 식인지는 **9.7절**에서):

- non-solution 세 개 ($a = \frac{1}{2}$): $\frac{1}{2} - \frac{1}{2} = 0$
- solution ($a = -\frac{1}{2}$): $\frac{1}{2} - \left(-\frac{1}{2}\right) = 1$

결과 amplitude는 $(0,\ 0,\ 1,\ 0)$ — state가 **정확히 solution** ω 이 되었습니다! measurement를 하면 solution이 나올 probability가 $|1|^2 = 1$, 즉 **단 한 번의 iteration으로 100% 확실하게** 찾았습니다. 두 동작만으로 solution을 얻은 거예요.

(4 states는 한 번에 딱 맞아떨어지는 특별히 운 좋은 크기입니다. state가 많으면 한 번으론 부족하고 여러 번 흔들어야 하는데, 그 얘기가 바로 다음 절입니다.)

solution이 둘이면 — 같은 계산, 두 막대

이제 방금 계산에서 **solution만 하나 더 늘려** 봅시다. circuit은 그대로 씁니다. 셋째와 넷째 칸이 둘 다 조건을 만족한다고 하죠(2 solutions). oracle은 조건에 걸리는 칸을 **전부** 뒤집으므로:

$$\left(\tfrac{1}{2},\ \tfrac{1}{2},\ -\tfrac{1}{2},\ -\tfrac{1}{2}\right)$$

mean은 $\bar{a} = \frac{1}{4}\left(\frac{1}{2} + \frac{1}{2} - \frac{1}{2} - \frac{1}{2}\right) = 0$ 이고, diffusion operator $a \mapsto 2\bar{a} - a = -a$ 를 적용하면:

$$\left(-\tfrac{1}{2},\ -\tfrac{1}{2},\ \tfrac{1}{2},\ \tfrac{1}{2}\right)$$

두 solution 막대의 값이 **정확히 같습니다.** 우연이 아니에요. 시작이 같았고(uniform superposition이니 모든 칸이 같은 값), oracle이 둘을 똑같이 뒤집었고, diffusion operator $a \mapsto 2\bar{a} - a$ 는 애초에 index를 구별하지 않는 global operation이라 **같은 값에 같은 function을 적용한 것**뿐입니다. 이 세 줄이 그대로 induction이고, iteration을 몇 번 하든 결론은 바뀌지 않습니다 — **solution들의 amplitude는 매 iteration 후 정확히 같습니다.**

(다만 4 states에 solution이 둘이면 처음부터 절반이 solution이라 더 짜낼 것이 없습니다. 실제로 solution 쪽 probability는 $\frac{1}{2}$ 에서 $\frac{1}{2}$ 로 제자리예요. 이건 **9.6절** "지나치면 안 된다"의 최소 사례이기도 합니다 — 한 번의 rotation이 목표를 그대로 지나쳐 버린 것이죠.)

막대가 나란히 자라는 것까지 보려면 8 states. 여덟 칸 중 두 칸이 조건을 만족한다고 하고 같은 계산을 반복합니다. 시작 amplitude는 여덟 칸 모두 $\frac{1}{\sqrt{8}} \approx 0.35355$.

- oracle 뒤: solution 둘이 -0.35355, non-solution 여섯이 $+0.35355$.
- mean: $\bar{a} = \dfrac{6(0.35355) - 2(0.35355)}{8} = 0.17678$.
- diffusion operator: solution $2(0.17678) - (-0.35355) = 0.70711 = \frac{1}{\sqrt{2}}$, non-solution $2(0.17678) - 0.35355 = 0$.

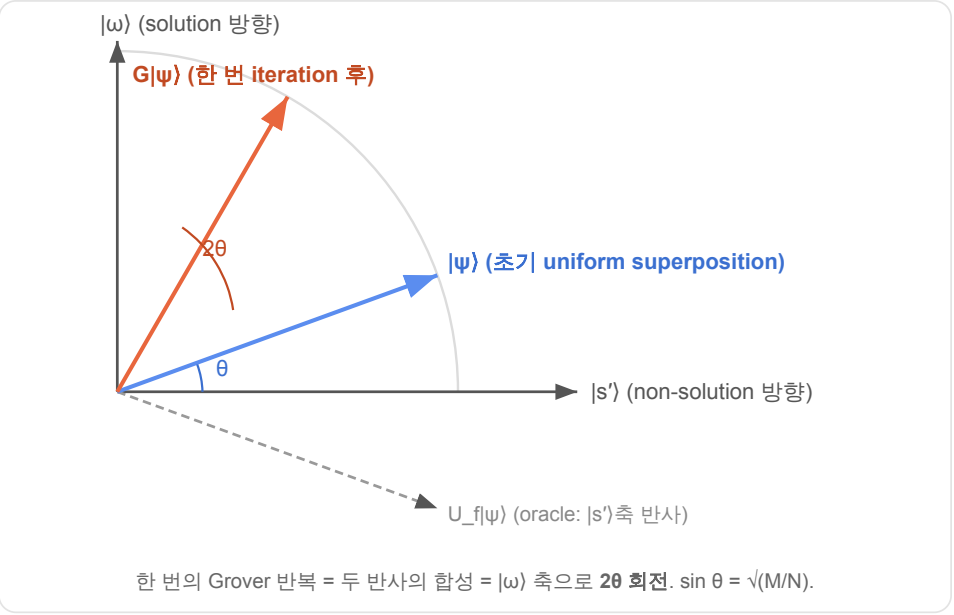
두 solution 막대가 **나란히** $\frac{1}{\sqrt{2}}$ **까지 자랐고**, non-solution 여섯은 정확히 0으로 눌렸습니다. 각 solution의 probability는 $\frac{1}{2}$ 이고 합치면 1 — 여기서도 한 번에 100%입니다. 두 막대의 높이는 정확히 같습니다.

이 "solution들의 amplitude가 항상 같다"는 사실은 이론에서는 사소해 보이지만 hardware에서는 결정적입니다. amplitude가 가장 큰 칸을 고르는 comparator circuit은 동률이 나오면 구조가 정한 **하나**만 내놓기 때문에, solution이 둘 이상이면 나머지는 영원히 출력되지 않습니다(14.3절). 일반적인 경우의 증명은 **9.7절** 끝에 있습니다.

9.5 왜 여러 번? · 왜 $\sqrt{\text{number of states}}$ 번? · 왜 π?

---

state가 많으면 solution이 그 많은 state 가운데 딱 하나 묻혀 있어, 처음 막대가 $\frac{1}{\sqrt{\text{number of states}}}$ 로 아주 작습니다. 한 번 흔들어선 충분히 안 커져요. 그래서 Grover iteration을 여러 번 돌리는데, **몇 번**이 적당한지를 알려면 Grover를 **rotation**으로 보는 게 제일 깔끔합니다.



state를 화살표로. 전체 space는 state마다 축이 하나씩 있는 고차원이지만, Grover의 계산이 이루어지는 공간은 딱 **2차원 평면**이에요. 두 방향만 잡으면 됩니다:

- **solution 방향** $|\omega\rangle$ — 우리가 가고 싶은 곳.
- **non-solution 방향** $|s'\rangle$ — solution을 뺀 나머지 전부.

초기 uniform superposition은 이 평면 안의 화살표인데, 거의 **non-solution 방향**을 가리키고 solution 방향에서 살짝 기울어 있어요(solution 성분이 $\frac{1}{\sqrt{\text{number of states}}}$ 로 작으니까). 그 기울어진 각을 θ 라 하면 $\sin\theta = \frac{1}{\sqrt{\text{number of states}}}$, state가 많으면 θ 는 아주 작습니다.

한 번 흔들 때마다 2θ rotation. "oracle + diffusion operator" 한 번은 이 화살표를 solution 방향으로 **정확히 2θ 만큼 돌립니다**(왜 딱 rotation인지는 **9.7절**에서). 그러니 반복할수록 화살표가 non-solution 방향에서 solution 방향으로 조금씩 돌아가요. solution 방향에 화살표가 정렬되면, measurement 시 solution이 나올 probability가 (거의) 1이 됩니다.

그래서 몇 번? — 여기서 π 가 나옵니다. solution 방향은 non-solution 방향과 **직각(90°)** 입니다. 화살표를 거기까지 돌리는 건 원의 **4분의 1 바퀴**, 각도로는 $\frac{\pi}{2}$ 예요 (π 는 "반 바퀴 = π, 4분의 1 바퀴 = π/2"인 그 원의 숫자입니다). 한 번에 2θ 씩 도니까, 필요한 횟수는:

$$\text{number of iterations} \approx \frac{\text{돌아야 할 각}}{\text{한 번에 도는 각}} = \frac{\pi/2}{2\theta} = \frac{\pi}{4\theta}$$

solution이 하나면 $\theta \approx \frac{1}{\sqrt{\text{number of states}}}$ 이므로:

$$\boxed{\text{number of iterations} \approx \frac{\pi}{4}\sqrt{\text{number of states}}}$$

$\frac{\pi}{4}$ 의 4는 "직각이라 π 의 **절반**(=π/2)" × "한 번에 2θ 라 **2×**" 가 겹쳐 나온 것뿐이에요. solution이 여러 개면 $\sin\theta = \sqrt{\text{number of solutions}/\text{number of states}}$ 가 되어 $\text{number of iterations} \approx \frac{\pi}{4}\sqrt{\text{number of states}/\text{number of solutions}}$. 이것이 $O(\sqrt{\text{number of states}})$ 의 정확한 근원입니다. (solution이 여러 개일 때 이 $\sqrt{\text{number of states}/\text{number of solutions}}$ 가 hardware에서 무엇을 뜻하는지는 14.1절에서 다룹니다 — multiple solutions는 oracle과 diffusion operator에서는 공짜이고, 값을 치르는 곳은 iteration 제어와 measurement입니다.)

운전대 비유: 목표가 옆으로 90° 방향에 있고, 운전대를 한 번 꺾으면 일정 각도씩 돈다고 해봐요. 몇 번 꺾어야 하냐? "90°(=π/2) ÷ 한 번에 도는 각도"죠. π 는 그저 "90°를 각도(radian)로 쓰면 π/2"라서 등장한 겁니다 — 신비로운 constant가 아니라 **rotation의 기하학**이에요.

✏️ **연습 9-1**: 64 states 가운데 solution이 4개일 때, iteration을 대략 몇 번 돌리는 것이 최적일까요? *(답: $\text{number of iterations} \approx \frac{\pi}{4}\sqrt{64/4} = \frac{\pi}{4}\sqrt{16} = \pi \approx 3$ 회. 10장에 풀이.)*

9.6 지나치면 안 된다

rotation 관점이 주는 중요한 경고입니다. iteration은 화살표를 2θ 씩 **계속** 돌립니다. 최적 횟수를 넘겨 돌리면 solution 방향을 **지나쳐** 버리고, 그러면 solution probability가 다시 **떨어집니다**(그리고 진동합니다).

> ⚠️ **함정 — Grover는 "많이 돌릴수록 좋은" 게 아니다**: classical algorithm은 오래 돌릴수록 확실해지지만, Grover는 $\frac{\pi}{4}\sqrt{\text{number of states}}$ 근처에서 probability가 최대이고 그 뒤엔 오히려 감소합니다. **정확한 횟수**를 지키는 게 중요합니다. number of solutions를 모르면 횟수를 정하기 어려운데, 이를 다루는 기법들이 따로 연구돼 있습니다.

solution이 하나일 때 최적 횟수 근처의 오류(solution이 안 나올) probability는 약 $\frac{1}{\text{number of states}}$ 정도로 아주 작습니다. 그리고 8장 Deutsch–Jozsa는 interference **한 번**으로 답이 떨어졌지만, Grover iteration을 **여러 번** 돌리는 이유는 이제 분명하죠 — solution 하나가 많은 state 가운데 묻혀 초기 amplitude가 $\frac{1}{\sqrt{\text{number of states}}}$ 로 작으니, rotation을 여러 번 쌓아야 solution 방향에 닿기 때문입니다.

방금 ⚠️ 상자가 "기법들이 따로 연구돼 있다"고만 하고 넘어간 그 기법에 이름을 붙여 둡시다. 표준 해법은 **BBHT**(Boyer–Brassard–Høyer–Tapp, 1998)입니다. number of solutions를 모르니 number of iterations를 한 번에 정하지 않고, 대신 여러 번의 **shot**으로 나눠 매 shot마다 number of iterations를 무작위로 뽑습니다. 뽑는 범위인 sampling range를 1 에서 시작해 $\text{number of iterations} \sim U[0,\ \text{sampling range})$ 로 뽑고, 그 shot이 실패하면 $\text{sampling range} \leftarrow \min(1.2 \cdot \text{sampling range},\ \sqrt{\text{number of states}})$ 로 넓혀 다시 뽑습니다($\text{growth factor} = 6/5 = 1.2$). rotation angle을 모르니 **여러 각도를 무작위로 시도하는** 전략이고 성공 여부는 measurement로 나온 후보 하나가 정말 predicate를 만족하는지 다시 확인하는 **solution check**로 판정합니다 — 값 한 칸을 읽는 것이라 사실상 공짜입니다.

초반 shot이 거의 다 실패하는 것은 고장이 아니라 이 절차의 정상 동작입니다. sampling range가 1에서 시작해 growth factor 1.2로만 커지므로 처음 몇 shot은 number of iterations가 0~2 근처에서만 뽑히는데, state가 많으면 그 정도 iteration은 rotation을 겨우 두어 번 한 state라 solution probability가 사실상 0이거든요. 우리 규모인 14 qubits(16,384 states)에서 solution이 하나라면 최적은 $\frac{\pi}{4}\sqrt{16{,}384} - \frac{1}{2} = 100.03 \to 100$ iterations인데, sampling range가 거기까지 커지기 전의 shot들은 성공에 이르지 못합니다. 그래서 shot이 꽤 쌓입니다 — 성공까지 **mean 22.7 shots**입니다(14 qubits·1 solution, board benchmark와 같은 100 workloads). "네댓 번이면 끝난다"가 아니에요. 이 shot loop를 FSM으로 어떻게 돌리는지, 실패한 shot이 남긴 amplitude array를 버리지 않고 재사용하는 방법은 16장에서 다룹니다. 원논문은 10.8절 대조표 끝에 적어 둔 Boyer et al. (1998) *Tight bounds on quantum searching* 입니다.

hardware에서는 "지나치면 안 된다"는 성질에 더 주의해야 합니다 — number of iterations를 integer로 반올림해야 하고, state가 적으면 그 rounding error가 성공률에 크게 반영됩니다(14.8절).

9.7 이제 수식으로 — 두 개의 reflection

여기부터는 앞의 두 그림(막대·rotation)을 **operator(matrix) 언어**로 정확히 옮깁니다. 직관이 이미 잡혔다면 이 절은 "왜 그게 맞는지"의 증명이라고 보면 됩니다.

oracle = solution 방향의 reflection. 7.4절에서 phase oracle을 이렇게 썼습니다(solution이 하나인 경우):

$$U_f = I - 2|\omega\rangle\langle\omega|$$

이건 "solution ω 성분만 sign을 뒤집는" 연산이에요. 확인: ω 에 작용하면 $-|\omega\rangle$(뒤집힘), ω 와 orthogonal인 state엔 그대로. 기하로는 **non-solution 축 $|s'\rangle$ 에 대한 reflection**입니다.

diffusion operator = inversion about the mean. diffusion operator는 이렇게 씁니다. $|\psi\rangle$ 은 uniform superposition입니다:

$$U_\psi = 2|\psi\rangle\langle\psi| - I$$

이게 왜 "inversion about the mean"($a \mapsto 2\bar{a} - a$)인지 봅시다. 임의의 state $\sum_x a_x|x\rangle$ 에 작용하면, $\langle\psi|\big(\sum a_x|x\rangle\big) = \frac{1}{\sqrt{\text{number of states}}}\sum a_x = \sqrt{\text{number of states}}\,\bar{a}$ (mean의 $\sqrt{\text{number of states}}$ 배)이므로:

$$U_\psi \sum_x a_x|x\rangle = 2|\psi\rangle(\sqrt{\text{number of states}}\,\bar{a}) - \sum_x a_x|x\rangle = \sum_x (2\bar{a} - a_x)|x\rangle$$

정확히 $a_x \mapsto 2\bar{a} - a_x$ — **9.4절**에서 손으로 쓴 그 규칙입니다. ✓

기하로 보면, U_ψ 는 $|\psi\rangle$ 방향은 그대로 두고($U_\psi|\psi\rangle = |\psi\rangle$) 거기에 수직인 성분만 sign을 뒤집습니다. 즉 $|\psi\rangle$ **방향을 거울로 삼는 reflection**이에요. oracle이 $|s'\rangle$ 축을 거울로 삼았던 것과 짝을 이룹니다 — 이제 Grover 한 번은 "거울 두 개에 연달아 비추기"가 됩니다.

gate로 만들기. $|\psi\rangle = H^{\otimes \text{number of qubits}}|0\rangle^{\otimes \text{number of qubits}}$ 이므로 diffusion operator를 실제 gate로 풀 수 있어요:

$$U_\psi = H^{\otimes \text{number of qubits}}\big(2|0\cdots0\rangle\langle 0\cdots0| - I\big)H^{\otimes \text{number of qubits}}$$

가운데 $2|0\cdots0\rangle\langle 0\cdots0| - I$ 는 "$|0\cdots0\rangle$ 만 빼고 sign flip"인데, 실제로는 7.5절 Z trick처럼 모든 qubit에 X 를 걸어 $|0\cdots0\rangle$ 을 $|1\cdots1\rangle$ 로 옮긴 뒤 multi-controlled Z 로 sign을 뒤집고 다시 X 로 되돌리면 됩니다. 즉 diffusion operator는 **Hadamard → ($|0\cdots0\rangle$만 sign flip) → Hadamard**. 이 **Hadamard–diagonal–Hadamard sandwich** 구조가 13.5절에서 "accumulation → shift → subtraction"이라는 multiplication 없는 datapath로 구현됩니다.

두 reflection을 이으면 rotation — 왜 하필 2θ인가. 한 번의 Grover iteration은

$$G = U_\psi\, U_f = (2|\psi\rangle\langle\psi| - I)(I - 2|\omega\rangle\langle\omega|)$$

즉 "oracle reflection → diffusion operator reflection", **거울 두 개에 연달아 비추는 것**입니다. 두 거울은:

- oracle U_f: **non-solution 축** $|s'\rangle$ 을 거울로 삼는 reflection.
- diffusion operator U_ψ: **초기 state** $|\psi\rangle$ **방향**을 거울로 삼는 reflection.

그리고 이 두 거울은 정확히 θ **만큼 벌어져** 있어요 — $|\psi\rangle$ 이 $|s'\rangle$ 축에서 θ 기울어 있으니까(**9.5절**, $\sin\theta = \frac{1}{\sqrt{\text{number of states}}}$). 여기서 기하학의 깔끔한 사실 하나를 씁니다:

> **거울 두 개에 연달아 비추면, 그 결과는 "두 거울 사이 각의 2×"만큼 돌린 것과 같다.**

왜 하필 2배인지, 각도를 직접 따라가 보면 바로 보입니다. 각도를 non-solution 축 $|s'\rangle$(=0°) 기준으로 재고, 화살표가 지금 각도 ϕ 에 있다고 하죠. **거울 하나(각도 a)에 비추면** 그 거울에 대해 대칭 이동하니 새 각도는 $2a - \phi$ 가 됩니다. 이걸 두 번 적용합니다:

- 시작: 화살표는 $|\psi\rangle$ 위치, 즉 $\phi = \theta$.
- ① **oracle** (거울 = $|s'\rangle$ 축, $a = 0$): $\theta \;\to\; 2\cdot 0 - \theta = -\theta$. → solution 성분 sign이 뒤집혀 축 **아래로** 접힘.
- ② **diffusion operator** (거울 = $|\psi\rangle$, $a = \theta$): $-\theta \;\to\; 2\theta - (-\theta) = 3\theta$.

시작 θ 에서 끝 3θ — solution 축 쪽으로 **정확히** 2θ 돌았습니다. 게다가 시작 각 ϕ 가 무엇이든 두 번 접으면 $\phi \to 2\theta - (2\cdot 0 - \phi) = \phi + 2\theta$ 라, **언제나** $+2\theta$ 만큼 돕니다. 이게 "두 거울 사이 각(θ)의 2×만큼 rotation"의 정체예요.

9.5절의 rotation 그림이 바로 이 세 걸음입니다: 파란 화살표 $|\psi\rangle$ 이 θ, 회색 점선이 oracle로 축 아래 $-\theta$ 로 접힌 것, 주황 $G|\psi\rangle$ 이 3θ. 그러니 반복할 때마다 화살표는 $\theta \to 3\theta \to 5\theta \to \cdots \to (2\cdot\text{number of iterations}+1)\theta$ 로 solution 축(90°=$\frac{\pi}{2}$)을 향해 2θ 씩 다가갑니다. solution 축에 닿을 조건 $(2\cdot\text{number of iterations}+1)\theta \approx \frac{\pi}{2}$ 에서 $\text{number of iterations} \approx \frac{\pi}{4\theta} - \frac{1}{2} \approx \frac{\pi}{4}\sqrt{\text{number of states}}$ 가 나오고요.

solution이 여럿이어도 평면은 2차원. 지금까지 $|\omega\rangle$ 하나를 solution 축으로 삼았는데, solution 집합 A 에 원소가 여럿이면 oracle은 항이 그만큼 늘어난 꼴이 됩니다:

$$U_f = I - 2\sum_{x\in A}|x\rangle\langle x|$$

항이 늘었으니 rotation 평면도 그만큼 많은 차원으로 퍼질 것 같지만 그렇지 않습니다. solution 성분을 하나로 묶은 **solution subspace의 unit vector**

$$|\omega_A\rangle = \frac{1}{\sqrt{\text{number of solutions}}}\sum_{x\in A}|x\rangle$$

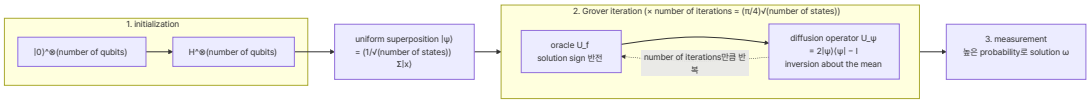
를 새 solution 축으로 잡아 보세요. $U_f|\omega_A\rangle = -|\omega_A\rangle$ 이고 $|\omega_A\rangle$ 에 orthogonal인 성분은 그대로이니, U_f 는 여전히 **한 축에 대한 reflection**입니다. uniform superposition은 $|\psi\rangle = \sin\theta\,|\omega_A\rangle + \cos\theta\,|s'\rangle$ 로 분해되고 이번엔 $\sin\theta = \sqrt{\text{number of solutions}/\text{number of states}}$ 이고요. diffusion operator는 원래부터 $|\psi\rangle$ 하나만 보는 연산이라 손댈 것이 없습니다. 두 거울이 여전히 같은 평면 안에 있으니 **rotation 평면은 solution이 몇 개든 2차원**이고, 앞의 유도가 $\sqrt{\text{number of states}} \to \sqrt{\text{number of states}/\text{number of solutions}}$ 만 바꿔 그대로 성립합니다.

여기서 다음 성질도 알 수 있습니다. 평면이 2차원이라는 것은 state가 언제나 $\alpha|\omega_A\rangle + \beta|s'\rangle$ 꼴이라는 뜻인데, $|\omega_A\rangle$ 는 A 의 모든 칸에 **똑같은 계수** $\frac{1}{\sqrt{\text{number of solutions}}}$ 를 주는 vector입니다. 따라서 모든 solution의 amplitude는 어느 시점에서나 $\frac{\alpha}{\sqrt{\text{number of solutions}}}$ 로 **전부 같습니다.** 9.4절에서 2

solutions로 손계산해 본 그 동률이, 일반적인 경우에 대해 여기서 증명된 셈입니다. 이 정리가 14.3절에서 measurement circuit의 설계를 통째로 바꿉니다.

> 🔑 Grover의 전부는 결국 **두 reflection** U_f 와 U_ψ 를 번갈아 거는 것입니다. U_f 는 solution에 표식을 달고, U_ψ 는 그 표식을 큰 amplitude로 키웁니다.

9.8 전체 그림 정리 — amplitude는 왜 real number뿐인가



요소	수식	역할
초기 state	$\lvert\psi\rangle = H^{\otimes \text{number of qubits}}\lvert 0\rangle^{\otimes \text{number of qubits}} = \frac{1}{\sqrt{\text{number of states}}}\sum_x \lvert x\rangle$	uniform superposition (모든 후보 동시에)
oracle	$U_f = I - 2\sum_{x\in A}\lvert x\rangle\langle x\rvert$	조건에 걸리는 막대를 전부 아래로
diffusion operator	$U_\psi = 2\lvert\psi\rangle\langle\psi\rvert - I$	inversion about the mean (증폭)
한 iteration	$G = U_\psi U_f$	solution 방향으로 2θ rotation
number of iterations	$\text{number of iterations} \approx \frac{\pi}{4}\sqrt{\text{number of states}/\text{number of solutions}}$	solution 방향에 정렬될 때까지
complexity	$O(\sqrt{\text{number of states}})$	oracle 호출 횟수 기준(real quantum hardware)

표를 세로로 훑으면 성질 하나가 눈에 들어옵니다. **여기 어디에도 imaginary number가 없습니다.** 세 가지가 겹쳐서 그렇습니다. 첫째, 시작 state $\lvert\psi\rangle = H^{\otimes \text{number of qubits}}\lvert 0\rangle^{\otimes \text{number of qubits}}$ 의 amplitude는 전부 $\frac{1}{\sqrt{\text{number of states}}}$ 로 real number입니다. 둘째, oracle은 각 amplitude에 $+1$ 또는 -1 을 곱할 뿐이라 real number를 real number로 보냅니다. 셋째, diffusion operator $a_x \mapsto 2\bar{a} - a_x$ 는 real number 계수만 쓰는 linear operation이라 역시 real number를 벗어나지 않습니다. real number 집합이 이 세 연산에 대해 닫혀 있으므로, 귀납적으로 **amplitude는 algorithm 처음부터 끝까지 real number**입니다. 일반적인 state의 amplitude는 complex number이고 그 relative phase가 곧 정보인데(1장), 표준 Grover는 그 자유도를 한 번도 쓰지 않습니다 — 등장하는 phase가 0 과 π 둘뿐이고, 그건 sign bit 하나로 적히니까요.

이 성질 덕분에 hardware에서는 비용이 절반으로 줄어듭니다. amplitude 하나당 real part·imaginary part 두 칸을 잡을 이유가 없으니 **imaginary part memory가 아예 없고**, complex number multiplication(real number multiplication 4번 + addition 2번)도 datapath에 등장하지 않습니다. 대신 조건이 하나 붙습니다 — 이 논증은 "초기 state가 real number + phase가 ± 1 + diffusion operator가 real linear"라는 **세 전제에 근거하므로**, 그 중 하나라도 깨는 기능을 얹는 순간 datapath를 complex number에 맞게 다시 설계해야 합니다. 실제로 number of solutions를 세는 표준 기법인 **quantum counting**은 phase estimation을 쓰고, phase estimation은 $e^{2\pi i k/2^t}$ 꼴의 complex phase를 amplitude에 직접 곱합니다. 그래서 우리 설계는 counting 계열을 아예 채택하지 않고, number of solutions를 모르는 채로 도는 9.6절의 BBHT를 택했습니다. real number 전용 datapath를 지키는 것이 그 선택의 근거 가운데 하나입니다.

9.9 여기서부터 hardware로 — 어느 개념이 어느 장으로 가는가

이 장에서 배운 개념은 모두 circuit의 각 부분에 대응합니다. 숫자와 circuit diagram은 해당 장에 맡기고, 여기서는 **어느 개념이 어디서 circuit이 되는지**의 지도만 그립니다.

9장의 개념	hardware에서 무엇이 되나	어디서
uniform superposition $\frac{1}{\sqrt{\text{number of states}}}\sum_x \lvert x\rangle$	amplitude array를 constant 하나로 채우는 Init	11장 memory 지도 · 13.2절
oracle U_f — predicate 판정	predicate circuit과 sign flip	13.3절
diffusion operator U_ψ — $a \mapsto 2\bar{a} - a$	accumulation → arithmetic shift → subtraction	13.5절
solution이 여러 개라는 것 · measurement	amplitude 동률과 Born-rule sampling	14장
number of iterations	number of solutions를 모를 때의 BBHT shot loop	16장
amplitude를 숫자로 적는 일	fixed-point 형식과 round-half-to-even·saturation 규칙	12장

이 지도에서 가장 자주 오해받는 칸은 **measurement**입니다. 9.3절의 bar chart를 보면 "제일 높은 막대를 고르면 되지 않나" 하는 생각이 자연스럽게 들고, 실제로 이 project도 한동안 maximum comparator를 Measurement Unit으로 상정했습니다. 그런데 9.4절의 손계산과 9.7절의 증명이 말하는 대로 모든 solution의 amplitude는 **정확히 같습니다.** maximum comparator는 동률에서 구조가 정한 index 하나만 결정적으로 내놓으므로, solution이 둘 이상이면 나머지는 probability가 낮아서가 아니라 **원리적으로** 출력될 수 없습니다. 그래서 Measurement Unit은 amplitude의 square를 probability로 삼아 random number로 뽑는 **Born-rule sampling**으로 설계합니다(14.4절). 이 절의 요약만 읽고 comparator를 만들면 나중에 전면 재작업이 필요하므로 이 차이를 기억해 두세요.

> 🔑 FPGA 설계자는 결국 "amplitude array에 **sign flip(oracle)**과 **inversion about the mean(diffusion operator)**을 $\sqrt{\text{number of states}}$ 번 적용"하는 circuit을 만듭니다. 이 한 문장의 모든 단어 — amplitude·sign flip·inversion about the mean·$\sqrt{\text{number of states}}$ — 가 1~9장에서 쌓아 온 개념이에요. algorithm을 정확히 이해해야 precision(fixed-point bit width)·resource·오류를 올바르게 설계할 수 있습니다.

그 "precision·resource·오류를 올바르게 설계"하는 이야기가 **Part 5 이후(11~18장)** 입니다. 위 표의 여섯 줄을 다 지나면 여섯 논문의 지도와 설계 결정표(15장), SoC 통합(17장), verification(18장)이 남습니다. 설계값의 source of truth는 15.4절의 결정표입니다.

9.10 이 장의 요약

- **문제**: 정렬 안 된 $\text{number of states} = 2^{\text{number of qubits}}$ 개 항목에서 조건을 만족하는 항목 찾기. solution 집합 A 가 비어 있지 않은 경우($\text{number of solutions} \geq 1$)가 정식 설정이고, solution이 하나뿐인 경우는 가장 단순한 special case.
- **핵심 장면**: 균등 막대에서 시작 → ① oracle로 조건에 걸리는 막대를 전부 negative로 → ② diffusion operator(inversion about the mean $a \mapsto 2\bar{a} - a$)로 그 막대들을 껑충. 이 둘을 매 iteration마다 되풀이.
- **4 states 예제**: amplitude $\left(\frac{1}{2}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2}\right)$ → oracle → diffusion operator → $(0, 0, 1, 0)$. 한 번에 probability 1. solution을 둘로 늘리면 두 amplitude가 **정확히 같은 값**으로 움직이고, 8 states·2 solutions에서는 나란히 $\frac{1}{\sqrt{2}}$ 까지 자란다.
- **왜 $\sqrt{\text{number of states}}$·왜 π**: Grover는 solution 방향(직각)으로 매번 2θ 씩 도는 **rotation**. 직각까지 = $\frac{\pi}{2}$, 나눗셈하면 $\text{number of iterations} \approx \frac{\pi}{4\theta} \approx \frac{\pi}{4}\sqrt{\text{number of states}/\text{number of solutions}}$. π 는 rotation angle이라서 나온 것.
- **함정**: 최적 횟수를 넘기면 solution을 지나쳐 probability가 다시 떨어짐. number of solutions를 모르면 number of iterations를 계산할 수 없어 **BBHT** shot loop가 shot마다 그 값을 무작위로 뽑는다.
- **수식**: 두 reflection $U_f = I - 2\sum_{x\in A}\lvert x\rangle\langle x\rvert$, $U_\psi = 2\lvert\psi\rangle\langle\psi\rvert - I$. 두 reflection의 composition = rotation. solution이 여러 개여도 rotation 평면은 2차원이고, 그래서 solution들의 amplitude가 **항상 동일**하다.
- **real number 전용**: 초기 uniform superposition이 real number + oracle이 ± 1 곱하기 + diffusion operator가 real linear → amplitude는 real number를 벗어나지 않는다. imaginary part memory가 없고, complex number를 강제하는 quantum counting(phase estimation) 계열은 설계에서 제외.
- **emulator와의 관계**: $\sqrt{\text{number of states}}$ 는 real quantum hardware의 oracle 호출 횟수 이야기다. classical circuit으로 흉내 내면 $O(\text{number of states} \cdot \sqrt{\text{number of states}/\text{number of solutions}})$ 이라 classical linear scan보다 느리므로, 비교 대상은 Qiskit Aer·NumPy state vector simulator이고 값어치는 속도가 아니라 **재현성과 verification**에 있다.

quantum computing 기초부터 Grover까지 살펴보았습니다. 다음 장은 배운 내용을 한눈에 보는 **cheat sheet와 glossary**입니다.

10장. 용어 정리와 cheat sheet

> **성격**: 참조용. 앞 장들을 복습하거나 계산 중 빠르게 찾아볼 때 쓰세요.

이 장은 1~9장의 핵심을 **한눈에** 모은 참조 자료입니다. gate matrix, 핵심 공식, glossary, 연습문제 답, 원문 대조표를 담았습니다.

10.1 gate cheat sheet

gate	matrix	작용	장
I	$\begin{bmatrix}1 & 0\\0 & 1\end{bmatrix}$	identity (quantum wire)	3
X (NOT)	$\begin{bmatrix}0 & 1\\1 & 0\end{bmatrix}$	$\|0\rangle \leftrightarrow \|1\rangle$	3
Y	$\begin{bmatrix}0 & -i\\i & 0\end{bmatrix}$	bit flip + phase flip	3
Z	$\begin{bmatrix}1 & 0\\0 & -1\end{bmatrix}$	$\|1\rangle$ sign flip (phase)	3
H	$\frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\1 & -1\end{bmatrix}$	computational basis↔superposition, interference	3
CNOT	$\begin{bmatrix}1 & 0 & 0 & 0\\0 & 1 & 0 & 0\\0 & 0 & 0 & 1\\0 & 0 & 1 & 0\end{bmatrix}$	control qubit=1이면 target qubit 뒤집기	5
Toffoli	(8×8)	control qubit 둘 다 1이면 target qubit 뒤집기 = 가역 AND	6

꼭 외울 것: X, Z, H 의 matrix와 작용. $HH = I$, $XX = I$, $Z|1\rangle = -|1\rangle$.

10.2 핵심 state

$$|0\rangle = \begin{bmatrix}1\\0\end{bmatrix}, \quad |1\rangle = \begin{bmatrix}0\\1\end{bmatrix}, \quad |+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

- $H|0\rangle = |+\rangle$, $H|1\rangle = |-\rangle$, $H|+\rangle = |0\rangle$, $H|-\rangle = |1\rangle$.
- $X|-\rangle = -|-\rangle$ (phase kickback의 핵심, 7장).
- Bell state (entanglement): $\frac{|00\rangle+|11\rangle}{\sqrt{2}}$ (5장).

10.3 핵심 공식

normalization / measurement (1·4장)

$$|\alpha|^2 + |\beta|^2 = 1, \qquad \langle\psi|\psi\rangle = 1, \qquad P(\text{outcome } k) = |\text{amplitude}_k|^2$$

uniform superposition / Hadamard (6장)

$$H^{\otimes \text{number of qubits}}|0\rangle^{\otimes \text{number of qubits}} = \frac{1}{\sqrt{2^{\text{number of qubits}}}}\sum_x |x\rangle, \qquad H^{\otimes \text{number of qubits}}|u\rangle = \frac{1}{\sqrt{2^{\text{number of qubits}}}}\sum_v (-1)^{u\cdot v}|v\rangle$$

oracle / phase kickback (7장)

$$U_f|x\rangle = (-1)^{f(x)}|x\rangle, \qquad U_f|x\rangle|-\rangle = (-1)^{f(x)}|x\rangle|-\rangle, \qquad U_f = I - 2|\omega\rangle\langle\omega|$$

Grover (9장)

$U_\psi = 2|\psi\rangle\langle\psi| - I$ (inversion about the mean $a \mapsto 2\bar{a} - a$), $\sin\theta = \sqrt{\frac{\text{number of solutions}}{\text{number of states}}}$, $P(\text{number of iterations}) = \sin^2((2 \cdot \text{number of iterations} + 1)\theta)$

number of iterations — 세 가지 경우 (9·14·16장)

- number of solutions를 **알 때**의 최적 number of iterations: $\frac{\pi}{4}\sqrt{\frac{\text{number of states}}{\text{number of solutions}}} - \frac{1}{2}$. (14 qubits·1 solution이면 $100.03 \to 100$ iterations.)
- number of solutions를 **모를 때** (이 project의 기본): BBHT recurrence — $\text{sampling range} \leftarrow 1$ 로 시작해 shot마다 $\text{number of iterations} \sim U[0, \text{sampling range})$ 를 뽑고, 실패하면 $\text{sampling range} \leftarrow \min(\text{growth factor} \cdot \text{sampling range}, \sqrt{\text{number of states}})$, $\text{growth factor} = 6/5$ (16.2절).
- 조건을 만족하는 원소를 **전부 enumeration**할 때의 총비용: $\sum_{k=1}^{\text{number of solutions}} \sqrt{\text{number of states}/k} \approx 2\sqrt{\text{number of states} \cdot \text{number of solutions}}$ — solution이 하나씩 줄어 뒤 enumeration round가 비싸집니다 (14.7절).

10.4 algorithm 한 줄 요약

algorithm	문제	핵심	complexity (classical→quantum)
Deutsch–Jozsa	constant vs balanced 판별	$H^{\otimes\text{number of qubits}} \to U_f \to H^{\otimes\text{number of qubits}}$, interference 1회	$O(2^{\text{number of qubits}}) \to O(1)$
Grover	정렬 안 된 search	$(U_\psi U_f)$ iteration, $\frac{\pi}{4}\sqrt{\text{number of states}}$ 회	$O(\text{number of states}) \to O(\sqrt{\text{number of states}})$
Grover **emulation** (이 project)	같은 algorithm을 classical circuit으로 흉내	iteration당 $O(\text{number of states}/\text{number of lanes})$ cycles × $O(\sqrt{\text{number of states}/\text{number of solutions}})$ iterations	$O(\text{number of states} \cdot \sqrt{\text{number of states}/\text{number of solutions}})$ — classical scan보다 **느림**

마지막 줄을 오해하지 마세요. $\sqrt{\text{number of states}}$ 이득은 **real quantum hardware**의 oracle 호출 횟수 이야기이고, 모든 state의 amplitude를 매 iteration 전부 훑어야 하는 emulator에는 해당하지 않습니다. 그래서 이 해설서는 우리 accelerator를 classical linear scan과 비교하지 않습니다 — 비교 대상은 같은 일을 하는 **Qiskit `AerSimulator`·NumPy state vector simulator**이고, 이 장치의 가치는 속도 우위에 있지 않습니다. quantum algorithm의 동작을 **결정적이고 재현 가능하게 재현·검증하는 전용 hardware**라는 점에 있습니다(11.1절).

10.5 용어집

용어	뜻	장
qubit	quantum 정보의 단위. state는 2차원 complex vector	1
computational basis	$\|0\rangle, \|1\rangle$ (고전 bit 0, 1에 대응)	1
amplitude	state의 complex 성분 α, β. absolute value의 square가 probability	1
superposition	여러 computational basis state의 linear combination	1
normalization	$\|\alpha\|^2 + \|\beta\|^2 = 1$, state vector의 길이가 1	1
ket · bra	column vector $\|\psi\rangle$ · 그 conjugate transpose인 row vector $\langle\psi\|$	2
inner product	$\langle\phi\|\psi\rangle$, 두 state가 닮은 정도 (0이면 orthogonal)	2
conjugate transpose	transpose한 뒤 complex conjugate를 취한 것. adjoint, dagger(†)라고도 부름	2
unitary	$U^\dagger U = I$. 길이를 보존. 모든 gate가 unitary	3
Hadamard gate	superposition을 만드는 gate H	3
Pauli gate	X, Y, Z	3
measurement	state를 고전 bit로 collapse시켜 읽음	4
Born rule	결과 probability $= \|\text{amplitude}\|^2$	4
collapse	measurement 뒤 computational basis state 하나로 확정됨 (비가역)	4
global phase	state 전체에 곱해지는 $e^{i\theta}$. measurement 결과에 영향 없음	4
tensor product	qubit를 합치는 연산 $\otimes$	5
CNOT	controlled-NOT. control qubit가 1이면 target qubit를 뒤집음	5
entanglement	tensor product로 분해되지 않는 state. measurement 결과가 서로 상관됨	5
quantum parallelism	$2^{\text{number of qubits}}$ 개 입력을 동시에 처리 (단, measurement로는 하나만 얻음)	6
interference	amplitude끼리 상쇄 · 보강	6
Toffoli gate	controlled-controlled-NOT. 가역 AND (control qubit 2개)	6
universal gate set	single-qubit gate + CNOT	6
oracle	solution을 판별하는 black box f	7
phase kickback	출력 쪽 뒤집힘이 입력 phase로 되돌아오는 현상	7
diffusion operator	Grover의 inversion about the mean U_ψ	9
amplitude amplification	solution amplitude를 키우는 기법	9

10.6 연습문제 답

- **1-1** (1장): $|c| = \frac{\sqrt{3}}{2}$. ($\frac{1}{4} + |c|^2 = 1$.)
- **2-1** (2장): $\langle +|-\rangle = 0$ (orthogonal). $\langle +|-\rangle = \frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\end{bmatrix} \cdot \frac{1}{\sqrt{2}}\begin{bmatrix}1 \\ -1\end{bmatrix} = \frac{1}{2}(1 \cdot 1 + 1 \cdot (-1)) = 0$. 그래서 $|+\rangle, |-\rangle$ 도 orthonormal basis를 이룹니다.
- **3-1** (3장): $Z|+\rangle = |-\rangle$. (Z 가 $|1\rangle$ sign을 뒤집어 $\frac{|0\rangle - |1\rangle}{\sqrt{2}}$.)
- **4-1** (4장): $P(1) = \left|\frac{\sqrt{3}}{2}\right|^2 = \frac{3}{4}$. (Born rule: amplitude의 absolute value square. 참고로 $P(0) = \frac{1}{4}$, sum은 1.)

- **5-1** (5장): **product state**. $\frac{|00\rangle+|01\rangle}{\sqrt{2}} = |0\rangle \otimes \frac{|0\rangle+|1\rangle}{\sqrt{2}} = |0\rangle|+\rangle$ 로 첫 qubit를 깔끔히 묶어낼 수 있으므로 entanglement가 아닙니다.
- **6-1** (6장): $\frac{1}{\sqrt{8}} \cdot H^{\otimes 3}|000\rangle$ 은 8개 computational basis를 모두 amplitude $\frac{1}{\sqrt{2^3}} = \frac{1}{\sqrt{8}}$ 로 균등하게 담으므로, $|101\rangle$ 도 예외 없이 $\frac{1}{\sqrt{8}}$.
- **7-1** (7장): $|x\rangle|-\rangle$ (그대로). $f(x) = 0$ 이면 $(-1)^{f(x)} = (-1)^0 = 1$ 이라 phase kickback으로 붙는 sign이 $+1$ 입니다. sign이 뒤집히는 것은 solution($f(x) = 1$)뿐.
- **8-1** (8장): 0. balanced function은 $|0\cdots0\rangle$ 의 amplitude $\frac{1}{2^{\text{number of qubits}}} \sum_x (-1)^{f(x)}$ 이 $+1$ 절반과 -1 절반의 완벽한 destructive interference로 0이 되므로, measurement에서 절대 나오지 않습니다.
- **9-1** (9장): $\text{number of iterations} \approx \frac{\pi}{4}\sqrt{\text{number of states}/\text{number of solutions}} = \frac{\pi}{4}\sqrt{64/4} = \frac{\pi}{4} \cdot 4 = \pi \approx 3.14$ → 약 **3 iterations**.

10.7 자주 하는 실수 모음

실수	바로잡기	장
$\|0\rangle$ 을 zero vector로 착각	$\|0\rangle = (1, 0)^T$, unit vector	1
superposition을 "모르는 0/1"로 해석	진짜 동시 존재. amplitude는 complex number	1
amplitude를 probability로 착각	probability는 $\|\text{amplitude}\|^2$ (square)	4
circuit 순서 = matrix 순서로 착각	반대. 나중 gate가 matrix 왼쪽	3
measurement를 가역으로 착각	measurement는 비가역, superposition 파괴	4
global phase와 relative phase 혼동	global phase는 무의미, relative phase는 결정적	4
CNOT이 target qubit만 바꾼다고 일반화	computational basis에서만. $\|+\rangle\|-\rangle$ 은 control qubit도 바뀜	5
Grover를 많이 돌릴수록 좋다고 착각	최적 초과 시 probability 감소(진동)	9
amplitude가 가장 큰 칸을 고르면 된다고 착각	모든 solution의 amplitude는 매 iteration **정확히 같음.** maximum comparator는 늘 같은 index 하나만 반환	9·1 4
$\|\alpha\| \le 1$ 이니 integer bits 1-bit면 된다고 착각	diffusion operator 중간값 $2 \cdot \text{mean} - \text{amplitude}$ 는 최대 $1 + 2/\sqrt{\text{number of states}}$ 까지 커져 1을 넘음	12

마지막 두 줄은 교과서적 오해가 아니라 **이 project에서 실제로 겪은 문제**입니다. Measurement Unit을 한동안 maximum comparator로 잡아 두었다가 solution이 둘 이상일 때 동률 때문에 실패하는 것을 확인하고 Born-rule sampling으로 바꾸었습니다(14.3절). 또 amplitude의 크기만 보고 integer bits를 sign 1-bit로 잡았다가 diffusion operator 중간값이 표현 범위를 넘는 것을 보고 한때 integer bits를 2 bits로 늘렸습니다. 최종 RTL은 저장 data word를 23-bit Q1.22로 두고 중간값 register만 넓혀 symmetric saturation을 거는 쪽으로 정리했습니다(12.3절).

10.8 원문 자료 대조표

이 해설서가 재구성한 원문(`documents/study_references/papers/1. 양자컴퓨터 기초이론/`):

원문	다룬 주제	대응 장
Quantum Computing for the Very Curious (Matuschak & Nielsen)	qubit·gate·measurement·unitary·CNOT·entanglement·universality·Toffoli	1~6
[기초이론] 5. 그로버 알고리즘과 도이치 알고리즘 (hgmin1159)	oracle·amplitude amplification·inversion about the mean·diffusion operator·Grover circuit·Deutsch–Jozsa	7~9
양자 컴퓨팅 입문 (2) - Grover's Algorithm (infossm)	phase kickback·diffusion operator·geometric rotation·number of iterations 유도	7, 9

더 깊이 보려면: Nielsen & Chuang, *Quantum Computation and Quantum Information* (표준 교과서); Sutor, *Dancing With Qubits*; Bernhardt, *Quantum Computing for Everyone*; Grover (1996) 원논문; Boyer et al. (1998) *Tight bounds on quantum searching* (number of Grover iterations의 엄밀한 분석).

10.9 다음 단계 — Part 5: 이론에서 hardware로

이론을 마쳤으니, 이 algorithm을 **FPGA hardware로 흉내 내는** 이야기로 넘어갈 준비가 되었습니다. 11장부터 18장까지 여덟 장에서 이 project가 실제로 만들려는 장치를 다룹니다. Part 5(11·12장)가 기초가 되는 memory 지도와 수 표현을 설명하고 Part 6(13·14장)이 한 shot 안의 datapath를, Part 7(15장)이 설계 좌표를, Part 8(16~18장)이 shot 바깥의 제어·system·verification을 다룹니다.

- 11장 state vector emulator — ket $|\psi\rangle$ 이 array가 되는 원리, `amp`·`data`·`mask` 세 array의 BRAM 예산, 32-lane banking, pass 단위 cycle 비용 model.
- 12장 amplitude를 숫자로 — 왜 floating-point가 아니라 fixed-point인가. Q1.22(23 bits)와 넓은 중간값·symmetric saturation, round-half-to-even, 그리고 bit width는 공식이 아니라 **재서** 정한다는 것.
- 13장 oracle과 diffusion operator — 한 iteration 안의 FSM, predicate 4종($<\cdot>\cdot=$·범위)이 subtractor 하나를 공유하는 방법, accumulation·shift·subtraction만으로 계산하는 diffusion operator(0 multipliers).
- 14장 multiple solutions — solution이 여럿일 때의 amplitude 동률, Born-rule sampling, row·lane layer measurement, Found Mask로 중복 없이 전부 찾는 enumeration mode.
- 15장 논문 지도와 설계 결정표 — 여섯 논문의 계보와 여덟 개의 대립축, 확정 설계 결정표, 인용 시 주의할 정오표.
- 16장 몇 번 돌릴지 모를 때 — number of solutions를 모를 때의 BBHT shot loop(mean 22.7 shots, 14 qubits·1 solution)와, 실패한 shot이 남긴 amplitude를 세 벌까지 남겨 두고 이어 도는 checkpoint.
- 17장 RVX SoC에 붙이기 — accelerator를 system에 얹는 일. APB CSR로 제어하고, AHB master로 data를 실어 오고, host와는 UART로 통신합니다.
- 18장 3단 reference model과 bit-exact verification — float64 → fixed-point model → RTL의 verification ladder로 "무엇이 틀렸나"를 층마다 나눠 묻는 방법.

hardware 쪽 용어(LUT·FF·BRAM·DSP·APB·AHB 같은 것들)는 15.7절 hardware glossary에 모아 두었습니다. 위 10.5절의 glossary는 quantum 이론 용어 전용입니다. 그리고 설계 수치 — bit width·BRAM 개수·number of lanes·shot 통계 — 의 source of truth는 15.4절 확정 설계 결정표입니다. 다른 장이 같은 값을 본문에 다시 적어 두는 것은 읽는 도중 link를 따라가지 않고도 확인할 수 있게 하기 위해서입니다. 값이 어긋나면 설계 결정표를 기준으로 합니다.

이제 "state vector·oracle·diffusion operator·$\sqrt{\text{number of states}}$ iterations·measurement probability"가 무슨 뜻인지 알고 있으므로, 이 이론이 Verilog RTL과 datapath로 바뀌는 것을 따라갈 수 있습니다. 각 논문을 원문으로 정독하고 싶다면 한국어 해설본이 `../papers/papers_ko/`에 있습니다(원문 대조용 보조 자료).

11장. state vector emulator — memory map과 resource 예산

> **이 장을 읽기 위한 준비**: 1장 amplitude·superposition, 5장 tensor product·$2^{\text{number of qubits}}$ dimension, 9장 Grover. FPGA 기초 용어(LUT·FF·BRAM·DSP)는 15.7절 hardware 용어집에 모아 두었습니다.

10장까지 우리는 **quantum algorithm이 수학적으로 무엇을 하는지**를 배웠습니다. 이제 방향을 바꿉니다. 여기서부터(Part 5)는 그 algorithm을 **classical hardware(FPGA)로 흉내 내는 기계** — 이 project의 실제 목표물 — 를 만드는 과정입니다. 새로 배우는 논문은 여섯 편이고, 이 장은 그 여섯 편이 공유하는 **기본적인 질문 하나**에서 출발합니다: *quantum computer가 없는데 어떻게 quantum algorithm을 "돌린다"는 걸까요?*

답은 단순합니다. state는 결국 **숫자 array**(amplitude vector)이고, gate는 그 array에 가하는 **산술 연산**입니다. 진짜 quantum computer가 물리적으로 하는 일을, classical computer는 그 숫자 array를 memory에 펼쳐 놓고 계산으로 재현합니다. 이것이 **state vector emulation**이고, 이 장은 그 원리와 **비용**을 다룹니다. 비용을 알아야 이 방식으로 다룰 수 있는 qubit 수가 왜 제한되는지 알 수 있고, board에서 검증된 우리 IP가 실제로 몇 개의 BRAM과 몇 cycle을 쓰는지도 이 장에서 숫자로 확인합니다.

11.1 emulation·simulation·진짜 quantum computer

세 단어가 자주 뒤섞여 쓰이니 먼저 정리하겠습니다. 07번 논문(El-Araby 외, IEEE TC 2023)의 용법을 따릅니다.

- **진짜 quantum computer**: 물리적 qubit(초전도 circuit·이온 등)가 실제로 superposition·entanglement·interference를 겪습니다. amplitude는 어디에도 "저장"되지 않고 물리 state 그 자체입니다.
- **simulation**: software로 state vector를 계산합니다. 보통 CPU/GPU에서 Qiskit 같은 library로 돌립니다. 유연하지만 느립니다.
- **emulation**: **전용 hardware(FPGA)** 로 state vector 계산을 circuit 수준에서 가속합니다. simulation과 하는 일(숫자 array 계산)은 같지만, software가 아니라 **맞춤 datapath**로 처리한다는 점이 다릅니다.

셋 중 뒤의 둘은 **똑같이 classical computer가 숫자 array를 계산**합니다. 차이는 "얼마나 빠른 hardware로 하느냐"뿐입니다. 우리 project가 만드는 것은 세 번째 — **FPGA emulator**입니다.

우리 설계의 구체적인 구성을 먼저 살펴보겠습니다. 이 IP는 독립적으로 동작하는 chip이 아니라 **RVX SoC 안에 얹히는 accelerator**입니다. 제어 register(CSR)는 **APB slave**로 노출되어 RISC-V core가 명령·threshold·state를 읽고 쓰고, search 대상 data는 IP가 **AHB master**가 되어 직접 실어 옵니다. host는 **UART** 너머에서 RISC-V app에게 명령을 보낼 뿐, amplitude array를 한 칸도 만지지 않습니다. 즉 연산은 전부 IP 안에서 끝나고 host는 시작·정지·결과 회수만 합니다(자세한 것은 17장).

여기서는 emulator가 quantum computer가 아니라는 사실을 반드시 구분해야 합니다. **quantum 이득($O(\sqrt{\text{number of states}})$ 같은)을 물리적으로 누리지 못합니다.** Grover iteration 한 번이 이미 모든 amplitude를 두 번 훑으므로 총비용은 $O\left(\text{number of states} \cdot \sqrt{\text{number of states}/\text{number of solutions}}\right)$ 이고, 이것은 같은 문제를 classical program이 array 한 번 훑어 푸는 $O(\text{number of states})$ 보다 **느립니다.** 이 사실을 흐리지 마십시오 — 우리 IP를 classical linear scan과 나란히 놓고 속도를 자랑하는 문장은 이 해설서 어디에도 없습니다.

그럼 무엇과 비교해야 할까요. 같은 일을 하는 것, 즉 **state vector simulator**입니다. 구체적으로는 Qiskit의 `AerSimulator` 나 NumPy로 짠 state vector code가 우리의 비교 대상입니다. 그쪽도 모든 amplitude를 memory에 펼쳐 놓고 매 iteration 전부 훑으니, 우리와 같은 $O\left(\text{number of states} \cdot \sqrt{\text{number of states}/\text{number of solutions}}\right)$ 이 듭니다. 다만 그 일을 범용 CPU의 명령어로 하느냐 전용 datapath로 하느냐가 다를 뿐이죠. 같은 board 위에서 accelerator를 끄고 RISC-V core 하나로 같은 계산을 돌린 순수 software baseline도 정당한 대조군입니다(18.8절). 그리고 이 IP의 값어치는 애초에 속도 우위가 아니라, **quantum algorithm의 동작을 결정적이고 재현 가능하게 재현·검증하는 전용 hardware**라는 데 있습니다. 같은 seed를 주면 같은 shot 수열과 같은 후보가 나오고, amplitude array는 언제든 덤프해 software reference model과 bit 단위로 대조할 수 있습니다(18장). 실제 quantum computer에서는 둘 다 불가능한 일입니다.

한 가지 더 짚어 둘 것이 있습니다. 문헌에서 "FPGA로 quantum을 다룬다"는 말은 사실 두 방식으로 갈립니다 — FPGA 자체가 quantum computer 역할을 맡고 host가 거기에 접속해 쓰는 방식과, 무거운 state vector 연산만 FPGA로 떼어내 가속하는 보조 accelerator 방식입니다(Cicero 외 2025 survey). 우리 IP는 뒤쪽입니다. 용어에는 주의가 필요합니다. 이 survey는 앞의 것을 "emulation", 뒤의 것을 "acceleration"이라 불러, 07번을 따라 "emulation = FPGA 가속 simulation"으로 써 온 이 해설서와 이름이 정반대입니다. 문헌마다 같은 단어가 다른 것을 가리킬 수 있다는 점을 기억해 두세요.

11.2 세 개의 array — amp·data·mask

9장에서 Grover의 시작 state를 이렇게 썼습니다:

$$|\psi\rangle = H^{\otimes \text{number of qubits}}|0\rangle^{\otimes \text{number of qubits}} = \frac{1}{\sqrt{\text{number of states}}} \sum_{x=0}^{\text{number of states}-1} |x\rangle$$

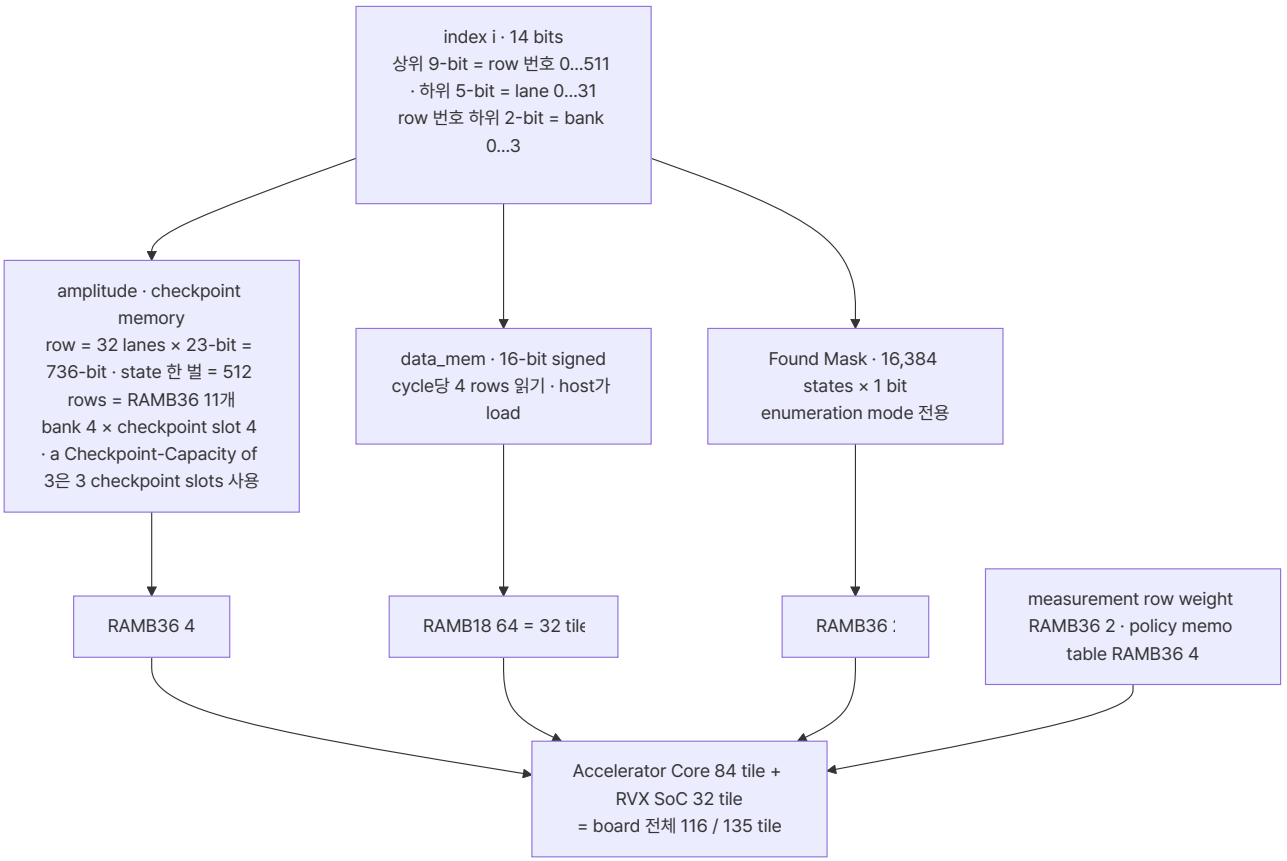
이 ket을 hardware에서 계산하려면 hardware가 다루는 **memory에 담긴 숫자**로 표현해야 합니다. 그래서 emulator는 이 state를 **amplitude array**로 펼칩니다:

$$|\psi\rangle = \sum_{i=0}^{\text{number of states}-1} \alpha_i |i\rangle \;\longrightarrow\; \texttt{amp_mem}[0..\text{number of states} - 1] = [\alpha_0,\ \alpha_1,\ \ldots,\ \alpha_{\text{number of states}-1}]$$

`amp_mem[i]` 는 computational basis state $|i\rangle$ 의 amplitude α_i 를 담는 memory 한 칸입니다. uniform superposition은 이 array의 **모든 칸을** $\frac{1}{\sqrt{\text{number of states}}}$ **로 채우는 것**과 정확히 같습니다.

그런데 우리 IP에는 array가 하나가 아니라 **셋**입니다. 우리가 푸는 문제가 추상적인 "표시된 state 찾기"가 아니라 **memory에 담긴 data 중 predicate를 만족하는 index 찾기**이기 때문입니다.

array	data word width	깊이	담는 것	누가 건드리나
`amp_mem`	**23b** (Q1.22)	16,384	amplitude α_i	Init·oracle·diffusion operator가 **씁니다**. measurement는 읽기만
`data_mem`	**16b signed**	16,384	search 대상 값 `value[i]`	host가 load. IP는 **읽기만**
`found_mask`	**1b**	16,384	이미 찾은 solution 표시 `found[i]`	enumeration mode에서 solution이 하나 나올 때마다 한 bit씩



amplitude 폭을 왜 23 bits로 잡았는지는 12장이 통째로 답합니다. 여기서는 array가 셋이라는 사실과, 그중 amplitude만 쓰기 대상이라는 사실만 기억해 두면 됩니다 — 뒤의 사실은 16.5절 checkpoint의 근거가 됩니다. 하나 미리 적어 두면, 실제 chip 안의 `amp_mem` 은 한 벌이 아닙니다. amplitude array 여러 벌을 같은 memory에 두고 그중 하나에서 계산을 이어 가는 **checkpoint slot** 구조인데, 그 이유는 16장이 설명합니다.

이제 gate가 array 연산이 되는 모습을 봅니다. 9장에서 본 Grover 한 바퀴 $G = U_\psi U_f$ 는 다음과 같은 연산이 됩니다.

oracle U_f. 9장에서는 "solution index의 amplitude를 뒤집는다"고 썼습니다. 그것은 답을 이미 아는 사람의 서술입니다. 우리 circuit은 solution index를 **모릅니다.** circuit이 가진 것은 threshold와 predicate뿐이고, 하는 일은 이것입니다:

$$\texttt{amp_mem}[i] \;\leftarrow\; -\,\texttt{amp_mem}[i] \quad \text{if } \mathrm{pred}\big(\texttt{data_mem}[i],\ \texttt{thr}\big)$$

predicate pred 는 네 종류입니다 — `value < thr_a`, `value > thr_a`, `value == thr_a`, 그리고 범위 `thr_a < value < thr_b`. predicate를 만족하는 칸이 하나일 수도 수백 개일 수도 있고, circuit은 **몇 개인지조차 모른 채** array 전체를 훑으며 칸마다 판정합니다. 이 명제 — **oracle은 solution을 아는 circuit이 아니라 predicate를 판정하는 circuit** — 는 해설서 전체에서 일관되게 따르는 설명이고, 14.2절에서 다시 다룹니다. predicate가 실제로 어떤 gate가 되는지는 13.3절입니다.

diffusion operator U_ψ. 이쪽은 predicate와 무관하게 전 칸에 똑같이 걸립니다 — inversion about the mean `amp_mem[i] ← 2*mean - amp_mem[i]` 입니다. index를 가리지 않으므로 data를 볼 필요조차 없습니다.

즉 ket과 unitary matrix로 쓰던 모든 것이, emulator에서는 **array에 대한 산술**로 표현됩니다. Part 5 이후에는 이 대응 관계를 바탕으로 설명합니다.

11.3 qubit 하나 = memory 두 배

이 구조의 가장 중요한 성질은 **비용이 number of qubits에 지수적**이라는 것입니다. 5장에서 qubit가 여럿인 state의 dimension이 $2^{\text{number of qubits}}$ 임을 배웠습니다. amplitude를 명시적으로 펼치면, array 크기가 곧 number of states이고 $\text{number of states} = 2^{\text{number of qubits}}$ 입니다:

number of qubits	number of states
4	16
10	1,024
14	**16,384** ← 우리 IP
20	약 100만
30	약 10억

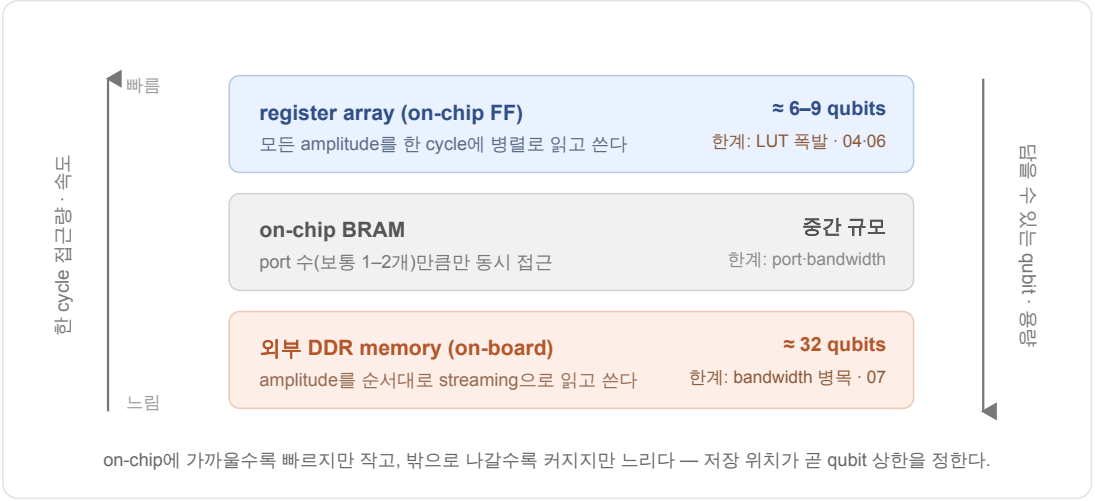
qubit를 **하나** 늘릴 때마다 저장해야 할 amplitude가 **두 배**가 됩니다. 진짜 quantum computer라면 qubit 하나를 추가하는 것이 state space를 두 배로 키우는 "공짜 지수 이득"이지만, emulator에서는 정반대로 **memory·연산이 두 배로 드는 지수 비용**입니다. qubit를 늘리는 것이 아니라 memory를 지수적으로 늘리는 구조인 셈이고, 이것이 emulator가 넘을 수 없는 근본 한계입니다.

엄밀히 말하면 이 지수 비용은 물리 법칙이 아니라 **우리가 택한 방식의 결과**입니다. 복잡도 이론에는 amplitude를 저장하는 대신 필요할 때마다 다시 계산해 memory를 number of qubits에 선형($O(s+t)$)으로 줄이는 공간–시간 맞바꿈이 알려져 있습니다(Frank 외 2009, SEQCSim). 다만 그 대가로 이번엔 **시간**이 지수적으로 늘어나고, 특히 Grover에는 불리합니다 — diffusion operator가 매 iteration amplitude **전체**의 mean을 요구하므로 결국 모든 amplitude가 동시에 필요하고, "한 번에 하나씩 재계산"하는 그 방식과 정면으로 충돌하기 때문입니다. 그래서 우리는 memory를 사용하는 state vector 전개를 택합니다 — 부담을 다른 resource로 옮길 수는 있어도, Grover accelerator에서는 그 맞바꿈이 손해입니다.

그래서 이 분야의 모든 논문은 사실상 **"이 지수 벽을 몇 qubit까지 미룰 수 있는가"** 의 경쟁입니다. 그리고 그 경쟁의 첫 갈림길이 바로 다음 절 — amplitude를 **어디에** 저장할 것인가입니다.

11.4 amplitude를 어디에 둘 것인가

FPGA에는 숫자를 저장할 곳이 여러 종류이고, 어느 곳을 고르느냐가 다룰 수 있는 qubit 수를 결정합니다. 여섯 논문과 우리 설계는 세 가지 저장 방식을 씁니다.



on-chip에 가까울수록 빠르지만 작고, 밖으로 나갈수록 커지지만 느리다 — 저장 위치가 곧 qubit 상한을 정한다.

저장 위치	누가	접근 방식	number of qubits
register array(FF/LUT)	04, 06	모든 amplitude를 온칩 register에 — 한 cycle에 전부 병렬 접근	**6** (04, Kintex) · **8~9** (06)
온칩 BRAM	**우리 IP**	32칸짜리 row 단위로 저장하고 매 cycle 4 rows(128칸)씩 순차 streaming	**14**
외부 DDR memory	07	board에 붙은 대용량 DRAM에 — 순차 streaming	**32**

세 방식의 차이는 **속도와 용량의 맞바꿈**입니다. register는 가장 빨라서(모든 칸을 같은 cycle에 읽고 씀) 연산을 극단적으로 병렬화할 수 있지만, FPGA에 register는 적으므로 금방 바닥납니다. 외부 DDR은 수 GB로 방대하지만 한 번에 몇 data word씩만 읽어 와야 해서 순차 처리를 강요합니다. 07번 논문이 32 qubits라는 압도적 규모를 달성한 것은 순전히 **외부 DDR4를 썼기 때문**입니다(16GB DIMM × 4). 04·06이 한 자릿수 qubit에 머문 것은 반대로 **전부 온칩 register에 저장했기 때문**이고요.

우리는 가운데를 골랐습니다. 이 선택의 이유를 먼저 살펴보겠습니다. **board에서 검증된 우리 IP에는 외부 DDR이 등장하지 않습니다.** 첫째, 14 qubits라면 세 array가 checkpoint slot까지 포함해 온칩에 전부 들어갑니다 — 용량이 충분하므로 외부에 둘 필요가 없고, DRAM controller와 그 latency를 감출 buffer 구조는 통째로 순수한 추가 비용입니다. 둘째, 그리고 이쪽이 더 결정적인데, 우리는 amplitude만이 아니라 **data array까지** 온칩에 둬야 합니다. oracle이 한 cycle에 128칸을 판정하려면 `value[i]` 128개가 amplitude 128칸과 **같은 cycle에** 나와야 하기 때문입니다. 외부 DRAM 한 channel로는 그런 접근을 줄 수 없습니다. (저장소에는 iteration마다 amplitude를 DRAM에 전량 저장하는 두 번째 variant인 DRAM Variant(`hardware_dram`)가 초안으로 있지만, 거기서도 계산 자체는 BRAM 위에서 하고 DRAM은 amplitude 보관 창고로만 씁니다. Board-Verified Build는 이 장이 다루는 BRAM Variant입니다.)

그 대가로 우리는 용량 예산을 빡빡하게 씁니다. 2026-09-07 implementation report에서 잰 실제 사용량은 이렇습니다(값의 source of truth는 15.4절):

block	구성	BRAM
amplitude · checkpoint memory	state 한 벌 = 512 rows × 736-bit = RAMB36 11개. row phase 4 banks에 checkpoint slot 4벌	RAMB36 44
`data_mem`	4 rows 동시 읽기용	RAMB18 64 (= 32 tile)
`found_mask`	16,384 × 1-bit, 4 rows 동시	RAMB36 2
measurement용 row-weight memory	512 × 51-bit	RAMB36 2
checkpoint policy memo table	16장의 policy 표	RAMB36 4
Accelerator Core 계		**84 tile**
RVX SoC	system SRAM 128 KB 가 대부분	32 tile
board 전체		**116 / 135**

board는 Arty A7-100T(`xc7a100tcsg324-1`)이고 BRAM tile(BRAM36)이 **135개** 있습니다. 116개를 쓰니 남는 것이 19개입니다. 여기서 qubit를 하나 늘리면 어떻게 될까요. state 한 벌이 두 배(RAMB36 22개)가 되어 checkpoint slot 4벌만 88개, `data_mem` 도 64 tile 로 불어나 Accelerator Core 계가 대략 160 tile, board 전체를 넘깁니다(구조를 그대로 둔 채 깊이만 두 배로 잡은 추정입니다). 즉 **이 구조와 이 board에서 14 qubits가 천장**입니다. 14 qubits로 굳힌 근거는 이 resource 계산 하나가 아니라 precision benchmark도 함께인데, 그쪽은 12.7절에 있습니다.

> ⚠️ 여기서 흔히 하는 오해를 하나 짚고 갑니다. "register에 두니까 resource(LUT)가 터진다"는 설명은 **틀렸습니다.** 다음 절이 이 오해를 data로 반박합니다.

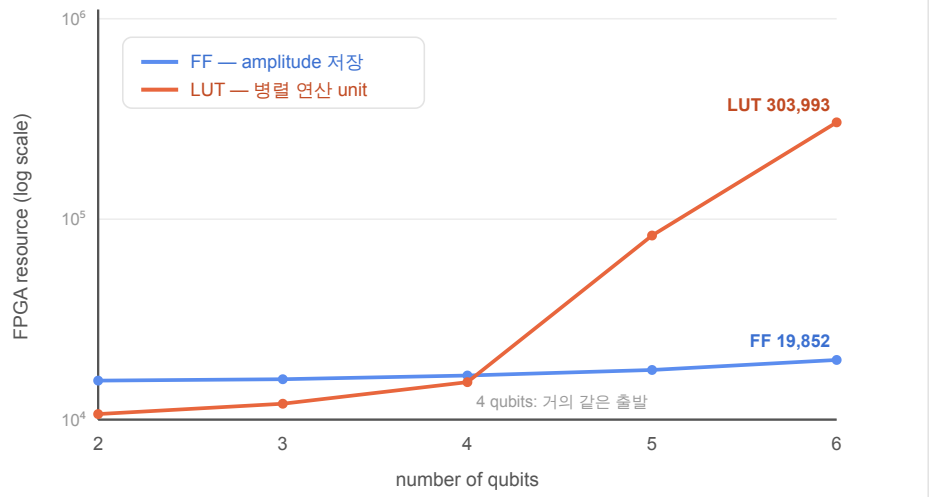
11.5 register의 진짜 대가

04번 논문은 qubit를 늘리며 FPGA resource 사용량을 실측했습니다. Kintex UltraScale+ board 기준으로, 4 qubits에서 6 qubits로 갈 때 두 resource가 이렇게 움직입니다:

resource	4 qubits	6 qubits	배율
FF(flip-flop = 저장)	16,606	19,852	**1.20×**
LUT(lookup table = 연산 logic)	15,386	303,993	**19.8×**

이 결과에서 원인을 구분할 수 있습니다. 만약 resource 폭발의 원인이 "amplitude를 register에 저장하는 것"이라면, **저장을 담당하는 FF가 폭증해야 합니다.** 그런데 FF는 겨우 1.2× 늘었습니다. 폭증한 것은 **LUT** — 즉 **연산 logic**입니다.

원인은 저장이 아니라 **병렬 연산 unit**입니다. 04번 논문은 $O(\text{number of states})$ 짜리 연산(amplitude 전체를 더하는 accumulation, 전체를 비교하는 measurement 등)을 **한두 cycle에 끝내려고** adder·comparator·shifter를 number of states에 비례해 통째로 펼쳐 놓습니다. qubit가 하나 늘면 number of states가 두 배가 되고, 그만큼 연산 unit도 두 배가 되어 LUT 사용량이 지수적으로 늘어납니다. 논문 스스로 "늘어난 연산 부담을 실행 시간이 아니라 hardware resource로 떠넘겼다"고 밝힙니다(이 맞바꿈은 11.7절에서 우리 선택과 나란히 놓고 다시 봅니다).



04 Table 4 (Kintex UltraScale+). 저장은 1.2×로 평평, 병렬 연산은 20×로 치솟는다 — LUT 폭발의 범인은 저장이 아니다.

06번 논문(같은 연구실의 QAOA emulator)이 이 현상을 더 선명하게 보여줍니다. 8~9 qubits에서 한계에 부딪히는데, 그 시점에 **DSP(multiplier)는 전체의 3.8%밖에 안 쓰고** LUT가 먼저 100%를 넘겨 버립니다. multiplication resource에는 여유가 있는데 연산 logic routing이 한계에 이른 것입니다.

이 구분이 왜 중요할까요? 우리가 만약 "register 대신 BRAM에 두면 resource가 해결된다"고 착각하면 원인을 잘못 판단하기 때문입니다. BRAM에 두는 진짜 효과는 **저장 압박 해소**가 아니라, amplitude를 한꺼번에 병렬 접근할 수 없게 만들어 **연산을 순차화하도록 강제**하는 것입니다 — 그래서 병렬 unit이 줄고 LUT가 회수됩니다. 대가는 속도(cycle 증가)이고요. 즉 저장 위치 선택은 **저장 용량 문제가 아니라 병렬성 예산 문제**입니다. 우리 IP의 resource 기록이 이것을 그대로 보여 줍니다 — iteration 한 번에 협력하는 PE를 1 PE에서 4 PEs로 늘렸을 때(16장), memory는 그대로인데 Accelerator Core의 LUT가 13,804에서 30,294로 2.2× 뛰었습니다(synthesis 기준). 병렬도를 높이면 logic 사용량도 늘어납니다.

11.6 memory-bound의 두 얼굴 — 용량과 port

07번 논문의 핵심 주장이자 이 분야를 관통하는 진단입니다. **memory-bound**란, 계산의 속도가 산술 능력이 아니라 **memory에서 data를 꺼내 오는 능력**에 제한되는 상황을 뜻합니다.

비유하자면 이렇습니다. 요리사(연산 unit)를 아무리 늘려도, 재료(amplitude)를 냉장고(memory)에서 꺼내 오는 문 하나가 좁으면 요리 속도는 그 문이 결정합니다. emulator에서 diffusion operator 한 번은 모든 amplitude를 읽어야 하는데, number of states가 지수적으로 크니 결국 "amplitude를 얼마나 빨리 실어 나르느냐"가 전체 속도를 지배합니다. multiplier(DSP)를 더 넣는 것은 속도 개선으로 이어지지 않는 경우가 많습니다 — 06번에서 DSP가 3.8%만 쓰인 것도 같은 이야기입니다.

이 진단에 대한 최근의 응답은 "bandwidth 자체를 높이자"입니다. 07번이 외부 DDR4에서 "memory-bound다"로 멈춘 지점을, 2026년 C-DAC의 Alveo U55C implementation(Tiwari 외)은 이 제약을 완화합니다 — DDR4보다 약 10× 빠른 **HBM**(high-bandwidth memory, 460 GB/s)을 16 banks로 나누고, amplitude i 를 `i mod 16` 번째 bank에 분산해(연속된 amplitude를 서로 다른 bank로) 병렬로 읽습니다. 이렇게 30 qubits(amplitude 약 10.7억 개, 8 GB)를 단일 FPGA에서 돌렸습니다. 특히 눈길을 끄는 것은 이들이 관측한 **성능 절벽**입니다 — 12 qubits에서 13 qubits로 넘어가는 순간 state vector가 온칩 BRAM을 넘쳐 HBM으로 밀려나며 실행 시간이 81% 튑니다. 11.4에서 개념으로만 이야기한 "storage layer의 경계"가 benchmark graph에 그대로 찍힌 셈입니다. (다만 이 논문은 Grover 전용이 아닌 범용·complex number simulator이고 아직 비심사 preprint이며, number of qubits 자체는 07번의 32를 넘지 않습니다 — 새로운 것은 규모가 아니라 HBM이라는 storage layer와 bank 병렬 구조입니다.)

그렇다면 우리는 어떨까요. 우리 IP는 계산 중에 외부 memory를 쓰지 않으므로 **bandwidth는 병목이 아닙니다.** 온칩 BRAM은 요청한 cycle에 답을 줍니다. 그 대신 memory-bound는 서로 다른 두 가지 제약으로 나타납니다.

첫째는 용량 천장입니다. Accelerator Core가 84 tile, SoC가 32 tile로 board 135 tile 중 116 tile을 씁니다(11.4). qubit 하나가 memory 두 배이므로 15 qubits는 곧바로 예산 밖입니다. 07번이 "온칩 resource에만 의존해서 2~7 qubits에 갇혔다"고 지적한 그 한계가 우리에게도 적용됩니다. 다만 register 대신 BRAM에, 그것도 23-bit amplitude 셋을 69-bit 폭에 빈틈없이 담아(12.8절) 상한을 14까지 높인 것이고, **number of qubits의 상한을 정하는 것은 연산 circuit이 아니라 memory 용량**이라는 구조는 똑같습니다. 우리 경우는 checkpoint slot(16장)이 amplitude array를 여러 벌 요구하기 때문에 이 천장이 한층 더 빨리 옵니다.

둘째는 port 천장입니다. BRAM 하나에는 port가 둘뿐입니다. 한 cycle에 amplitude 128칸을 읽고 쓰고 싶으면 방법이 하나뿐입니다 — array를 여러 개의 독립된 block로 쪼개 폭을 넓히는 것. 즉 우리 병렬도의 상한을 정하는 것도 adder 개수나 DSP가 아니라 **memory의 number of ports와 port width**입니다. "multiplier를 늘려도 소용없다"는 07번의 진단이 우리 조건에서는 **"lane을 늘리려면 memory를 넓혀야 하고, 폭을 넓히면 필요한 BRAM 개수가 늘어난다"** 로 번역됩니다. 다음 절이 그 계산입니다.

11.7 병렬도와 banking

우리 IP는 amplitude를 **row** 단위로 다룹니다. 한 row는 연속된 index **32개**, 곧 **32 lanes**이고, row 하나의 32칸이 한 번에 읽히고 한 번에 써집니다. index를 row와 lane으로 나누는 규칙은 가장 단순한 것을 씁니다 — **index의 하위 5 bits가 lane 번호, 상위 9 bits가 row 번호**입니다(14 qubits):

$$i = \underbrace{\texttt{row}[8{:}0]}_{\text{row } 0\ldots511} \;\|\; \underbrace{\texttt{lane}[4{:}0]}_{\text{lane } 0\ldots31}$$

이렇게 잘라 두면 array를 훑는 일이 row 번호 counter 하나로 끝납니다. row r 를 읽으면 index $32r \ldots 32r+31$ 이 한꺼번에 나오고, 두 lane이 서로 다른 address를 동시에 원하는 일이 원리적으로 일어나지 않으니 **충돌이 0**입니다.

row 하나는 memory data word 하나입니다. amplitude 32칸 × 23-bit = **736 bits**짜리 data word죠. BRAM 한 개는 이렇게 넓은 data word를 담지 못하므로 여러 개를 옆으로 붙입니다. 7 시리즈의 RAMB36은 한 port를 최대 72-bit 폭으로 쓸 수 있고, 23-bit lane 셋이 69 bits라 거기에 딱 들어갑니다. 그래서 32 lanes는 RAMB36 **11개**(3 lanes짜리 10개 + 2 lanes짜리 1개)에 담기고, 이것이 amplitude state 한 벌의 값입니다.

여기에 한 겹이 더 붙습니다. final configuration은 Grover iteration 한 번 안에서 **4 PEs**(PE 하나가 32 lanes)가 협력해 **한 cycle에 4 rows**를 처리합니다. 4 rows를 동시에 읽으려면 row들을 **row phase**로 나눠 네 개의 bank에 흩어 둡니다 — row 번호의 하위 2 bits가 bank를, 나머지 7 bits가 bank 안 address를 정합니다. cycle c 에는 네 bank가 모두 같은 address c 를 읽어 row $4c, 4c+1, 4c+2, 4c+3$ 이 한꺼번에 나옵니다. 512 rows가 128 cycles에 끝나고, 여기서도 충돌은 원리적으로 0입니다.

다른 simulator에서는 "충돌 0"을 만들기 위해 추가 비용이 듭니다. 범용 gate simulator는 임의의 qubit에 임의의 gate를 걸어야 하므로 amplitude를 bank 사이로 **재배치**해야 하고, 그러려면 crossbar나 permutation network가 필요합니다. Belfore(2024)의 확장형 FPGA architecture는 그 **Beneš**

permutation network 하나에 전체 logic의 36% 를 씁니다. 우리는 그 36%를 통째로 쓰지 않습니다. Grover의 두 연산이 둘 다 index 순서를 건드리지 않는 **순차 streaming**이기 때문입니다 — oracle은 index i 의 amplitude를 index i 의 data로 판정하고, diffusion operator는 모든 칸에 같은 값을 적용합니다. algorithm의 규칙성 덕분에 이 project에서는 적은 비용으로 logic를 절약합니다.

row 구조는 resource 계산도 바꿉니다. 12.8절의 표는 **용량**만 센 것이었는데, 실제로 잡히는 BRAM 개수는 data word를 몇 개로 쪼갰느냐가 함께 정합니다. state 한 벌이 RAMB36 11개인 것은 폭(736 bits)이 정한 값이고, 깊이 512 rows는 RAMB36 한 개의 깊이(72-bit 폭에서 512)와 딱 맞습니다. 그래서 **13 qubits나 12 qubits로 낮춰도 개수가 줄지 않습니다** — 512 rows가 256 rows·128 rows로 줄 뿐 폭은 그대로라, BRAM을 절반만 채운 채 11개를 그대로 사용하기 때문입니다. resource 계산을 지배하는 것이 number of states가 아니라 **폭(number of lanes × data word width)** 인 범위인 셈이고, resource까지 아끼며 축소하려면 number of qubits가 아니라 **number of lanes를 함께 낮춰야** 합니다.

마지막으로 04번 논문과의 차이를 살펴보겠습니다. 11.5에서 본 04번의 전략은 연산 unit을 **number of states에 비례해 펼치는 것**이었습니다. 64 states(6 qubits) 이하에서만 성립하는 architecture입니다. 14 qubits에 그대로 적용하면 adder·comparator를 16,384벌 펼치라는 뜻이고, LUT 19.8× 곡선을 그대로 연장하면 어떤 FPGA로도 불가능합니다. 우리는 정반대를 택했습니다 — unit을 number of states가 아니라 **constant 128개**(32 lanes × 4 PEs)만 두고 array를 $\mathrm{number\ of\ states}/128$ cycles에 걸쳐 흘려 보냅니다. 그러면 logic resource는 number of qubits와 **무관하게 일정**하고, 늘어나는 것은 시간뿐입니다. 04번이 시간을 resource로 바꿨다면 우리는 resource를 시간으로 되바꾼 것이고, 그 시간이 얼마인지가 다음 절입니다.

11.8 cycle 비용 model

병렬도를 정하고 나면 이 IP의 시간은 전부 **pass** 단위로 셀 수 있습니다. pass 하나는 array를 처음부터 끝까지 한 번 훑는 것입니다. 1 PE면 한 cycle에 1 row라 512 cycles, final configuration(4 PEs)이면 한 cycle에 4 rows라 128 cycles입니다.

실제 cycle은 이 이상값에 pipeline 채우기·mean 확정·measurement 같은 고정비가 붙습니다. 그래서 여기서는 이론값 대신, 500 workloads RTL campaign(Six-Stage Ablation, 16.3절)의 총 cycle을 "number of iterations × iteration당 비용 + number of shots × shot당 비용 + constant"라는 직선에 맞춘 값을 적습니다. coefficient of determination R^2 가 모든 configuration에서 0.99 이상이라 이 model이 실제 cycle을 거의 그대로 설명합니다.

단위	Stage 1 (1 PE, measurement 최적화 없음)	final configuration	근거
pass 1회 (array 한 번 훑기)	512	128	512 rows ÷ cycle당 rows
Grover iteration 1회 = 2 passes + 고정비	**≈ 1,033**	**≈ 268**	13.6 · campaign 직선 맞춤
shot 1회의 나머지 (Init·measurement·solution check·policy)	≈ 1,327	≈ 354	14.5 · campaign 직선 맞춤

iteration 하나가 왜 하필 두 pass인지는 13.6절의 주제입니다. 짧게만 적으면, diffusion operator가 요구하는 mean은 **모든 amplitude를 다 더해야** 나오므로 "전부 읽어 더하는 pass"와 "그 값으로 되쓰는 pass"가 순서상 겹칠 수 없습니다. oracle은 이 model에서 **별도의 항이 아닙니다** — Pass 1이 amplitude를 읽어 되쓰는 김에 predicate 판정 결과대로 sign을 뒤집으므로 cycle을 더 쓰지 않습니다.

여기까지가 **한 shot** 안쪽의 비용입니다. 그런데 Grover는 몇 번 돌릴지를 미리 알 수 없고(number of solutions를 모르기 때문입니다), 그래서 실제 실행 시간에는 **number of shots가 곱해집니다.** 이 multiplication이 16장의 주제입니다. 결과만 미리 적어 두면, 14 qubits · 1 solution인 100 workloads에서 mean **22.7 shots**(median 23)가 들고, requested iterations가 mean 173.5 iterations입니다. 매 shot 처음부터 도는 Restart mode는 그 173.5 iterations를 전부 물리적으로 돌려 mean 209,277 cycles(100 MHz에서 약 2.1 ms)이고, final configuration은 checkpoint 덕에 executed iterations를 mean 51.9 iterations로 줄이고 4 PEs와 measurement 최적화까지 더해 mean 25,345 cycles(약 0.25 ms)입니다. 이 값은 **RTL cycle 축**입니다 — board에서 잰 wall-clock time은 다른 축이라 섞어 speedup을 만들면 안 됩니다(18.8절). 그리고 이 숫자가 "data를 고전적으로 한 번 훑는 비용"(128 rows를 읽는 128 cycles 남짓)보다 훨씬 크다는 사실도 함께 봐야 합니다(11.1). 우리의 목표는 속도 우위를 내세우는 데 있지 않습니다.

11.9 emulator가 실기보다 쉬운 것들

emulator는 진짜 quantum computer의 흉내이지만, 어떤 면에서는 **진짜보다 훨씬 편합니다.** 실제 quantum computer에서 값비싼 여러 연산이, classical array 계산에서는 공짜에 가깝기 때문입니다. 이 역설은 응용 논문(08·09)을 읽을 때 특히 중요하니 미리 정리해 둡니다.

- **phase 반전.** 실기에서 특정 computational basis state의 phase만 뒤집으려면 multi-controlled Z gate가 필요하고, 이는 qubit가 늘수록 비싸집니다(08번 논문이 다루는 일반화된 phase 반전은 별도의 보조 circuit block까지 요구합니다). emulator에서는 sign을 뒤집는 two's complement negate 한 줄이 전부입니다(13.4절). 다만 "그래서 1 cycle에 끝난다"고 쓰면 틀립니다 — 우리 oracle은 solution index를 모르므로 **모든 칸을 훑으며 predicate를 판정**해야 하고, 128 lanes로도 16,384 / 128 = 128 cycles입니다. 정확한 서술은 "gate 비용이 사라지고 scan 비용만 남는다"이고, 그 scan이 diffusion operator의 accumulation pass와 같은 pass라 **oracle이 따로 더 쓰는 cycle은 없습니다** (11.8).
- **되돌리기(uncomputation).** 실기에서는 ancilla qubit를 쓴 뒤 entanglement를 풀려고 circuit을 **역순으로 다시 실행**해야 합니다(09번 논문의 oracle은 $A_f \to N_f \to U_\delta$ 를 걸고 다시 역순으로 풀어 circuit depth가 두 배가 됩니다). emulator에서는 그냥 array를 **덮어쓰면** 끝이라 되돌리기 자체가 불필요합니다.
- **ancilla qubit와 phase kickback.** 7장에서 본 phase kickback은 $|-\rangle$ state의 ancilla qubit가 있어야 성립합니다. 실기에서는 이 ancilla qubit를 실제로 마련하고 관리해야 하지만, emulator에서는 "sign을 뒤집는다"는 **결과만 array에 직접 쓰면** 되므로 ancilla qubit가 필요 없습니다.
- **state initialization.** 07번 논문은 논문의 **절반가량을 "classical data를 state로 싣는(Classical-to-Quantum, C2Q)" 문제**에 씁니다. 임의의 data를 amplitude로 encoding하는 circuit이 algorithm 본체보다 깊어질 만큼 어려운 문제이기 때문입니다. 그런데 Grover의 초기 state는 **uniform superposition** — 모든 칸을 같은 값으로 채우는 `memset` 한 번입니다. Grover는 이 최대 난제를 **구조적으로 우회**합니다.
- **중간 state 관찰.** 실기에서 계산 도중 state를 들여다보면 measurement가 되어 superposition이 collapse합니다(4장). emulator에서는 array를 언제든 덤프해 볼 수 있어 debugging이 자유롭고, 이것이 11.1에서 말한 "verification 가능성"의 실체입니다.

이 마지막 항목에는 **일부러 사용하지 않는 간단한 방법**이 있습니다. oracle 직후의 amplitude array에서 sign이 음수인 칸이 곧 solution 집합이므로, 그 array를 한 번 훑기만 하면 iteration도 measurement도 없이 solution을 **전부** 얻을 수 있습니다. 이것은 Grover가 아니라 classical scan이고, 그 사실이 바로 11.10절이 말하는 왜곡의 극단입니다. 그래서 이 경로는 **동작 경로에 넣지 않습니다.** 실기라면 measurement 전까지 amplitude를 볼 수 없고 measurement하면 collapse하므로, 우리도 그 제약을 스스로 지켜야 재현한 것이 algorithm이 됩니다 — 결과가 밖으로 나가는 길이 measurement 한 곳뿐이고, 중간 amplitude는 verification용 덤프로만 꺼내는 이유가 이것입니다.

11.10 emulator가 숨기는 것

편한 만큼 위험도 있습니다. emulator는 실기의 **gate 비용을 왜곡**합니다. 방금 본 것처럼 multi-controlled Z gate가 emulator에서는 XOR 한 줄로 보이니, 실기라면 엄청나게 비쌀 circuit이 값싸 보입니다. 그래서 **emulator로 "algorithm의 논리가 맞다"는 것은 검증할 수 있어도, "이 circuit이 실기에서 실현 가능하다"는 것은 검증하지 못합니다.** emulator가 재현하는 것은 algorithm의 수학이지 물리적 gate 예산이 아닙니다.

그리고 이 역설에는 **반대 방향**도 있습니다. 실기에서 값싼 trick이 emulator에서는 오히려 손해가 되는 경우입니다. 08번 논문(Byrnes 외, PRL 2018)은 diffusion operator 연산을 여러 solution state로 **일반화**한 $G = I - 2P_S$ 를 제안합니다. 실제 quantum computer에서는 이 일반화가 gate를 아껴 주는 좋은 아이디어입니다. 하지만 emulator에서 이 일반화된 projector P_S 를 계산하려면 모든 state 각각에 대해 D dimension inner product를 해야 해서 $O(\text{number of states} \cdot D)$ 짜리 multiply-accumulate와 $\text{number of states} \cdot D$ data words의 추가 저장이 듭니다 — **순수한 손해**입니다.

그래서 08번 논문의 일반화 diffusion operator는 우리에게 **implementation 대상이 아니라, "왜 우리는 표준 diffusion operator 하나($a \mapsto 2\bar{a} - a$)를 고수하는가"의 근거**입니다. 표준 diffusion operator는 multiplication 없이 accumulation·shift·subtraction만으로 끝나거든요(13.5절·13.8절).

한 문장으로: 실제 quantum computer에서 싼 것이 emulator에서 비싸고, 그 반대도 참입니다. 이 감각을 갖고 논문을 읽어야, 어떤 아이디어를 우리 설계에 가져오고 어떤 것을 버릴지 판단할 수 있습니다.

11.11 이 장의 요약

- **emulator**는 state vector를 숫자 array로 펼쳐 classical hardware로 계산하는 기계입니다. quantum 이득을 누리지 못하며, 총비용 $O\left(\text{number of states} \cdot \sqrt{\text{number of states}/\text{number of solutions}}\right)$ 은 classical linear scan보다 **느립니다.** 비교 대상은 Qiskit `AerSimulator`·NumPy state vector simulator와 같은 board의 순수 software baseline이고, 값어치는 속도가 아니라 **결정적·재현 가능한 verification**에 있습니다.

- 우리 IP는 **RVX SoC의 APB slave(CSR) + AHB master(DMA)** 이고, host는 UART 너머에서 명령만 내립니다(17장).
- array는 셋입니다 — `amp_mem`(23b, 쓰기 대상, 실제로는 checkpoint slot 여러 벌) · `data_mem`(16b signed, 읽기 전용) · `found_mask`(1b, enumeration용). oracle은 `amp[i] ← −amp[i] if pred(value[i], thr)` 이고 **solution index를 모릅니다.**
- qubit 하나 = memory 두 배. 14 qubits에서 Accelerator Core가 **84 tile**, board 전체가 **116 / 135 tile**(2026-09-07 implementation report)이고, 이 구조에서 15 qubits는 예산 밖입니다.
- "register라서 resource가 터진다"는 **틀렸습니다.** FF는 1.20×, LUT는 19.8× — 원인은 저장이 아니라 **병렬 연산 unit**입니다. 저장 위치 선택은 용량 문제가 아니라 **병렬성 예산** 문제입니다.
- 우리의 **memory-bound는 bandwidth가 아닙니다.** (a) BRAM 용량 천장, (b) BRAM의 number of ports와 port width = 병렬도의 상한 — 두 얼굴로 나타납니다.
- **row = 32 lanes**, index 하위 5 bits가 lane, 상위 9 bits가 row. row 하나(736 bits)가 RAMB36 11개에 담기고, 4 PEs가 row phase 4 banks에서 한 cycle에 4 rows를 읽습니다. 순차 streaming이라 **충돌 0**이고 범용 simulator가 logic 36%를 쓰는 **Beneš permutation network를 통째로 회피**합니다.
- **비용 model**(500 workloads campaign 직선 맞춤): iteration 1회 ≈ 1,033 cycles(1 PE) → **≈ 268 cycles**(4 PEs), shot 1회의 나머지 ≈ 1,327 → **≈ 354 cycles**(final configuration). 여기에 number of shots가 곱해집니다 — 14 qubits·1 solution이면 mean 22.7 shots, final configuration에서 mean 약 **0.25 ms**(RTL cycle 축, 16장).
- emulator는 실기의 여러 난제(multi-controlled Z, uncompute, ancilla, C2Q initialization)를 **공짜로** 우회하지만, 실기의 gate 비용도 **숨깁니다.** 그리고 반대로, 08의 일반화 diffusion operator처럼 실기에서 좋은 trick이 emulator에서는 손해가 되기도 합니다.

다음 장에서는 array의 각 칸 — amplitude 하나 — 를 **실제로 몇 bit의 어떤 숫자로 담을 것인가**를 파고듭니다. fixed-point와 Q1.22, 즉 이 datapath의 가장 실무적인 첫 결정입니다.

12장. amplitude를 숫자로 — fixed-point와 Q1.22

> **이 장을 읽기 위한 준비**: 1장 amplitude·normalization, 4장 Born rule, 11장 state vector emulator.

11장에서 state vector를 array `amp[0..16383]` 로 펼쳤습니다. 그런데 array의 한 칸에 amplitude α_i 를 담으려면 **결국 몇 bit짜리 어떤 숫자 형식으로 담을지**를 정해야 합니다. 이것은 FPGA emulator를 설계할 때 처음 정해야 할 사항입니다. bit를 너무 적게 잡으면 error가 쌓여 solution을 찾기 어려워지고, 너무 많이 잡으면 memory와 logic를 낭비합니다(11장에서 봤듯 amplitude bit 폭은 곧 memory 폭이니까요).

결론부터 적어 둡니다. 우리 amplitude 한 칸은 **23-bit fixed-point, Q1.22**입니다. rounding은 round-half-to-even, 넘치면 symmetric saturation, norm renormalization은 하지 않습니다. 계산 도중의 중간값만 더 넓은 register에서 다루고, memory에 되쓸 때 23 bits의 범위로 제한합니다. 이 장은 그 결정 하나하나가 **무엇을 재서** 나왔는지를 따라갑니다(값의 source of truth는 15.4절 설계 결정표).

길잡이가 되는 것은 두 편의 중앙대 연구실 논문입니다. 04번(Choi & Lee, 2024)은 "fixed-point로 하자, Q1.30으로"라는 **선택**을 내렸고, 05번(Choi 외, QIP 2026)은 "그럼 fraction bits는 정확히 몇 bit여야 하는가"라는 **이론**을 세웠습니다. 우리는 둘 중 어느 쪽 숫자도 그대로 쓰지 않는데, 왜 그런지가 이 장의 절반입니다.

12.1 왜 floating-point를 안 쓰는가

프로그래밍에서 fractional number를 다룰 때 우리는 보통 `float`·`double` 같은 **floating-point**를 씁니다. radix point 위치가 "떠다니며" 아주 큰 수와 아주 작은 수를 모두 표현하죠. 편리하지만, hardware로 만들면 **비쌉니다.** floating-point는 숫자를 $\text{mantissa} \times 2^{\text{exponent}}$ 로 쪼개 저장하기 때문에, addition 한 번에도 exponent를 맞추고(alignment) 결과를 다시 normalization하고 rounding하는 부수 작업이 딸려 옵니다.

04번 논문은 이 비용을 정확히 셉니다. floating-point **addition 한 번**은 내부적으로 real addition 4회 + 비교 4회 + shift 3회를 요구하고, **multiplication 한 번**은 real addition 4 + real multiplication 1 + 비교 4 + shift 2 + XOR 1이 필요합니다. 반면 **fixed-point**는 같은 addition이 그냥 integer addition 1회, power of two를 곱하는 것은 shift 1회로 끝납니다. emulator는 diffusion operator 한 번에 모든 amplitude를 더하고 빼므로 (13.5절), 이 constant 배의 차이가 number of states에 곱해져 resource·전력·latency로 고스란히 나타납니다.

우리에게는 이유가 하나 더 있습니다. 13.8절에서 보겠지만 이 IP의 diffusion operator·oracle·initialization 경로에는 **multiplier가 하나도 없습니다** — 전부 addition·subtraction·shift뿐입니다. floating-point를 택하는 순간 그 결론은 성립하지 않습니다. alignment와 normalization이 곧 가변 shift와 priority encoder를 뜻하고, 그것이 128 lanes마다 하나씩 붙기 때문입니다.

12.2 fixed-point와 Q format 명명 규약

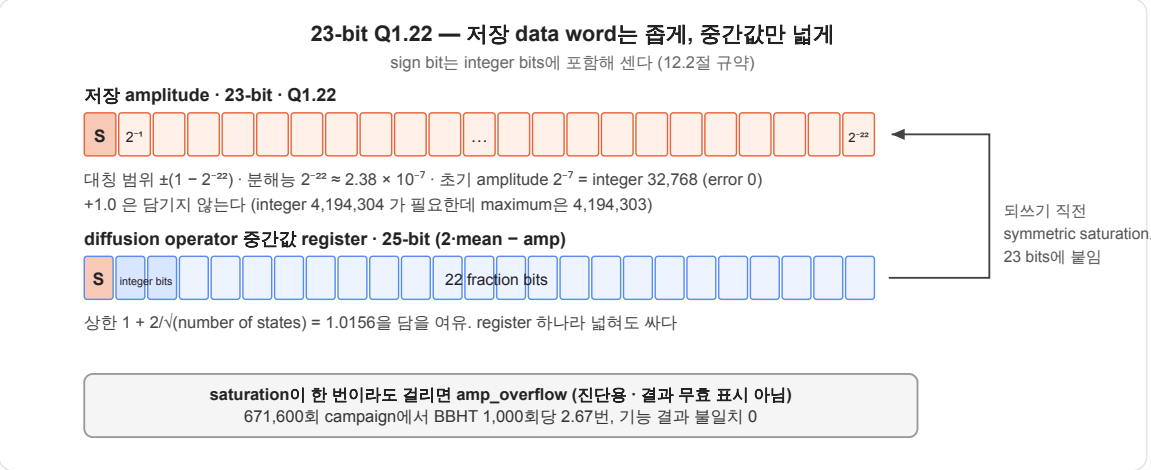
fixed-point는 이름 그대로 radix point 위치를 **한 자리에 고정해** 두는 방식입니다. radix point가 떠다니지 않으므로 alignment·normalization이 필요 없고, 숫자를 그냥 **integer처럼** 다루되 "이 integer의 아래 몇 bit는 fraction bits"라고 약속만 해 두면 됩니다.

일상의 비유를 들면, 돈을 다룰 때 우리는 12,345원을 굳이 1.2345×10^4 로 쓰지 않습니다. 그냥 "원" 단위 integer로 적고 radix point는 생각하지 않죠. fixed-point도 똑같습니다. 예를 들어 아래 22 bits를 fraction bits로 약속하면, memory에 저장된 integer $2{,}097{,}152$ 는 실제로는 $2^{21}/2^{22} = 0.5$ 를 뜻합니다. radix point는 "약속" 속에만 있고 hardware는 integer 연산만 합니다.

이 약속을 **Q format**으로 표기합니다. `Q<integer bits>.<fraction bits>` 꼴입니다. 그런데 여기에 문헌마다 갈리는 함정이 있습니다 — **sign bit를 integer bits에 넣어 세는 곳과 따로 빼는 곳이 있어서**, 똑같은 32-bit data word가 어디서는 Q1.30으로, 어디서는 Q2.30으로 적힙니다.

이 해설서에서는 다음 규약으로 통일합니다. **sign bit는 integer bits에 포함합니다.** 따라서 Q format의 data word width는 언제나 integer bits + fraction bits이고, 표현 범위는 $\left[-2^{\text{integer bits}-1},\ 2^{\text{integer bits}-1} - 2^{-\text{fraction bits}}\right]$, resolution은 $2^{-\text{fraction bits}}$ 입니다.

이 규약으로 우리 설계를 적으면 이렇게 됩니다. amplitude data word width는 23 bits, fraction bits는 22 bits, 그래서 **Q1.22** — sign 1 + fraction 22이고, integer bits에 sign 말고는 아무것도 없습니다. 23-bit two's complement의 표현 범위는 $[-1,\ 1 - 2^{-22}]$ 이고 resolution은 $2^{-22} \approx 2.38 \times 10^{-7}$ 입니다. 여기서 한 가지를 눈여겨보십시오 — $+1.0$ **은 담기지 않습니다.** integer로 $2^{22} = 4{,}194{,}304$ 가 필요한데 23-bit two's complement의 maximum은 $4{,}194{,}303$ 입니다. 우리는 여기에 한 칸을 더 버려, 음수 쪽 끝 $-4{,}194{,}304$ 도 쓰지 않고 $[-4{,}194{,}303,\ +4{,}194{,}303]$ **의 대칭 범위**만 합법 값으로 둡니다. 왜 그런지는 12.5절과 13.4절에서 드러납니다.



04번 논문의 표기도 이 규약으로 옮겨 읽어야 합니다. 논문은 32-bit data word를 "Q1.30"이라 부르는데, 이는 sign을 밖으로 뺀 표기입니다. 우리 규약으로는 **Q2.30**(sign 1 + integer 1 + fraction 30 = 32 bits)입니다. 앞으로 Q format 표기를 만나면 **"integer bits + fraction bits = data word width"가 성립하는지** 확인하십시오. 성립하지 않으면 sign을 따로 센 표기입니다. 이 사소해 보이는 구분이 memory 예산 계산에서 bit를 한 칸씩 어긋나게 만듭니다.

마지막으로 이 규약이 곧바로 쓰이는 constant 하나. uniform amplitude는 $1/\sqrt{\text{number of states}}$ 인데, 14 qubits면 $1/\sqrt{16384} = 1/128 = 2^{-7}$ 로 **power of two**입니다. Q1.22 integer로는 $2^{22}/2^{7} = \mathbf{32{,}768}$ 이고, rounding이 전혀 없는 **정확한** 값입니다. initialization이란 array 16,384칸을 전부 32,768로 채우는 일입니다(13.2절).

12.3 integer bits는 몇 bit인가 — 재는 대상이 셋이다

integer bits를 몇 bit로 잡을지는 **표현할 수 있어야 하는 maximum**이 정합니다. 여기서 혼동하기 쉬운 점이 있습니다. datapath가 만들어 내는 값이 **한 종류가 아니라 셋**이고, 셋의 maximum이 서로 다르다는 것입니다.

(a) array에 저장되는 amplitude. 4장 Born rule에서 measurement probability는 $|\alpha_i|^2$ 이고 전체 probability의 sum이 1이므로, 어떤 amplitude도 $|\alpha_i| \le 1$ 입니다. 여기까지만 보면 integer bits는 ± 1 근방만 담으면 되니 **sign bit 하나로 충분**해 보입니다. 04번이 integer bits를 1-bit로 잡고 남는 30 bits를 전부 fraction bits로 몰아준 것이 이 논리입니다.

(b) diffusion operator 도중에 만들어지는 중간값 $2\bar{a} - \alpha_i$. 그런데 amplitude는 memory에 저장된 뒤에도 매 iteration inversion about the mean을 거치며 바뀝니다. 그 결과값의 상한을 재 보면 이렇습니다. Cauchy–Schwarz inequality로 mean의 크기는

$$|\bar{a}| = \frac{1}{\text{number of states}}\left|\sum_i \alpha_i\right| \le \frac{1}{\text{number of states}}\sqrt{\text{number of states}}\sqrt{\sum_i \alpha_i^2} = \frac{1}{\sqrt{\text{number of states}}}$$

이고 $|\alpha_i| \le 1$ 이므로

$$|2\bar{a} - \alpha_i| \le 1 + \frac{2}{\sqrt{\text{number of states}}}$$

이 값은 1을 넘을 수 있습니다. 넘는 폭이 크지는 않습니다 — 14 qubits면 $2/\sqrt{16384} = 2/128 = 0.0156$ 이니 상한이 1.0156입니다. 하지만 Q1.22의 천장은 $1 - 2^{-22}$ 이라, 이 값을 23-bit data word에 그대로 담을 수는 없습니다.

(c) 전체 sum을 담는 accumulator $\sum_i \alpha_i$. mean을 구하려면 먼저 모든 amplitude를 더해야 합니다. 최악의 경우 자릿수가 $\log_2(\text{number of states}) = \text{number of qubits}$ bits만큼 늘어납니다.

integer bits 폭을 하나만 정해서 셋에 다 쓰겠다는 발상이 틀린 것입니다. **재는 대상이 다르면 필요한 폭도 다릅니다.** 그래서 우리 datapath는 셋을 따로 잡습니다.

값	폭	이유
저장 amplitude (a)	**23 bits** (Q1.22)	수학적으로 $\lvert\alpha\rvert \le 1$. memory에 들어가는 유일한 값
$2\bar{a}$ 와 diffusion operator 중간값 (b)	**25 bits**	상한 1.0156 을 넉넉히 담는 register. 되쓸 때 23 bits로 symmetric saturation
sum accumulator (c)	**39 bits**	data word width + number of qubits + 2 = 23 + 14 + 2

(b)를 어떻게 처리하느냐가 이 절의 핵심 결정입니다. 길은 둘입니다. 저장 data word 자체에 integer bit를 하나 더 두는 길(Q2.22, 24 bits)과, 저장 data word는 Q1.22 로 두고 **중간값만 넓은 register에서 계산한 뒤 되쓸 때 saturation하는** 길입니다. 우리 RTL은 뒤를 택했습니다. 근거는 셋입니다.

첫째, **되쓰는 값은 수학적으로 1을 넘지 않습니다.** $2\bar{a} - \alpha_i$ 의 상한 $1 + 2/\sqrt{\text{number of states}}$ 은 Cauchy–Schwarz inequality가 주는 느슨한 한계일 뿐이고, diffusion operator를 마친 값은 정확한 산술에서 여전히 unit vector의 성분입니다 — Grover iteration은 unitary이기 때문입니다(9장). 1을 넘는 양은 rounding이 만든 몇 LSB 뿐이고, 그것도 solution amplitude가 1에 거의 닿는 iteration에서만 생깁니다. 둘째, 중간값은 **register 하나**라 두 bit 넓히는 비용이 미미하지만, 저장 data word를 넓히면 그 비용이 amplitude array 전체와 checkpoint slot 전부에 곱해집니다. 셋째, 그리고 결정적으로 **benchmark가 뒷받침합니다.** 같은 산술(14 qubits · Q1.22 · 같은 rounding과 saturation 규칙)로 돌린 2026-08 reference model campaign 671,600 runs에서 saturation이 걸린 경우는 이렇습니다(실험 02 amplitude precision).

search	run	saturation event	1,000 runs당
number of iterations 수동 지정 (최적 근처를 일부러 포함)	41,280	2,520	61.05
BBHT Restart mode	137,600	368	2.67
BBHT Checkpoint mode	137,600	368	2.67

saturation이 일어나도 **기능 결과가 달라진 경우는 0**이었고, Restart mode와 Checkpoint mode에서 saturation event가 똑같이 368번 나왔다는 것은 두 경로가 같은 state를 지난다는 추가 증거입니다. saturation은 조용히 넘어가지 않고 `amp_overflow` 라는 sticky bit로 남습니다(12.5절). 진단용이지 결과 무효 표시가 아닙니다.

다만 이 선택으로 BRAM을 아낀 것은 아닙니다. 24-bit lane 셋은 72 bits로 RAMB36 한 port 폭에 정확히 들어가서, Q2.22로 잡았어도 state 한 벌이 RAMB36 11개로 같았을 것입니다(12.8절). 그러니 23 bits의 근거는 memory가 아니라 위의 셋 — 수학적 상한, register와 memory의 비용 차이, 그리고 benchmark입니다. (이 해설서의 옛 revision은 integer bits 2 bits를 결론으로 적었지만, 최종 RTL은 이 절의 방식을 택했습니다.)

(c)의 accumulator는 amplitude와 같은 data word에 담을 수 없으니 따로 잡습니다. Cauchy–Schwarz 상한만 보면 7 bits(number of qubits / 2)면 되지만 최악값인 14 bits(number of qubits)로 잡았는데, accumulator는 memory가 아니라 register라서 몇 bit 더 잡는 비용이 미미하기 때문입니다. 나머지 2 bits는 sign extension과 뒷단 연산을 위한 여유입니다. Measurement Unit에는 또 다른 accumulator(amplitude square의 sum)가 있는데, 그 폭은 measurement circuit의 구조와 함께 14장에서 정합니다.

12.4 rounding — 왜 half-to-even인가

integer bits가 넘침을 막는 역할이라면, **fraction bits는 precision**을 정합니다. fraction bits가 유한하므로 어딘가에서는 아래 bit를 버려야 합니다. 어디서 버려질까요? 05번 논문이 이를 정확히 셋으로 짚습니다.

1. **initialization** — uniform amplitude $1/\sqrt{\text{number of states}}$ 를 유한 bit로 표현할 때. loop 시작 전 **한 번**. 다만 우리 14 qubits에서는 $1/\sqrt{\text{number of states}} = 2^{-7}$ 이라 **버리는 bit가 없습니다**(12.2절).
2. **diffusion operator의 right shift** — diffusion operator가 실제로 필요로 하는 값은 mean이 아니라 **2×mean**이라, accumulator를 number of qubits − 1 bits, 곧 13 bits만큼 arithmetic right shift해(`sum >>> 13`) 한 번에 구합니다(13.5절). 이 shift에서 아래 13 bits가 떨어져 나갑니다. **매 iteration**.
3. **measurement의 square** — probability를 만들 때 amplitude를 제곱합니다. 05번은 여기서도 product의 아래 bit를 버리지만 우리는 자르지 않습니다. 23-bit끼리의 product는 46 bits이고 그 폭 그대로 누적하므로, 이 자리에서 precision을 좌우하는 것은 truncation이 아니라 **accumulator 폭**입니다. 폭 결정은 14.5절에 있습니다. loop가 끝난 뒤 **한 번**.

addition과 subtraction은 fraction bits를 깎지 않고(자릿수만 늘 뿐 아래 bit를 버리지 않습니다), 2×는 shift 칸수를 14 대신 13으로 잡는 것으로 흡수되어 별도의 연산 자체가 없습니다. 그래서 diffusion operator 전체(accumulation → shift → subtraction)에서 error를 만드는 곳은 **가운데 shift 한 곳** 뿐입니다. 되쓸 때의 saturation은 그보다 훨씬 드문 두 번째 자리입니다(12.3·12.5절).

그런데 그 한 곳이 결정적입니다. mean은 그 iteration의 **모든 amplitude에 2×로 실려** 들어갑니다 — mean의 rounding error ϵ 은 곧바로 모든 칸에 2ϵ 으로 퍼집니다. 그리고 이 일이 매 iteration 되풀이됩니다. 14 qubits · 1 solution이면 최적은 100 iterations입니다.

따라서 rounding 방식도 설계 결정의 하나입니다. 후보는 셋입니다.

- **truncation** — arithmetic right shift 결과를 그대로 쓰는 것. two's complement에서 truncation은 언제나 $-\infty$ 방향이므로 평균적으로 -0.5 LSB의 **한 방향 편향**이 남습니다.
- **half-up** — 정확히 절반인 지점을 늘 위로 올림. 대부분의 값에서는 편향이 사라지지만, 절반인 경우에만 $+$ 방향으로 치우친 것이 남습니다.
- **round-half-to-even** — 절반일 때는 결과의 최하위 bit가 0이 되는 쪽으로 붙임. 절반이 위·아래로 반반 갈리므로 편향이 정확히 0입니다.

편향이 있느냐 없느냐가 error의 **성장 속도**를 바꿉니다. 방향이 같은 error는 서로 상쇄되지 않고 number of iterations에 **비례**해 쌓입니다. 반면 방향이 무작위인 error는 서로 상쇄되며 $\sqrt{\text{number of iterations}}$ 규모로만 자랍니다. 100 iterations를 도는 loop에서 이 차이는 열 배입니다. 그래서 **truncation은 쓰지 않습니다.**

남은 둘 중 half-to-even을 고른 이유는 error 크기가 아닙니다. diffusion operator의 shift는 13 bits라 버려지는 bit가 정확히 절반이 되는 경우 자체가 2^{-13} probability로만 나오고, half-up과 갈리는 자리도 딱 거기뿐이기 때문입니다. 고른 이유는 규칙이 대칭이라 편향의 상한이 원리적으로 0이라는 것, 그리고 수식으로 적었을 때 모호함이 없어 software reference model과 RTL이 tie에서 갈릴 여지가 없다는 것입니다(18.3절).

hardware 비용은 사실상 0입니다. 버릴 하위 bit 중 최상위 하나를 g(guard), 그 아래 전부의 OR를 r(sticky), 남는 값의 최하위 bit를 ℓ 이라 하면 올림 조건은

$$g \wedge (r \vee \ell)$$

입니다. gate 몇 개짜리 combinational logic이고, diffusion operator 한 번에 한 번만 씁니다.

두 가지를 미리 밝혀 둡니다. 첫째, 다음 두 절에서 인용할 05번 논문의 error 이론은 **truncation을 전제로** 세워졌습니다. 규칙을 바꾸는 순간 그 이론의 exponent와 constant를 그대로 가져다 쓸 수 없게 되는데, 이것이 12.7절의 주제입니다. 둘째, 규칙을 못 박는 것 자체가 verification의 전제조건입니다. reference model과 RTL이 서로 다른 rounding을 쓰면 두 결과는 영원히 bit 단위로 일치하지 않습니다(18.3절).

12.5 overflow policy — saturation·wraparound 금지·renormalization 없음

12.3절에서 중간값만 넓게 잡고 되쓸 때 23 bits로 붙이기로 했습니다. 그러면 "붙인다"가 정확히 무엇인지 정해야 합니다.

wraparound는 금지입니다. two's complement 산술에서 넘침을 방치하면 maximum 바로 위가 minimum이 됩니다 — $+0.99\ldots$ 에 조금 더하면 -1 이 나옵니다. amplitude가 **sign까지 뒤집혀 정반대 값**이 되는 것입니다. Grover에서 sign은 장식이 아닙니다. oracle이 하는 일 전부가 sign 반전이고, diffusion operator는 sign이 뒤집힌 칸을 mean 위로 끌어올립니다(9장). 한 칸의 sign이 잘못 뒤집히면 그 칸은 다음 iteration부터 solution처럼 처리되고 오염된 값이 mean에 섞여 **모든 칸으로 번집니다.** 조용히 틀린 답을 내는 circuit이 되는 것이고, 이런 고장은 simulation에서도 찾아내기 어렵습니다.

그래서 symmetric saturation을 씁니다. 25-bit 중간값이 $+4{,}194{,}303$ 을 넘으면 $+4{,}194{,}303$ 에, $-4{,}194{,}303$ 을 밑돌면 $-4{,}194{,}303$ 에 붙입니다. error는 남지만 sign과 크기 순서는 보존되고, 무엇보다 error가 그 칸에 **국소적으로** 머뭅니다. 비용은 comparator 하나와 mux 하나입니다. 아래쪽 끝을 $-4{,}194{,}304$ 가 아니라 $-4{,}194{,}303$ 으로 잡는 이유는 13.4절에서 보듯 **모든 합법 값의 sign 반전이 다시 합법 값**이 되게 하려는 것입니다.

saturation이 한 번이라도 일어나면 `amp_overflow` bit가 서고, 다음 search가 시작될 때까지 내려가지 않습니다. saturation은 값을 상한에 붙일 뿐 아무 signal도 내지 않으므로, 이 bit가 없으면 saturation이 실제로 발동했는지 관측할 방법이 없습니다. 이 bit는 **진단용이고 결과 무효 표시가 아닙니다** — 12.3절의 benchmark대로 saturation이 기능 결과를 바꾼 경우는 없었고, 뽑힌 후보는 어차피 solution check를 통과해야 채택됩니다(16.4절).

norm renormalization은 하지 않습니다. 매 iteration 끝에 $\sum_i \alpha_i^2$ 를 다시 1로 맞추는 것은 수치해석에서 흔한 처방이지만, 우리는 세 가지 이유로 하지 않습니다. 첫째, Grover iteration은 이론상 unitary라 norm이 보존되고 실제로 어긋나는 양은 rounding error 규모뿐입니다. 둘째, norm을 1로 되돌리려면 $1/\|a\|$ 를 곱해야 하는데 inverse square root와 multiplication이 들어옵니다 — 12.1절에서 확보한 "diffusion operator·oracle 경로에 0 multipliers"가 그 자리에서 무너집니다. 셋째, 애초에 필요가 없습니다. measurement가 Born-rule sampling이고, sampler는 amplitude square를 **전체 sum으로 나눈 상대 비율**만 쓰므로(14.4절) norm이 정확히 1이 아니어도 probability distribution은 제대로 정의됩니다.

12.6 error는 어떻게 자라나

05번 논문에서 제시한 분석 결과입니다. truncation error가 Grover iteration을 거치며 최종 measurement probability distribution을 얼마나 왜곡하는지를, 두 state의 amplitude만 추적하는 방법으로 닫힌 형태로 유도합니다.

그 방법의 출발점은 **two-amplitude theorem**입니다 — Grover의 어느 iteration 시점에서도 state vector에는 딱 **두 종류의 값**만 존재한다는 것입니다: solution amplitude α_{solution} 하나와, 나머지 non-solution amplitude $\alpha_{\text{non-solution}}$ 하나. 9장에서 4 states를 손으로 돌릴 때 non-solution 세 칸이 항상 같은 값이었던 것을 기억하시나요? 그게 우연이 아니라 **number of states와 무관하게 성립하는 성질**입니다. 덕분에 $2^{\text{number of qubits}}$ dimension 문제가 2 scalars 문제로 줄어 분석이 가능해집니다.

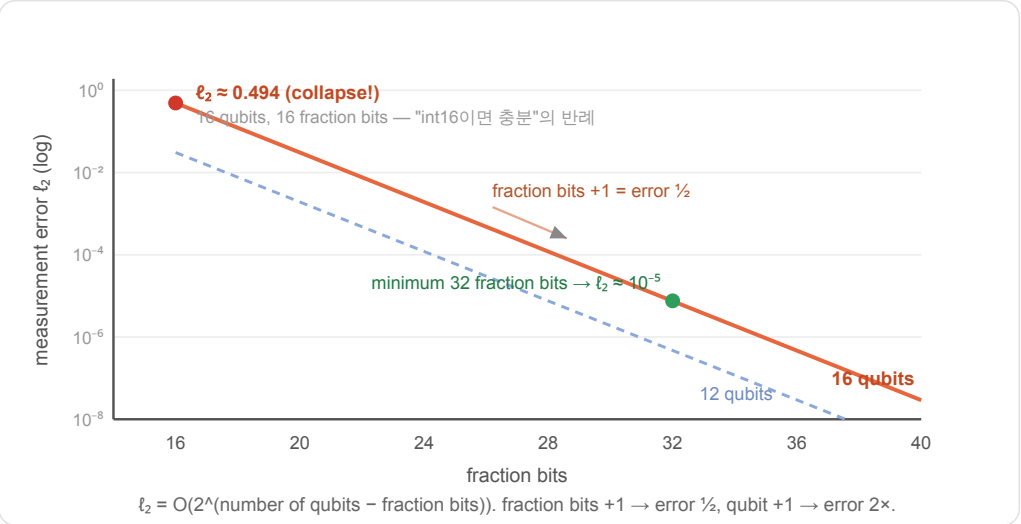
이 theorem은 이 해설서에서 두 번 쓰입니다. 여기서는 error 분석을 가능하게 하고, 14.3절에서는 **solution이 여럿일 때 그 amplitude들이 매 iteration 정확히 같다**는 사실을 뒷받침해 measurement 방식 자체를 결정하는 근거가 됩니다. 9.4절 손계산에서 숫자로 확인할 수 있습니다.

분석의 결론을 한 줄로 쓰면:

$$\ell_2 \text{ error} = O\left(2^{\text{number of qubits} - \text{fraction bits}}\right)$$

measurement probability distribution의 ℓ_2 거리(참값과 얼마나 벌어졌는가)가 $2^{\text{number of qubits} - \text{fraction bits}}$ 규모로 자란다는 뜻입니다. 이 exponent는 다음 관계를 뜻합니다:

- **fraction bits를 1 bit 늘리면** → exponent가 1 줄어 **error가 절반**.
- **qubit를 1개 늘리면** → exponent가 1 늘어 **error가 두 배**.



왜 하필 solution amplitude에 이렇게 빠르게 error가 쌓일까요? solution amplitude는 작게 시작해 iteration마다 커지는데, 그 커지는 과정이 **inversion about the mean**($a \mapsto 2\bar{a} - a$)라서 매 iteration non-solution들의 truncation error까지 함께 끌어와 증폭하기 때문입니다. number of iterations가 $\frac{\pi}{4}\sqrt{\text{number of states}} \approx O(2^{\text{number of qubits}/2})$ 이고 error가 대략 그 square로 누적되므로 $O(2^{\text{number of qubits}})$, 여기에 한 번의 truncation 크기 $2^{-\text{fraction bits}}$ 가 곱해져 $O(2^{\text{number of qubits} - \text{fraction bits}})$ 가 됩니다.

이 공식이 실무에 주는 경고가 하나 있습니다. 05번 논문의 실험에서 16 qubits, 16 fraction bits를 넣으면 ℓ_2 error가 무려 **0.494** 로 치솟습니다. measurement probability의 총합이 1인데 error가 0.494라면, 결과는 사실상 **무의미**합니다. "amplitude를 그냥 16 bits에 담으면 충분하겠지"라는 순진한 가정이 정확히 어디서 무너지는지 보여주는 반례입니다 — qubit가 늘면 fraction bits도 그만큼 따라 키워야 한다는 것이죠.

그리고 여기서 반드시 붙여 두어야 할 단서가 있습니다. 위 유도에서 error가 number of iterations의 square로 누적되는 것은 **매 iteration 같은 방향으로 밀리는 error**의 그림입니다. 12.4절에서 본 그대로, 그것은 **truncation**의 성질입니다. 05번의 이론 전체가 truncation을 전제로 합니다.

12.7 몇 bit면 되나 — minimum fraction bits 공식과 그 전제

이제 필요한 폭을 정하는 문제를 봅시다. "목표 정확도를 맞추려면 fraction bits를 최소 몇 bit로 잡아야 하는가?" 05번 논문은 위 error 공식을 뒤집어 닫힌 형태의 답을 냅니다. 허용 가능한 최대 error를 $\ell_{2,\max}$ 라 하면:

$$\boxed{\text{minimum fraction bits} = \lceil \text{number of qubits} - \log_2 \ell_{2,\max} - 1.03 \rceil}$$

qubit가 많을수록 bit가 더 필요하고, 더 엄격한 정확도를 원할수록 bit가 더 필요합니다. constant 1.03 은 error 계수를 실험으로 보정한 값입니다. 예를 들어 6 qubits에서 error를 10^{-5} 이하로 잡고 싶다면 $\log_2 10^{-5} \approx -16.6$ 이므로 minimum fraction bits는 $\lceil 6 + 16.6 - 1.03 \rceil = 22$, 곧 22 bits입니다. 04번의 30 fraction bits는 같은 조건에서 **8 bits가 과잉**인 셈입니다.

이 공식을 우리 설계에 대입해 봅시다. 14 qubits, 22 fraction bits이므로

$$\ell_2 \approx 2^{\text{number of qubits} - \text{fraction bits} - 1.03} = 2^{14-22-1.03} = 2^{-9.03} \approx 1.9 \times 10^{-3}$$

입니다. 흔히 쓰는 기준 10^{-3} 을 조금 넘는 값이라, 공식만 믿으면 아슬아슬해 보입니다. 그러나 이 공식에는 **세 가지 전제**가 있고, 우리 설계는 셋 다 다릅니다.

1. **rounding이 truncation입니다.** error가 매 iteration 같은 방향으로 쌓이는 것이 square 누적의 근거인데, 우리는 round-half-to-even으로 그 편향을 없앴습니다(12.4절).
2. **data word 구성이 다릅니다.** 05번은 integer bits를 number of qubits만큼 넉넉히 잡은 **하나의 넓은 data word**를 전제합니다. 우리는 저장 data word를 23 bits로 좁히고 중간값과 accumulator만 따로 넓혔습니다(12.3절).
3. **판정 지표가 다릅니다.** 05번의 ℓ_2 는 measurement probability distribution **전체**가 참값에서 얼마나 벌어졌는가입니다. 우리 합격 기준은 그것이 아닙니다 — shot 하나가 후보 index를 하나 내면, 그 index는 **solution check**를 거쳐 predicate를 실제로 만족할 때만 채택됩니다(16.4절). distribution이 조금 흐려지면 shot이 몇 번 더 필요할 뿐, 틀린 답이 출력되지는 않습니다. precision error가 **정확도가 아니라 속도로** 나타나는 구조입니다.

세 전제가 모두 다르니 공식의 수를 우리 설계의 예측으로 읽어서는 안 됩니다. 그래서 우리는 fraction bits를 공식으로 정하지 않고 **직접 쟀습니다.** 두 가지 benchmark가 있습니다.

첫째, float64 기준과 같은 mask·같은 iterations로 fraction bits 폭만 바꿔 돌린 precision 분석입니다(1 · 4 · 64 solutions와 state 절반이 solution인 경우). probability distribution ℓ_2 가 10^{-3} 이하가 되는 것은 fraction bits 20 bits부터, 10^{-5} 이하가 되는 것은 **22 bits**부터였습니다. 같은 분석에서 15 qubits · 16 qubits는 fraction bits 24 bits까지 늘려도 10^{-5} 에 닿지 못했습니다 — 14 qubits로 굳힌 근거 중 하나가 이것입니다(실험 02 amplitude precision). 둘째, 12.3절의 671,600-run campaign입니다. Q1.22로 돈 모든 run에서 amplitude ℓ_2 · probability ℓ_2 · success probability error · norm error의 maximum이 전부 10^{-3} 아래였습니다(같은 실험 02).

즉 22 fraction bits는 공식이 준 답이 아니라 **잰 답**입니다. 공식으로 경향을 파악하되 전제가 다르면 benchmark로 확인합니다. 18장 verification ladder도 이 원칙을 따릅니다.

12.8 data word와 BRAM 환산

amplitude 말고 폭을 정해야 할 것이 하나 더 있습니다. search 대상이 되는 data array `value[i]` 입니다. 확정값은 **16-bit signed** 입니다.

여기에는 precision 이론이 필요 없습니다. `value` 는 계산되는 값이 아니라 host가 실어 주는 **입력 data**이고, oracle이 하는 일은 `value - thr` 의 최상위 bit 하나를 보는 것뿐이라(13.3절) truncation error가 누적될 자리가 아예 없습니다. 16 bits는 $-32{,}768 \sim 32{,}767$ 을 담아 sensor 값·pixel·index 같은 흔한 data를 그대로 받기에 자연스럽고, 32-bit bus data word 하나에 두 칸이 딱 들어가 load도 간단합니다(17.5절).

왜 폭이 개수를 정하는가. 7 시리즈 FPGA의 BRAM은 RAMB18(18Kb)·RAMB36(36Kb) 단위로 잡히고, RAMB36은 한 port를 최대 **72-bit 폭 × 512 깊이**로 쓸 수 있습니다. 23-bit amplitude 셋이 69 bits라 그 폭에 들어가고, 우리 row 하나(32 lanes × 23-bit = 736 bits)는 3 lanes짜리 10개 + 2 lanes짜리 1개, 곧 **RAMB36 11개**에 담깁니다. 깊이 512도 우리 512 rows와 딱 맞습니다. 그래서 state 한 벌이 11개입니다.

14 qubits(16,384 states)에서 array들의 예산입니다.

array	data word width	용량	실제 BRAM (implementation report)
amplitude state 한 벌	23b × 32 lanes = 736b × 512 rows	368 Kb	RAMB36 11
amplitude · checkpoint memory (checkpoint slot 4벌)		1,472 Kb	RAMB36 44
`data_mem` (data)	16b	256 Kb	RAMB18 64
`found_mask` (enumeration mask)	1b	16 Kb	RAMB36 2
Accelerator Core 계 (measurement·policy memory 포함)			**84 tile**

board는 Arty A7-100T(`xc7a100tcsg324-1`)이고 BRAM tile이 **135개** 있습니다. RVX SoC가 32 tile을 써서 board 전체가 116 / 135입니다. `data_mem` 이 용량(256 Kb)에 비해 BRAM을 많이 먹는 것은, oracle이 한 cycle에 4 rows 128칸을 읽어야 해서 폭이 개수를 정하기 때문입니다.

만약 04번을 따라 amplitude를 **32 bits**로 잡았다면 어땠을까요. 32-bit lane은 72-bit 폭에 둘밖에 안 들어가서 state 한 벌이 RAMB36 16개, checkpoint slot 4벌이 64개가 됩니다. 지금보다 20개가 더 들어 board 전체가 136 / 135 — **넘칩니다.** 12.7절에서 본 대로 30 fraction bits는 이론적으로도 과잉이었으니, 우리가 04번의 data word width를 그대로 베끼지 않은 데에는 이론과 예산 양쪽의 이유가 있는 셈입니다.

표를 읽을 때 한 가지 주의할 점이 있습니다. 용량 열은 bits만 센 것이고, 실제 BRAM 개수는 data word를 몇 개로 쪼개 동시에 읽느냐가 함께 정합니다. 그래서 qubit를 줄여도 개수가 그대로일 수 있습니다. 그 계산은 11.7절에 있습니다.

12.9 반론 — "fixed-point는 부정확하다"

다른 선택을 한 연구도 살펴보겠습니다. 07번 논문(El-Araby 외)은 fixed-point를 **명시적으로 "low accuracy(낮은 정확도)"라고 비판**하며, 기존 FPGA quantum emulator들이 쓴 16 bits·24-bit fixed-point를 정확도 부족의 사례로 지목합니다. 그리고 자신은 **32-bit floating-point complex number**를 씁니다. 04·05번과 정면으로 부딪히는 입장이죠. 이 차이를 검토하면 우리가 fixed-point를 택한 근거를 더 분명히 설명할 수 있습니다. 해소 논리는 셋입니다.

첫째, Grover amplitude는 dynamic range가 좁습니다. floating-point의 진짜 강점은 아주 큰 수와 아주 작은 수를 동시에 다루는 **넓은 exponent 범위**입니다. 그런데 우리 datapath에 등장하는 값은 저장 amplitude가 $[-1, 1]$, 중간값이 $[-1.016, 1.016]$ 안입니다(12.3절). float32의 exponent 8 bits는 $10^{\pm 38}$ 을 담는데, 우리가 실제로 쓰는 범위는 한 자릿수도 되지 않습니다. 필요 없는 범위에 exponent 처리 circuit을 얹는 것은 낭비입니다.

둘째, 요구 precision은 오히려 올라갔지만 그것이 floating-point의 근거가 되지는 않습니다. measurement가 **Born-rule sampling**이기 때문에 (14.4절) sampler는 amplitude square의 **상대적인 probability mass**을 그대로 출력 distribution으로 삼습니다. 순위뿐 아니라 크기의 **비율**까지 맞아야 하고, solution보다 훨씬 작은 non-solution amplitude도 수천 개가 모이면 probability mass의 상당 부분을 차지하므로 작은 값도 제대로 담겨야 합니다. 요구가 올라간 것은 분명합니다. 다만 올라간 것은 **resolution**(fraction bits)이지 **exponent 범위**가 아닙니다. fixed-point는 fraction bits를 늘려 resolution을 높이면 되지만 float32는 24-bit mantissa를 얻으려고 8 bits를 exponent에 떼어 줍니다 — 같은 data word width에서 우리에게는 버려지는 8 bits입니다. 07번이 절대 정확도에 민감했던 이유도 짚어 둘 만합니다. 그들의 대상은 **quantum Haar transform(QHT)을 통한 image 재구성**이었고, 재구성된 pixel 값 자체가 결과물이라 절대 정확도가 필수였습니다. 우리 출력은 index 하나이고, 그 index는 solution check를 통과해야 채택됩니다 (16.4절).

셋째, 근거의 무게가 반대입니다. 07번은 fixed-point를 "부정확하다"고 비판하면서도, 정작 **bit 폭에 따른 error 곡선을 하나도 측정하지 않았습니다.** 반면 05번은 minimum fraction bits 공식과 그것을 뒷받침하는 verification table을 냅니다. 한쪽은 정성적 비판, 다른 쪽은 정량적 이론입니다. 07번의 비판은 "**아무렇게나** fixed-point를 쓰면 부정확하다"로 읽는 것이 옳고, 05번은 그 "아무렇게나"를 숫자로 바꾸는 처방입니다. 우리는 여기서 한 걸음 더 갔습니다 — 우리 조건이 05번의 전제마저 벗어나므로(12.7절), 공식조차 그대로 믿지 않고 직접 쟀고, 그 결과가 모든 run에서 10^{-3} 아래였습니다.

12.10 우리 project의 결정

지금까지의 논증을 설계 언어로 정리하면 이렇습니다. 값의 source of truth는 15.4절 설계 결정표입니다.

항목	값	근거
수 체계	real number 전용 fixed-point	12.1 · 9.8
amplitude data word	23 bits, **Q1.22** (sign 1 + fraction 22)	12.2 · 12.3
범위 / resolution	대칭 $[-(1-2^{-22}),\ 1-2^{-22}]$ / $2^{-22} \approx 2.38 \times 10^{-7}$	12.2 · 12.5
`INIT_AMP` (14 qubits)	**32768** $= 2^{22}/128$, error 0	12.2
2×mean · diffusion operator 중간값	25-bit register	12.3
sum accumulator	39 bits (data word width + number of qubits + 2)	12.3
rounding	**round-half-to-even**, 13-bit right shift 한 곳	12.4
overflow	**symmetric saturation** + `amp_overflow` 진단 bit. wraparound 금지. norm renormalization 없음	12.5
fraction bits 충분성	공식이 아니라 benchmark — 모든 run에서 error $< 10^{-3}$	12.7
data word	16-bit signed	12.8
amplitude memory	state 한 벌 RAMB36 11개, checkpoint slot 4벌 44개 (Accelerator Core 84 / board 116 tile)	12.8

이 장에 미결로 남긴 항목은 없습니다. 초기 revision에서 "아직 하지 않은 benchmark"로 남겨 두었던 fraction bits 충분성은 이제 precision 분석과 671,600-run campaign으로 확인했습니다.

12.11 이 장의 요약

- **floating-point는 비쌉니다.** addition 하나에 alignment·normalization·rounding이 딸려 옵니다(04번: addition 4 + 비교 4 + shift 3). fixed-point는 integer addition 1회이고, 이것이 "0 multipliers diffusion operator"의 전제입니다.
- **명명 규약: sign은 integer bits에 포함.** Q format의 data word width = integer bits + fraction bits. 우리는 **Q1.22**(23 bits), 04번의 "Q1.30"은 우리 규약으로 **Q2.30**(32 bits). Q1.22는 $+1.0$ 을 담지 못하고, 우리는 -1.0 도 버려 **대칭 범위**만 씁니다.
- 14 qubits에서 $1/\sqrt{\text{number of states}} = 2^{-7}$ 이라 **초기 amplitude 32,768이 정확**합니다. initialization에서는 error가 생기지 않습니다.
- **폭은 재는 대상이 셋**입니다: 저장 amplitude(23 bits) · diffusion operator 중간값(상한 $1 + 2/\sqrt{\text{number of states}} = 1.0156$, 25-bit register) · sum accumulator(39 bits). 저장 data word를 넓히는 대신 중간값만 넓히고 되쓸 때 **symmetric saturation**합니다. 되쓰는 값은 수학적으로 1을 넘지 않고, 671,600-run campaign에서 saturation은 드물었고(BBHT 1,000 runs당 2.67) 기능 결과를 바꾸지 않았습니다.
- **rounding은 round-half-to-even.** loop 안에서 매 iteration 새로 생기는 truncation은 diffusion operator의 13-bit right shift 하나뿐인데, 그 error가 mean을 통해 전 칸에 퍼지고 1 solution이면 100 iterations 동안 되풀이됩니다. 한 방향 편향은 number of iterations에 비례해 쌓이고, 무편향은 $\sqrt{\text{number of iterations}}$ 로만 쌓이므로 truncation은 쓰지 않습니다.
- **overflow는 symmetric saturation**과 `amp_overflow` 진단 bit. wraparound는 sign을 뒤집어 non-solution을 solution으로 만들고 mean을 통해 전체로 번집니다. **renormalization은 하지 않습니다** — Born sampler가 상대 비율만 쓰므로 필요가 없고, 하려면 multiplier가 들어옵니다.
- **error 성장**: 05번의 $\ell_2 = O(2^{\text{number of qubits}-\text{fraction bits}})$, fraction bits 1 bit = error 절반, qubit 1개 = error 두 배. 단 이 이론은 **truncation 전제**입니다.
- **minimum fraction bits 공식의 전제 셋**(truncation · 넓은 단일 data word · distribution 전체의 ℓ_2)이 모두 우리와 다릅니다. 그래서 fraction bits를 공식으로 정하지 않고 **직접 쟀습니다** — 14 qubits에서 22 fraction bits면 probability $\ell_2 \le 10^{-5}$, campaign의 모든 run에서 error $< 10^{-3}$.
- **data word 16-bit signed**, amplitude 23 bits. 23-bit 셋이 RAMB36 한 port(72 bits)에 들어가 state 한 벌이 11개, checkpoint slot 4벌이 44개. amplitude를 32 bits로 잡았다면 64개로 board를 넘겼을 것입니다.
- **07번의 float32 비판**에 대한 답 셋: (1) dynamic range가 좁아 exponent field가 낭비, (2) Born-rule sampling으로 precision 요구가 **올라갔지만** 필요한 것은 resolution이지 exponent 범위가 아님, (3) 07번은 error 실험이 없고 우리는 쟀음.

다음 장에서는 이 숫자 형식 위에서 oracle·diffusion operator가 **실제 circuit**이 되는 과정을 봅니다. 12장에서 "error의 주범"이라 부른 그 right shift가 13.5절에서 gate로 구현되는 모습을 보고 12.3절의 25-bit 중간값과 symmetric saturation이 왜 그 자리에 필요했는지도 그때 눈으로 확인하게 됩니다.

13장. oracle과 diffusion operator — multiplier 없는 산술

> **이 장을 읽기 위한 준비**: 9.4절 4 states 손계산, 9.7절 두 개의 반사, 11.7절 병렬도와 banking, 11.8절 cycle 비용 model, 12장 fixed-point와 Q1.22.

11장에서 state vector를 세 개의 array로 펼쳤고, 12장에서 그 array의 한 칸을 Q1.22 fixed-point로 정했습니다. 이 장은 그 array 위에서 **Grover iteration 한 바퀴**가 실제로 어떤 circuit이 되는지를 봅니다.

범위를 먼저 좁혀 두겠습니다. 이 장이 다루는 것은 **한 iteration 안쪽**뿐입니다. 몇 번 돌릴지 정하는 shot loop, 뽑힌 후보를 확인하는 solution check, amplitude array를 재활용할지 정하는 checkpoint는 전부 16장의 소관입니다. amplitude array에서 답 하나를 뽑는 measurement는 14장이고요. 여기서는 "목표 number of iterations를 받아 그만큼 돌고 빠져나오는" 안쪽 FSM만 봅니다.

이 장에서 확인할 성질은 다음과 같습니다 — **initialization·oracle·diffusion operator 경로에는 multiplier가 한 개도 없습니다.** 9장에서 배운 inversion about the mean이 어떤 연산으로 구현되는지 살펴보겠습니다. 동시에 multiplier가 정말로 필요한 곳이 어디인지도 13.8절에서 정확히 짚습니다. "0 multipliers"라고만 쓰면 어느 경로를 가리키는지 알 수 없어 부정확하기 때문입니다.

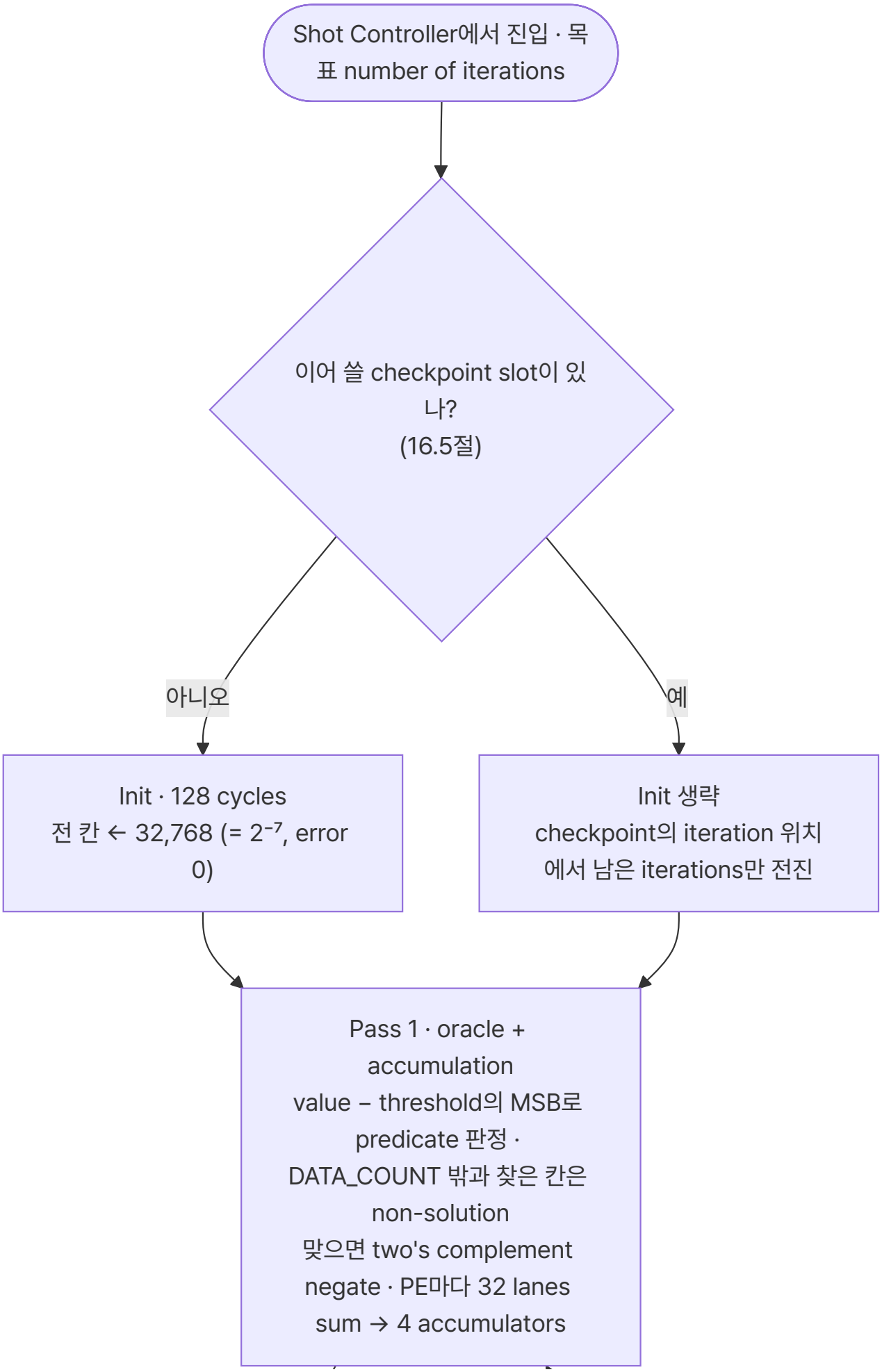
13.1 한 iteration 안의 FSM

안쪽 FSM은 네 종류의 state뿐입니다. initialization 한 번, 그다음 **Pass 1과 Pass 2를 iteration마다 한 번씩 왕복**하고 끝입니다.

state	cycle (4 PEs)	하는 일
`S_INIT`	128	amplitude array의 모든 칸을 `INIT_AMP` 로 (13.2)
`S_PASS1`	128 + pipeline	predicate 판정 → sign 반전 → 되쓰기. **같은 흐름에서** 전체 accumulation (13.3·13.4)
`S_MEAN`	1	accumulator를 13칸 right shift하고 rounding해 $2\bar{a}$ 확정 (13.5)
`S_PASS2`	128 + pipeline	각 칸을 $2\bar{a} - a_i$ 로 갱신하고 saturation (13.5)

final configuration은 4 PEs(PE 하나가 32 lanes)가 한 cycle에 4 rows(128칸)를 처리하므로 array 한 바퀴가 $\mathrm{number\ of\ states}/128 = 128$ cycles입니다(11.7절). 여기에 pipeline을 채우고 비우는 몇 cycle이 더해져 500 workloads campaign에서 잰 **Grover iteration 1회는 약 268 cycles**, 100 MHz에서 약 2.7 μs입니다(11.8절). Pass 1에는 100 MHz timing을 맞추려고 adder tree 중간에 pipeline register를 넣은 탓에, 마지막 row를 되쓴 뒤 partial sum이 accumulator에 도착하기를 기다리는 **drain cycle**이 하나 있습니다. 결과 산술은 바뀌지 않고 latency만 늘어납니다.

oracle이 독립된 pass를 갖지 않는다는 점이 중요합니다 — sign 반전은 amplitude를 읽어 되쓰는 김에 끼워 넣을 수 있으므로, oracle은 **cycle을 한 개도 쓰지 않습니다.**


Shot Controller에서 진입 · 목표 number of iterations
이어 쓸 checkpoint slot이 있나?
(16.5절)
아니오
예
Init · 128 cycles
전 칸 ← 32,768 (= 2⁻⁷, error 0)
Init 생략
checkpoint의 iteration 위치에서 남은 iterations만 전진
Pass 1 · oracle + accumulation
value − threshold의 MSB로 predicate 판정 ·
DATA_COUNT 밖과 찾은 칸은 non-solution
맞으면 two's complement negate · PE마다 32 lanes
sum → 4 accumulators

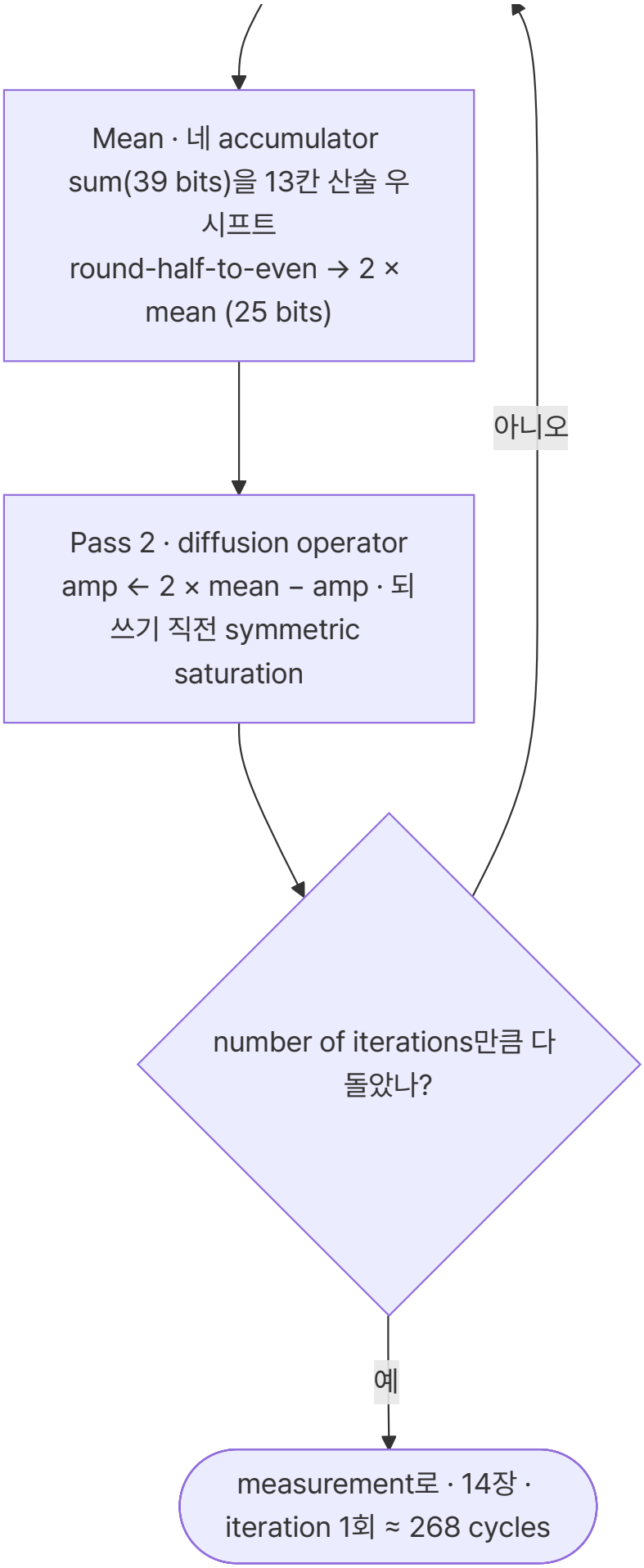


그림 맨 위의 "Init을 할지 말지"만 이 장 바깥의 결정입니다. checkpoint slot 중 하나에 이어 쓸 수 있는 amplitude array가 있으면 `S_INIT` 을 건너뛰

고 곧바로 Pass 1로 들어가는데, 그 판정 규칙과 정당화는 16.5절에 있습니다. iteration이 끝난 뒤 amplitude array를 **지우지 않고 그대로 두는 것**도 같은 이유입니다.

13.2 Init — uniform amplitude를 square-root unit 없이

initialization은 $H^{\otimes \text{number of qubits}}|0\rangle$ 을 만드는 일, 즉 array의 모든 칸을 $1/\sqrt{\text{number of states}}$ 로 채우는 일입니다. 문제는 $\sqrt{}$ 가 hardware로 비싸다는 것입니다. 그런데 $\text{number of states} = 2^{\text{number of qubits}}$ 라는 사실을 쓰면 square-root unit이 통째로 사라집니다:

$$\frac{1}{\sqrt{\text{number of states}}} = \frac{1}{\sqrt{2^{\text{number of qubits}}}} = \begin{cases} 1 \times \dfrac{1}{2^{\text{number of qubits}/2}}, & \text{number of qubits even} \\ \dfrac{1}{\sqrt{2}} \times \dfrac{1}{2^{(\text{number of qubits}-1)/2}}, & \text{number of qubits odd} \end{cases}$$

number of qubits가 짝수면 $1/\sqrt{\text{number of states}}$ 가 딱 power of two라 constant 1.0 을 그만큼 right shift하면 되고, 홀수면 $1/\sqrt{2}$ 라는 **constant 하나**만 있으면 나머지는 역시 shift입니다. 04번 논문(Choi & Lee, 2024)이 Q1.30에서 쓰는 code가 이 꼴입니다:

```
init_coefficient = (NUM_QUBIT % 2 == 0)
                 ? (0x4000_0000 >> (NUM_QUBIT / 2))         // 0x4000_0000 = 1.0
                 : (0x2D41_3CCE >> ((NUM_QUBIT - 1) / 2));  // 0x2D41_3CCE = 1/√2
```

우리는 14 qubits, 즉 **짝수 분기**입니다. Q1.22에서 1.0 에 해당하는 integer는 2^{22} 이고, 이것을 7칸(14 / 2) right shift하면 $2^{15} = 32{,}768$ 이 나옵니다. 그래서 `INIT_AMP` **= 32768** 이고, $32768/2^{22} = 1/128 = 1/\sqrt{16384}$ 가 **정확히** 성립합니다. rounding이 한 번도 일어나지 않으므로 초기 state의 norm도 정확히 1입니다. measurement 쪽 숫자로 확인하면, amplitude square의 총합이 $16384 \times 32768^2 = 2^{44}$ 이고 이것이 Q1.22 square의 1.0 에 해당하는 값 그대로입니다(14.5절). constant가 필요 없으니 RTL은 이 값을 `grover_param.vh` 의 integer 하나로 정의합니다.

number of qubits가 홀수였다면 사정이 다릅니다. $1/\sqrt{2}$ 를 22 fraction bits로 rounding하는 순간 error가 생기고, 그 error가 모든 칸에 똑같이 실려 초기 norm이 1을 살짝 벗어납니다. 짝수일 때는 이 오차가 생기지 않습니다.

다만 원문의 값을 그대로 옮겨 쓰면 안 됩니다. 논문에 인쇄된 constant `0x2D41_3CCE` 는 실제 $\frac{1}{\sqrt{2}} \times 2^{30}$ = `0x2D41_3CCD` 와 **1 LSB 어긋나** 있습니다. 같은 수법으로 만든 number of iterations constant `0x1_1C58_31AC` ($\frac{\pi\sqrt{2}}{4} \times 2^{32}$) 도 실제 값 `0x1_1C58_31AE` 와 **2 LSB 어긋나** 있고요 (`0x4000_0000` 과 `0xC90F_DAA2` = $\frac{\pi}{4} \times 2^{32}$ 는 정확합니다). error가 $\sim 10^{-9}$ 라 실용적으로는 무해하지만, 교훈은 constant 자체가 아니라 **1차 자료도 틀린다**는 것입니다. magic number는 인용하지 말고 직접 계산해 넣으십시오. 위의 32768도 직접 계산해 유도한 값입니다. (논문 constant·수식의 오류 목록 전체는 15.6절에 모여 있습니다. number of iterations constant를 우리가 어떻게 다루는지는 16.2절입니다 — 우리는 최적 number of iterations를 고정 constant로 정의하지 않기 때문에 이 constant를 쓸 일 자체가 별로 없습니다.)

13.3 oracle — predicate 4종이 subtractor 하나를 공유한다

이 project의 oracle은 **solution이 어디 있는지 아는 circuit이 아니라, 한 칸을 보고 predicate를 만족하는지 판정하는 circuit**입니다(14.2절). 우리 IP가 판정하는 predicate는 네 가지입니다:

$$\texttt{LT}: v_i < t_a \qquad \texttt{GT}: v_i > t_a \qquad \texttt{EQ}: v_i = t_a \qquad \texttt{RANGE}: t_a < v_i < t_b$$

여기서 v_i 는 `data_mem` 의 i 번 칸이고 t_a, t_b 는 CSR로 받은 threshold입니다. data word는 16-bit signed입니다(12.8절).

조건마다 comparator IP를 하나씩 두는 방법을 먼저 생각할 수 있지만, 네 개가 모두 필요하지는 않습니다. **비교는 subtraction의 sign bit 하나**이기 때문입니다. 16-bit operand를 sign extension해 17 bits로 빼면

$$d = \underbrace{\{v_{15}, v\}}_{17\text{b}} - \underbrace{\{t_{15}, t\}}_{17\text{b}}$$

이고, d 의 최상위 bit `d[16]` 이 곧 $v < t$ 의 답입니다. sign extension을 빠뜨리면 $v = -32768, t = 1$ 같은 입력에서 subtraction이 overflow해 sign bit가 뒤집히므로, 17 bits로 넓히지 않으면 circuit이 잘못된 결과를 냅니다. 나머지 predicate는 이 subtractor를 재활용합니다.

predicate	subtractor 입력 $A - B$	`hit_i` combinational logic	추가 hardware
`LT` $v_i < t_a$	$A = v_i, B = t_a$	`d1[16]`	없음
`GT` $v_i > t_a$	$A = t_a, B = v_i$ (**operand 교환**)	`d1[16]`	입력 mux
`EQ` $v_i = t_a$	(subtractor를 쓰지 않음)	`~|(v_i ^ t_a)` (XOR-NOR)	XOR 16개 + NOR 1단
`RANGE` $t_a < v_i < t_b$	$A = t_a, B = v_i$ / $A = v_i, B = t_b$	`d1[16] & d2[16]`	subtractor 하나 더

`GT` 는 추가 비교 연산이 필요 없다는 점을 눈여겨보십시오. $v > t$ 는 $t < v$ 와 같은 말이므로, subtractor에 들어가는 두 입력을 **바꿔 넣기만** 하면 됩니다. comparator를 새로 만들 이유가 없습니다. `EQ` 는 subtractor조차 필요 없습니다 — 두 값이 같다는 것은 XOR 결과가 전부 0이라는 뜻이니 XOR 한 층과 NOR 한 층이면 끝이고, 이게 subtraction 결과가 0인지 보는 것보다 latency가 짧습니다. `RANGE` 만 비교가 둘이라 subtractor를 하나 더 둡니다. 17-bit subtractor는 LUT 수십 개 수준이라 128 lanes에 깔아도 부담이 크지 않습니다 — implementation report에서 lane 하나의 predicate circuit이 LUT 한 자릿수로 잡힙니다. 경계가 열린 interval($<$)인지 닫힌 interval($\leq$)인지는 circuit이 아니라 **약속**의 문제이고, 우리는 위 표대로 네 predicate 모두 strict inequality로 고정합니다. 경계값 test는 18.5절에서 따로 다룹니다.

판정 앞에는 조건이 하나 더 있습니다. host가 array를 다 채우지 않고 `DATA_COUNT` 개만 올렸다면, index가 `DATA_COUNT` 이상인 칸은 memory에 무엇이 남아 있든 **항상 non-solution**입니다(padding gate). 이 조건이 없으면 지난 실험에서 남은 값이 solution으로 판정될 수 있습니다.

마지막으로 **enumeration mode**가 이 판정에 한 겹을 더합니다. 이미 찾아낸 index를 다시 뽑지 않으려면 `found_mask` 의 bit를 AND로 걸면 됩니다:

$$\texttt{hit}_i = \texttt{predicate}(v_i) \ \wedge \ \overline{\texttt{found}[i]}$$

AND gate 하나가 전부이고, 이것이 왜 정당한 oracle인지는 14.7절에서 설명합니다.

solution index를 parameter로 고정하고 그 칸만 뒤집는 circuit — predicate 판정이 아예 없는 형태 — 도 값어치가 있긴 합니다. 9장의 수식을 **최소 circuit으로 확인하는 한 단계**이기 때문입니다. 판정 logic가 없으니 diffusion operator와 iteration만 남고, 그러면 9.4절의 4 states 손계산과 waveform을 한 칸씩 대조할 수 있습니다(18.4절). 다만 그것은 verification을 위한 중간 단계이며 실제 설계는 아닙니다. 우리 oracle은 처음부터 위 표의 predicate circuit입니다.

13.4 sign 반전 — two's complement negate 한 번

판정 뒤의 연산은 간단합니다. oracle U_f 가 하는 일은 predicate를 만족하는 칸의 **sign을 뒤집는 것**뿐이고, fixed-point는 two's complement이므로 sign 반전은 bit 반전 + 1입니다.

$$a_i \leftarrow (-1)^{\texttt{hit}_i} \, a_i$$

조건부 반전이라는 점을 이용하면 gate가 더 줄어듭니다. `hit` 를 XOR의 한 입력과 adder의 carry-in에 동시에 넣으면 됩니다:

```
// hit=0 이면 amp 그대로, hit=1 이면 ~amp + 1 = -amp
wire signed [DW-1:0] amp_o = (amp ^ {DW{hit}}) + hit;
```

multiplication도, ancilla qubit도, 별도의 state도 없습니다. 실기에서 multi-controlled Z gate로 만들어야 하는 phase 반전이 emulator에서는 XOR 한 줄이 되는 것 — 11.9절에서 설명한 차이를 잘 보여 주는 사례입니다.

two's complement에는 원래 **sign 반전이 불가능한 값**이 하나 있습니다. 23-bit container의 minimum $-4{,}194{,}304 (= -1.0)$를 반전하면 $+4{,}194{,}304$ 인데 이 값은 23 bits에 담기지 않습니다. wraparound로 두면 -1.0 이 -1.0 으로 되돌아와 **sign 반전이 항등이 되어 그 칸에서는 oracle이 작동하지 않습니다.** 우리 설계에서는 이 실패 mode가 구조적으로 사라집니다. 12.5절의 symmetric saturation이 amplitude를 $[-4{,}194{,}303, \ +4{,}194{,}303]$ 안에만 두기 때문에, array에 들어 있는 어떤 값을 뒤집어도 다시 합법 범위 안입니다. 아래쪽 끝을 한 칸 버린 이유가 여기 있습니다.

13.5 diffusion operator — 2·mean − amp가 circuit이 되는 순간

9장에서 diffusion operator를 "inversion about the mean" $a \mapsto 2\bar{a} - a$ 로 배웠습니다(9.4·9.7절). 이 한 줄이 circuit에서 어떻게 펼쳐지는지 봅니다. 9.7절에서 쓴 diffusion operator U_ψ 를 matrix로 펼치면 이렇게 생겼습니다($|\psi\rangle$ 은 uniform superposition):

$$U_\psi = H^{\otimes \text{number of qubits}}\left(2|0\cdots0\rangle\langle 0\cdots0| - I\right)H^{\otimes \text{number of qubits}} = 2|\psi\rangle\langle\psi| - I = \begin{bmatrix} 2^{-\text{number of qubits}+1} & \cdots & 2^{-\text{number of qubits}+1} \\ \vdots & \ddots & \vdots \\ 2^{-\text{number of qubits}+1} & \cdots & 2^{-\text{number of qubits}+1} \end{bmatrix} - I$$

"모든 원소가 $2^{-\text{number of qubits}+1}$ 인 matrix에서 identity matrix를 뺀" 꼴입니다. 언뜻 $\text{number of states} \times \text{number of states}$ matrix-vector product($O(\text{number of states}^2)$)가 필요해 보이지만, 이 특수 구조 덕분에 **세 단계**로 분해됩니다. 지금 array에 저장된 state를 $|\phi\rangle = \sum_i \beta_i |i\rangle$ 라고 두면:

$$U_\psi|\phi\rangle = 2^{-\text{number of qubits}+1}\begin{bmatrix} \sum_i \beta_i \\ \vdots \\ \sum_i \beta_i \end{bmatrix} - \begin{bmatrix} \beta_0 \\ \vdots \\ \beta_{\text{number of states}-1} \end{bmatrix}$$

즉 ① 모든 amplitude를 **더하고**(accumulation $\sum \beta_i$), ② $2^{-\text{number of qubits}+1}$ 을 **곱하고**(= $\text{number of qubits} - 1$ bits right shift), ③ 각 원소를 그 값에서 **빼면**($\text{sum} \times 2^{-\text{number of qubits}+1} - \beta_i$) 끝입니다. 이것이 정확히 inversion about the mean입니다 — 왜냐하면 $2^{-\text{number of qubits}+1}\sum \beta_i = \frac{2}{\text{number of states}}\sum \beta_i = 2\bar{\beta}$ 이고, 따라서 세 번째 단계가 $2\bar{\beta} - \beta_i$ 이니까요.

"왜 $\text{number of qubits} - 1$ 칸 shift인가" 가 여기서 풀립니다. mean은 sum을 $2^{\text{number of qubits}}$ 으로 나누는 것(= number of qubits 칸 right shift)이지만, 우리가 실제로 필요한 것은 mean이 아니라 **2×mean**($2\bar{a}$)입니다. $\frac{2}{2^{\text{number of qubits}}} = \frac{1}{2^{\text{number of qubits}-1}}$ 이므로, number of qubits 칸 shift 후 다시 2를 곱하는 대신 처음부터 $\text{number of qubits} - 1$ **칸만** shift하면 한 번에 $2\bar{a}$ 가 나옵니다. 14 qubits면 13칸입니다.

두 번에 나눠 하지 않는 것이 중요합니다. `mean = sum >>> n` 으로 밀었다가 `two_mean = mean <<< 1` 로 되미는 implementation은 결과값은 같아 보여도 **최하위 bit 하나를 버립니다**(오른쪽으로 밀 때 떨어져 나간 bit는 왼쪽으로 되밀어도 0으로 채워집니다). 한 번에 13칸을 미는 쪽이 precision이 낫고, 무엇보다 **rounding이 일어나는 지점이 정확히 한 곳으로 고정**됩니다.

그 지점을 확인해 두겠습니다. 한 번의 Grover iteration에서 fraction bits가 줄어드는 곳은 **accumulator를 13칸 right shift하는 이 한 순간뿐**입니다. oracle의 sign 반전은 정확하고, 반사의 subtraction도 두 operand가 모두 22 fraction bits라 정확합니다. 그러므로 12.4절의 round-half-to-even은 여기, 오직 여기에만 적용됩니다:

```
localparam K = NBITS - 1;                    // n=14 → 13
wire [ACCW-1:0] trunc = $signed(sum) >>> K;  // 버림
wire            half  = sum[K-1];            // 버려지는 bit 중 최상위
wire            rest  = |sum[K-2:0];         // 그 아래에 1이 하나라도 있나
wire [ACCW-1:0] two_mean = trunc + (half & (rest | trunc[0]));
```

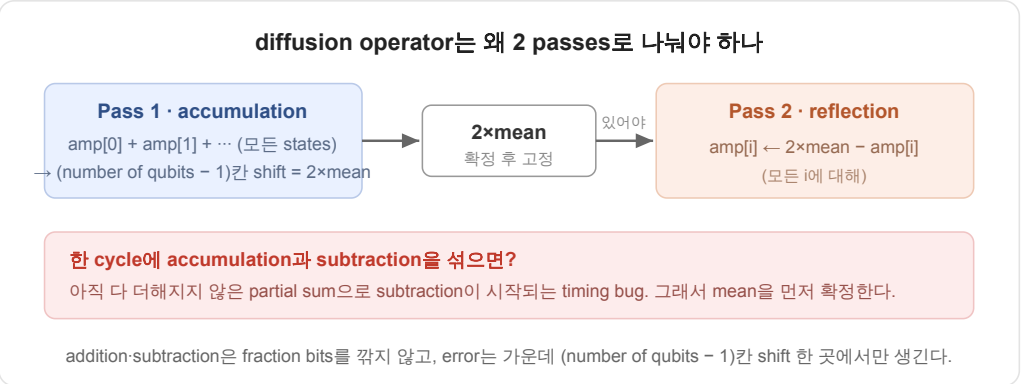
마지막 줄이 half-to-even 그 자체입니다. 버려지는 부분이 정확히 절반이면(`half=1, rest=0`) 결과의 최하위 bit가 1일 때만 올림해서 짝수로 맞춥니다. RTL은 이 결과를 25-bit `two_mean` register에 담습니다(12.3절). 매 iteration 한 번씩, 1 solution이면 100 iterations 안팎 동안 적용되는 이 규칙이 reference model과 RTL이 bit 단위로 일치하기 위한 첫 번째 조건입니다(18.3절).

subtraction 결과 $2\bar{a} - a_i$ 는 25-bit 폭에서 계산되고, 되쓰기 직전에 23-bit 대칭 범위로 saturation됩니다. saturation이 걸리면 그 lane이 signal을 내고, 한 번이라도 걸린 search는 `amp_overflow` 가 섭니다(12.5절).

그리고 이 세 단계 어디에도 **multiplier가 없습니다.** accumulation은 addition, $2^{-\text{number of qubits}+1}$ 을 곱하는 것은 shift, 반사는 subtraction — 전부 addition 계열입니다.

13.6 왜 2 passes인가

diffusion operator가 왜 굳이 두 번의 array 순회로 쪼개지는지는 한 문장으로 답할 수 있습니다. **$2\bar{a} - a_i$ 를 계산하려면 $\bar{a}$ 가 이미 확정돼 있어야 하는데, $\bar{a}$ 는 모든 a_i 를 다 더해야 나오기 때문입니다.**



accumulation과 subtraction을 한 pass에서 섞으면 아직 다 더해지지도 않은 partial sum으로 subtraction이 시작됩니다. array 앞쪽 칸은 거의 0에 가까운 partial sum으로 반사되고 뒤쪽 칸은 거의 전체 sum으로 반사되어, 결과는 diffusion operator의 계산과 달라집니다. simulation에서 잡기 어려운 종류의 버그는 아니지만(amplitude의 오차가 뚜렷합니다), 이 순서 제약이 **algorithm에서 오는 것이지 implementation 편의가 아니라는** 점이 중요합니다. diffusion operator는 전역 연산이고, 전역 연산은 global reduction을 먼저 끝내야 합니다.

그래서 Pass 1은 oracle을 건 amplitude를 memory에 되쓰면서 sum을 모으고, Pass 2는 그 되쓴 값을 다시 읽어 $2\bar{a}$ 에서 뺍니다. checkpoint가 출발점을 준 경우에는 Pass 1이 출발 checkpoint slot에서 읽어 목적 checkpoint slot에 쓰고, Pass 2가 목적 checkpoint slot을 읽고 덮어쓰는 식으로 두 pass가 checkpoint slot을 나눠 쓰기도 합니다(16.5절). 어느 쪽이든 순서 제약은 같습니다.

이 제약이 남기는 흔적은 cycle만이 아닙니다. 12장에서 폭을 따로 잡았던 것이 전부 여기서 나옵니다. 첫째, Pass 1이 만드는 sum은 모든 amplitude의 총합이라 amplitude 하나보다 훨씬 큽니다. 그래서 accumulator는 amplitude container보다 넓어야 하고, 그 폭이 data word width + number of qubits + 2 = 23 + 14 + 2 = 39 bits입니다. 둘째 — 그리고 이쪽이 놓치기 쉬운데 — **넓혀야 하는 것이 accumulator만이 아닙니다.** Pass 2가 만드는 값 $2\bar{a} - a_i$ 자체가 이론상 $1 + 2/\sqrt{\text{number of states}} = 1.0156$ 까지 커져 Q1.22의 표현 범위를 넘을 수 있습니다. 그래서 이 값은 25-bit register에서 계산하고, memory에 되쓸 때만 23-bit 대칭 범위로 붙입니다(12.3절). 되쓰는 값은 수학적으로 1을 넘지 않으므로 saturation이 걸리는 것은 rounding이 한두 LSB 넘긴 드문 경우뿐입니다.

13.7 32-lane datapath와 accumulation 순서

지금까지의 circuit은 amplitude 한 칸을 다루는 이야기였습니다. 실제로는 PE 하나가 한 cycle에 row 하나(32칸)를 처리하고, final configuration에는 **4 PEs**가 있어 128칸이 한꺼번에 흐릅니다(11.7절).

address 생성부터 봅니다. index의 **하위 5 bits가 lane, 상위 9 bits가 row**이고, row 번호의 하위 2 bits가 네 row phase bank 중 하나를 고릅니다(14 qubits):

$$i = \underbrace{\texttt{row}[8{:}2]}_{\text{address in bank}} \;\|\; \underbrace{\texttt{row}[1{:}0]}_{\text{row phase bank}} \;\|\; \underbrace{\texttt{lane}[4{:}0]}_{\text{lane}}$$

이렇게 잘라 두면 address generator가 7-bit counter 하나로 끝납니다. cycle c 에는 네 bank가 **모두 같은 address** c 를 읽어 row $4c \ldots 4c+3$ 이 한꺼번에 나오므로 충돌이 원리적으로 0이고, crossbar나 permutation network가 필요 없습니다. amplitude를 임의로 재배치해야 하는 범용 gate simulator가 permutation network 하나에 logic의 3분의 1을 쓰는 것과 대조됩니다 — Grover의 두 연산(전역 균일 반사, index별 predicate 판정)이 둘 다 **순차 streaming**으로 충분하기 때문에 우리는 그 비용을 통째로 피합니다.

lane 하나는 지금까지 만든 것을 그대로 복제한 것입니다. `data_mem` 에서 v_i, amplitude memory에서 a_i 를 같은 cycle에 읽고, subtractor가 predicate를 판정하고, XOR-carry-in 한 줄이 sign을 뒤집어 되씁니다. 여기에 Pass 1에서만 쓰이는 것이 하나 더 붙습니다 — **lane 간 adder tree**입니다. PE 하나마다 32 amplitudes를 5단 adder tree로 더하면 단마다 폭이 1-bit씩 늘어 $23 + 5 = 28$-bit partial sum이 되고, PE마다 accumulator가 하나씩 이 partial sum을 128 cycles 동안 모읍니다. mean을 확정할 때는 **네 accumulator를 더한 전체 sum**에서 13.5절의 shift를 한 번 합니다. 39 bits는 이 모든 경우를 최악값으로 담는 폭입니다.

여기에 함정이 하나 있습니다. **addition 순서를 고정하지 않으면 bit-exact가 흔들릴 수 있습니다.** integer addition은 폭이 충분하면 순서에 무관하게 같은 값을 주므로 언뜻 신경 쓸 일이 없어 보입니다 — 실제로 우리 RTL은 partial sum을 자르지 않고 최악 폭으로 담기 때문에, tree 모양이 달라도 integer 결과는 같습니다. 하지만 두 가지 경우에 순서가 결과를 바꿉니다. 첫째, 폭을 아끼려고 중간 partial sum을 좁은 register에 담아 자르거나 saturation하는 순간, 어느 쌍을 먼저 더했는지가 잘리는 bit를 바꿉니다. 둘째, float64 model과 대조할 때는 **floating-point addition은 associativity를 만족하지 않으므로** 순서가 곧 결과입니다. `sum += a[i]` 로 차례로 더한 float model과 tree로 더한 RTL은 마지막 bit가 다를 수 있습니다. 그래서 bit-exact 대조

의 상대는 float64 model이 아니라 **integer로만 계산하는 fixed-point reference model**이고, 이 항목이 18.3절의 고정 대상 목록에 들어 있는 이유입니다.

Pass 2에는 adder tree가 없고, 대신 확정된 $2\bar{a}$ 가 128 lanes에 broadcast되어 각 lane이 subtraction과 saturation 하나씩을 합니다. broadcast signal이 128곳으로 퍼지므로 fanout이 커지는데, pipeline register를 한 단 두어 나눠 주면 100 MHz에서 문제가 되지 않습니다. 실제로 이 IP는 100 MHz에서 timing을 닫았습니다(WNS +0.126 ns, 17장).

13.8 multiplier는 어디에 있고 어디에 없는가

이제 이 장에서 설명한 연산 경로를 정리합니다.

initialization·oracle·diffusion operator 경로에는 multiplier가 0개입니다. initialization은 constant 하나, oracle은 subtractor와 XOR, diffusion operator는 adder tree·arithmetic right shift·subtraction·saturation뿐입니다. 임의 실수를 곱하는 자리가 한 군데도 없으니 DSP가 잡힐 이유가 없습니다. 이것은 우연이 아니라 Grover라는 algorithm의 성질입니다 — oracle이 거는 phase가 0 아니면 π(sign $+$ 아니면 $-$)뿐이고, diffusion operator가 곱하는 계수가 $2^{-\text{number of qubits}+1}$ 이라는 power of two뿐이기 때문입니다. 같은 연구실의 QAOA emulator(06번)와 대조하면 선명해집니다. QAOA는 임의 각도 $e^{i\theta}$ 회전이 필요해서 CORDIC과 complex number multiplier를 넣어야 했고 DSP를 31~32개 고정으로 씁니다. Grover는 그 회전 hardware가 통째로 필요 없습니다.

그러나 Measurement Unit에는 multiplier가 있습니다. Born-rule sampling은 amplitude square a_i^2 의 prefix sum을 만들어야 하고, square는 shift로 바뀌지 않습니다(14.5절). 23×23 multiplication은 Artix-7의 DSP48(25×18 multiplier) 하나에 들어가지 않아 squarer 하나가 DSP **2개**를 씁니다. 32 lanes면 64개이고, final configuration은 measurement 준비 단계에서 한 cycle에 2 rows를 제곱하도록(Dual-Row Build) 32-lane squarer를 두 벌 두므로 **DSP 128개**입니다. board가 주는 240개 중 절반을 조금 넘고, SoC가 쓰는 4개를 더해 board 전체가 132 / 240입니다.

그래서 예전 revision에 있던 "합성했는데 DSP가 잡히면 잘못 짠 것"이라는 점검 기준은 **틀렸습니다.** 우리 IP를 합성하면 DSP가 128개 잡히고, 그것이 정상입니다. 올바른 점검 기준은 경로를 붙인 쪽입니다:

경로	기대 DSP	잡히면 의심할 것
Init	0	constant를 shift가 아니라 multiplication으로 썼는가
oracle (predicate·sign 반전)	0	comparator IP나 multiplication 기반 비교를 불렀는가
diffusion operator (accumulation·shift·subtraction·saturation)	0	`two_mean` 을 `sum >>> (n−1)` 대신 multiplication·division으로 썼는가
measurement (sum of squares)	**128**	128보다 많으면 squarer가 중복 instantiate됐는가
Measurement PRNG	0	multiplication이 필요한 PRNG를 골랐는가 (xorshift는 shift와 XOR뿐)

`two_mean` 을 `2 * (sum / N)` 처럼 풀어 써 두면 synthesis tool이 대개는 shift로 알아서 바꿔 주지만, 폭이 애매하거나 sign이 섞이면 multiplier·divider가 생성될 수 있습니다. shift로 쓸 수 있는 것은 **shift로 쓰는 편**이 synthesis 결과를 예측 가능하게 만듭니다 — 13.5절의 `sum >>> (n−1)` 한 줄이 그 자리입니다. 그리고 synthesis report를 볼 때는 총 DSP 사용량이 아니라 **어느 module에서 잡혔는지**를 보십시오. 우리 implementation report에서 128개는 전부 measurement sampler의 squarer 두 벌 아래에 잡혀 있습니다. 이 표는 그 확인용입니다.

13.9 이 장의 요약

- 안쪽 FSM은 `Init → (Pass 1: oracle + accumulation → Mean → Pass 2: diffusion operator) × iterations` 뿐입니다. shot loop·solution check·checkpoint는 16장, measurement는 14장입니다.
- **pass 1회 = 128 cycles**(4 PEs가 한 cycle에 4 rows), **iteration 1회 ≈ 268 cycles**(2 passes + pipeline, campaign 직선 맞춤), 100 MHz에서 약 2.7 µs. **oracle은 cycle을 쓰지 않습니다** — amplitude를 읽어 되쓰는 김에 sign을 뒤집습니다.
- **Init**: 14 qubits는 짝수라 $1/\sqrt{\text{number of states}} = 2^{-7}$. Q1.22에서 $2^{22} \gg 7 = \mathbf{32768}$, error 0, 초기 norm 정확히 1.
- **oracle**: comparator IP 없이 **sign extension한 subtraction의 MSB 하나**가 `<` 의 답. `>` 는 operand 교환(공짜), `=` 는 XOR-NOR, 범위는 두 비교의 AND, `DATA_COUNT` 밖은 항상 non-solution, enumeration mode는 `!found[i]` 를 AND. 뒷단은 two's complement negate

한 줄이고, symmetric saturation 덕에 negate가 늘 합법입니다.

- **diffusion operator**: $a \mapsto 2\bar{a} - a$ = accumulation → **13칸(number of qubits – 1) shift**(∵ 필요한 건 mean이 아니라 2×mean) → subtraction → symmetric saturation. round-half-to-even이 적용되는 지점은 **이 shift 한 곳뿐**입니다.
- **2 passes는 필연**입니다. mean이 전체 sum을 요구하기 때문입니다. 그 대가로 accumulator가 39 bits, 반사 중간값이 25 bits로 넓어지고, 되쓸 때 23 bits로 붙입니다.
- **128 lanes**: index 하위 5 bits가 lane, row 번호 하위 2 bits가 row phase bank → 충돌 0, crossbar 불필요. PE마다 5단 adder tree와 accumulator가 하나씩이고, 네 accumulator의 sum에서 mean을 확정합니다. partial sum을 자르지 않아 integer 결과는 순서에 무관하지만, bit-exact 대조의 상대는 float64가 아니라 integer reference model입니다(18.3절).
- **multiplier**: Init·oracle·diffusion operator 경로 **0개**, measurement 경로 **128개**(23×23 squarer가 DSP 2개씩, 32 lanes × squarer 두 벌). "DSP가 잡히면 잘못 짠 것"이 아니라, **어느 경로에서** 잡혔는지를 보십시오.

다음 장은 이 datapath가 만들어 낸 amplitude array에서 답을 뽑습니다. predicate를 만족하는 칸이 여럿일 때 그 amplitude들이 매 iteration 정확히 같아진다는 사실, 그래서 maximum을 고르는 방식이 원리적으로 실패한다는 사실, 그리고 Born-rule sampling을 circuit으로 만드는 방법을 봅니다.

14장. multiple solutions — Born-rule sampling과 enumeration

> **이 장을 읽기 위한 준비**: 4장 measurement의 Born rule, 9장 Grover algorithm의 회전 그림, 13장 oracle과 diffusion operator의 datapath.

13장에서는 oracle과 diffusion operator를 circuit으로 구현했습니다. 한 iteration 안에서 벌어지는 일은 이제 전부 정해졌고, 남은 단계는 하나입니다 — **measurement**. amplitude array에서 답 하나를 꺼내는 circuit을 이 장에서 설계합니다.

그런데 이 단계는 앞의 두 단계와 성격이 다릅니다. 우리가 푸는 문제는 "`value[i] < thr_a` 를 만족하는 index를 찾아라" 같은 **predicate search** 이고, predicate를 만족하는 칸이 몇 개인지는 data가 정합니다. 하나일 수도, 300개일 수도, 0개일 수도 있습니다. 즉 우리 문제는 처음부터 **multiple solutions 문제**입니다. 그리고 solution이 여럿이라는 사실은 oracle에서는 추가 비용이 들지 않지만, measurement에서는 설계를 바꿔야 합니다. 이 장은 그 인과 — solution이 여럿이다 → amplitude가 똑같아진다 → 순위로는 못 고른다 → probability로 뽑아야 한다 → 그래야 전부 열거할 수 있다 — 를 따라갑니다.

먼저 혼동하기 쉬운 표기를 정리해야 합니다.

14.0 기호 충돌 주의

이 장이 인용하는 세 자료가 **같은 글자를 다른 뜻으로** 씁니다. 이걸 정리하지 않으면 수식을 따라가다 반드시 헷갈립니다. 그래서 이 해설서는 **글자 대신 용어를 그대로 쓰고**(아래 표의 둘째 열), 원문을 인용할 때만 환산하겠습니다.

개념	본 해설서(통일)	08번 논문	09번 논문
search space의 크기	number of states	D	$K = 2^N$
solution의 개수	number of solutions	M	S (또는 n_s)
(08의 N)	—	diffusion operator가 뒤집는 state의 개수	—
(09의 N)	—	—	사용자 수

특히 조심할 것 두 가지입니다. **08번의 N 은 number of qubits도 search space도 아닙니다** — diffusion operator가 phase를 뒤집는 state의 개수라는, 그 논문만의 variable입니다. 08번에서 search space는 D 입니다. 그리고 **09번의 N 은 "사용자 수"이고 search space는 $K = 2^N$** 입니다. 그래서 09번의 optimal number of iterations 공식에 $S = 1$ 을 넣으면 우리에게 익숙한 $\frac{\pi}{4}\sqrt{K} = \frac{\pi}{4}\sqrt{2^N}$ 이 나오지, $\frac{\pi}{4}\sqrt{N}$ 이 아닙니다. 교과서의 "$\frac{\pi}{4}\sqrt{N}$"은 N 을 search space의 크기(이 해설서의 number of states)로 쓴 표기이니, 09번을 읽을 때는 반드시 $N \to K$ 로 환산하세요.

이 표를 참고하면서, 아래부터는 search space의 크기를 number of states, solution의 개수를 number of solutions로 읽으시면 됩니다.

14.1 solution이 여럿이면 — oracle에서는 공짜

solution이 여러 개일 때 optimal number of iterations는 9.5절의 회전 그림에서 그대로 나옵니다. 초기 각이 $\sin\theta = \sqrt{\text{number of solutions}/\text{number of states}}$ 가 되어, solution이 많을수록 시작부터 solution 축 쪽으로 더 기울어 있고 그만큼 덜 돌아도 됩니다:

$$\text{optimal number of iterations} = \frac{\pi}{4}\sqrt{\frac{\text{number of states}}{\text{number of solutions}}} - \frac{1}{2}$$

oracle에서는 추가 비용이 들지 않습니다. 13.3절의 Predicate Unit은 `value[i] - thr` 의 MSB 한 bit로 각 칸을 독립적으로 판정합니다. predicate를 만족하는 칸이 하나든 300개든 circuit은 같고, 걸리는 cycle도 같습니다 — 어차피 128 lanes가 array 전체를 한 번 훑으니까요. **diffusion operator에서도 추가 비용이 들지 않습니다.** inversion about the mean $a_i \mapsto 2\bar{a} - a_i$ 는 어느 칸이 solution인지 아예 보지 않는 전역 연산이라, solution이 몇 개로 바뀌어도 circuit은 달라지지 않습니다.

그러나 iteration 제어에서는 해결할 문제가 있습니다. 위 식에는 number of solutions가 들어 있는데 우리는 그 값을 모릅니다. 몇 번 돌릴지를 모르는 채로 시작해야 하고, 이 문제를 푸는 것이 16장의 BBHT shot loop입니다.

measurement에서도 설계를 다시 살펴봐야 합니다. solution이 여럿이면 measurement circuit이 지금까지의 방식으로는 **작동하지 않습니다.** 왜 그런지 보려면 oracle이 무엇을 아는 circuit인지부터 다시 확인해야 합니다.

14.2 oracle은 solution을 아는 circuit이 아니다

이 해설서는 이 설명을 일관되게 따릅니다. Grover를 처음 배울 때 교과서는 흔히 "solution index w 의 amplitude에 -1 을 곱한다"고 씁니다. 그 표기는 w 를 constant로 못 박은 것처럼 읽히지만, 그렇게 만든 circuit은 **search를 하지 않습니다** — 답을 이미 알고 있으니까요. 7장에서 oracle을 정의할 때 강조한 대로, oracle은 solution이 무엇인지 **모르면서** "이게 predicate를 만족하느냐"는 물음에만 답하는 Predicate Unit입니다.

	index를 아는 oracle	**predicate를 판정하는 oracle**
solution을	**안다** — constant로 못 박음	**모른다** — data가 정함
circuit	index comparator	`value - thr` 의 MSB 한 bit
number of solutions	설계 시점에 1로 고정	실행 시점에 결정, 0일 수도 모든 state일 수도
쓸모	algorithm 골격 확인	실제 search

우리 accelerator는 오른쪽입니다. host가 CSR에 쓰는 것은 **threshold** `THRESHOLD_A` (그리고 범위 predicate면 `THRESHOLD_B`)이지 solution index가 아닙니다. threshold를 아는 것과 solution을 아는 것은 다릅니다 — threshold는 **질문**이고, 어느 칸이 그 질문에 걸리는지는 `data_mem` 안의 값들이 정합니다. circuit은 실행 전에 어느 칸이 뒤집힐지 모르고, 알 필요도 없습니다.

이 명제에서 두 가지가 곧바로 따라 나옵니다. 첫째, **solution이 몇 개인지 미리 셀 수 없습니다.** 그 개수는 predicate·threshold·data 셋 모두에 의존하므로 설계 시점에는 물론이고 실행 시작 시점에도 알 수 없습니다(16.1절). 둘째, **solution들 사이에는 아무 우열이 없습니다.** oracle이 보기에 모두 똑같이 "predicate를 만족하는 칸"입니다. 이 두 번째 성질 때문에 기존 measurement 방식이 작동하지 않습니다.

14.3 amplitude 동률 — argmax가 무너지는 지점

solution 두 칸 i, i' 를 골라 그 amplitude가 매 순간 어떻게 되는지 따라가 봅시다.

- 시작은 uniform superposition이므로 $a_i = a_{i'} = 1/\sqrt{\text{number of states}}$ 입니다. **같습니다.**
- oracle은 predicate를 만족하는 **모든** 칸에 똑같이 -1 을 곱합니다. 둘 다 solution이니 둘 다 sign이 뒤집힙니다. **여전히 같습니다.**
- diffusion operator $a \mapsto 2\bar{a} - a$ 는 index를 구별하지 않습니다. 같은 값에 같은 function을 적용했으니 결과도 같습니다. **여전히 같습니다.**

이 관계는 반복해도 유지됩니다. oracle과 diffusion operator가 solution 집합을 **완전히 대칭으로** 다루므로, 모든 solution의 amplitude는 매 iteration 후 정확히 같습니다. 게다가 우리는 fixed-point를 쓰고(12장) 두 칸이 지나온 연산열이 문자 그대로 동일하므로, rounding error까지 똑같이 생겨 **bit 단위로 같습니다.** 우연히 갈릴 여지가 없습니다.

숫자로 보겠습니다. 8 states 가운데 predicate를 만족하는 칸이 $\{1, 3, 5\}$ 인 경우(3 solutions, $\theta = 37.761^\circ$, optimal number of iterations $0.692 \to 1$):

number of iterations	solution amplitude	non-solution amplitude	solution 하나의 probability	solution 전체 probability
0	$+0.35355$	$+0.35355$	0.1250 (= 4/32)	0.3750
1	$+0.53033$	-0.17678	**0.2812 (= 9/32)**	0.8437
2	-0.08839	-0.44194	0.0078	0.0234
3	-0.57452	-0.04419	0.3301	0.9902

한 번 돌리고 나면 1번·3번·5번 칸의 amplitude가 **셋 다 정확히** 0.53033 이고, 각각 $9/32 = 28.1\%$ 의 probability를 나눠 가집니다. non-solution 다섯 칸은 -0.17678 로 눌려 있고요. Grover는 의도한 대로 동작했습니다 — solution 쪽에 84.4%의 probability가 모였습니다.

문제는 여기서 답을 꺼내는 circuit입니다. 이 project가 한동안 상정했던 measurement는 **argmax**, 즉 $|a_i|$ 가 가장 큰 index를 고르는 comparator tree였습니다(4.2절이 이미 기각해 둔 그 circuit입니다). 그런데 지금 maximum을 갖는 칸이 셋이고 그 값이 bit까지 같습니다. comparator tree는 동률

에서 무엇을 내놓을까요? 구조가 정한 우선순위 — 순차 scan이면 먼저 만난 칸, tournament tree면 leaf 배치 순서 — 에 따라 **index 1** 하나입니다. circuit이 결정론적이므로 몇 번을 다시 돌려도 1입니다.

즉 argmax를 쓰면 **3번과 5번은 영원히 나오지 않습니다.** random number seed를 바꿔도, iteration을 늘려도, clock을 아무리 오래 돌려도 나오지 않습니다. 이것은 probability가 낮아서 못 찾는 것이 아니라 **원리적으로 도달할 수 없는** 것입니다. 우리 accelerator의 두 번째 출력 요구 — "predicate를 만족하는 원소를 전부 열거한다" — 를 이 방식으로는 충족할 수 없습니다.

동률을 감지해서 random number로 하나 고르면 되지 않느냐고 물을 수 있습니다. 그러면 $\{1, 3, 5\}$ 는 균등하게 나오겠지만, 그건 이미 argmax가 아니라 probability sampling의 제한적인 특수 경우입니다. 게다가 "동률"을 정확 일치로 판정해야 하는데 amplitude가 조금이라도 다른 경우가 섞이면 곧바로 무너지고, 무엇보다 **순위만 남기고 probability의 크기 정보를 버린다**는 근본 문제가 그대로입니다. 순위로 뽑는 대신, 처음부터 probability로 뽑아야 합니다.

(solution 두 개짜리 최소 예제는 9.4절의 4 states 손계산에 2 solutions 변형으로 붙여 두었습니다. 두 막대의 높이가 **정확히 같아지는 것**을 네 줄짜리 계산으로 확인할 수 있습니다. 다만 4 states에 solution이 둘이면 처음부터 절반이 solution이라 probability가 $\frac{1}{2}$ 에서 제자리인 퇴화 사례입니다 — 동률인 채로 **자라는** 모습은 이 절 앞의 8 states · 3 solutions 예에서 이미 봤습니다.)

14.4 Born-rule sampling — probability를 circuit으로

4.1절의 Born rule을 그대로 적용하면 됩니다. state $|\psi\rangle = \sum_i a_i |i\rangle$ 를 computational basis로 측정하면 결과 i 가 나올 probability는

$$\text{probability}(i) = |a_i|^2$$

입니다. amplitude가 실수뿐인 우리 datapath에서는 그냥 a_i^2 이고요. 우리가 할 일은 이 분포를 따르는 random number를 circuit으로 뽑는 것 — 실기 quantum에서 자연이 공짜로 해 주는 일을, emulator에서는 우리가 만들어야 합니다.

교과서적인 방법은 **inverse transform sampling** 입니다. prefix sum

$$\text{prefix sum}(i) = \sum_{k \le i} a_k^2$$

를 만들어 두고, uniform random number를 하나 뽑아 $\text{prefix sum}(i) > \text{random number}$ 가 되는 **첫 번째** i 를 답으로 씁니다. index i 가 뽑힐 조건은 random number가 $[\text{prefix sum}(i-1),\ \text{prefix sum}(i))$ interval에 떨어지는 것이고, 그 interval의 길이가 정확히 a_i^2 이므로 $\text{probability}(i) = a_i^2$ 가 그대로 성립합니다. 4장의 규칙과 circuit의 동작이 일치합니다.

여기서 세 가지 실용적 결정을 합니다.

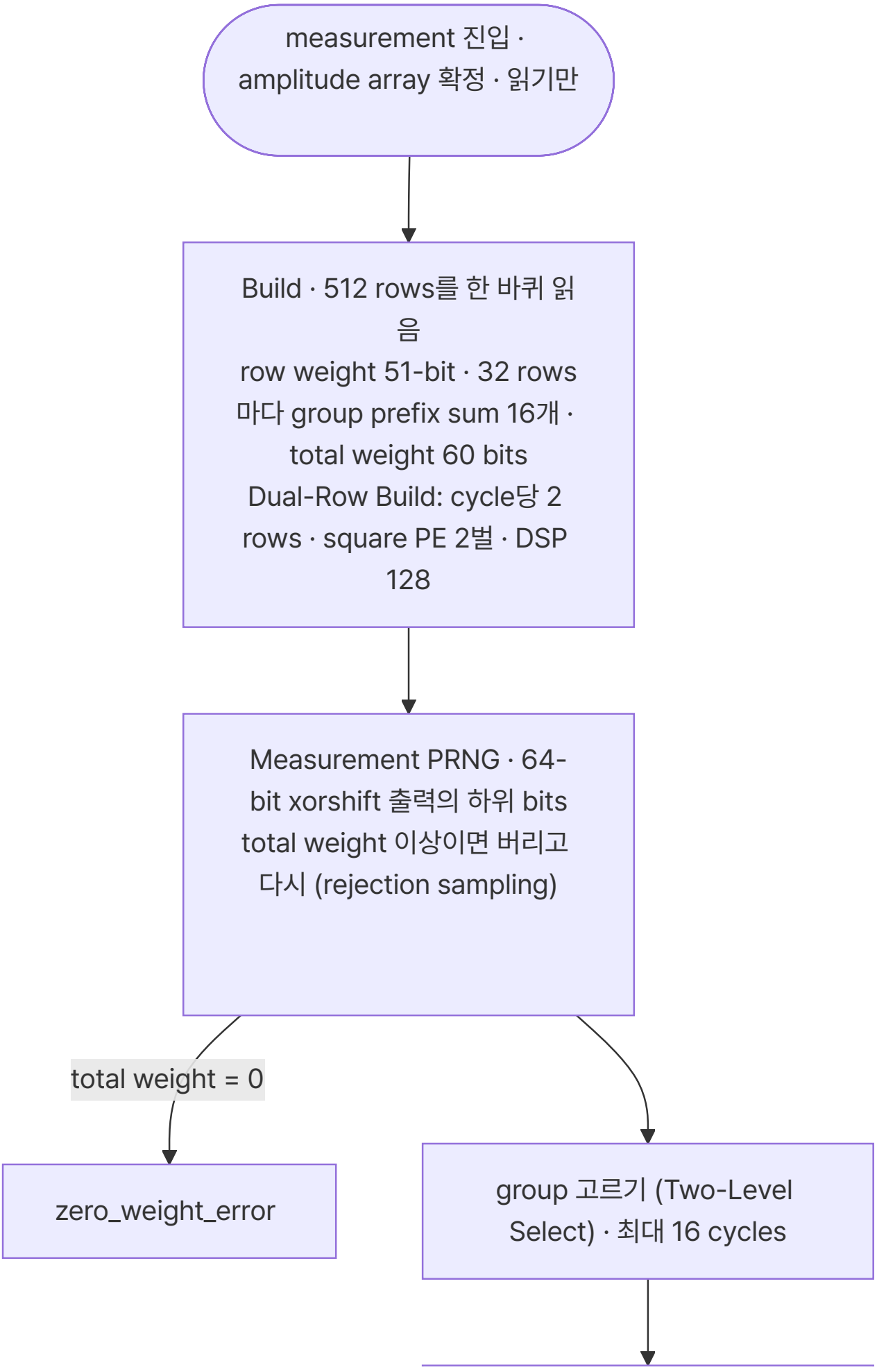
전부 integer로, 자르지 않고 계산합니다. amplitude가 23-bit integer이므로 square는 46-bit integer이고, 우리는 이 곱을 한 bit도 자르지 않습니다. 잘린 하위 bit가 곧 probability의 왜곡이고, 값이 작은 칸일수록 상대 error가 커져 분포가 체계적으로 기울기 때문입니다. prefix sum과 total weight도 필요한 폭 그대로 담습니다(14.5절). integer라서 좋은 점이 하나 더 있습니다 — prefix sum이 array 끝에서 **정확히** total weight에 닿으므로, random number가 total weight 미만이기만 하면 어디선가 반드시 멈춥니다. floating-point prefix sum이라면 rounding 때문에 끝까지 가도 random number를 못 넘는 경우를 대비한 안전망이 필요했겠지만, integer 설계에서는 그 경로 자체가 없습니다. 남는 예외는 total weight가 0인 경우 하나이고, 이때 circuit은 `zero_weight_error` 를 세우고 끝냅니다.

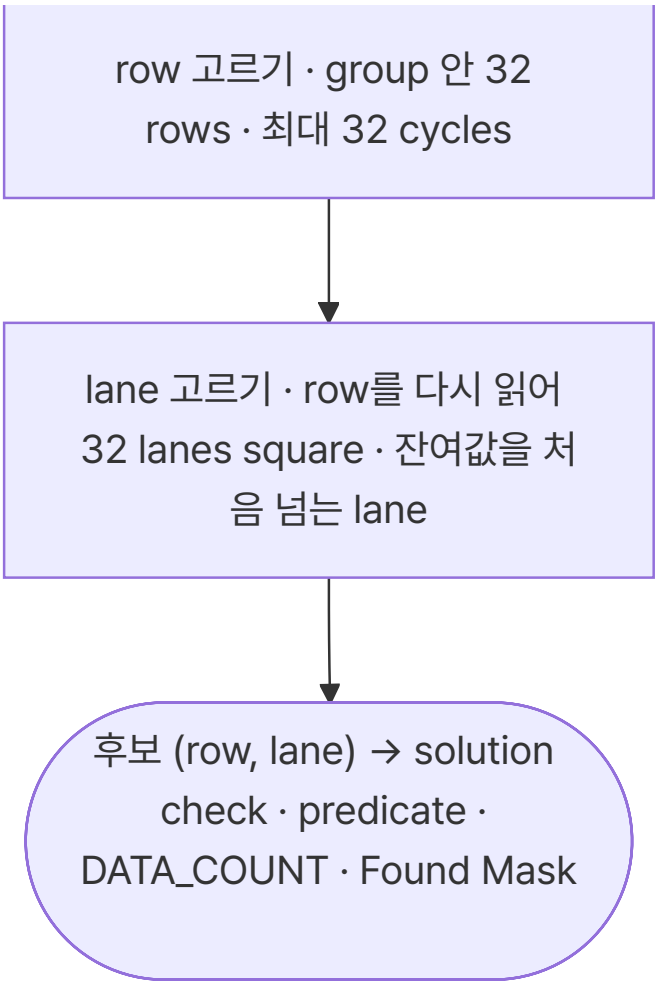
나눗셈을 하지 않습니다. 보통은 probability가 되도록 prefix sum을 전체 sum of squares로 나누지만, 우리는 나누는 대신 random number를 $[0,\ \text{total weight})$ 범위에서 뽑습니다($\text{total weight} = \sum_k a_k^2$). 비교의 양변에 같은 constant를 곱한 것과 같으므로 분포는 동일하고, divider가 통째로 사라집니다. 다만 이 동치는 random number가 $[0,\ \text{total weight})$ 에서 **정확히 균등**할 때만 성립하는데, total weight는 2의 거듭제곱이 아니라서 PRNG 출력을 그대로 잘라 쓰면 균등해지지 않습니다. 그래서 **rejection sampling**을 씁니다 — 64-bit PRNG 출력에서 total weight − 1 의 bit 길이만큼 하위 bit를 후보로 꺼내, 후보가 total weight 이상이면 버리고 다시 뽑습니다. comparator 하나면 되니 divider가 도로 들어오지는 않고, 후보가 받아들여질 probability가 늘 절반을 넘으므로 mean 두 번 안에 끝납니다. 64-bit 출력은 Measurement PRNG(xorshift64)가 한 block씩 내며, 32-bit seed 하나에서 결정적으로 확장됩니다. 초기 state에서 total weight는 정확히 2^{44} 인데(13.2절), iteration을 거친 뒤에는 rounding 때문에 정확히 그 값이 아니어도 이 방식은 그때의 total weight를 그대로 쓰므로 영향을 받지 않습니다.

squarer는 필요합니다. argmax 시절에는 $|a|$ 의 최대와 a^2 의 최대가 같아서 square를 건너뛸 수 있었지만, probability로 뽑으려면 크기 자체가 필요하므로 square를 피할 수 없습니다. 13.8절의 "0 multipliers"는 **diffusion operator·oracle·Init 경로에 한정된** 이야기이고, Measurement Unit은 squarer를 씁니다. 이것이 우리 설계에서 DSP가 잡히는 유일한 자리입니다.

남은 문제는 속도입니다. 위 절차를 곧이곧대로 짜면 amplitude를 한 칸씩 훑는 직렬 scan이라 16,384 cycles이 듭니다. Grover iteration 한 번이 약 268 cycles인데(11.8절) measurement 한 번에 그 60×의 시간이 들고, 128 lanes datapath는 활용되지 않습니다.

14.5 2단 병렬 measurement


measurement 진입 · amplitude array 확정 · 읽기만
Build · 512 rows를 한 바퀴 읽음
row weight 51-bit · 32 rows마다 group prefix sum 16개 · total weight 60 bits
Dual-Row Build: cycle당 2 rows · square PE 2벌 · DSP 128
Measurement PRNG · 64-bit xorshift 출력의 하위 bits
total weight 이상이면 버리고 다시 (rejection sampling)
total weight = 0
zero_weight_error
group 고르기 (Two-Level Select) · 최대 16 cycles



11.7절의 row 구조를 measurement에도 그대로 씁니다. sampling을 **row를 먼저 고르고 그 안에서 lane을 고르는** 두 단으로 쪼개는 것입니다. row가 512개나 되므로, final configuration은 row 고르기를 다시 **group → row**의 두 층으로 나눕니다.

1. **Build — row weight 만들기.** amplitude memory를 한 바퀴 읽으며 row마다 32 lanes의 sum of squares(row weight)를 만들어 작은 memory에 적고, total weight도 함께 모읍니다. 512 rows를 32 rows씩 16개 **group**으로 묶어, group 경계를 지날 때마다 그때까지의 prefix sum을 group prefix sum으로 적어 둡니다 — 추가 square도 추가 cycle도 없이 따라 나오는 값입니다. squarer 한 벌이면 한 cycle에 1 row라 512 cycles인데, final configuration(**Dual-Row Build**)은 checkpoint memory가 한 번에 내주는 4 rows를 받아 **한 cycle에 2 rows씩** 제곱해 256 cycles로 줄입니다. squarer가 두 벌인 이유가 이것입니다(DSP 128개).
2. **random number — rejection sampling.** 14.4절의 규칙대로 total weight 미만인 random number를 얻을 때까지 64-bit block를 뽑습니다.
3. **group 고르기 (Two-Level Select).** 16 groups의 group prefix sum을 앞에서부터 보며 random number를 처음 넘는 group을 고릅니다. 최대 16 cycles입니다.
4. **row 고르기.** 그 group의 32 rows 안에서 row weight의 prefix sum이 random number를 처음 넘는 row를 고릅니다. 최대 32 cycles입니다. (Linear Select라면 512 rows 전체를 앞에서부터 훑어야 합니다.)
5. **lane 고르기.** 고른 row를 다시 읽어 32 lanes를 제곱하고, 그 row 안의 잔여값(random number에서 앞 row들까지의 prefix sum을 뺀 값)을 lane의 prefix sum이 처음 넘는 lane을 고릅니다. 후보 index는 `{row, lane}` 입니다.
6. **verification.** `data_mem` 의 두 번째 port로 후보 칸의 값을 다시 읽어 oracle과 같은 predicate(와 padding·`found_mask`)로 판정합니다 (16.4절).

비교는 전부 **엄격한** `>` 이고, 순서는 늘 group → row → lane입니다. 이 규칙이 하나라도 다르면 같은 random number에서 다른 칸이 나오므로, 18.3절에서 bit-exact 조건으로 명시한 항목입니다.

얼마나 줄었는지는 shot 한 번의 나머지 비용(measurement·verification·policy 등)으로 봅니다. 500 workloads campaign에 직선을 맞춘 값으로, Single-Row Build · Linear Select configuration에서 shot당 약 821 cycles이던 것이 Dual-Row Build에서 약 567, Two-Level Select까지 더해 약 **354 cycles**이 되었습니다. Six-Stage Ablation에서 Dual-Row Build와 Two-Level Select가 각각 총 cycle을 18.15%, 18.94% 줄였고, 둘을 합치면 Stage 4 대비 33.7%입니다 — 눈에 잘 띄지 않는 자리였지만 이득의 큰 몫이 Measurement Unit에 있었습니다.

이 쪼개기가 분포를 바꾸지 않는다는 것을 반드시 확인해야 합니다. index i 가 row b 에 속할 때, 이 절차가 i 를 뽑을 probability는 두 단의 곱입니다:

$$\text{probability}(i) = \underbrace{\frac{\text{row weight}_b}{\text{total weight}}}_{\text{row } b \text{ 선택}} \times \underbrace{\frac{a_i^2}{\text{row weight}_b}}_{\text{row } b \text{ 안에서 } i \text{ 선택}} = \frac{a_i^2}{\text{total weight}}$$

row weight가 약분되어 원래의 Born-rule sampling이 그대로 남습니다. group 층을 하나 더 얹어도 같은 약분이 한 번 더 일어날 뿐입니다. 더 근본적으로는 — **inverse transform sampling은 index를 어떤 순서로 훑든, 몇 층으로 나눠 고르든 결과 분포가 같습니다.** probability 질량은 index에 붙어 있지 훑는 순서에 붙어 있지 않으니까요(4.3절). group·row·lane 순의 layer 선택은 순회 방식을 바꾼 것이고, 그래서 순차 prefix sum scan과 통계적으로 구별되지 않습니다. 이 성질은 verification에서도 쓰입니다(18.6절).

accumulator width. 12장에서 미뤄 둔 논의가 여기서 결론이 납니다. square는 46 bits, 32개를 더한 row weight는 51 bits, 16,384개를 더한 total weight는 60 bits입니다. 실제 값은 norm이라 초기에 정확히 2^{44} 이고 그 근처에 머무르니 상위 bit는 거의 쓰이지 않지만, saturation 없이 bit-exact를 보장하려면 최악 경계로 잡는 것이 맞습니다. diffusion operator 경로의 accumulator width `ACCW = 39` 와는 **별개의 accumulator**라는 점에 주의하세요. 재는 대상이 amplitude의 sum이 아니라 amplitude square의 sum이라 자릿수 계산이 처음부터 다릅니다.

마지막으로, **Build도 lane 고르기도 amplitude memory를 읽기만 합니다.** 제곱해서 prefix sum을 만들 뿐 amplitude를 한 칸도 고쳐 쓰지 않습니다. 이 읽기 전용 성질이 16장 checkpoint의 근거가 됩니다(16.5절).

14.6 기각한 대안들

measurement를 hardware로 만드는 길은 여럿이고, 우리가 고르지 않은 셋을 왜 안 골랐는지 남겨 둡니다. 셋 다 나름의 논리가 있어서, 이유를 알아야 나중에 같은 대안을 다시 검토하는 일을 줄일 수 있습니다.

argmax. 가장 싸고 가장 매력적입니다. comparator 하나로 array를 훑으면 끝이고, $|a|$ 의 최대와 a^2 의 최대가 같으니 squarer조차 필요 없으며, absolute value도 sign bit로 조건부 negate 한 번입니다. solution이 하나라고 확신할 수 있다면 지금도 맞는 선택입니다 — optimal number of iterations만큼 돈 뒤에는 그 한 칸이 압도적으로 크니까요. 우리가 버린 이유는 정확히 하나, **14.3절의 동률**입니다. predicate search는 solution이 몇 개인지 모르는 문제이고 solution이 둘 이상이면 argmax가 index 하나에 영원히 고착됩니다. 성능이 아니라 **기능**이 안 되는 것이라 절충의 여지가 없습니다.

04번의 직렬 prefix sum + linear approximation. 04번 논문은 squarer와 divider를 피하려고 근사를 3중으로 쌓습니다 — solution probability를 $2|\alpha| - 1$ 로 linear approximation하고, 나머지 probability를 geometric series shift로 배분하고, prefix sum을 병렬로 구한 뒤 weighted random number generator로 뽑습니다. 정교하지만 전제가 있습니다: **solution 하나가 probability를 거의 다 차지한다**는 것. multiple solutions에서는 이 전제가 깨집니다. 앞의 8 states · 3 solutions · 1 iteration 예에서 solution amplitude는 0.53033 인데, $2|\alpha| - 1 = 0.0607$ 인 반면 참값은 0.2812 입니다 — 4.6× 어긋납니다. non-solution 쪽은 $2(0.17678) - 1 = -0.646$ 으로 아예 음수가 나옵니다. probability로 쓸 수 없는 수치입니다. 우리는 근사를 버리고 정확한 integer square를 택했습니다. Artix-7의 DSP에는 여유가 있고(128 / 240), 근사 error와 달리 square는 검증할 수 있습니다.

von Neumann식 collapse 후 renormalization. Pilch–Długopolski(2018)의 범용 emulator는 진짜 measurement를 circuit으로 합니다 — qubit별로 0이 나올 probability를 계산하고, LFSR random number와 비교해 state를 collapse시킨 뒤, 살아남은 amplitude를 sum of squares의 square root로 나눠 **renormalization**합니다(4.4절). 범용 simulator에는 필수입니다. measurement 뒤에도 circuit이 이어지니 collapse 후 state가 정확해야 하니까요. 우리는 다릅니다. **measurement는 한 shot의 마지막 단계이고 그 뒤에 gate가 없습니다.** collapse된 state를 쓸 일이 없으므로 square-root unit과 divider를 통째로 생략할 수 있습니다. 게다가 renormalization은 amplitude array를 **덮어씁니다** — 그러면 방금 확인한 "measurement는 읽기 전용"이 깨지고 checkpoint가 성립하지 않습니다. renormalization을 안 하는 것이 circuit을 줄이는 데서 그치지 않고 checkpoint도 사용할 수 있게 합니다.

14.7 enumeration mode — mask로 중복 없이

이제 두 번째 출력 요구를 채웁니다. predicate를 만족하는 원소를 **전부** 찾는 것입니다.

Born-rule sampling을 그냥 반복하면 안 됩니다. 매번 같은 분포에서 뽑으니 이미 찾은 답이 계속 다시 나옵니다(solution이 셋이면 세 답이 각각 28%씩이니 같은 답이 연달아 나오는 일이 흔합니다). 그래서 **찾은 답을 solution 집합에서 빼야** 합니다. 이를 위해 `found_mask`(16,384 × 1-bit, RAMB36 2개)를 두고, oracle의 predicate에 한 항을 AND합니다:

$$\texttt{mark}(i) = \texttt{predicate}(i) \ \wedge \ \neg\, \texttt{found}[i]$$

13.3절의 Predicate Unit 출력에 AND gate 하나가 붙는 것이 전부이고, 후보를 검증할 때도 같은 mask를 봅니다. 새 답 `cand` 가 나오면 Result FIFO에 넣고, `found[cand] ← 1` 을 쓰고, checkpoint를 무효화한 뒤(16.7절) BBHT를 처음부터 다시 시작합니다. 이 모든 것이 Accelerator Core 안에서 일어나며, firmware가 할 일은 FIFO에서 결과를 뽑는 것뿐입니다.

이것이 정당한 oracle인 이유를 짚어야 합니다. 7장은 oracle을 "고정된 블랙박스"로 정의했는데, mask는 enumeration round마다 바뀌지 않느냐는 의문이 자연스럽습니다. 답은 — mask를 걸면 **solution 집합이 하나 줄어든 새 문제를 처음부터 다시 푸는 것**입니다. 한 enumeration round 안에서 oracle은 고정되어 있고, enumeration round가 바뀌면 그것은 같은 문제를 이어 푸는 것이 아니라 **다른 문제**입니다. 그래서 uniform superposition부터 다시 시작하는 것이 당연하고, 7장 전제와도 충돌하지 않습니다.

앞의 8 states · $\{1, 3, 5\}$ 예를 끝까지 돌리면 이렇습니다:

enumeration round	남은 solution	number of solutions	optimal number of iterations	success probability
1	$\{1, 3, 5\}$	3	1	84.4%
2	$\{3, 5\}$	2	1	100.0%
3	$\{5\}$	1	2	94.5%
4	$\{\}$	0	—	uniform superposition 유지 → 종료 판정

enumeration round가 진행될수록 남은 solution이 줄어 optimal number of iterations $\frac{\pi}{4}\sqrt{\text{number of states}/\text{number of solutions}} - \frac{1}{2}$ 가 커집니다. **뒤로 갈수록 비싸집니다.** 총비용은 $\sum_{k=1}^{\text{number of solutions}} \sqrt{\text{number of states}/k} \approx 2\sqrt{\text{number of states} \cdot \text{number of solutions}}$ 로, 마지막 몇 enumeration round가 전체 시간을 지배합니다.

결과는 256칸 FIFO로 나옵니다. 찾은 index가 FIFO에 쌓이고, FIFO가 가득 차면 hardware가 멈춰 기다립니다(full stall). 그래서 solution이 256개를 넘을 수 있는 enumeration에서는 firmware가 search가 끝나기를 기다리지 말고 **돌고 있는 동안에도** FIFO를 비워야 합니다. 멈춘 cycle은 `FIFO_STALL` 로 남아, firmware가 늦게 뽑았는지를 나중에 확인할 수 있습니다. solution이 너무 많아 enumeration을 포기하는 별도의 signal은 두지 않았습니다 — predicate를 만족하는 원소가 수천 개라면 애초에 Grover로 열거할 문제가 아니라 predicate를 좁혀야 할 문제이고, 그 판단은 firmware 몫입니다.

마지막 enumeration round에는 solution이 하나도 없습니다. 남은 solution이 없으면 oracle이 아무 칸도 뒤집지 않고, diffusion operator는 uniform superposition을 그대로 두므로 amplitude 분포가 변하지 않습니다 — 몇 번을 돌려도 uniform random sampling과 구별되지 않고, 뽑힌 후보는 solution check에서 전부 탈락합니다. 그러면 언제 "다 찾았다"고 선언할까요. hardware의 규칙은 이렇습니다 — 한 번의 BBHT search가 shot cap이나 BBHT budget(16.3절)에 닿도록 새 답을 하나도 못 내면 그것을 **실패 한 번**으로 세고, 실패가 `fail_repeat_limit` 번(기본 3번) **연속**되면 enumeration을 끝냅니다. 새 답이 하나라도 나오면 연속 실패 수는 0으로 돌아갑니다. 즉 enumeration의 끝은 언제나 "solution이 없음을 몇 번 연속 확인하는 비용"을 치르는데, 이것이 solution이 몇 개인지 모르는 채 전부를 열거하는 대가입니다.

새 답이 나와 mask가 바뀔 때는 checkpoint가 무효가 되지만, **실패한 BBHT search 사이에는 mask가 그대로이므로 checkpoint가 유효합니다.** 그래서 enumeration에서도 checkpoint를 활용합니다. software reference model로 `EQ` · 4 solutions · 실패 한계 3 조건을 돌려 보면, 네 답을 다 찾고 끝나기까지 requested iterations는 2,422였는데 실제로 계산한 executed iterations는 635였습니다. 매 shot 처음부터 도는 Restart mode라면 2,422 iterations를 전부 돌았을 것입니다.

14.8 두 개의 함정

함정 1 — overshoot. 9.6절의 "지나치면 안 된다"가 hardware에서는 더 날카롭게 나타납니다. optimal number of iterations $\frac{\pi}{4}\sqrt{\text{number of states}/\text{number of solutions}} - \frac{1}{2}$ 는 대개 integer로 떨어지지 않아 반올림해야 하는데, state가 적으면 그 error가 success probability에 그대로 반영됩니다. 위 8 states · 3 solutions 표가 그 자체로 경고입니다 — 최적값 0.692를 1로 올리면 84.4%지만, 한 번 더 돌려 2 iterations로 가면 solution 전체 probability가 **2.3%로 추락**합니다. probability가 단조 증가하다 포화하는 것이 아니라 **진동**하기 때문입니다. FSM의 종료 counter를 한 칸 잘못 세는 것이 success probability를 36× 깎을 수 있다는 뜻이니, qubit가 적을 때는 특히 조심해야 합니다.

함정 2 — 역주행(reverse amplification). solution이 **너무 많으면**(전체 state의 절반 초과) 초기 각 θ 가 이미 45°를 넘어, 첫 iteration부터 solution 축을 지나쳐 non-solution 쪽으로 갑니다. 실기에서는 드문 상황이지만 **predicate search에서는 쉽게 일어납니다.** `value < thr_a` 에서 `thr_a` 를 data에 실린 maximum보다 크게 잡으면 **모든 칸이 predicate를 만족**해 모든 state가 solution이 되고, oracle이 사실상 $-I$ 가 되어 diffusion operator와 합쳐도 global phase만 남습니다 — iteration이 measurement 분포를 전혀 바꾸지 못합니다. 범위 predicate `thr_a < value <`

`thr_b` 를 너무 넓게 잡아도 정도만 다를 뿐 같은 일이 벌어집니다. 다행히 이 극단은 BBHT가 자연스럽게 견딥니다 — 초반 shot은 iteration을 거의 돌지 않아 사실상 uniform random sampling인데, solution이 많으면 그 sampling만으로 곧바로 맞으니까요. 대신 이 현상을 이해하지 못한 채 "iteration을 늘리면 정확해지겠지"라고 접근하면 판단을 그르치게 됩니다.

반대 극단도 짝으로 기억해 두세요. enumeration mode는 enumeration round가 진행될수록 남은 solution이 하나로 줄어들어 함정 2에서 함정 1 쪽으로 건너갑니다. 14 qubits · 1 solution이면 optimal number of iterations가 100인데, 이 근처에서 probability 곡선의 골짜기가 깊습니다 — 100 iterations에서 100%인 success probability가 두 배쯤 지나친 200 iterations에서는 사실상 0입니다. 우리 hardware는 number of iterations를 $[0, 128)$ 에서만 뽑으므로 그 골짜기까지 가지는 않지만(16.2절), 이 상한 자체가 overshoot를 막으려고 BBHT가 둔 장치입니다. 즉 enumeration의 앞쪽 enumeration round는 "solution이 너무 많아서" 위험하고 뒤쪽 enumeration round는 "optimal number of iterations가 너무 커서" 위험합니다. 양쪽 다 solution이 몇 개인지 모르는 채로 통과해야 하고, 그 통과 방법이 16장입니다.

14.9 확장 — threshold를 낮추면 minimum search

여기까지 오면 최적화 문제로 가는 길이 한 줄로 보입니다. `value < thr_a` predicate로 solution을 하나 찾을 때마다 `thr_a ← value[cand]` 로 threshold를 낮추고 다시 search하면, 기준이 계단처럼 내려가며 minimum에 수렴합니다 — 이것이 **Dürr–Høyer algorithm**이고, 09번 논문(Habibie 외, IEEE IoT J. 2024)이 무선 IoT의 사용자 탐지에 적용한 것이 이 구조입니다. 우리 accelerator는 이 loop를 hardware에 넣지 않습니다. 넣을 필요가 없기 때문입니다 — `THRESHOLD_A` 는 host가 CSR에 쓰는 값이고 결과는 어차피 host로 돌아가므로, RVX 위의 RISC-V app이 while loop 한 개를 도는 것만으로 같은 algorithm이 됩니다(17.7절). hardware가 제공해야 할 것은 "threshold를 받아 solution 하나를 돌려주는" 원자적 연산 하나뿐이고, 그것은 이미 있습니다. 09번을 인용할 때 주의할 오류 두 가지는 15.6절에 정리해 두었습니다.

14.10 이 장의 요약

- **기호 충돌 주의**: 본 해설서는 글자 대신 number of states(search space의 크기)와 number of solutions라는 용어를 씁니다. 08번의 N (diffusion operator가 뒤집는 state의 개수)·D(search space), 09번의 N(사용자 수)·$K = 2^N$(search space)은 뜻이 서로 다르니 원문을 읽을 때 환산합니다.
- **multiple solutions는 oracle과 diffusion operator에서 공짜**입니다 — Predicate Unit도 inversion about the mean도 solution이 몇 개인지 보지 않습니다. **iteration 제어와 measurement에서는 공짜가 아닙니다.**
- **oracle은 solution을 아는 circuit이 아닙니다.** threshold는 질문이고 solution은 data가 정합니다. 귀결 둘 — solution이 몇 개인지 미리 셀 수 없고, solution 사이에 우열이 없습니다.
- **amplitude 동률**: oracle·diffusion operator가 solution 집합을 완전 대칭으로 다루므로 모든 solution의 amplitude가 매 iteration **bit까지 같습니다.** 8 states · $\{1, 3, 5\}$ · 1 iteration에서 셋 다 0.53033, 각 $9/32 = 28.1\%$, non-solution은 -0.17678.
- **그래서 argmax는 무너집니다.** comparator tree 우선순위가 정한 index 하나만 영원히 반환하고, 나머지 solution은 원리적으로 도달 불가 → **enumeration이 불가능합니다.**
- **Born-rule sampling**: prefix sum에 uniform random number를 떨어뜨리는 inverse transform sampling. 전부 integer로 자르지 않고 계산하므로 반드시 어딘가에서 멈추고(예외는 total weight가 0일 때의 `zero_weight_error`), total weight로 나누는 대신 64-bit block의 하위 bit와 **rejection sampling**으로 random number를 $[0, \text{total weight})$ 에 맞춥니다. squarer는 필요합니다.
- **Measurement Unit**: Build(512 row weights · 16 group prefix sums · total weight, **Dual-Row Build**는 cycle당 2 rows) → random number → group(≤16) → row(≤32, **Two-Level Select**) → lane(32) → verification. shot당 나머지 비용이 약 821 → 567 → **354 cycles**. joint probability $\frac{\text{row weight}_b}{\text{total weight}} \cdot \frac{a_i^2}{\text{row weight}_b} = \frac{a_i^2}{\text{total weight}}$ 로 분포가 정확히 보존됩니다.
- **accumulator**: square 46 bits, row weight 51 bits, total weight 60 bits(초기에는 정확히 2^{44}). diffusion operator 경로의 `ACCW=39` 와는 별개입니다.
- **measurement는 읽기 전용**입니다 → 16장 checkpoint의 근거.
- **기각한 대안**: argmax(동률로 기능 불능) / 04번 linear approximation $2|\alpha| - 1$(multiple solutions에서 0.0607 vs 참값 0.2812) / von Neumann식 renormalization(measurement 뒤에 gate가 없어 불필요하고, array를 덮어써 checkpoint를 깹니다).
- **enumeration mode**: `found_mask` 로 `predicate(i) ∧ ¬found[i]`. enumeration round마다 **solution 집합이 줄어든 새 문제**를 다시 푸는 것이라 7장 전제와 충돌하지 않습니다. 결과는 **256칸 FIFO**(가득 차면 멈춤 — 도는 동안 비울 것), 종료는 **BBHT 실패가 `fail_repeat_limit` (기본 3)번 연속**될 때. 실패 사이에는 checkpoint가 유효하므로 enumeration에서도 executed iterations가 줄어듭니다.
- **두 함정**: overshoot(8 states · 3 solutions에서 1 iteration 84.4% → 2 iterations 2.3%) / 역주행(solution이 전체 state의 절반 초과, `thr` 을 data 범위 밖으로 잡으면 발생). enumeration 후반에 solution이 하나만 남는 경우가 반대편 극단입니다.
- **확장**: solution을 찾을 때마다 threshold를 낮추면 그대로 Dürr–Høyer minimum search. hardware가 아니라 host software의 loop로 충분합니다.

다음 15장은 여섯 논문의 지도를 펼치고, 지금까지의 결정을 한 표로 모으며, 논문을 인용할 때 조심해야 할 오류 목록을 정리합니다.

15장. 논문 지도와 설계 결정표

> **이 장을 읽기 위한 준비**: 11장 state vector emulator · 12장 fixed-point와 Q1.22 · 13장 oracle과 diffusion operator · 14장 multiple solutions. **성격**: 참조용(Part 7). 여섯 논문의 관계를 지도로 펼치고, **확정된 설계값을 한 표에 모읍니다.** 10장이 1~9장(quantum 이론)의 cheat sheet라면, 이 장은 hardware 쪽의 cheat sheet입니다.

이 장은 앞 장들에서 배운 내용을 **지도 한 장과 표 몇 개**로 압축한 참조 자료입니다. 논문을 다시 읽으러 갈 때, 설계 결정을 되짚을 때, 그리고 무엇보다 — 논문을 **인용하기 전에** 펼쳐 보세요. 15.6절이 "1차 자료도 틀린다"는 것을 실물 목록으로 보여줍니다.

이 장의 숫자는 board에서 검증된 **final configuration**(Board-Verified Build: 2026-09-07 build, 2026-09-08 benchmark)을 따릅니다. 해설서 초판은 2026-08-15 에 설계를 한 번 확정했지만, 그 뒤 RTL을 실제로 짓고 board에 올리면서 number of qubits·amplitude 형식·checkpoint 구조가 바뀌었습니다. 15.4절이 그 최종값의 해설서 안 **단일 출처(source of truth)** 이고, 해설서 어느 장에 같은 숫자가 적혀 있든 어긋나면 15.4절을 기준으로 합니다. 해설서 바깥까지 포함한 저장소 전체의 기준은 board build 그 자체이고, 이 표는 그것을 해설서의 언어로 옮긴 것입니다.

15.1 여섯 논문 한눈에

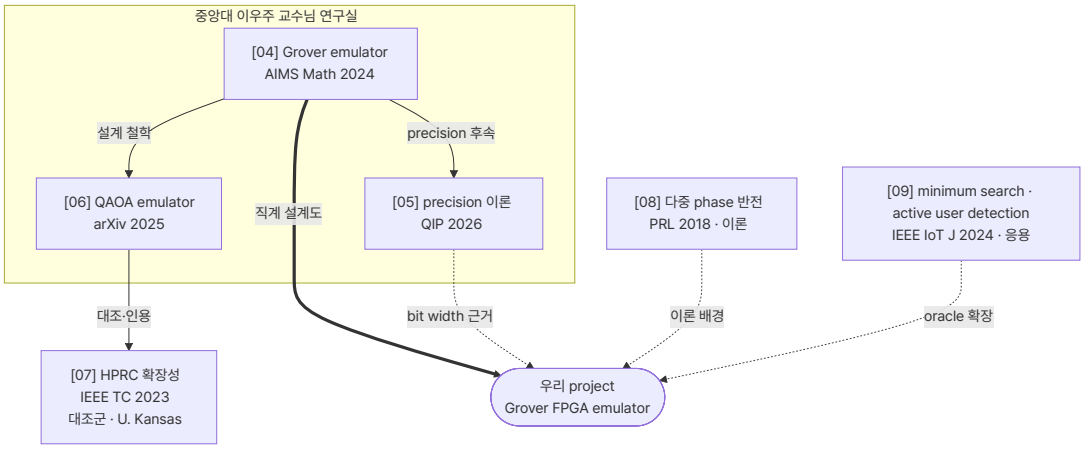
#	논문	출처	한 줄	우리와의 거리
04	Developing a Grover's Emulator on Standalone FPGAs	**중앙대**, AIMS Math. 2024	Grover 전용 최적화 4종 + RISC-V SoC 통합	**직계 설계도**
05	Precision-aware Fixed-point Emulation	**중앙대**, QIP 2026	truncation error $O(2^{\text{number of qubits}-\text{fraction bits}})$ 증명 + minimum fraction bits 공식	**bit width 판단의 나침반**
06	Standalone FPGA QAOA Emulator	**중앙대**, arXiv 2025	같은 골격을 QAOA로 (multiplier 대비)	multiplier 없는 이유
07	Towards Complete & Scalable Emulation (HPRC)	Kansas, IEEE TC 2023	외부 DDR로 32 qubits, C2Q initialization	**대조군** (반대 진영)
08	Generalized Grover for Multiple Phase Inversion	PRL 2018	diffusion operator를 multiple solutions로 일반화	이론 배경 (implementation X)
09	Quantum Minimum Searching for AUD in IoT	INSA Lyon, IEEE IoT J. 2024	minimum search(DHA)를 무선 사용자 탐지에	**확장 참고 사례**

핵심 사실 하나: **04·05·06은 이 project 지도교수 연구실(중앙대 이우주 교수님)의 논문**입니다. 특히 04번은 우리가 만들려는 것과 정확히 같은 물건(state vector를 fixed-point memory에 두고 oracle+diffusion operator를 FSM으로 $\sqrt{\text{number of states}}$ 번 도는 emulator)이라, 참고문헌이 아니라 사실상 **설계도**에 가깝습니다.

이번 개정에서는 09번의 참조 비중을 줄였습니다. 이전 판은 09번을 14장의 원문처럼 다뤘지만, 우리 accelerator는 threshold를 계단처럼 낮추는 바깥 loop를 hardware에 넣지 않고 host firmware의 `while` 한 줄로 처리하기로 했고(14.9·17.7), iteration schedule의 근거도 09번이 아니라 아래 15.2절의 BBHT 원전으로 옮겼습니다. 09번이 여전히 남기는 것은 **"oracle은 index가 아니라 predicate를 판정한다"** 는 관점 하나와, 그 관점을 실제 응용에 적용한 사례 하나입니다. 이 두 내용을 참고하려고 인용합니다.

15.2 계보도 — 누가 누구를 딛고 섰나

여섯 논문은 서로 인용하며 **연관된 연구 흐름**을 이룹니다.



- **04 → 06 → 05** 가 중앙대 연구실의 3부작입니다. 04번(Grover emulator)이 뿌리이고, 06번이 그 설계 철학을 QAOA라는 다른 algorithm으로 옮겼으며, 05번이 04번의 fixed-point precision 문제에 이론적 근거를 댔습니다. 04번이 중심이고 05·06은 그와 관련된 연구입니다.
- **07** 은 반대 진영(미국 Kansas 대학, HPC accelerator 계보)에서 온 **대조항**입니다. on-chip resource 대신 외부 DDR을, fixed-point 대신 floating-point를 택해 우리와 정반대의 선택을 합니다 — 그래서 12장에서 선택의 근거를 비교할 때 참고합니다.
- **08·09** 는 algorithm 축의 확장입니다. 08번은 Grover 자체의 이론적 일반화(FPGA 무관), 09번은 Grover를 통신 문제에 응용한 사례입니다.

04번의 최적화 4종 중 기법 1(imaginary part 제거)과 기법 3(matrix multiplication 분해)의 **착상 자체는 선행연구 인용**입니다(각각 Ionescu 2019, Bag 2022). 04번의 실제 기여는 이들을 통합·정리하고 SoC로 만들고 자동 생성 tool(QEX)까지 붙인 데 있습니다. 그런데 04번은 자기가 인용한 그 선행연구와 정량 비교를 하지 않습니다 — 논문을 읽을 때 감안할 점입니다.

덧붙여, 이 여섯 편에는 공통으로 참고한 선행연구가 하나 더 있습니다 — Pilch–Długopolski(2018)의 범용 FPGA quantum emulator입니다. 04·05·06·07번이 **모두** 이 논문을 인용합니다(각각 참고문헌 [18]·[48]·[11]·[41]). 여섯 편의 계보도에는 그리지 않았지만, FPGA quantum emulator라는 분야의 공통 출발점 격이라 알아 둘 값어치가 있습니다(더 읽을거리는 15.9절).

이 계보 밖에도 우리 설계 결정 하나의 직접적인 근거가 되는 논문이 한 편 있습니다. **Boyer·Brassard·Høyer·Tapp(1998)**, ***Tight bounds on quantum searching*** — solution이 몇 개인지 모르는 채로 Grover를 돌리는 표준 절차(이른바 BBHT)의 원전입니다. 우리가 쓰는 schedule, 곧 sampling range를 1에서 시작해 실패할 때마다 growth factor 6/5(1.2×)로 넓히는 방식이 전부 이 논문에서 나왔습니다. 09번 논문의 Algorithm 1·2도 이 절차를 그대로 옮겨 온 것이고 — 옮기는 과정에서 한 줄이 빠졌습니다(15.6절). 그러니 iteration schedule을 인용할 때는 09번이 아니라 1998년 원전을 인용하십시오.

15.3 여덟 개의 대립축

Part 5~8의 각 장은 사실 "설계 질문 하나"였고, 그 질문마다 논문들이 다른 답을 냈습니다. 그 대립을 한 표로 모읍니다. 굵게 표시한 것이 우리 좌표입니다.

축	질문	답들	다룬 곳
A	state vector를 어디에 두는가	register array(04·06, 6~9 qubits) ↔ **on-chip BRAM, row 32 lanes × 4 PEs(우리, 14 qubits)** ↔ 외부 DDR(07, 32 qubits)	11.4 · 15.5
B	수를 어떻게 담는가	**fixed-point(04·05·우리)** ↔ float32(07). minimum fraction bits 공식(05)	12장
C	multiplier를 어떻게 없애는가	**2의 거듭제곱은 shift → diffusion operator DSP 0개(04·우리)** ↔ 임의 각도 CORDIC(06)	13.5 · 13.8
D	measurement를 어떻게 하는가	**Born-rule sampling(04의 WRNG · 우리 — 단 우리는 integer 정확 계산과 group→row→lane layer 선택)** ↔ argmax(우리가 기각) ↔ Q2C 미해결(07)	14.3 · 14.4
E	oracle은 무엇인가	index를 안다(04) ↔ **predicate를 판정한다(09·우리)**	13.3 · 14.2
F	실기 vs emulator 비용	실기에서 싼 것이 emulator에서 비싸고, 그 반대도 참	11.9 · 11.10
G	iteration을 몇 번 돌릴지 어떻게 정하는가	number of solutions를 안다고 두고 $\text{optimal number of iterations} = \frac{\pi}{4}\sqrt{\text{number of states}/\text{number of solutions}}$(04·08·09) ↔ **BBHT로 모르는 채 시작(우리)** ↔ quantum counting(QPE, 우리가 기각)	16.1 · 16.2
H	shot 사이에 state를 버리는가 남기는가	shot마다 버리고 Init부터(여섯 논문 전부) ↔ **state를 세 벌 남겨 두고 policy가 고른 지점에서 차이만큼만 전진(우리)**	16.5 · 16.6

앞의 여섯 축(A~F)은 논문들끼리 갈리는 자리이고, 뒤의 둘은 성격이 다릅니다. G축은 여섯 논문이 대체로 "solution이 몇 개인지는 주어진 것"으로 놓고 지나가 버린 자리라 우리가 직접 세워야 했고, **H축은 여섯 논문 어디에도 없습니다.** shot을 여러 번 수행하는 구조 자체가 G축을 세운 뒤에야 생기고, 실기 quantum에서는 measurement가 state를 collapse시켜 애초에 물음이 성립하지 않기 때문입니다. emulator에서만 물을 수 있는 질문이고, 이 project가 내놓는 답이 checkpoint입니다. board에서 잰 가속의 가장 큰 몫도 이 축에서 나왔습니다.

15.4 우리 project의 좌표 — 확정 설계 결정표

이 표가 해설서 안에서 이 project 숫자의 source of truth입니다. 다른 장이 같은 값을 본문에 다시 적어 두는 것은 읽는 도중에 link를 따라가지 않아도 되게 하려는 배려일 뿐이고, 값이 어긋나면 여기가 기준입니다.

다만 순서를 오해하지 마십시오. **이 장은 뒤 세 장(16·17·18)의 결론을 미리 모은 색인이며, 왜 그렇게 정했는지의 근거는 그쪽에 있습니다.** 표의 각 줄에 있는 "상세" 열이 그 근거를 가리킵니다. 표만 읽고 이유까지 안다고 생각하면, 나중에 설계를 바꿔야 할 때 무엇을 다시 계산해야 하는지 알 수 없게 됩니다.

결정	우리 선택	따른 논문	버린 것	근거	상세
amplitude 표현	**실수 전용**	04	imaginary part array	초기 uniform superposition이 실수 + oracle이 ± 1 + diffusion operator가 실수 선형 → amplitude가 실수를 벗어나지 않음. memory가 절반	9.8 · 11.2
number format	**fixed-point**	04·05	07의 float32	실수 전용이라 동적 범위가 좁고, diffusion operator가 shift·subtraction만으로 닫힘. 지수·normalization circuit이 통째로 불필요	12.1 · 12.9
number of qubits	**14 qubits** (16,384 states)	—	더 많은 qubit	$1/\sqrt{\text{number of states}} = 2^{-7}$ 이 정확. checkpoint slot까지 on-chip에 들어가는 천장이고, 22 fraction bits로 probability $\ell_2 \le 10^{-5}$ 가 되는 크기	11.4 · 12.7
data word width	**23 bits, Q1.22**, round-half-to-even, symmetric saturation(중간값은 25 bits), renormalization 없음	05(방향)	04의 30 bits, 저장 data word의 integer bits 확장	되쓰는 값은 수학적으로 ≤ 1 이라 중간값만 넓히면 됨. fraction bits 충분성은 공식이 아니라 benchmark(전 실행 error $< 10^{-3}$)	12.3 · 12.4 · 12.5 · 12.7
저장	**전부 on-chip BRAM**, Accelerator Core 84 BRAM tiles (amplitude·checkpoint RAMB36 44 · `data_mem` RAMB18 64 · `found_mask` 2 · measurement 2 · policy 4)	04	07의 외부 DDR, register array	14 qubits면 checkpoint slot까지 on-chip에 들어감. DDR latency·controller도, register의 LUT 폭발도 피함	11.4 · 15.5
병렬도	**row당 32 lanes × 4 PEs** = cycle당 128칸. index 하위 5 bits가 lane, row 번호 하위 2 bits가 row phase bank	—	crossbar·permutation network	순차 streaming이라 충돌 0. routing 비용이 붙지 않음	11.7 · 13.7
diffusion operator	$2 \cdot \text{mean} - \text{amp}$**, 0 multipliers**	04	08의 일반화 P_S	number of states가 2의 거듭제곱이라 2×mean을 accumulator의 13칸(number of qubits − 1) arithmetic right shift **한 번**으로 만들고, 나머지는 subtraction. 일반화는 (08번 표기로) $O(N \cdot D)$ 손해	13.5 · 13.8

결정	우리 선택	따른 논문	버린 것	근거	상세
measurement	**Born-rule sampling**, integer 정확 계산, 64-bit random number rejection sampling, group(16) → row(32) → lane(32). Build는 cycle당 2 rows(Dual-Row Build), row 선택은 16×32 layer(Two-Level Select). DSP 128개	04(WRNG)	argmax	solution들의 amplitude가 매 iteration **정확히 같아** argmax는 늘 같은 index 하나만 반환 → enumeration이 원리적으로 불가능	14.3 · 14.4 · 14.5
oracle	**predicate 4종 판정**($<$, $>$, $=$, 범위) + `DATA_COUNT` 밖은 항상 non-solution	09	index hardcoding, comparator IP	solution을 모르는 것이 정상. $\mathrm{value} - \mathrm{thr}$ 의 MSB 하나가 $<$ 의 답이라 subtractor 하나를 4종이 공유	13.3 · 14.2
number of iterations	**BBHT가 기본**(sampling range는 1에서 시작, growth factor 6/5, 28항 ROM, sampling range 상한 128 $= \sqrt{\mathrm{number\ of\ states}}$), 보호 한도는 shot cap 100 · BBHT budget 576	BBHT (1998) ·09	quantum counting(QPE)	solution이 몇 개인지는 predicate·threshold·data에 의존해 사전에 알 수 없음. QPE는 complex number를 강제해 실수 datapath가 무너짐	16.1 · 16.2 · 16.3
shot loop	**BBHT autonomous mode**(hardware 바깥 FSM)가 기본, firmware가 number of iterations를 정하는 **Manual mode** 병존	—	한쪽만 두는 안	autonomous mode는 shot마다의 CSR 왕복을 없애고, Manual mode는 solution이 몇 개인지 아는 조건의 비교와 reference model과의 한 shot 단위 대조에 계속 필요	16.4 · 17.7
checkpoint	**a Checkpoint-Capacity of 3 + 3-shot Lookahead DP policy**, 가장 가까운 앞선 state에서 차이만큼 전진. 500 workloads executed iterations −69.5%	— (우리 고유)	1칸 checkpoint(초판), a Checkpoint-Capacity of 4 and 4-shot Lookahead·a Checkpoint-Capacity of 4 and 8-shot Lookahead, 되감기, `hit[]` caching	Born-rule sampling이 amplitude를 읽기만 하므로 실패한 shot의 state가 그대로 남음. $(D \cdot O)^{\mathrm{number\ of\ iterations}}$ 의 associative law로 결과가 bit 단위로 동일. Checkpoint-Capacity·Lookahead-Shots를 줄이면 결정 비용이 줄어 오히려 빠름	16.5 · 16.6 · 16.8
enumeration mode	**`found_mask` 16,384 × 1-bit**, predicate $\wedge\ \neg\mathrm{found}[i]$, 결과 **FIFO 256칸**(가득 차면 멈춤), BBHT 실패 **3연속**(기본값)으로 종료	—	결과를 host가 걸러내기, 별도 포기 signal	mask를 걸면 solution 집합이 줄어든 **새 문제**를 다시 푸는 것이라 중복이 원천 차단됨	14.7 · 16.7
SoC 형태	**RVX `rvc_orca`** + APB slave(CSR 38개, 4-byte 간격) + AHB master(SINGLE load), host는 UART	04(RISC-V SoC)	MicroBlaze, AXI4-Lite, AXI-Stream, INCR16 burst	제어는 register 몇 개라 APB로 충분하고, data load는 master가 필요해 AHB. RVX에는 AXI-Stream shim이 없음	17.3 · 17.4 · 17.5

결정	우리 선택	따른 논문	버린 것	근거	상세
verification	**단계별 verification**(float64·Qiskit → Q1.22 bit-exact reference model → RTL → board), 합격 기준 **bit-exact**와 BBHT trajectory 일치	05	눈으로 waveform 확인	층을 나눠야 "algorithm이 틀렸나 / 수 표현이 틀렸나 / circuit이 틀렸나"를 분리해 물을 수 있음. reference model과 board가 500/500 일치	18.2 · 18.3

초판이 이 표 밖에 미정으로 남겨 두었던 아홉 가지는 이제 전부 정해졌습니다. 무엇이 어떻게 정해졌는지는 15.8절에 모았습니다.

15.5 확정 platform과 resource 예산

항목	값
board	**Arty A7-100T**, part `xc7a100tcsg324-1` (Artix-7)
board resource	BRAM36 **135개** · DSP **240개** · LUT 63,400
number of qubits	**14 qubits** (16,384 states)
data word	16-bit signed
clock	accelerator **100 MHz**(전용 clock `clk_accel`) / RISC-V core와 bus 50 MHz
SoC · bus	RVX(`rvc_orca` RV32 · `munoc` interconnect), APB slave + AHB master
host	PC — UART — RISC-V app — CSR — accelerator
configuration	**final configuration** — a Checkpoint-Capacity of 3 · 3-shot Lookahead · 4 PEs · Dual-Row Build · Two-Level Select

이전 판의 XC7S100은 폐기합니다. RVX에 `xc7s100` imp class 가 없기 때문이고, 이 판단은 그대로입니다.

resource는 이제 추정이 아니라 **2026-09-07 implementation report의 실제 값**입니다.

부분	LUT	FF	BRAM tile	DSP
Accelerator Core (`u_main_ip`)	32,242	28,223	84	128
Bus Interface · RVX SoC 나머지	13,042	17,000	32	4
board 전체	**45,284 / 63,400**	**45,223 / 126,800**	**116 / 135**	**132 / 240**

Accelerator Core의 84 BRAM tiles는 amplitude·checkpoint memory(RAMB36 44개, state 한 벌이 11개이고 checkpoint slot 네 벌 분량), `data_mem`(RAMB18 64개 = 32 tiles), `found_mask`·measurement row weight·policy memo table(RAMB36 2·2·4개)입니다. SoC 몫 32 tiles는 대부분 system SRAM 128 KB입니다. qubit를 하나 늘리면 state 한 벌이 두 배가 되어 amplitude·checkpoint memory만 88개가 되고 board를 넘깁니다(11.4절).

DSP 사용처는 둘로 갈립니다. diffusion operator·oracle·Init 경로에는 multiplier가 **하나도** 없고, Measurement Unit에만 23×23 squarer(DSP 2개씩)가 32 lanes × 두 벌, 즉 **DSP 128개**가 잡힙니다. synthesis report에 DSP가 잡혔다고 잘못 짠 것이 아닙니다 — Born-rule sampling을 쓰기로 한 순간 정상적으로 예상되는 값입니다. timing은 100 MHz에서 닫혔습니다(WNS +0.126 ns).

시간은 축을 나눠 적어야 합니다. **RTL cycle 축**으로, Grover iteration 한 번이 약 268 cycles(100 MHz에서 2.7 µs)이고, 14 qubits · 1 solution search 한 번이 mean 25,345 cycles(약 0.25 ms)입니다. 매 shot 처음부터 도는 Restart mode는 같은 조건에서 mean 209,277 cycles이었습니다. 500 workloads 전체로는 42,308,335 → 6,890,470 cycles, **6.1401×**입니다. **board wall-clock time 축**으로는 같은 500 workloads가 Restart mode 425,502 µs → 55,798 µs, **7.626×**입니다. 두 축은 BBHT trajectory가 500/500 같지만 cycle 값 자체가 달라, 섞어 speedup을 만들면 안 됩니다.

이 숫자들을 고전 선형 scan과 나란히 놓지 마십시오. `value[]` 를 한 번 훑는 데는 128 rows를 읽는 128 cycles 남짓이면 끝납니다. emulation 비용이 $O\left(\text{number of states} \cdot \sqrt{\text{number of states}/\text{number of solutions}}\right)$ 이라 원리적으로 그렇게 됩니다 — Grover의 $\sqrt{\text{number of states}}$ 이득은 **실기 quantum**의 이야기이고, amplitude array를 전부 훑어야 하는 emulator에는 해당하지 않습니다. 우리 accelerator의 비교 대상은 같은 일을 하는 **Qiskit AerSimulator·NumPy state vector simulator**, 그리고 같은 board 위에서 accelerator를 끄고 RISC-V core 하나로 같은 계산을 돌린 **순수 software baseline**입니다. 마지막 것과의 차이는 wall-clock time으로 116,426×인데, 이는 32 lanes 병렬과 두 배 빠른 clock 덕이지 algorithm 우위가 아닙니다. 값어치는 속도 우위가 아니라 **quantum algorithm의 동작을 결정적이고 재현 가능하게 재현·검증하는 전용 hardware**라는 데 있습니다.

비교의 공정성에 관한 주의도 하나 — 04번의 resource 수치(LUT/FF)는 RISC-V SoC infrastructure를 포함한 값이라, 우리가 accelerator만 따로 재면 훨씬 작게 나옵니다. 게다가 04번 논문 스스로 "Vivado의 cross-optimization 때문에 이론적 역산이 불가능하다"고 경고합니다. 그러니 우리 수치를 04번과 나란히 놓을 때는 **무엇을 포함한 값인지**를 반드시 밝혀야 부당한 비교를 피할 수 있습니다. 위 표가 Accelerator Core와 board 전체를 따로 적은 이유가 이것입니다.

15.6 인용 시 주의 — 논문의 오류와 모호점

여섯 논문을 원문과 한 줄씩 대조한 결과, 전부에 인용할 때 주의해야 할 오류·모호점이 있었습니다. "논문은 옳다"고 믿고 베끼면 circuit이 동작하지 않거나 틀린 값을 인용하게 됩니다. 특히 ★ 표시는 **그대로 옮기면 결과가 틀어지는** 것들입니다.

논문	오류·모호점	인용 시 대응
04	★ pseudocode 오타 다수: `>> NUM_STATE`(→`NUM_QUBIT`), `amp_state` index 누락, `i-1 << k` 괄호(operator 우선순위)	**그대로 짜면 동작 안 함** — 직접 고쳐서
04	★ magic number 2개 LSB 어긋남: `0x2D41_3CCE`(→`…CCD`), `0x1_1C58_31AC`(→`…AE`)	직접 계산해 넣을 것(13.2·16.2)
04	Table 1의 Pauli-Y = $\begin{bmatrix}0 & -i\\ i & 1\end{bmatrix}$ (→ 우하단 0)	3장과 충돌 — 3장이 옳음
04	"2 qubits~6에서 error 2^{-12} 이하"가 Table 3과 모순 (2 qubits는 2^{-4})	본문 서술 인용 금지, Table 3 값 사용
04	"4 qubits $p=3$ → $\epsilon_r = 0.4\%$"가 자기 식과 모순 (실제 0.024%; 0.4%는 $p=2$)	이 수치 인용 금지
04	Table 6 단위 불명("clock cycles"인데 값이 10.21)	비율(191×)만 유효, 절대값 인용 주의
04	Theorem 6 증명이 실제 algorithm(shift) 아닌 나눗셈 version을 증명 — 정리 형식이 장식적	"정리+증명"을 근거로 신뢰하지 말 것
05	12 qubits · 28 fraction bits의 ℓ_2 가 세 곳에서 다름 (7.49 / 7.492 / 7.493 $\times 10^{-6}$)	reference model 목표값을 하나로 지정 후 재현
05	solution amplitude $\alpha_S \approx n_s^{-1}$ (→ $n_s^{-1/2}$ 로 보임)	solution이 여럿인 경우로 확장 시 직접 재유도
05	$r_{\text{num_q}}$ 가 2.04→2.24로 drift하는데 설명 없음	qubit가 더 많은 쪽으로 외삽 시 safety margin 주의
06	★ 식 (6)에서 l 과 (i,j) 의 역할이 뒤바뀌어 인쇄	§IV-B code가 정답지
06	852× vs 854× (초록/서론 불일치), Fig.1 caption $\gamma \leftrightarrow \beta$	두 값 병기
07	식 (21) `2n`을 `2^n`으로 오독하기 쉬움; `n_{d-1}`은 아래첨자(multiplication 아님)	지수 아님에 주의
07	Fig. 8의 (b)·(c)가 caption과 뒤바뀌어 배치	그림·caption 대조 확인
07	"32 qubits가 memory 천장"은 논문에 없는 추정	논문 주장으로 인용 금지
08	**초록은 $\alpha \approx 1$, 본문은 $\alpha \approx 0.9$**	반드시 병기
08	식 (6) $\delta_{nn'}$ 이 원문엔 δ_{nn} 으로 인쇄 (논문 오타로 판단)	주석 달고 인용
09	★ 식 (9)(12)의 sign 규약이 본문과 어긋남 — 그대로면 **minimum 아닌 maximum**	sign은 우리가 정하고 simulation으로 verification(13.3)
09	★ Algorithm 2(DHA)에 sampling range 갱신 줄 누락 (BBHT에만 있음)	본문 기준으로 sampling range 증가 직접 추가(16.2)
09	N(사용자 수)과 $K = 2^N$(search space) 혼동 위험	14.0절 표기 통일 규약 적용

이 목록에서 확인하듯 **1차 자료도 틀립니다** — 그러니 hardware로 옮기기 전에 반드시 원문을 의심하고, 수치는 직접 계산해 보고, sign은 simulation으로 확인하라는 것입니다. 예를 들어 04번의 pseudocode를 그대로 옮기면 circuit이 동작하지 않습니다.

15.7 hardware 용어집

hardware 쪽에서 처음 나온 용어들을 모았습니다(quantum 이론 용어는 10.5절에 있습니다).

용어	뜻	장
emulation	전용 hardware(FPGA)로 state vector 계산을 가속하는 것. software simulation과 대비	11장
LUT	look-up table. FPGA의 조합 logic resource. **연산 logic**이 여기 잡힘	11.5
FF	flip-flop. FPGA의 순차(저장) resource. **register 저장**이 여기 잡힘	11.5
BRAM	block RAM. FPGA 안의 block memory. port 수만큼만 동시에 접근. RAMB36 port 하나는 최대 72-bit 폭	11.4
DSP	전용 multiplier block. 우리는 Measurement Unit에만 128개를 씀	13.8
row · lane	연속한 index 32개가 row 하나, 그 안의 한 칸이 lane. index 하위 5 bits가 lane	11.7
banking	array를 여러 memory로 쪼개 한 cycle에 여러 칸을 읽는 것. 우리는 row 번호 하위 2 bits로 4 banks 중 하나를 고름	11.7
PE · number of PEs	Iteration Datapath에서 한 cycle에 row 하나를 처리하는 circuit 한 벌과 그 개수. final configuration은 4 PEs	13.1
Dual-Row Build · Two-Level Select	Measurement Unit에서 한 cycle에 2 rows의 row weight를 만드는 Build와, 16 groups → 32 rows 두 단으로 고르는 Select. final configuration의 Measurement Microarchitecture	14.5
memory-bound	속도가 산술이 아니라 memory bandwidth · port에 묶인 state	11.6
fixed-point	소수점 위치를 고정해 integer처럼 다루는 수 표현	12.2
Q format	`Q<integer bits>.<fraction bits>`. sign은 integer bits에 포함 → 우리는 Q1.22	12.2
round-half-to-even	정확히 0.5인 값을 짝수 쪽으로 보내 rounding 편향을 없애는 규칙	12.4
saturation	표현 범위를 넘으면 wraparound 대신 maximum · minimum에 붙이는 처리. 우리는 symmetric saturation	12.5
truncation error	fraction bits가 유한해 아래 bit가 잘려 나가며 생기는 error	12.6
FSM	finite-state machine. state를 순서대로 밟으며 제어하는 circuit. emulator의 골격	13.1
datapath	data가 흐르며 연산되는 경로(adder · shifter · comparator 등)	13장
pipeline	연산을 여러 단으로 나눠 매 cycle 하나씩 흘려 throughput을 높이는 구조	13.7
CORDIC	shift와 addition만으로 임의 각도 회전을 계산하는 algorithm. 우리는 쓰지 않음	13.8
Born-rule sampling	amplitude square에 비례하는 probability로 index 하나를 뽑는 measurement. 우리 방식	14.4
prefix sum	앞에서부터 차례로 더한 partial sum. 우리 Measurement Unit은 row weight의 prefix sum을 32 rows마다 group prefix sum으로 적어 두고 group → row → lane 순으로 좁혀 감	14.5
rejection sampling	넓은 범위에서 뽑아 원하는 범위 밖이면 버리고 다시 뽑아 정확한 균등분포를 얻는 방법. number of iterations를 뽑을 때와 Measurement PRNG 둘 다 씀	14.4 · 16.2
argmax	array에서 maximum의 **index**를 찾는 것. amplitude 동률 때문에 **기각한** measurement 방식	14.3
weighted random number generator	probability distribution을 따라 뽑는 random number 발생기(04번 논문의 measurement). 우리와 같은 방식	14.4
LFSR	linear-feedback shift register. shift와 XOR만으로 만드는 의사난수 발생기. Iteration-Count LFSR이 씀	16.2
BBHT	Boyer–Brassard–Høyer–Tapp. number of solutions를 모를 때 쓰는 iteration 스케줄(sampling range를 growth factor 6/5씩 키움)	16.2
checkpoint	실패한 shot의 amplitude state를 몇 벌 남겨 두었다가 가장 가까운 것에서 이어 도는 기법. 우리는 a Checkpoint-Capacity of 3	16.5
policy	앞으로 올 request 몇 개를 보고 checkpoint에 무엇을 남길지 정하는 규칙. Policy Engine이 계산. 우리는 3-shot Lookahead	16.5
SoC	system-on-chip. CPU · memory · 주변장치 · accelerator를 한 chip에 담은 system	17.2

용어	뜻	장
CSR	control/status register. software가 읽고 쓰는 제어 · status register. APB로 접근	17.4
APB · AHB	저속 · 단순 bus와, master를 둘 수 있는 고속 bus. 제어는 APB, data load는 AHB	17.3
bit-exact	두 implementation의 출력이 bit 하나까지 같음. reference model과 RTL의 합격 기준	18.3
classical-to-quantum loading	고전 data를 quantum amplitude로 싣는 initialization. 07번 논문의 절반	11.9

15.8 되돌아보며

여기까지 이론을 **hardware로 구현**하는 과정을 살펴보았습니다. state vector가 array가 되고(11장), amplitude가 Q1.22 fixed-point 숫자가 되고(12장), oracle과 diffusion operator가 multiplier 없는 datapath가 되고(13장), measurement가 probability를 뽑는 circuit이 되어 enumeration까지 열린(14장) 과정을 따라왔습니다. 각 과정에서 여섯 논문의 선택을 비교했습니다.

이제 여러분은 04번 논문을 "설계도"로 읽을 수 있고, 07번의 반대 선택이 왜 그럴듯한지 판단할 수 있으며, 논문의 수치를 그대로 믿지 않고 검증할 수 있습니다. 그리고 15.4절의 표는 이제 계획이 아니라 **board에서 동작을 확인한 설계의 기록**입니다.

초판은 아홉 가지를 **미정**으로 열어 두었습니다. RTL을 짓고 board에 올리면서 전부 정해졌고, 어떻게 정해졌는지를 그대로 적어 둡니다. 앞의 넷은 16.9절이, 뒤의 다섯은 17.8절이 자세히 다룹니다.

1. **solution이 없을 때의 종료 조건** — array를 고전적으로 훑어 확정하지 않고, shot cap 100과 BBHT budget 576 중 먼저 닿는 쪽으로 끝냅니다. enumeration mode는 이 "실패로 끝난 BBHT"가 연속 3번(기본값)이면 끝냅니다.
2. $\text{number of iterations} \sim U[0,\ \text{sampling range})$ **의 균등성** — LFSR 후보를 경계 이상이면 버리고 다시 뽑는 rejection sampling으로 정확한 균등을 얻습니다.
3. **random number 소비 규약** — number of iterations 쪽은 32-bit Iteration-Count LFSR로 후보마다 한 번(버린 후보 포함) 추첨하고, measurement 쪽은 독립인 Measurement PRNG(64-bit xorshift)로 rejection sampling 후보마다 64-bit block 하나를 씁니다. group → row → lane 선택은 한 random number의 잔여를 이어 씁니다. reference model과 RTL이 이 규약을 똑같이 따르고, 500 workloads 전부에서 BBHT trajectory가 같습니다.
4. $\text{sampling range} \leftarrow \min(1.2 \cdot \text{sampling range},\ \sqrt{\text{number of states}})$ **의 hardware 산술** — 28항 ROM입니다. 14 qubits에서 $\sqrt{\text{number of states}} = 128$ 이 정확한 integer라 clamp가 깔끔합니다.
5. **enumeration 결과 반환 경로** — accelerator 안의 Result FIFO 256칸. 가득 차면 hardware가 멈추고 firmware가 돌고 있는 동안에도 뽑습니다.
6. `M_max` **값** — 별도의 포기 signal을 두지 않았습니다. 상한은 FIFO 깊이가 사실상 정하고, 그 이상은 firmware가 비우는 속도의 문제입니다.
7. **RVX SoC의 실제 resource 점유** — implementation report로 실측했습니다. SoC 몫은 32 BRAM tiles이고, board 전체가 116 / 135입니다.
8. **CSR map** — 32-bit register 38개, 4-byte 간격으로 확정했습니다.
9. **실제 clock** — accelerator 전용 100 MHz clock을 두고, RISC-V core와 bus는 50 MHz로 둡니다. 두 clock domain 사이의 경계 처리는 RVX가 생성하는 비동기 bridge가 맡습니다.

남은 큰 결정은 하나입니다. 이 해설서가 설명한 BRAM Variant(checkpoint 방식)와, number of iterations마다의 amplitude를 DRAM에 전량 저장해 두는 DRAM Variant 가운데 무엇을 최종으로 할지입니다. DRAM Variant는 RTL 초안과 regression까지 있고 물리 DRAM 연결과 board verification이 남아 있습니다(16.8절).

원문을 한 편씩 처음부터 끝까지 읽고 싶다면, 각 논문의 한국어 해설본이 `../papers/papers_ko/`에 있습니다(원문 대조용). 이 해설서(study_references)가 **주교재**이고, 그쪽은 원문을 옆에 두고 볼 때의 보조 자료입니다.

15.9 여섯 논문 너머 — 더 읽을거리

이 해설서의 중심 자료는 여섯 편이지만 같은 문제(quantum algorithm을 고전 hardware로 흉내 내기)를 다른 각도에서 다룬 논문이 많습니다. 함께 참고한, 여섯 편 밖의 자료들을 여기 모읍니다.

먼저 전체적인 분류를 보면, 지금까지 당연한 듯 쓴 "state vector를 array로 펼쳐 계산"하는 방식은 사실 **유일한 길이 아니라 세 갈래 중 하나**입니다. Cicero 외(2025) survey는 algorithm 수준 simulation을 **Schrödinger식(state vector)·Feynman식(path sum)·tensor network** 으로 나눕니

다. 우리 Grover emulator는 전형적인 Schrödinger식으로, 셋 중 memory를 가장 많이 쓰지만($2^{\text{number of qubits}}$ amplitudes를 전부 저장) implementation이 가장 단순합니다. tensor network는 qubit가 많고 circuit이 얕을 때 소수의 출력 amplitude만 뽑아 훨씬 효율적이지만, Grover는 iteration마다 amplitude 전체를 건드리는 깊은 circuit이라 tensor network의 이점이 약해집니다 — 즉 Grover에 state vector 방식을 쓰는 데에는 이런 근거가 있습니다.

논문	무엇을 더하나	본문 연결
Boyer, Brassard, Høyer, Tapp (1998) — Tight bounds on quantum searching	**BBHT의 원전.** solution이 몇 개인지 모를 때의 iteration schedule(growth factor 6/5)과 success probability의 lower bound가 여기서 나옴	15.2 · 16.2
Pilch & Długopolski (2018) — FPGA quantum emulator	04·05·06·07이 공통 인용하는 **baseline**. qubit=지수 hardware의 benchmark(2 qubits 8k → 3 qubits 25k ALM). hardware von Neumann measurement	15.2 · 14.6
Frank 외 (2009) — SEQCSim	저장 대신 **재계산**으로 memory를 $O(s+t)$ 로 (공간–시간 맞바꿈). 단 Grover엔 자멸적	11.3
Tiwari 외 (2026) — 30 qubits HBM simulator	memory-bound의 다음 수 = **HBM**. bank interleaving + BRAM→HBM 성능 절벽 benchmark	11.6
Belfore (2024) — Scalable FPGA Architecture	병렬화의 **제3비용 = routing**(Beneš permutation network가 logic 36%). 순차 streaming인 우리는 회피	11.7
Cicero 외 (2025) — simulation survey	방식 3분류(Schrödinger/Feynman/tensor network), FPGA의 두 역할(emulation↔acceleration)	11장 · 위 문단
Häner 외 (2016) — High-Performance Emulation	"emulation vs simulation" 용어의 원전. 단 이들의 "emulation"은 QFT→FFT 같은 **software 지름길** 뜻이라 우리(state vector를 hardware에 펼침)와 다른 갈래	—
AMARETTO (2024) — 저가 FPGA emulator	**algorithm을 RTL에 고정(04·우리)** vs **gate를 명령어로 실행**(butterfly amplitude pair 선택, KV260에서 16 qubits)의 대조	—

이 논문들의 원문 PDF는 `../papers/5. 추가 수집 (OpenAlex)/`에, 더 넓은 목록은 `../papers/references.md`에 있습니다. 다만 이들은 여섯 편처럼 정독·검증한 자료가 **아니라 보조 참고 자료**이니 인용하기 전에는 원문을 직접 확인하세요.

15.10 이 장의 요약

- 여섯 논문의 지형은 **04번이 중심**입니다. 05·06은 그 위성이고, 07은 반대 진영의 대조군, 08은 이론 배경, 09는 oracle을 predicate 판정으로 넓힌 **확장 참고 사례**입니다. iteration schedule의 근거는 09번이 아니라 BBHT 원전(1998)입니다.
- 논문들이 갈라지는 자리를 여덟 개의 축으로 정리했고, 그중 **H축("shot 사이에 state를 버리는가 남기는가")은 여섯 논문 어디에도 없는 우리 고유의 축**입니다. 우리 답은 a Checkpoint-Capacity of 3과 3-shot Lookahead policy입니다.
- 15.4절이 해설서 안에서 **이 project 숫자의 source of truth**입니다. board에서 검증된 final configuration을 따르고, 근거는 표의 각 줄이 가리키는 뒤 장들에 있습니다.
- platform은 **Arty A7-100T · 14 qubits · accelerator 100 MHz** 입니다. Accelerator Core가 84 BRAM tiles · 128 DSPs(전부 Measurement Unit), board 전체가 BRAM 116 / 135 · DSP 132 / 240이고 timing은 닫혔습니다. 성능은 RTL cycle 6.1401×와 board wall-clock time 7.626×로 **축을 나눠** 적습니다.
- 15.6절은 인용 전에 반드시 펼쳐 볼 정오표입니다. **1차 자료도 틀립니다.**
- 초판의 미정 아홉 가지는 **전부 정해졌습니다**(15.8절). 남은 큰 결정은 BRAM Variant와 DRAM Variant 중 하나를 고르는 일입니다.

16장. 몇 번 돌릴지 모를 때 — BBHT shot loop와 checkpoint

> **이 장을 읽기 위한 준비**: 4장 measurement, 6.6절 circuit의 순효과는 하나의 unitary, 9.6절 지나치면 안 된다, 13장 oracle과 diffusion operator의 안쪽 FSM(13.1절), 14장 multiple solutions.

13장은 한 번의 initialization에서 시작해 iteration을 몇 바퀴 돌고 measurement로 끝나는 **한 shot** 안을 들여다봤고, 14장은 그 shot의 마지막 단계인 measurement를 확률적으로 만들었습니다. 그런데 13장은 내내 "number of iterations는 정해져 있다"고 가정했습니다. 이 장은 그 가정을 풉니다.

optimal number of iterations를 계산하려면 number of solutions가 필요한데, 우리 accelerator는 solution이 몇 개인지 모릅니다. 모르면 shot 하나로 끝나지 않고 **여러 번 실행해야** 하고 그러면 곧바로 다음 질문이 따라옵니다 — 실패한 shot이 남긴 amplitude array를 그냥 버릴 것인가? 이 장의 후반부는 그 array를 버리지 않고 몇 벌 남겨 두었다가 이어 돌리는 **checkpoint**를 다룹니다. 이 점이 이 project와 기존 Grover emulator들의 차이이며 board에서 잰 가속의 가장 큰 몫이 여기서 나옵니다.

16.1 왜 solution이 몇 개인지 모르는가

우리 accelerator가 푸는 문제는 "`data_mem` 에 담긴 16,384개 값 중 predicate를 만족하는 index를 찾아라"입니다. predicate는 `value < thr_a` 같은 부등식이고, threshold는 host가 CSR로 넘겨줍니다. 그러면 number of solutions는 무엇에 달려 있을까요 — **predicate의 종류, threshold, 그리고 그 순간 memory에 실려 있는 data**입니다. 셋 다 실행 시점에 정해지므로, synthesis 시점에는 물론이고 시작 명령을 쓰는 순간에도 solution이 몇 개인지 알 수 없습니다.

number of solutions를 모르면 optimal number of iterations를 계산할 수 없습니다:

$$\text{optimal number of iterations} = \frac{\pi}{4}\sqrt{\frac{\text{number of states}}{\text{number of solutions}}} - \frac{1}{2}$$

number of solutions가 분모 안에 있으니까요(14 qubits · 1 solution이면 optimal number of iterations는 $100.03 \to 100$; 값의 source of truth는 15.4절). 그렇다고 "모르니까 넉넉히 많이 돌리자"가 통하지도 않습니다. 9.6절에서 확인했듯 Grover는 최적점을 지나치면 success probability가 **다시 떨어지고 진동**하기 때문입니다. 많이 돌리는 것이 안전한 쪽이 아니라는 점이 이 문제를 어렵게 만듭니다.

quantum algorithm에는 이 문제를 다루는 표준 방법이 있습니다. **quantum counting** — Grover iteration의 eigenvalue에 담긴 각 θ 를 phase estimation(QPE)으로 읽어 내 solution이 몇 개인지 추정하는 방법입니다. 우리는 이것을 설계에서 제외했습니다. 이유는 성능이 아니라 **수 표현**입니다. QPE는 inverse quantum Fourier transform을 쓰고, QFT는 $e^{2\pi ik/2^t}$ 형태의 complex phase를 곱합니다. 9.8절이 정리한 "표준 Grover는 amplitude가 실수를 벗어나지 않는다"는 성질이 여기서 깨지고, 그러면 imaginary part array가 하나 더 생겨 amplitude memory가 두 배가 되며, 12장에서 세운 실수 전용 23-bit Q1.22 datapath와 13장의 multiplier 없는 산술을 그대로 쓸 수 없게 됩니다. counting을 위해 accelerator 전체를 complex number로 바꾸는 것은 비용에 비해 이득이 작습니다.

여기서 구분해 둘 점이 있습니다. **emulator에서는 solution을 그냥 세면 됩니다.** `data_mem` 을 한 번 훑으면 128 cycles이고, 이는 shot 하나에 드는 시간보다도 짧습니다. 순수 속도만 놓고 보면 세어 놓고 optimal number of iterations를 딱 계산해 도는 쪽이 BBHT보다 빠릅니다. 그럼에도 BBHT를 택하는 이유는 속도가 아니라 **실기 충실도**입니다. 진짜 quantum computer는 amplitude array를 들여다볼 수 없고, oracle을 호출해 보는 것 말고는 solution이 몇 개인지 알 방법이 없습니다. solution을 미리 세어 놓고 시작하는 것은 amplitude array를 직접 읽는 것과 마찬가지로 실제 장치에서 얻을 수 없는 정보를 쓰는 일입니다. 그렇게 만든 기계는 Grover의 외형만 따를 뿐 Grover의 동작을 충실히 재현하지 못합니다. 이 accelerator의 값어치는 고전 code보다 빠른 데 있지 않고 **quantum algorithm의 동작을 결정적이고 재현 가능하게 재현·검증하는 전용 hardware**라는 데 있으므로, 속도보다 충실도를 우선합니다.

다만 number of iterations를 바깥에서 정해 주는 **Manual mode**는 남겨 둡니다(`MANUAL_SINGLE`). firmware가 number of iterations를 직접 적어 한 shot만 돌리는 mode라, solution이 몇 개인지 아는 조건에서 optimal number of iterations로 딱 맞게 도는 비교 실험과 software reference model과의 한 shot 단위 대조에 씁니다.

16.2 BBHT — 무작위로 뽑는 number of iterations와 1.2× growth factor

solution이 몇 개인지 모를 때 쓰는 표준 절차가 BBHT(Boyer–Brassard–Høyer–Tapp)입니다. 네 줄이면 끝납니다.

1. sampling range를 1로 두고 시작한다.
2. shot마다 number of iterations를 $[0,\ \text{sampling range})$ 에서 **무작위로** 뽑는다.
3. 뽑은 횟수만큼 iteration을 돌고 측정한 뒤, 나온 후보로 solution check를 한다.
4. 실패하면 $\text{sampling range} \leftarrow \min(\text{growth factor} \cdot \text{sampling range},\ \sqrt{\text{number of states}})$ 로 sampling range를 넓히고 2로 돌아간다. growth factor는 6/5다.

학부생이 가장 먼저 걸리는 지점은 2번입니다 — **왜 number of iterations를 무작위로 뽑는가?** 그냥 sampling range − 1 처럼 가장 큰 값을 쓰거나, 매번 1씩 늘리면 안 되나요?

success probability의 모양을 보면 이유를 알 수 있습니다. iteration을 돈 뒤 solution이 나올 probability는

$$\text{success probability} = \sin^2\big((2 \cdot \text{number of iterations} + 1)\,\theta\big), \qquad \sin\theta = \sqrt{\text{number of solutions}/\text{number of states}}$$

로, number of iterations에 대해 **진동**합니다. 14 qubits · 1 solution이면 $\theta = 0.4476^\circ$ 이고 optimal number of iterations는 100인데, number of iterations를 바꿔 가며 재 보면 이렇습니다:

number of iterations	0	1	10	25	50	75	100	125	127	150	175	200
success probability	0.0 %	0.1 %	2.7 %	15.1 %	50.4 %	85.5 %	100.0 %	85.5 %	83.3 %	50.5 %	15.1 %	0.0 %

100 iterations 근처는 100%지만 200 iterations는 **0%** 입니다. 최적점을 두 배쯤 지나치면 probability가 거의 0인 골짜기로 떨어진다는 뜻이죠. 만약 number of iterations를 고정한다면, 하필 그 값이 골짜기에 놓였을 때 그 shot은 몇 번을 반복해도 실패합니다. sampling range 안에서 무작위로 뽑으면 한 shot의 success probability가 골짜기 하나에 걸리지 않고 그 interval의 **mean**이 되고, BBHT의 핵심 lemma는 sampling range가 충분히 커진 뒤 이 mean이 $1/4$ 아래로 내려가지 않음을 보장합니다. number of iterations를 무작위로 뽑는 것은 편의가 아니라 algorithm의 정당성을 보장하는 요소입니다.

4번의 growth factor 6/5도 임의의 숫자가 아닙니다. growth factor가 1보다 크고 $4/3$ 보다 작으면 총 비용의 상한이 보장된다는 것이 BBHT의 결과이고, $6/5$ 는 그 interval 안의 관습적 선택입니다. sampling range를 너무 천천히 넓히면 shot을 더 많이 실행해야 하고 너무 빨리 넓히면 최적점을 건너뛴 큰 number of iterations만 뽑히기 때문에 양쪽 다 손해입니다. 상한 $\sqrt{\text{number of states}}$ 는 solution이 하나일 때 optimal number of iterations가 대략 $\frac{\pi}{4}\sqrt{\text{number of states}}$ 라는 데서 옵니다 — 그보다 넓은 sampling range는 골짜기만 더 많이 담을 뿐입니다. 우리 14 qubits에서는 $\sqrt{\text{number of states}} = 128$ 이 **정확히** integer라, number of iterations는 7-bit $[0, 128)$ 안에서만 뽑힙니다. 위 표의 200 iterations 골짜기는 hardware가 아예 요청하지 못하는 자리이고, sampling range의 끝인 127 iterations에서도 success probability가 83%를 넘습니다. 이렇게 상한이 overshoot를 막습니다.

hardware에서 1.2×를 매번 곱하지는 않습니다. sampling range의 수열을 미리 계산해 **28항 ROM**에 저장해 둡니다 — $\lceil 1.2^k \rceil$ 을 128에서 자른 값으로, $1, 2, 2, 2, 3, 3, 3, 4, 5, 6, 7, 8, 9, 11, 13, 16, 19, 23, 27, 32, 39, 47, 56, 67, 80, 96, 115, 128$ 이고 그 뒤로는 계속 128입니다. number of iterations는 이 경계 안에서 **정확히 균등하게** 뽑습니다. 32-bit Iteration-Count LFSR의 하위 bit를 경계 이상의 가장 작은 2의 거듭제곱 폭으로 잘라 후보로 쓰고, 후보가 경계 이상이면 버리고 다시 뽑는 rejection sampling입니다. 후보를 하나 뽑을 때마다 LFSR을 7 steps씩 전진시켜, 이어 뽑는 후보들이 같은 bit 창을 겹쳐 쓰지 않게 합니다.

16.3 shot 통계 — mean이 20 shots를 넘는 이유

"한 shot success probability가 $1/4$ 이상이니 mean 네 번쯤 던지면 되겠네"는 이 장에서 가장 흔한 오해입니다. 실제 수치는 이렇습니다. board benchmark와 같은 500 workloads(1 · 4 · 16 · 64 · 256 solutions마다 100 seeds)를 RTL simulation으로 돌린 Six-Stage Ablation campaign의 mean입니다(값의 source of truth는 15.4절).

number of solutions	mean shots	requested iterations	executed iterations (Restart mode)	executed iterations (a Checkpoint-Capacity of 3 and 3-shot Lookahead)	줄어든 몫	mean cycles (Stage 1 → final configuration)
1	**22.7**	173.5	173.5	51.9	70%	209,277 → 25,345
4	18.5	78.9	78.9	23.9	70%	106,128 → 16,368
16	15.1	40.0	40.0	12.1	70%	61,458 → 11,993
64	10.6	15.9	15.9	5.5	65%	30,498 → 8,590
256	7.2	6.0	6.0	2.5	59%	15,723 → 6,610

여기서 executed iterations는 datapath가 실제로 돈 Grover iteration의 개수입니다. 매 shot 처음부터 도는 Restart mode에서는 requested iterations와 같고, checkpoint를 쓰면 줄어듭니다(16.5절). number of shots와 requested iterations는 두 mode에서 같습니다 — 같은 seed면 같은 number of iterations 수열을 요청하기 때문입니다(16.6절).

mean이 22.7 shots까지 올라가는 이유는 sampling range의 성장 곡선에 있습니다. sampling range는 1에서 출발해 1.2×씩 커지므로 처음 열 shot쯤은 아주 좁습니다 — 16.2절의 ROM 수열 그대로 $1, 2, 2, 2, 3, 3, 3, 4, 5, 6, \ldots$ 입니다. 그 동안 number of iterations는 0에서 5 사이에서만 뽑히는데, 위 표에서 10 iterations일 때조차 success probability가 2.7%였습니다. 즉 **초반 shot은 거의 전부 실패하도록 설계되어 있습니다.** 실제로 답을 얻기 쉬워지는 것은 number of iterations가 75~100 근처(success probability 85~100%)에 닿을 만큼 sampling range가 커진 뒤이고, ROM 수열에서 그 자리가 스물다섯 번째 항 안팎입니다. 관측된 mean 22.7 shots(median 23)이 정확히 그 지점입니다.

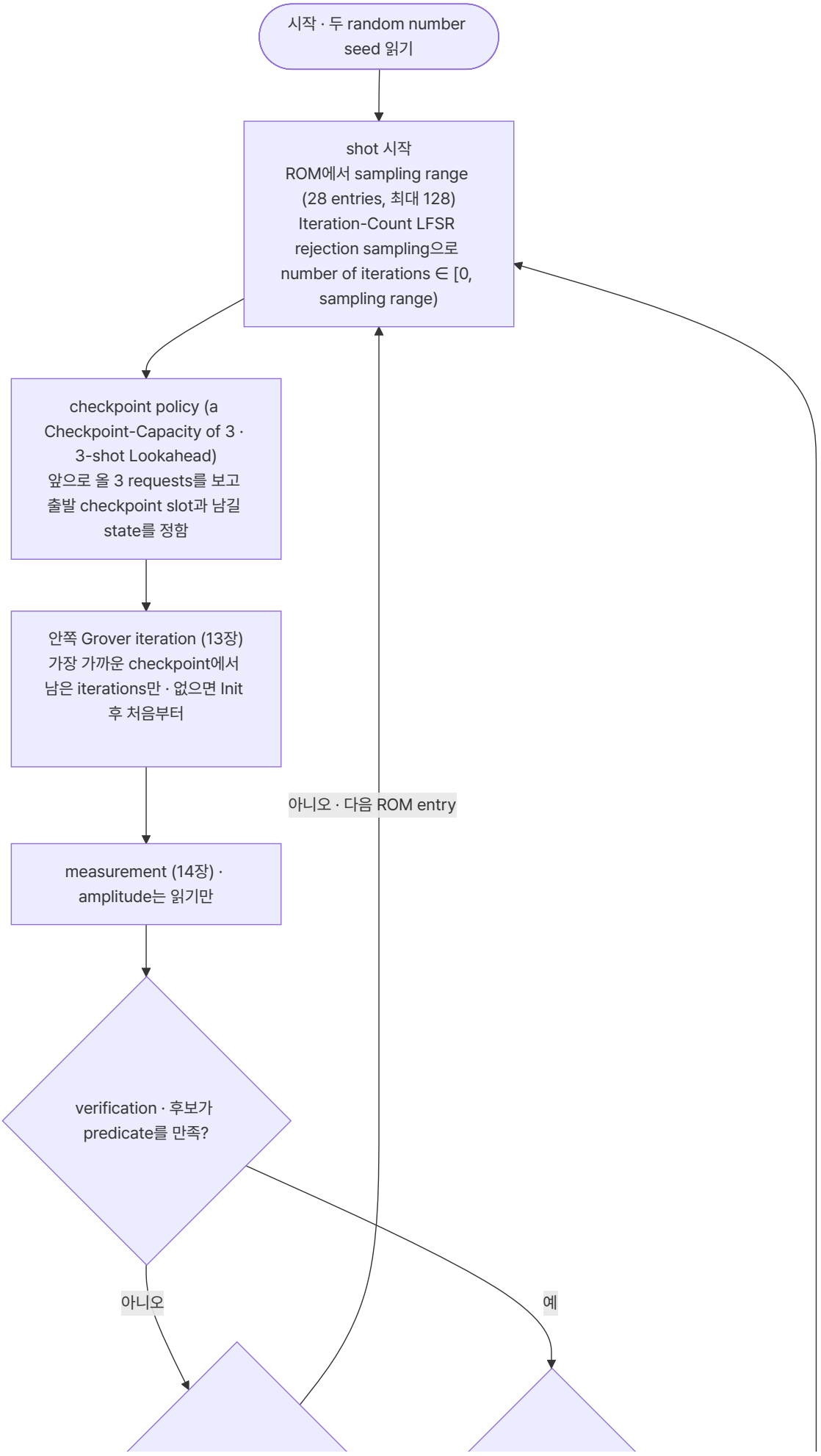
BBHT의 "$1/4$ 이상" 보장은 **sampling range가 충분히 커진 뒤의 lower bound**이지 첫 shot부터 성립하는 값이 아니라는 것 — 이 구분이 4~5 shots와 22.7 shots를 가릅니다.

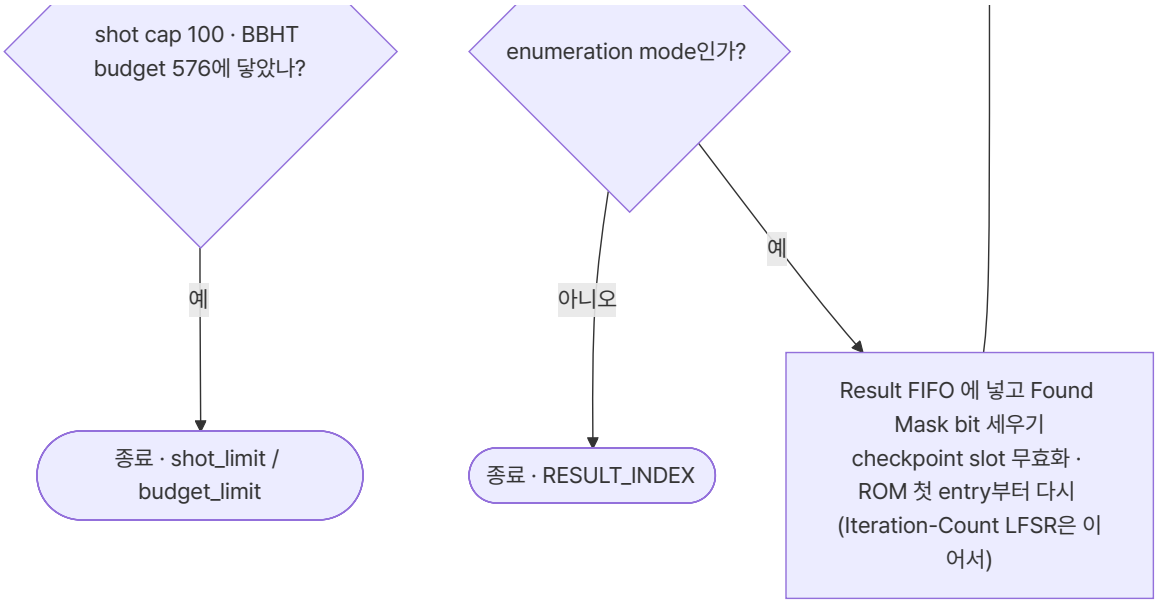
이 사실이 곧바로 **measurement를 싸게 만들어야 하는 동기**가 됩니다. shot이 22.7번이면 measurement도 22.7번이니까요. 1 solution에서 final configuration의 mean 25,345 cycles을 11.8절의 직선으로 쪼개 보면, executed iterations가 $51.9 \times 268 \approx 13{,}900$ cycles, shot마다의 measurement·verification이 $22.7 \times 354 \approx 8{,}000$ cycles, 나머지 고정비가 3,400 cycles 남짓입니다. checkpoint가 iteration을 70% 줄이고 나니 measurement가 전체의 3분의 1을 차지하게 된 것이고, 그래서 Measurement Unit을 줄인 Dual-Row Build와 Two-Level Select가 총 cycle을 각각 18% 넘게 더 줄였습니다(14.5절). **checkpoint와 measurement 최적화는 따로 고를 수 있는 option이 아니라 한 쌍입니다** — 한쪽을 줄이면 다른 쪽이 병목으로 드러납니다.

shot이 끝없이 이어지지 않도록 보호 한도가 둘 있습니다. number of shots가 shot cap(`SHOT_CAP`, 기본 100)에 닿거나, 실패한 shot 뒤에 $1 + \text{requested iterations}$ 가 **BBHT budget 576**에 닿으면 search를 끝내고 각각 `shot_limit` · `budget_limit` status bit를 세웁니다. solution이 하나도 없는 search는 이 둘 중 하나로 끝납니다.

16.4 바깥 FSM — shot loop와 solution check

지금까지의 이야기를 circuit으로 옮기면 13장의 FSM 위에 **바깥 loop**가 한 겹 얹힌 2 layer 구조가 됩니다. 아래 그림은 sampling range 갱신과 number of iterations sampling까지 FSM 안에 들어간 **BBHT autonomous mode 기준**입니다.


시작 · 두 random number seed 읽기
shot 시작
ROM에서 sampling range (28 entries, 최대 128)
Iteration-Count LFSR rejection sampling으로 number of iterations ∈ [0, sampling range)
checkpoint policy (a Checkpoint-Capacity of 3 · 3-shot Lookahead)
앞으로 올 3 requests를 보고 출발 checkpoint slot과 남길 state를 정함
안쪽 Grover iteration (13장)
가장 가까운 checkpoint에서 남은 iterations만 · 없으면 Init 후 처음부터
measurement (14장) · amplitude는 읽기만
verification · 후보가 predicate를 만족?
아니오 · 다음 ROM entry
아니오
예



그림에 보이듯 BBHT autonomous mode의 바깥 shot loop가 하는 일은 넷입니다. **ROM에서 sampling range 경계를 꺼내고, LFSR rejection sampling으로 number of iterations를 추첨하고, checkpoint policy에 물어 어느 checkpoint slot에서 출발해 어디까지 갈지 정하고**, 안쪽 iteration이 끝나면 **measurement 후보를 받아 solution check**를 합니다. 안쪽 loop는 13장에서 만든 것 그대로 — `S_PASS1`(predicate 판정·sign 반전·accumulation) → `S_MEAN` → `S_PASS2`(diffusion operator) 를 필요한 만큼 — 이고, 여기에는 손댈 것이 없습니다. oracle이 독립된 state로 보이지 않는 것은 13.1절이 oracle을 Pass 1에 흡수했기 때문입니다.

새로 등장하는 단계는 **solution check**입니다. measurement가 내놓은 후보 index `cand` 하나로 `data_mem[cand]` 를 읽어 predicate를 다시 판정하는(padding과 enumeration mask까지 포함해) 짧은 고전 확인입니다. 왜 필요한가 — Born-rule sampling은 확률적이고 success probability가 100%가 아니므로 **non-solution을 내놓을 수 있기** 때문입니다. 이 한 칸 확인이 있으면 accelerator의 출력이 "틀릴 수도 있는 확률적 답"에서 "맞다고 확인된 답, 또는 아직 못 찾음"으로 바뀝니다. 이것이 BBHT 4번의 "실패하면"을 circuit이 판정하는 방식이기도 합니다.

가장 바깥에는 **enumeration round loop**가 있습니다. solution check를 통과한 후보가 나왔을 때 enumeration mode라면 답 하나를 Result FIFO에 넣고 끝내는 대신, 그 index의 `found` bit를 세우고 BBHT를 ROM 첫 항부터 다시 시작합니다(14.7절). mask가 바뀌면 문제가 바뀐 것이므로 checkpoint도 무효가 됩니다(16.7절). 이때 Iteration-Count LFSR의 random number stream은 다시 seed를 읽지 않고 **이어서** 흐릅니다 — 같은 number of iterations 수열을 되풀이하지 않게 하려는 것입니다. seed는 host가 시작 명령을 쓸 때만 다시 읽힙니다.

hardware 비용은 크지 않습니다. 바깥 Shot Controller와 Iteration-Count LFSR을 합쳐도 implementation report에서 LUT 359개입니다. 비싼 것은 shot loop가 아니라 16.5절의 policy circuit입니다.

shot loop를 hardware가 돈다고 해서 firmware가 shot을 직접 모는 길이 없어진 것은 아닙니다. 16.1절의 Manual mode에서는 firmware가 number of iterations를 CSR에 적고 시작 명령을 쓸 때마다 **한 shot**만 돕니다. 이 mode에서는 checkpoint가 걸리지 않습니다(checkpoint는 autonomous mode와 checkpoint bit가 둘 다 켜졌을 때만 동작합니다). autonomous mode에서 firmware가 보는 것은 설정 · 시작 · polling · 결과 읽기 네 줄뿐이고, shot 사이의 state는 전부 hardware가 관리합니다.

16.5 checkpoint — measurement가 읽기 전용이라는 사실에서

22.7번의 shot 가운데 스물몇 번은 실패인데, 실패한 shot은 무엇을 남길까요?

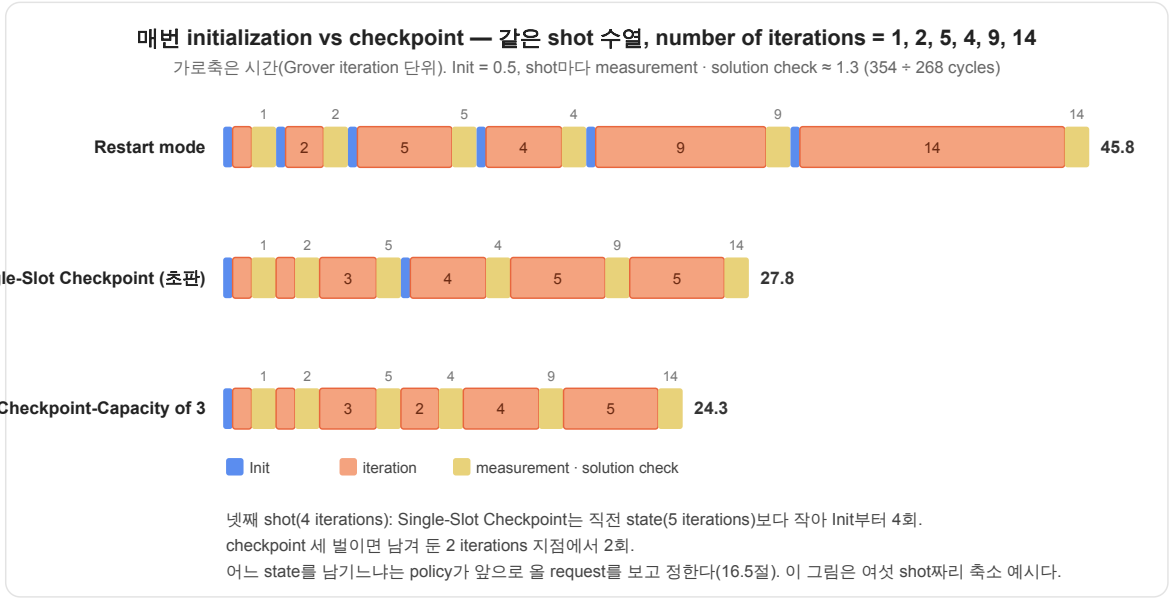
관찰 하나에서 출발합니다. **amplitude memory에 쓰는 주체는 셋뿐입니다** — Init, oracle, diffusion operator. 14장에서 만든 Born-rule sampling은 amplitude를 읽어 square의 prefix sum을 만들고 그 prefix sum이 random number를 넘는 지점을 찾을 뿐, amplitude에 **아무 값도 쓰지 않습니다.** 뒤이은 solution check도 `data_mem` 의 한 칸을 읽을 뿐입니다. 그러므로 shot이 실패해 바깥 loop로 돌아온 시점에도 amplitude memory에는 **그 shot이 요청한 만큼 iteration을 돈 state가 고스란히 남아 있습니다.**

그리고 BBHT의 number of iterations는 sampling range 안에서 뽑히는데 sampling range는 단조 증가합니다. 다음 shot이 요청하는 number of iterations가 이미 계산해 둔 state의 number of iterations보다 큰 경우가 흔하다는 뜻이고, 그럴 때는 처음부터 다시 돌 이유가 없습니다. 이 project의 초판 설계는 이 관찰을 **state 한 벌**로 옮겼습니다 — 직전 shot의 state를 지우지 않고, 다음 request가 그보다 크면 차이만큼만 더 돌리는 1칸짜리 **checkpoint**였습니다. 최종 설계는 이것을 **여러 벌**로 넓혔습니다.

checkpoint slot 세 벌. amplitude state를 최대 세 벌(a Checkpoint-Capacity of 3) 저장하고 각 checkpoint slot에는 "이 state는 iteration을 몇 번 돈 것인가"라는 number of iterations가 붙어 있습니다. 새 request가 오면, checkpoint slot 가운데 number of iterations가 request 이하이면서 가장 큰 것(가장 가까운 앞선 state)에서 출발합니다. 이번 request의 물리 비용은

$$\text{cost} = \text{number of iterations}_{\text{request}} - \text{number of iterations}_{\text{checkpoint slot}}$$

이고(출발할 checkpoint slot이 없으면 Init 뒤 request의 number of iterations 전부), request와 같은 number of iterations의 checkpoint slot이 이미 있으면 비용은 0 — 계산 없이 그 state를 다시 측정합니다. 전진하는 도중의 중간 state를 다른 checkpoint slot에 저장하는 데도 추가 비용이 거의 들지 않습니다. 그 state는 어차피 datapath를 지나가므로, 한 iteration만 다른 checkpoint slot으로 건너 쓰면(bridge) 됩니다. 계산은 checkpoint slot 안에서 제자리로 이어 가므로, chip 안에는 checkpoint slot과 따로 떨어진 작업용 amplitude memory가 없습니다.



무엇을 남길 것인가 — policy. checkpoint slot이 셋뿐이니, 이번 전진이 끝났을 때 어느 state들을 남겨 둘지 골라야 합니다. 여기서 emulator만의 사정이 하나 드러납니다. number of iterations를 뽑는 random number는 seed에서 시작하는 **결정적인** Iteration-Count LFSR이 내고, sampling range 경계도 ROM에 적혀 있습니다. 그러니 "이번 shot이 실패한다면" 다음에 올 request들은 **미리 계산할 수 있습니다.** hardware는 실제 LFSR state를 바꾸지 않고 사본을 전진시켜 앞으로 올 request를 미리 뽑아 보는 circuit(Iteration-Count Predictor)을 둡니다. shot이 성공하면 search가 끝나거나(단일) 문제가 바뀌므로(enumeration) 그 예측은 버리면 되고, 실패하면 예측이 정확히 맞습니다.

policy는 **지금 request 하나와 앞으로 올 3 requests**(3-shot Lookahead)를 한꺼번에 놓고, 어느 number of iterations의 state를 checkpoint slot에 남기면 이 request들 전체의 물리 비용 sum이 가장 작아지는지를 푸는 작은 DP입니다. 남길 후보는 이 request들의 값으로 제한하고, 지금 request의 state는 측정해야 하니 반드시 남깁니다. 비용이 같으면 전진하는 segment가 적은 쪽, 그다음 정해진 정렬 순서로 동률을 처리해 **언제나 같은 답**을 냅니다. 풀이 중간값은 BRAM 4개짜리 memo table에 적어 되풀이 계산을 줄입니다. 그리고 지금 shot이 도는 동안 다음 shot의 계획을 미리 풀어 작은 Plan Queue에 넣어 두어, shot 사이에 policy를 기다리는 시간을 숨깁니다. 미리 푼 계획이 실제 request와 어긋난 횟수(Plan Queue miss)는 `PLAN_FIFO_MISMATCH_COUNT` 로 남고, **0이어야 정상**입니다.

policy가 search 결과를 바꾸는 일은 없습니다. policy가 정하는 것은 "어느 state에서 출발해 무엇을 남기느냐" — 즉 요청된 state를 **어떻게 만드느냐**뿐이고, 어떤 number of iterations를 요청하느냐는 BBHT의 random number가 정합니다.

얼마나 줄었나. 500 workloads 전체에서 executed iterations가 31,424에서 9,582로 **69.5%** 줄었습니다(16.3절 표). 이 절감이 Six-Stage Ablation의 첫 단계(Stage 1 → Stage 2, 총 cycle −56.63%)이고, board wall-clock time 7.626× 가속의 가장 큰 몫입니다.

왜 a Checkpoint-Capacity of 3, 왜 3-shot Lookahead인가. 직관과 반대로, Checkpoint-Capacity와 Lookahead-Shots를 늘리는 것이 늘 이득은 아니었습니다. a Checkpoint-Capacity of 4 and 4-shot Lookahead에서 a Checkpoint-Capacity of 3 and 3-shot Lookahead로 줄였더니 executed iterations는 9,058에서 9,582로 5.8% **늘었는데** 총 cycle은 7.66% **줄었습니다**. 4 PEs로 iteration 한 번이 약 268 cycles까지 짧아진 뒤에는, policy가 결정을 내리는 데 드는 cycle이 아끼는 iteration보다 비싸졌기 때문입니다. 발표 자료의 직접 비교에서 policy가 평가한 action이 70%, policy cycle이 56%, policy 때문에 search가 멈춘 cycle이 73% 줄었습니다. 1 PE 조건의 Checkpoint-Capacity · Lookahead-Shots 단독 실험도 같은 방향입니다 — Checkpoint-Capacity만 4에서 3으로 줄이면 거의 차이가 없고(0.21%), 이득은 대부분 Lookahead-Shots를 줄일 때 나옵니다(2.58%). **비싼 쪽은 저장 공간이 아니라 결정 비용**입니다. 참고로 4 PEs configuration의 amplitude memory는 checkpoint slot 네 벌 분량(RAMB36 44개)으로 짜여 있어, Checkpoint-Capacity를 3으로 줄여도 BRAM은 줄지 않습니다(11.4절).

16.6 왜 결과가 달라지지 않는가

실패한 shot의 state를 물려받아 이어 돌리는 것이 정말 **같은 algorithm**인지, 두 층위로 나눠 답해야 합니다.

첫째, state 재현성. Grover 한 바퀴를 $G = D \cdot O$(oracle 뒤 diffusion operator)라 쓰면, checkpoint slot에서 이어 가는 경로가 만들어 내는 연산열은

$$(DO)^{\text{number of iterations}_{\text{request}}} = (DO)^{\text{number of iterations}_{\text{request}} - \text{number of iterations}_{\text{checkpoint slot}}} \cdot (DO)^{\text{number of iterations}_{\text{checkpoint slot}}}$$

입니다. 6.6절이 "circuit의 순효과는 하나의 unitary"라고 했던 그 associative law가 근거입니다 — unitary의 곱은 결합적이므로 어디서 끊어 읽든 순효과가 같습니다. 즉 이어 가는 경로는 새로 Init해서 같은 횟수만큼 돌린 것과 **문자 그대로 같은 연산열**이고, 우리 산술이 결정적(rounding은 round-half-to-even, saturation 지점과 accumulator width는 고정)이므로 결과는 근사적으로 같은 정도가 아니라 **bit 단위로 동일**합니다. checkpoint slot이 몇 벌이든, policy가 어떤 중간 state를 남기든 이 등식은 그대로입니다. 이 등식이 18.7절에서 verification의 합격 기준이 됩니다.

둘째, 통계적 정당성. 여기서 반드시 나오는 질문이 있습니다 — *"앞 shot의 계산을 물려받으면 그건 shot을 여러 번 던진 게 아니라 한 번 길게 돌린 거 아닌가? 반칙 아닌가?"*

아닙니다. 이유는 두 가지입니다. 하나, **같은 횟수만큼 iteration을 돈 state는 도달 경로와 무관하게 유일합니다.** 위 등식이 말하는 것이 정확히 그것이죠. 앞선 shot을 거쳐 왔든 방금 Init했든, 같은 횟수만큼 iteration을 돈 amplitude array는 같은 하나입니다. 둘, **measurement random number는 shot마다 Measurement PRNG에서 새로 뽑습니다.** 그래서 각 shot은 여전히 $\sin^2\left((2 \cdot \text{number of iterations} + 1)\,\theta\right)$ probability로 solution을 내는 **서로 독립인 shot**입니다. 결과적으로 후보 index의 분포도, shot당 success probability도, 총 number of shots의 분포도 checkpoint를 끄나 켜나 똑같습니다. **관측 가능한 출력은 하나도 바뀌지 않고 wall-clock time만 줄어듭니다.** 실제로 같은 500 workloads에서 board의 Restart mode와 final configuration이 결과 index·number of shots·requested iterations를 500/500 같게 냈습니다.

다만 제약도 있습니다. **실기 quantum computer는 이렇게 할 수 없습니다.** 4.4절의 collapse 때문입니다 — 실제 장치에서는 측정하는 순간 superposition이 무너져 amplitude가 사라지므로, 다음 shot은 무조건 $|0\cdots0\rangle$ 부터 다시 만들어야 합니다. 우리가 이어 돌릴 수 있는 것은 amplitude를 memory에 펼쳐 놓았고 Born-rule sampling을 **읽기 전용**으로 구현했기 때문이지, quantum mechanics에서 가능한 동작이어서가 아닙니다. 앞으로 올 request를 미리 계산하는 policy도 마찬가지로 emulator만 할 수 있는 일이지만, 그것이 들여다보는 것은 BBHT가 스스로 만든 number of iterations 수열뿐이고 amplitude나 solution은 아닙니다. 11장에서 말한 "emulator가 실기보다 쉬운 것들"의 대표적인 사례입니다.

그러므로 보고할 때 두 지표를 반드시 갈라 적습니다. **algorithm 지표(requested iterations)는 그대로이고, 줄어드는 것은 executed iterations(`EXECUTED_ITERATIONS`)와 wall-clock time뿐입니다.** 16.3절 표에서 두 열이 따로 적힌 것이 이 구분입니다. 이걸 섞어 "우리 방식이 Grover iteration을 70% 줄였다"고 쓰면 부정확합니다 — 줄어든 것은 emulation 시간이지 algorithm이 oracle에 던진 질의 횟수가 아닙니다.

16.7 무효화는 hardware가 한다

checkpoint slot에 붙은 number of iterations는 사실 "이 state는 **어떤 predicate·어떤 threshold·어떤 data·어떤 mask**로 몇 번 돌린 결과인가"라는 tuple 전체를 요약한 값입니다. 그 tuple 중 하나라도 바뀌면 남아 있는 amplitude는 지금 풀려는 문제의 중간 결과가 아니게 되므로, 즉시 무효가 되어야 합니다.

Accelerator Core는 다음 경우에 **스스로** checkpoint slot을 전부 비웁니다 — data load 시작, predicate 종류 변경, threshold A·B 변경, 유효 data 개수 변경, 그리고 enumeration에서 새 답이 나와 `found_mask` 가 바뀔 때. 반대로 두 random number seed, shot cap, autonomous mode와 Manual mode 사이의 전환, Manual mode의 number of iterations, checkpoint bit를 바꾸는 것은 amplitude의 의미를 바꾸지 않으므로 checkpoint slot을 비우지 않습니다. 미리 풀어 둔 policy 계획도 문제가 바뀌면 함께 버립니다.

"firmware가 threshold를 바꿀 때 무효화 bit도 같이 쓰면 되지 않나"는 유혹적이지만, software에 맡기면 **무효화 처리를 빠뜨리는 경우가 반드시 생깁니다.** 나중에 predicate를 하나 더 추가한 사람이 무효화를 빼먹는 순간, 이전 amplitude에 새 oracle이 적용되어 **조용히 틀린 답**이 나옵니다. crash도 아니고 timeout도 아니라 그럴듯한 index가 하나 나오는 종류의 bug라, simulation에서 잡기 가장 어렵습니다. 무효화 조건은 Accelerator Core가 이미 들고 있는 설정값에서 추가 비용 없이 판정할 수 있는데 그 신뢰를 software에 넘길 이유가 없습니다. 그래서 CSR에도 무효화 명령 register가 없습니다.

enumeration mode에서의 결말은 두 갈래입니다. 새 답이 나오면 mask가 바뀌므로 checkpoint slot이 비고 다음 enumeration round는 Init부터 시작합니다. 그러나 한 번의 BBHT search가 답을 못 내고 끝났을 때는 mask가 그대로이므로 **checkpoint slot이 살아 있고**, 다음 BBHT search가 그 state들을 이어 씁니다. 그래서 enumeration에서도 checkpoint가 일을 합니다(14.7절의 예에서 requested iterations 2,422 대 executed iterations 635).

16.8 기각한 대안

checkpoint에 도달하기까지 검토한 안들입니다. 왜 버렸는지가 왜 택했는지만큼 중요합니다.

1칸 checkpoint (초판). 직전 shot의 state 한 벌만 남기는 안입니다. 추가 memory가 0이라는 이점이 있었지만 request가 직전보다 작아지면 Init부터 다시 해야 하고 중간 state를 남길 자리도 없습니다. 여러 벌의 checkpoint slot과 policy를 둔 a Checkpoint-Capacity of 4 and 4-shot Lookahead가 board 250쌍에서 executed iterations를 71% 넘게 줄여 이 안을 대체했고, 최종 a Checkpoint-Capacity of 3 and 3-shot Lookahead가 그 뒤를 이었습니다.

되감기(양방향 이동). O 와 D 는 각각 자기 자신이 inverse이므로 $G^{-1} = (DO)^{-1} = OD$ 입니다. 즉 이론상으로는 request가 지금 state보다 작을 때 initialization 없이 **뒤로 감을** 수도 있습니다. 그런데 fixed-point에서 OD 는 DO 의 **정확한** 역이 아닙니다(round-half-to-even과 saturation은 비가역이니까요). 되감은 state는 bit 단위로 원래 state와 달라져 16.6절의 등식이 깨집니다. 게다가 checkpoint slot이 여러 벌이면 앞선 state를 이미 들고 있으므로 되감을 이유 자체가 줄어듭니다.

더 많은 Lookahead-Shots(8-shot Lookahead). 앞을 더 멀리 보면 iteration을 조금 더 아낄 수 있습니다. benchmark에서 8-shot Lookahead는 4-shot Lookahead보다 iteration을 2.7% 덜 썼지만, policy 때문에 search가 멈춘 cycle이 23×로 늘어 총 cycle에서 손해였습니다(2026-09-04 board benchmark). 이 관찰이 a Checkpoint-Capacity of 3 and 3-shot Lookahead로 가는 근거가 됐습니다(16.5절).

더 큰 Checkpoint-Capacity. Checkpoint-Capacity를 늘리면 policy가 고를 여지가 늘지만, 16.5절의 단독 실험처럼 4에서 3으로 줄여도 cycle 차이가 0.21%에 그쳤고, 반대로 늘리는 쪽은 state 한 벌이 RAMB36 11개라 BRAM 예산(11.4절)에 막힙니다.

`hit[]` bit vector caching. predicate 판정 결과를 bit vector로 저장해 두면 oracle을 다시 계산하지 않아도 되지 않을까 — 라는 발상입니다. cycle 절감이 **정확히 0**입니다. Predicate Unit은 subtractor 하나짜리 combinational logic이고(13.3절), `data_mem` 읽기가 amplitude 읽기와 **같은 cycle**에 나오기 때문에 아낄 시간이 애초에 없습니다. 무효화해야 할 대상만 하나 더 늘 뿐입니다.

한 가지는 기각이 아니라 **보류**입니다. number of iterations별 amplitude를 DRAM에 전량 저장해 두고 필요한 state를 burst 한 번에 가져오는 DRAM Variant(`hardware_dram`)는 Checkpoint-Capacity도 policy도 필요 없게 만드는 다른 답입니다. RTL 초안과 regression까지 있고 물리 DRAM 연결과 board verification은 아직이라, BRAM Variant와 DRAM Variant 중 어느 쪽을 최종으로 할지는 정해지지 않았습니다.

16.9 초판의 미정 넷은 이렇게 정해졌다

이 장의 초판은 네 가지를 미정으로 남겨 두었습니다. 지금은 RTL과 software reference model의 규칙을 동일하게 맞췄습니다.

1. solution이 없을 때의 종료 조건. predicate를 만족하는 원소가 하나도 없으면 BBHT는 영원히 실패합니다. hardware는 array를 고전적으로 훑어 "없음"을 확정하지 않고, 16.3절의 두 보호 한도 — shot cap 100, BBHT budget 576 — 중 먼저 닿는 쪽으로 끝냅니다. array에서 정보를 미리 읽지 않는다는 16.1절의 원칙이 그대로 지켜집니다. enumeration mode에서는 이 "실패로 끝난 BBHT"가 연속 `fail_repeat_limit` 번(기본 3번)이면 enumeration을 끝냅니다(14.7절).

2. $\text{number of iterations} \sim U[0,\ \text{sampling range})$ **의 균등성.** LFSR 출력을 masking만 하면 sampling range가 2의 거듭제곱이 아닐 때 분포가 균등하지 않습니다. 16.2절의 rejection sampling으로 해결했습니다 — 경계 이상의 후보는 버리고 다시 뽑습니다.

3. random number 소비 규약. number of iterations stream과 measurement stream은 **서로 독립인 두 generator**입니다. number of iterations 쪽은 32-bit Iteration-Count LFSR이고 후보 하나마다(버린 후보까지 포함해) 한 번 추첨하며 LFSR을 7 steps 전진시킵니다. sampling range 경계가 1이어도 sampling 한 번은 소비하고 0 iterations를 냅니다. measurement 쪽은 Measurement PRNG(64-bit xorshift)이고, rejection sampling의 후보마다 64-bit block 하나를 씁니다(14.4절). group → row → lane 선택은 받아들인 random number 하나의 잔여를 이어 쓰며 새 random number를 뽑지 않습니다. 두 seed는 host가 시작 명령을 쓸 때 다시 읽히고, seed가 0이면 정해진 대체 constant를 씁니다. 이 규약이 다르면 reference model과 RTL 사이에 **bit-exact가 성립하지 않는데**(18.3절), 지금은 둘이 500 workloads 전부에서 같은 BBHT trajectory를 냅니다.

4. $\text{sampling range} \leftarrow \min(1.2 \cdot \text{sampling range},\ \sqrt{\text{number of states}})$ **의 hardware 산술.** 1.2×는 shift로 떨어지지 않지만, $\sqrt{\text{number of states}} = 128$ 이 정확한 integer라 clamp는 쉬워졌습니다. 16.2절의 28항 ROM으로 정했습니다.

16.10 이 장의 요약

- **solution이 몇 개인지 모른다**: number of solutions는 predicate·threshold·data에 달려 실행 시점에야 정해지므로 optimal number of iterations $\frac{\pi}{4}\sqrt{\text{number of states}/\text{number of solutions}} - \frac{1}{2}$ 를 미리 계산할 수 없고, 9.6절 때문에 "넉넉히 많이"도 안 통합니다.
- **quantum counting(QPE)은 제외**: complex number를 강제해 실수 전용 23-bit Q1.22 datapath를 그대로 쓸 수 없기 때문. 성능 문제가 아니라 수 표현 문제입니다.
- **그래도 BBHT를 쓰는 이유는 속도가 아니라 충실도**: emulator에서 solution을 세는 비용은 shot 하나보다 싸지만, 그것은 amplitude array를 직접 읽는 것처럼 실제 장치에서 얻을 수 없는 정보를 쓰는 일입니다. 비교 실험용으로는 firmware가 number of iterations를 정하는 Manual mode가 따로 있습니다.
- **BBHT**: sampling range는 1에서 시작하고, shot마다 number of iterations를 $[0,\ \text{sampling range})$ 에서 **무작위**로 뽑고, 실패하면 sampling range를 growth factor 6/5만큼 넓힙니다(상한 $\sqrt{\text{number of states}}$). number of iterations를 무작위로 뽑는 이유는 success probability $\sin^2\left((2\cdot\text{number of iterations}+1)\,\theta\right)$ 가 진동하기 때문 — 14 qubits · 1 solution에서 100 iterations는 100%인데 200 iterations는 0%입니다. $\sqrt{\text{number of states}} = 128$ 이 integer라 number of iterations는 $[0, 128)$ 에서만 뽑히고, sampling range 수열은 28항 ROM, number of iterations는 LFSR rejection sampling으로 정확히 균등합니다.
- **mean 22.7 shots**(14 qubits · 1 solution · 500 workloads campaign): sampling range가 1부터 1.2×씩 커져 초반 shot은 거의 전부 실패하도록 되어 있습니다. "4~5회"가 아닙니다. 보호 한도는 shot cap 100과 BBHT budget 576입니다.
- **바깥 FSM 2 layer**: shot loop(ROM에서 sampling range · LFSR로 number of iterations · policy에 출발 checkpoint slot 문의) + 안쪽 iteration(13장) + solution check(한 칸 고전 확인) + enumeration round loop. shot loop 자체는 LUT 359개로 작습니다.
- **checkpoint**: amplitude에 쓰는 것은 Init·oracle·diffusion operator 셋뿐이고 Born-rule sampling은 읽기 전용이므로, 실패한 shot의 state가 그대로 남습니다. state를 **세 벌(a Checkpoint-Capacity of 3)** 저장해 두었다가 가장 가까운 앞선 state에서 **차이만큼만** 전진합니다. 무엇을 남길지는 지금 request와 **앞으로 올 3 requests(3-shot Lookahead)** 를 보는 DP policy가 정하고, 앞으로 올 request는 결정적인 Iteration-Count LFSR의 사본을 전진시켜 미리 압니다. 500 workloads executed iterations **−69.5%**.
- **a Checkpoint-Capacity of 3 and 3-shot Lookahead가 a Checkpoint-Capacity of 4 and 4-shot Lookahead를 이긴 이유**: iteration은 5.8% 더 쓰지만 결정 비용이 크게 줄어 총 cycle −7.66%. 비싼 쪽은 저장 공간이 아니라 결정 비용입니다.
- **결과는 안 바뀝니다**: Grover iteration의 곱은 결합적이라 checkpoint slot에서 이어 간 state가 bit 단위로 같은 state이고(6.6절), measurement random number는 shot마다 새로 뽑으므로 각 shot은 여전히 서로 독립입니다. board에서 Restart mode와 final configuration의 BBHT trajectory가 500/500 같았습니다. 단 **실기 quantum에서는 할 수 없는 동작**이고 algorithm 지표 requested iterations와 executed iterations·wall-clock time을 갈라 적습니다.
- **무효화는 hardware가**: data load·predicate·threshold·data 개수·enumeration mask가 바뀌면 Accelerator Core가 스스로 checkpoint slot을 비웁니다. software에 맡기면 조용히 틀리는 bug가 납니다. enumeration에서 실패한 BBHT 사이에는 checkpoint slot이 유효한 상태로 남습니다.
- **기각·보류한 대안**: 1칸 checkpoint(대체됨) · 되감기(fixed-point에서 정확한 역이 아님) · 8-shot Lookahead(stall 23×) · Checkpoint-Capacity 증설(이득 미미, BRAM 예산) · `hit[]` caching(cycle 절감 0). DRAM Variant는 보류.
- **초판의 미정 넷**은 전부 정해졌습니다: solution이 없으면 두 보호 한도로 종료 · number of iterations는 rejection sampling으로 균등 · 두 random number stream의 소비 규약 고정 · sampling range는 28항 ROM.

이것으로 한 shot 안(13·14장)과 shot 바깥(16장)의 제어가 모두 정리됐습니다. 다음 17장은 이 accelerator를 혼자 두지 않고 RVX SoC에 붙입니다 — host가 UART로 명령을 보내고, RISC-V core가 APB로 CSR을 쓰고, accelerator가 AHB master로 data를 끌어오는 경로입니다.

17장. RVX SoC에 붙이기 — APB CSR · AHB DMA · UART

> **이 장을 읽기 위한 준비**: 13장 oracle과 diffusion operator, 16장 BBHT shot loop와 checkpoint.

13·14·16장에서 IP 안쪽은 다 만들었습니다. amplitude array가 있고, predicate를 판정하는 oracle이 있고, mean으로 되비추는 diffusion operator가 있고, Born sampler가 있고, 그 위에 shot loop와 checkpoint가 얹혀 있습니다. 그런데 이 circuit에는 아직 시작 명령을 받고, data를 입력받고, 답을 내보내는 경로가 없습니다.

이 장에서는 외부 연결을 다룹니다. host가 UART로 명령을 보내고, RISC-V core가 APB로 설정 register를 쓰고, IP가 AHB master로 data를 끌어오는 경로입니다. FPGA를 다뤄 본 독자에게는 이 장이 가장 익숙한 장일 것입니다.

17.1 accelerator는 혼자 서지 않는다

우리 IP가 실제로 잘하는 일은 하나뿐입니다 — **모든 state의 amplitude를 한꺼번에 훑어 갱신하는 것**. 128 lanes가 매 cycle 128칸씩 sign을 뒤집고 mean으로 되비추는 일은 규칙적이고, 분기가 없고, data에 따라 흐름이 바뀌지 않습니다. hardware로 병렬 처리하기에 적합한 계산입니다.

반대로 이 IP를 실제로 **쓰는** 과정은 정반대의 성격을 가집니다. 어떤 predicate를 쓸지 정하고, threshold를 넘기고, data를 준비하고, enumeration mode라면 결과를 제때 빼 가고, 결과를 사람이 읽을 형태로 찍어 내는 일 — 전부 분기와 loop, 예외 처리가 필요한 작업입니다. 이런 것을 state machine으로 굳혀 넣으면 조건 하나 바뀔 때마다 재합성해야 하고, synthesis에 걸리는 시간이 실험 한 번의 시간을 압도하게 됩니다.

그래서 경계를 이렇게 긋습니다. **amplitude 갱신과 shot loop는 hardware가, 설정과 결과 처리와 표시는 software가.** 정확히 말하면 software가 하는 일은 "설정을 쓰고 시작을 누르고 status를 polling하고 결과를 읽는" 몇 줄이며, 그 몇 줄이 유연해야 하는 부분 전부를 흡수합니다.

이 구도가 임의로 고른 편의가 아니라는 점을 짚어 둘 필요가 있습니다. **실제 quantum computer도 똑같이 생겼습니다.** quantum processor는 circuit 하나를 실행하고 measurement 결과를 내보낼 뿐이고, "몇 번 더 돌릴지, 다음에 어떤 parameter를 넣을지"는 옆에 붙은 classical control computer가 정합니다. superconducting qubit든 trapped ion이든, 사용자가 실제로 명령을 주고받는 대상은 언제나 그 classical computer입니다. 우리가 RISC-V core를 옆에 두는 것은 emulator를 편하게 만들려는 타협이 아니라 **실기의 구조를 그대로 흉내 내는 것**입니다. 16.1절에서 "충실도가 속도를 이긴다"고 한 원칙이 system 층에서도 똑같이 적용됩니다.

17.2 RVX가 무엇인가

RVX(RISC-V eXpress)는 ETRI(한국전자통신연구원)에서 만든 MCU급 저전력 SoC 개발 키트입니다. 우리가 하려는 일 — "직접 만든 accelerator IP를 RISC-V SoC에 붙여 FPGA에 올린다" — 이 정확히 RVX가 상정한 사용 시나리오입니다.

RVX의 핵심은 **platform XML 하나에서 SoC 전체가 생성된다**는 점입니다. 우리 platform `bbht_grover_final` 의 XML은 짧습니다. accelerator 전용 clock `clk_accel` 을 100 MHz로 정의하고, system SRAM 크기를 128 KB로 잡고, IP instance를 셋 선언합니다 — RV32 core `rvc_orca` 를 `i_main_core` 로, 우리 IP의 제어 register 쪽을 APB slave shim `user_slaveif_apb_clkout` (memory map `0x1000`, address·data 32 bits)으로, data load 쪽을 AHB master shim `user_masterif_ahb_clkout` (32 bits)으로. core와 peripheral과 우리 IP 사이를 잇는 interconnect는 **munoc** 입니다.

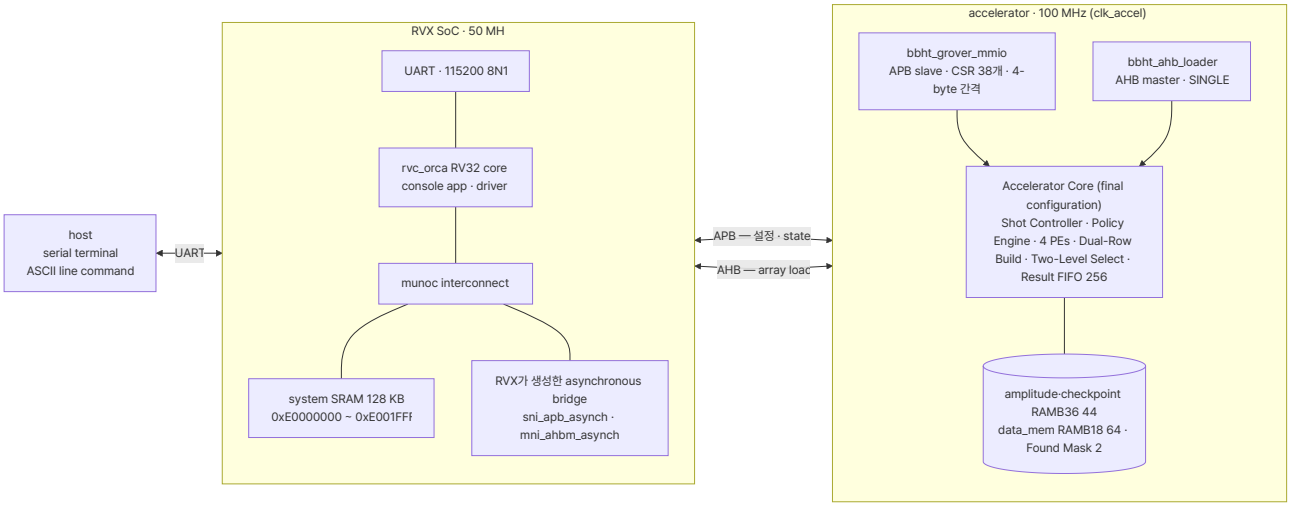
이 XML은 SoC RTL 외의 파일도 생성합니다. `make syn` 을 돌리면 다음이 전부 자동으로 나옵니다.

- **memory map header** `memorymap_info.h` — 우리 CSR의 base address `I_GROVER_CSR_SLAVE_BASEADDR` (이 환경에서 `0xE2020000`), system SRAM 범위 `0xE0000000` ~ `0xE001FFFF` (128 KB).
- **hardware 정보 header** `hw_info.h` — `SRAM_SIZE`, `SYSTEM_CLK_HZ` (50 MHz), 포함된 option macro들.
- 같은 내용의 Verilog version(`platform_info.vh`), Vivado용 TCL, boot ROM, SoC simulation 환경.

즉 "register address를 RTL과 C header에 두 번 적고 나중에 어긋나는" 고전적인 사고를 구조적으로 막아 줍니다. 이 점 하나만으로도 손으로 SoC를 조립하는 것보다 낫습니다. firmware는 base address를 숫자로 박지 않고 이 macro를 씁니다 — platform XML이 바뀌면 address가 옮겨 가기 때문입니다.

board 정의는 `imp_class_info/` 아래에 따로 있고, 우리가 쓰는 `arty-100t` 항목에는 `part` 가 `xc7a100tcsg324-1`, 외부 clock 100 MHz, configuration flash `s25fl128s` (SPI x4), USB-UART bridge `0403:6010:01` 로 적혀 있습니다. board를 바꾸는 것은 `TARGET_IMP_CLASS` 를 바꾸는 일이 됩니다.

이 project 환경에는 RVX가 `/opt/rvx` 에 **전체 설치**되어 있어, platform 생성부터 synthesis·implementation·bitstream·SoC 전체 RTL simulation까지 전부 local에서 돕니다. Vivado도 local에 있고 화면이 없는 batch mode로 씁니다. accelerator를 붙이는 방법은 RVX 예제 platform `tip_quantized_cnn` (accelerator + `user/rtl` · `user/api`)의 틀을 따랐습니다.



17.3 왜 APB와 AHB를 같이 쓰나

bus를 두 개 쓰는 것이 낭비처럼 보일 수 있으니, 우리 IP가 bus로 하는 일이 성격이 완전히 다른 두 가지라는 점부터 짚겠습니다.

첫째는 **설정과 status**입니다. predicate 두 bit, threshold 16 bits, 시작 한 bit, status 몇 bit, counter 몇 개. 한 번 실행에 몇백 byte가 오갈 뿐이고, 그나마 실행 전후에만 오갑니다. 여기에 필요한 것은 bandwidth가 아니라 **address로 register를 선택하는 기능**과 **적은 gate**입니다.

둘째는 **array load**입니다. 14 qubits면 `data_mem` 에 32 KB를 채워야 하고, 이것은 system SRAM의 연속된 address를 죽 읽어 오는 일입니다. 여기에 필요한 것은 IP가 스스로 bus에 나가 읽어 오는 **master 능력**입니다. core가 한 data word씩 읽어 CSR로 넘겨 주는 방식은 core를 붙잡아 두고 bus를 두 번씩 건넙니다.

AMBA 계열의 세 bus는 정확히 이 spectrum 위에 놓여 있습니다. **APB**는 가장 단순합니다 — setup/access 두 phase에 burst도 pipeline도 없고, 한 번에 data word 하나입니다. 대신 slave implementation이 combinational logic 몇 줄이면 끝납니다. **AHB**는 pipelined address/data phase와 burst를 지원하고, master를 직접 쓸 만한 복잡도입니다. **AXI**는 읽기와 쓰기 channel이 따로 있고 out-of-order completion과 multiple outstanding transaction을 지원합니다. 강력하지만 우리처럼 순차 load만 하는 IP에게는 쓰지 않을 기능에 대한 logic 비용입니다.

그래서 결론은 **CSR은 APB slave, data load는 AHB master**입니다. RVX가 두 shim을 모두 제공하니, 우리는 검증된 routing을 그대로 따라가면 됩니다.

load를 어떤 방식으로 하느냐는 따로 정했습니다. 초판은 AHB burst(`INCR16`)를 상정했지만, 최종 DMA Engine은 **32-bit SINGLE 읽기를 한 번에 하나씩(single outstanding)** 합니다. burst는 FIFO와 `BUSY` 처리가 붙어 복잡하고, 17.5절에서 보듯 SINGLE로도 load가 search 한 번과 비슷한 시간에 끝나기 때문입니다. burst는 benchmark에서 load가 실제 병목으로 드러난 뒤에 볼 일로 미뤄 두었습니다.

여기서 초기 설계안 하나가 폐기되었음을 명시해 둡니다. 처음에는 data를 **AXI-Stream** 으로 흘려 넣는 DMA Engine을 상정했습니다. 이 안은 못 씁니다 — **RVX library에 AXI-Stream shim이 아예 없기 때문입니다.** RVX에 등록된 사용자 interface shim을 전부 열거하면 master 쪽이 `moi` · `ahb` · `axi` · `axi4` · `axi4l`, slave 쪽이 `apb` · `ahb` · `axi` · `axi4` · `axi4l` 이고(각각 `_clkin` / `_clkout` 두 벌), stream 계열은 한 줄도 없습니다. shim이 없으면 interconnect에 붙일 방법이 없고, 그러면 그 IP를 SoC에 연결할 수 없습니다. 도구가 제공하는 것과 설계가 요구하는 것이 어긋날 때는 대개 설계를 고치는 쪽이 싸며, 이 경우는 특히 그렇습니다.

shim 이름 끝의 `_clkin` / `_clkout` 도 짚고 갈 값어치가 있습니다. `_clkin` 은 IP가 SoC의 clock을 **받아** core와 같은 도메인에서 도는 형태이고, `_clkout` 은 IP가 자기 clock을 **내보내** 별도 도메인에서 도는 형태입니다. 우리는 `_clkout` 을 씁니다. core와 bus는 50 MHz로 두고 accelerator만 100 MHz 전용 clock으로 돌리기 위해서입니다. 두 도메인 사이의 clock 경계 처리(CDC)는 RVX가 APB·AHB 쪽에 각각 생성해 주는 비동기 bridge가 맡

습니다. 그러니 **우리 쪽에 CDC를 또 넣으면 안 됩니다** — RVX가 만들어 주는 것이 실제 board에서 검증된 구조이고, 중복 삽입은 그 verification을 무효로 만듭니다.

17.4 CSR map

RVX에는 CSR block를 생성기로 뽑는 길과 손으로 짜는 길이 둘 다 있습니다. RVX 예제의 생성기는 register 간격을 **8 bytes**로 잡고, 설정 register의 쓰기 signal을 module 밖으로 내보내지 않습니다. 우리는 **손으로 짠 APB slave**(`bbht_grover_mmio`)를 쓰고 간격을 **4 bytes**로 잡았습니다. register가 38개라 8-byte 간격이면 address 공간이 두 배로 비고, 무엇보다 읽기 부작용(FIFO pop)이나 완료 표시 같은 동작을 우리가 직접 제어해야 했기 때문입니다. `pready` 는 constant 1이라 wait state가 없고, 정렬되지 않은 접근과 미할당 address는 `pslverr` 로 떨어집니다.

register는 32-bit 38개이고, 접근 성격이 넷입니다.

표기	뜻
`RW`	설정. 여기 저장되고 값이 IP로 상시 나감
`W1P`	명령. 저장 공간이 없고 쓰면 1 cycle pulse만 나감. 읽으면 0. 쓰는 행위 자체가 명령
`RO`	status. IP가 내보내는 signal을 읽기 응답에 실어 보냄
`RPOP`	읽기가 곧 Result FIFO에서 한 칸 꺼내기. 부작용이 있는 유일한 읽기

주요 register를 역할별로 모으면 이렇습니다. 전체 38개의 bit 정의는 CSR register 규격에 있습니다.

offset	이름	접근	뜻
0x000	COMMAND	W1P	search 시작
0x004	CONTROL	RW	autonomous mode(auto_shot) · checkpoint(burst_enable) · predicate 선택 2-bit
0x008	J_TARGET	RW	Manual mode의 number of iterations (7 bits, 0~127). autonomous mode에서는 무시
0x00C · 0x010	THRESHOLD_A · THRESHOLD_B	RW	threshold (signed 16 bits). range predicate는 열린 interval
0x014	DATA_COUNT	RW	유효 data 개수 1~16384. 그 뒤 index는 non-solution
0x018	SHOT_CAP	RW	shot cap (기본 100)
0x01C · 0x020	SEED_J · SEED_MEAS	RW	Iteration-Count LFSR과 Measurement PRNG의 seed. 서로 독립
0x024	STATUS	RO	실행 status 12-bit (아래)
0x028	RESULT_INDEX	RO	single search 결과 (14 bits)
0x02C ~ 0x038	SHOT_COUNT · REQUESTED_ITERATIONS · EXECUTED_ITERATIONS · CYCLE_COUNT	RO	shot count · requested iterations(요청된 iteration의 sum) · executed iterations · 100 MHz cycle count
0x03C	ENUM_CFG	RW	enumeration 켜기 · 연속 실패 한계 fail_repeat_limit (1~15)
0x040	FIFO_DATA	RPOP	Result FIFO 머리
0x044 ~ 0x054	FIFO_COUNT · FOUND_COUNT · CONSEC_FAIL · MAX_FIFO_OCC · FIFO_STALL	RO	enumeration status와 FIFO 통계
0x058 · 0x05C · 0x060	DATA_ADDR · DMA_COMMAND · DMA_STATUS	RW · W1P · RO	data load (17.5절)
0x064 ~ 0x094	policy telemetry 13개	RO	policy cycle · stall cycle · Plan Queue hit·miss 등 (16.5절)

STATUS 의 12 bits는 실행 중(busy), load 중(load_busy), 완료 표시(done_sticky), 결과 유효(result_valid), 그리고 sticky 오류·한도 bit들 — 설정 오류, shot cap 도달, requested iterations budget 도달, amplitude saturation(amp_overflow, 12.5절), measurement total weight 0, load 오류 — 과 enumeration 종료, FIFO 비었음입니다. saturation bit는 진단용이고 결과 무효 표시가 아닙니다.

초판의 CSR 초안과 달라진 점이 셋 있습니다. 첫째, **checkpoint 무효화용 register가 없습니다.** 16.7절대로 Accelerator Core가 설정값이 바뀐 것을 스스로 보고 checkpoint slot을 비우므로, 무효화 명령도, register마다 무효화 strobe를 모으는 routing도 필요 없어졌습니다. 둘째, **지금 몇 번의 iteration을 거친 state인지를 알려 주는 register가 없습니다.** checkpoint slot의 state는 전부 hardware 안에 있고, firmware는 알 필요가 없습니다. 셋째, 두 지표를 갈라 읽는 counter는 그대로 남았습니다 — REQUESTED_ITERATIONS 가 algorithm 지표(요청된 iteration의 sum), EXECUTED_ITERATIONS 와 CYCLE_COUNT 가 emulation 비용입니다(16.6절).

시작 명령에는 수락 조건이 있습니다. 실행 중이 아니고, load 중이 아니고, 직전 enumeration 결과가 FIFO에 남아 있지 않고, 직전 완료 표시가 아직 안 읽힌 채가 아니어야 합니다. **시작 queue는 두지 않았습니다.** 조건이 안 맞으면 pulse가 무시되므로, firmware가 COMMAND 를 쓰기 전에 STATUS 를 확인하는 것이 규약입니다. queue를 두면 "언제 시작했는지"가 software에서 보이지 않게 되고, 그런 때에 counter를 읽으면 서로 다른 시점의 값이 섞입니다.

17.5 data load — AHB master

14 qubits면 `data_mem` 은 16,384개 × 16-bit = **32 KB**, 32-bit data word로 8,192개입니다. 이것을 SoC의 system SRAM에서 IP 안으로 옮기는 것이 AHB master(`bbht_ahb_loader`)가 하는 유일한 일입니다.

DMA Engine은 data word 하나를 **SINGLE 읽기로 하나씩**(single outstanding) 읽고, 받은 32 bits를 16-bit 두 칸으로 나눠 `data_mem` 에 씁니다 — 하위 16 bits가 짝수 index, 상위 16 bits가 다음 index입니다. data word 하나에 address 1 + data 1 + 쓰기 1로 3 cycles이 들어 전체가 24,576 cycles, 100 MHz에서 **약 246 µs** 입니다. 같은 board에서 final configuration이 solution 하나를 찾는 search가 mean 약 0.23 ms(wall-clock time)이니, load 한 번이 search 한 번과 비슷한 시간입니다. data를 한 번 올려 두고 threshold만 바꿔 가며 여러 번 탐색하는 usage pattern이라면 load 비용은 더 희석됩니다.

load를 시작하기 전에 DMA Engine이 스스로 검사하는 것이 넷 있고, 어긋나면 `DMA_STATUS` 의 해당 오류 bit를 세우고 거절합니다.

검사	조건
정렬	`DATA_ADDR` 가 4-byte 정렬
개수	`DATA_COUNT` 가 1~16384
범위	읽을 interval이 system SRAM(`0xE0000000` ~ `0xE001FFFF`) 안
status	Accelerator Core가 실행 중이 아님

다음 세 가지 규약을 따릅니다.

load 중에는 시작을 받지 않습니다. load가 도는 동안은 `STATUS.load_busy` 가 서고, 17.4절의 수락 조건에 걸려 시작 pulse가 무시됩니다. 절반만 채워진 `data_mem` 위에서 oracle을 돌리면 아무 경고 없이 틀린 답이 나옵니다.

`DATA_COUNT` 가 홀수면 마지막 data word의 상위 16 bits는 쓰지 않습니다. 설령 쓰더라도 그 index는 `DATA_COUNT` 밖이라 oracle이 non-solution으로 봅니다(13.3절). 다만 Accelerator Core 쪽 load port는 같은 address를 두 번 써도 개수를 올리므로, 바깥의 DMA Engine이 각 address를 정확히 한 번씩 쓰도록 짜여 있습니다.

load는 checkpoint를 무효화합니다. data가 바뀌면 predicate가 같아도 solution 집합이 달라지므로, 남아 있던 amplitude는 의미가 없습니다(16.7절).

system SRAM 크기도 이 경로와 묶여 있습니다. 32 KB array에 firmware code와 stack까지 들어가야 하므로 SRAM을 128 KB로 잡았고, 그 몫이 SoC가 쓰는 32 BRAM tiles의 대부분입니다(15.5절).

data를 어디서 만드느냐는 firmware의 몫으로 정했습니다. benchmark용 data는 console app의 `GEN` 명령이 RISC-V core에서 **seed 둘로부터 결정적으로** 만들어 SRAM에 씁니다 — 배경값은 xorshift random number로, solution을 심을 위치는 또 다른 seed로 정하고, 배경값이 우연히 목표값과 같아지는 자리는 한 bit를 뒤집어 피해서 심은 개수가 곧 number of solutions가 되게 합니다. seed가 같으면 array가 같으므로 재현성도 확보됩니다. 초판은 IP 안에 PRNG를 두는 안을 적었지만, firmware에서 만들어도 load가 search와 비슷한 시간에 끝나므로 IP를 가볍게 두는 쪽을 택했습니다. (재현 package에는 RVX 없이 IP 안의 생성기로 data를 만드는 독립 FPGA revision도 있었지만, 임의 data를 올릴 수 없어 final revision에는 쓰지 않았습니다.)

17.6 host UART protocol

host와 SoC 사이의 물리 경로는 UART 하나입니다. Arty board의 micro USB 하나가 JTAG과 UART를 겸하고, 꽂으면 host에 serial port가 둘 잡히는데 뒤 번호가 UART입니다. 설정은 **115200 8N1, 흐름 제어 없음**이고, baud rate는 50 MHz system clock에서 분주됩니다. 표준 pin map에서 RX pin까지 물려 있어 **양방향**입니다.

board 쪽에는 **console app**(`bbht_console`)이 돌며, protocol은 **ASCII line 기반**입니다. binary frame보다 느리지만, 이 경로로 오가는 것이 원래 몇십 byte뿐이라 속도가 문제가 되지 않고, 대신 얻는 것이 큽니다 — terminal을 열어 손으로 명령을 쳐 볼 수 있고, log가 그대로 사람이 읽는 문서가 되며, parsing이 어긋났는지 값이 틀렸는지 눈으로 구분됩니다. bring-up 단계에서는 이 차이가 문제를 찾는 데 걸리는 시간을 크게 줄입니다.

명령은 `SET`(predicate·threshold·seed·mode·data 개수), `GEN`(board 위 data 생성), `LOAD`(DMA load), `RUN`(single search), `ENUM`(enumeration), `STAT`(counter), `REG`(CSR dump) 등이고, 응답 한 덩어리는 반드시 `OK` 나 `ERR` 로 끝납니다. 한 번 돌리면 이런 모양입니다.

```
> GEN COUNT=16384 TARGETS=1 VAL=12345
OK count=16384 targets=1 val=12345
> LOAD
OK loaded=16384 addr=0xe0000abc
> SET MODE=EQ A=12345 AUTO=1 BURST=0 SEEDJ=0x7b1dcdaf SEEDM=0x24370df2
OK
> RUN
HIT idx=507 val=12345 trials=25 l=308 iters=308 cyc=351394 us=3513
OK
> SET BURST=1
OK
> RUN
HIT idx=507 val=12345 trials=25 l=308 iters=70 cyc=30913 us=309
OK
```

두 `HIT` 줄의 수치는 같은 조건으로 2026-09-08 board benchmark app이 낸 값(Restart mode와 final configuration)을 옮긴 것입니다. 마지막 두 `RUN` 이 이 project의 주장을 그대로 보여 줍니다. **`idx` 와 `trials` 와 `l` 은 같고 `iters` 와 `cyc` 만 줄었습니다.** 답이 같고 algorithm이 oracle에 던진 질의 수도 같은데 emulation 비용만 줄어든 것입니다(16.6절). 명령과 응답의 전체 문법은 UART 명령 protocol에 있습니다.

enumeration mode의 흐름은 stream입니다. `ENUM` 을 치면 console app이 search를 시작하고, hardware가 답을 Result FIFO에 쌓는 동안 firmware가 계속 FIFO를 비우며 답마다 `FOUND` 줄을 내보내다가, enumeration이 끝나면 `END` 로 닫습니다. solution이 256개를 넘으면 firmware가 늦게 뽑는 동안 hardware가 멈추고, 그 cycle이 `FIFO_STALL` 에 남습니다.

이 경로로 data array를 보내지는 않습니다. 계산해 보면 이유가 분명합니다. 32 KB를 115200 bps · 8N1(byte당 10 bits)로 보내면 약 **2.8초**입니다. search 자체가 수백 µs인데 data를 넣는 데 그 만 배를 쓰는 것은 — 그것도 ASCII hex로 보내면 두 배 — 실험을 반복하는 데 큰 부담이 됩니다. 그래서 array는 board 위 `GEN` 으로 만들거나 SRAM에 미리 두고 DMA로 올립니다. UART로 오가는 것은 **명령과 결과뿐**이라는 원칙을 세워 두면 이 함정을 피할 수 있습니다.

host 쪽 도구는 serial terminal이면 충분합니다. 여러 명령을 한 번에 돌리고 싶다면, 한 줄 보내고 `OK` / `ERR` 을 받을 때까지 기다리는 loop를 짜면 됩니다 — 흐름 제어가 없으므로 **응답을 받은 뒤에 다음 명령**을 보내는 것이 규약입니다.

17.7 hardware와 firmware의 경계

지금까지 나온 역할 분담을 한 표로 모읍니다. 각 항목의 근거는 "왜 그쪽인가" 열에 있습니다.

하는 일	어디서	왜
amplitude 갱신 (oracle · diffusion operator · Init)	hardware	128 lanes 병렬, 분기 없음
Born-rule sampling · solution check	hardware	amplitude array 전체를 읽어야 하고 `data_mem` 이 IP 안에 있음 (14.5절)
number of iterations sampling · sampling range 갱신	hardware	shot마다 필요, 왕복 latency가 아까움 (Manual mode에서는 firmware가 `J_TARGET` 을 씀)
checkpoint policy · 무효화	hardware	state를 software에 맡기면 관리가 누락됨 (16.7절)
enumeration round · 종료 판정	hardware	새 답마다 Found Mask·checkpoint slot·BBHT를 되돌리는 일이 전부 IP 안 state
enumeration 종료 한계값	firmware	`fail_repeat_limit` 을 CSR로. 응용마다 다른 policy
Result FIFO 비우기	firmware	도는 동안에도 계속 (FIFO 256 entries)
predicate · threshold 결정	firmware	응용에 따라 바뀜
minimum search loop (threshold 낮추기)	firmware	`while` 한 줄로 충분 (14.9절)
UART parsing · 결과 출력	firmware	분기 덩어리
data 준비 (`GEN` · SRAM · load 시작)	firmware	policy 결정

shot loop는 hardware가 돕니다. 이 결정이 이 표에서 가장 논쟁적인 지점이라 근거를 적어 둡니다. 16.3절의 수치대로 14 qubits·1 solution에서 search 한 번에 mean 22.7 shots가 듭니다. shot마다 core를 깨워 status를 읽고 다음 shot이 돌 iteration을 계산해 CSR에 쓰면 왕복이 20번 넘게 생기는데, core가 50 MHz로 APB를 건너는 동안 100 MHz accelerator는 대기하고, 무엇보다 **checkpoint state가 hardware와 software 양쪽에 존재하게 됩니다.** 16.7절이 경계한 바로 그 구조입니다. 그래서 최종 형태는 시작 한 번에 "답을 찾거나 한도에 닿을 때까지" 돌고 완료 표시로 끝나는 **BBHT autonomous mode**이고, firmware가 보는 것은 "설정 → 시작 → polling → 결과 읽기" 네 줄입니다.

그렇다고 firmware가 shot을 직접 실행하는 방법을 없애지는 않았습니다. **Manual mode**(`MANUAL_SINGLE`)에서는 app이 `J_TARGET` 에 원하는 값을 써서 한 shot씩 돌릴 수 있습니다. 이 mode는 solution이 몇 개인지 알고 iteration을 정해진 횟수로 고정해 도는 비교 실험과, software reference model과의 한 shot 단위 대조(18장)에 쓰입니다.

초판과 달리 **enumeration round의 종료 판단도 hardware로 들어갔습니다.** 초판은 "몇 번 더 시도할지는 응용마다 다른 policy이니 firmware에 남긴다"고 적었는데, 새 답이 나올 때마다 Found Mask에 bit를 세우고 checkpoint slot을 비우고 BBHT를 처음부터 돌리는 일이 전부 IP 안 state라 경계를 IP 안에 두는 편이 깔끔했습니다. policy의 설정값인 연속 실패 횟수의 한계는 `ENUM_CFG` 로 firmware가 정합니다. 재합성 없이 바뀌어야 할 것은 여전히 firmware 쪽에 있는 셈입니다.

직접 CSR에 접근하는 firmware가 지켜야 할 규약도 여기 모아 둡니다. 시작 전에 `STATUS` 를 확인하고, 완료는 `done_sticky` 로 판정하며(이 bit는 `COMMAND` 쓰기로만 지워집니다), 결과와 counter는 `busy` 가 내려간 뒤에 읽고, load 중에는 시작하지 않고, `FIFO_DATA` 는 읽는 것 자체가 꺼내기이며 index 0도 정상 결과일 수 있으니 비었는지는 `STATUS` 나 `FIFO_COUNT` 로 봅니다. driver(`bbht_grover_driver`)가 이 규약을 이미 따릅니다.

17.8 초판의 미정 다섯은 이렇게 정해졌다

초판은 system 층에서 다섯 가지를 미정으로 남겨 두었습니다. 지금은 board에서 동작한 build를 기준으로 정해졌습니다.

1. CSR map. 32-bit register 38개, **4-byte 간격**, base `0xE2020000`(크기 `0x1000`)으로 확정했습니다. RVX 생성기 대신 손으로 짠 APB slave를 써서, 읽기 부작용과 완료 표시를 우리가 제어했습니다(17.4절).

2. enumeration 결과를 어떻게 돌려줄 것인가. IP 안에 **256-entry Result FIFO**를 두고, firmware가 도는 동안에도 비웁니다. enumeration round마다 완료를 세워 firmware가 한 번씩 읽어 가는 안보다 왕복이 적고, FIFO 깊이는 BRAM이 아니라 LUT memory로 잡혀 싸게 끝났습니다.

3. `M_max` 값. solution이 너무 많을 때 enumeration을 포기하는 별도의 상한은 두지 않았습니다. FIFO가 가득 차면 hardware가 멈추고 firmware가 비우는 만큼 진행합니다(14.7절).

4. RVX SoC가 실제로 얼마를 먹는가. implementation report로 쟀습니다. SoC 몫(core·interconnect·UART·SRAM 128 KB 등)이 32 BRAM tiles · 4 DSPs이고, board 전체가 BRAM 116 / 135 · DSP 132 / 240 · LUT 45,284 / 63,400입니다(15.5절).

5. clock이 정말 100 MHz인가. accelerator에 **전용 100 MHz clock**(`clk_accel`)을 주고, RISC-V core와 bus는 RVX 기본값인 50 MHz로 두었습니다. IP는 `_clkout` shim으로 자기 도메인에서 돌고, 경계는 RVX 생성 비동기 bridge가 맡습니다(17.3절). 100 MHz에서 timing이 닫혔습니다(WNS +0.126 ns). cycle counter `CYCLE_COUNT` 가 이 100 MHz 기준이라 **시간 = cycle ÷ 100 µs** 이고, 50으로 나누면 두 배 틀립니다.

17.9 이 장의 요약

- **경계**: amplitude 갱신과 shot loop처럼 규칙적이고 IP 안 state에 묶인 일은 hardware가, 설정·결과 처리·표시처럼 응용마다 바뀌는 일은 software가. 실제 quantum computer도 classical control computer가 circuit을 돌리는 구조라, 이 배치는 편의가 아니라 실기의 구조를 따르는 것입니다.
- **RVX**: ETRI의 RISC-V eXpress. **platform XML 하나**에서 SoC RTL·memory map C header·Verilog header·boot ROM·Vivado script가 전부 생성됩니다. 우리 platform은 core `rvc_orca`, APB slave shim, AHB master shim 셋에 accelerator 전용 100 MHz clock과 SRAM 128 KB. `/opt/rvx` 에 전체 설치되어 생성부터 bitstream까지 local에서 돕니다.
- **bus 두 개**: 설정·status는 **APB slave**, 32 KB array load는 **AHB master**. load는 SINGLE 읽기 하나씩으로 충분했고, **AXI-Stream은 RVX에 shim이 없어** 초기 원안을 폐기했습니다. IP는 `_clkout` shim으로 자기 clock domain에서 돌고, CDC는 RVX 생성 bridge가 맡습니다.
- **CSR map**: 손으로 짠 APB slave, 32-bit **38개 · 4-byte 간격**, 접근 성격은 `RW` · `W1P` · `RO` · `RPOP`. 무효화 register가 없고(Accelerator Core가 스스로), algorithm 지표 `REQUESTED_ITERATIONS` 와 비용 지표 `EXECUTED_ITERATIONS` · `CYCLE_COUNT` 를 따로 둡니다. 시작 queue는 없고 firmware가 `STATUS` 를 먼저 봅니다.
- **load**: 32-bit SINGLE, data word당 3 cycles → 8,192 data words **약 246 µs**(100 MHz). 정렬·개수·범위·실행 중 여부를 DMA Engine이 검사하고, load 중 시작 금지 · 홀수 개수의 마지막 상위 16-bit 생략 · load는 checkpoint 무효화. benchmark data는 firmware `GEN` 이 seed로부터 결정적으로 만듭니다.
- **UART**: console app과 ASCII line protocol, 115200 8N1, 흐름 제어 없음. 명령과 결과만 오갑니다 — 32 KB를 UART로 보내면 **약 2.8초**로 search 시간의 만 배라 data 직송은 비현실적입니다.
- **shot loop는 hardware**(BBHT autonomous mode)가 돌고, firmware가 number of iterations를 정하는 **Manual mode**도 비교 실험과 reference model 대조를 위해 남아 있습니다. **enumeration 종료 판정도 hardware**가 하고, 한계값만 firmware가 설정합니다.
- **초판의 미정 다섯**은 전부 정해졌습니다: CSR 38개 · 4-byte · 256-entry Result FIFO · 별도 `M_max` 없음 · SoC 32 BRAM tiles · accelerator 100 MHz / core 50 MHz.

system 경계까지 그렸으니 설계는 이제 board 위에서 동작합니다. 남은 질문은 하나입니다 — 이것이 맞다는 것을 어떻게 증명하나요? 다음 18장이 그 답입니다.

18장. 무엇을 어떻게 증명하나 — 3단 reference model과 bit-exact

> **이 장을 읽기 위한 준비**: 9.4절 4-state 손계산, 12장 amplitude를 숫자로의 Q1.22·round-half-to-even·saturation, 13장 oracle과 diffusion operator의 datapath, 14장 multiple solutions의 Measurement Unit(14.5절), 16장 BBHT shot loop와 checkpoint, 17장 RVX SoC에 붙이기의 CSR map.

앞의 일곱 장이 설계를 확정했습니다. 이 장은 그 설계가 맞다는 것을 **어떤 절차로 확인했는가**를 정리합니다.

이 장이 필요한 이유가 두 군데에 이미 적혀 있습니다. 12.7절은 05번 논문의 minimum fraction bits 공식이 전제하는 조건 셋을 우리가 전부 벗어난다고 결론 내리면서, 22 fraction bits가 충분한지는 "이론으로 정하지 않고 직접 잰다"고 했습니다. 15.6절의 정오표는 여섯 편의 1차 자료 모두에 인용할 때 주의해야 할 오류가 있음을 보이면서 논문 수치는 직접 재현해 확인하라고 못 박았습니다. 두 결론 모두 **재는 절차**가 필요하다는 뜻입니다. 초판에서 계획이었던 그 절차가 이제 실제로 돌았고, 이 장은 그 절차와 결과를 함께 씁니다.

18.1 왜 verification이 설계의 일부인가

보통 verification은 설계가 끝난 뒤에 붙는 일로 취급됩니다. 이 project에서는 그렇지 않습니다. 이유가 셋 있습니다.

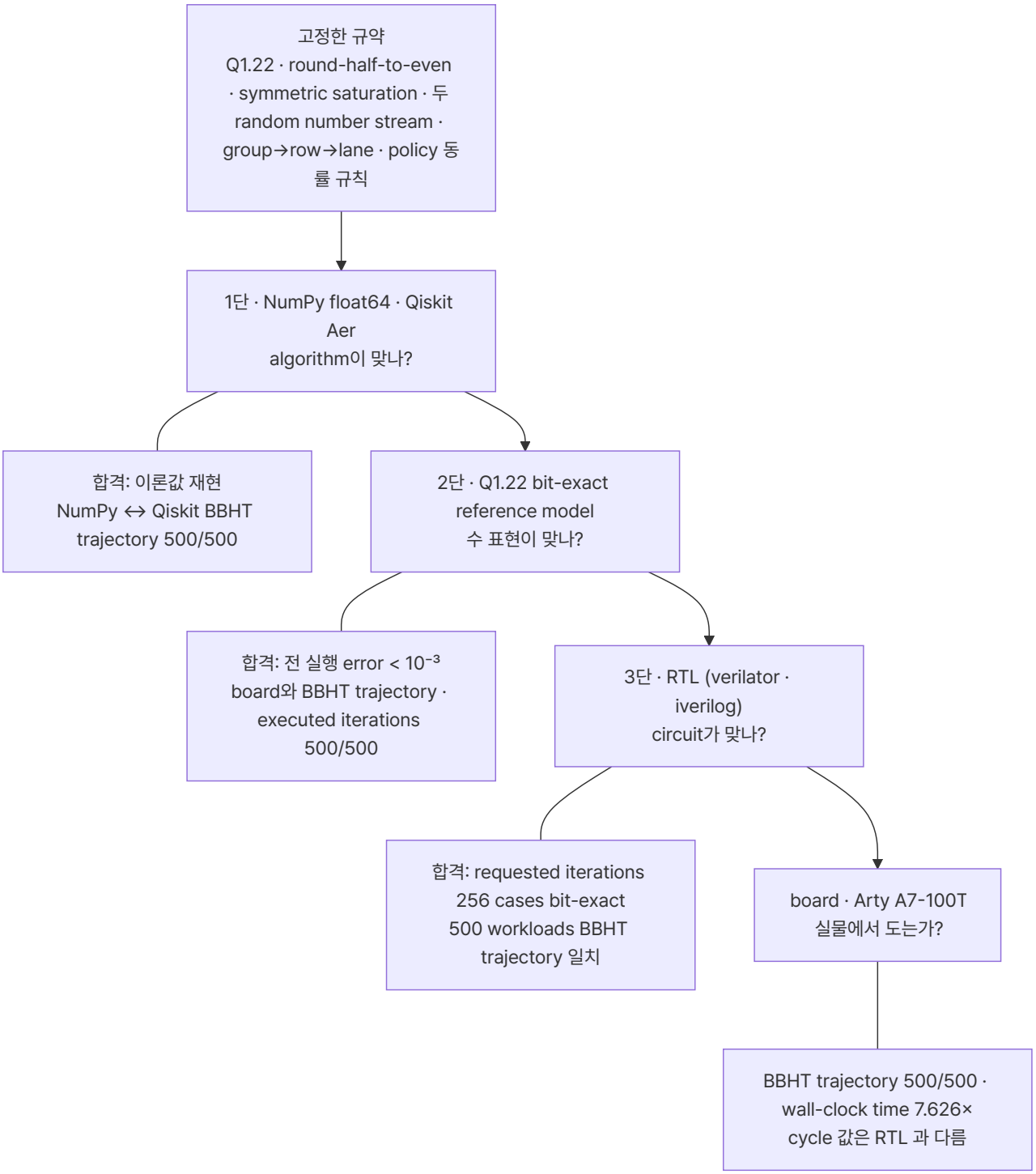
첫째, 1차 자료가 틀립니다. 15.6절의 정오표는 논문의 사소한 오탈자 모음이 아닙니다. 04번의 pseudocode는 그대로 옮기면 **circuit이 동작하지 않고**, 04번이 제시한 magic number 두 개는 최하위 bit가 어긋나 있으며, 09번 식 (9)·(12)의 sign 규약은 본문과 어긋나서 그대로 쓰면 minimum이 아니라 **maximum**을 찾습니다. 이 사례들은 검증할 때 다음과 같은 태도가 필요함을 보여줍니다 — hardware로 옮기기 전에 원문을 의심하고, 수치는 직접 계산해 보고, sign은 simulation으로 확인한다. 이 셋을 하려면 원문과 대조할 수 있는 참조 implementation을 갖추어야 합니다.

둘째, 이 설계에는 이론이 답을 주지 못하는 자리가 있습니다. 12.7절이 그 대표입니다. minimum fraction bits 공식은 truncation·넓은 단일 data word·distribution 전체의 ℓ_2 error를 전제하는데 우리는 round-half-to-even을 쓰고, 저장 data word가 23 bits로 좁고, 우리가 신경 쓰는 것은 distribution 전체가 아니라 "solution check를 통과할 답이 제때 나오는가"입니다. 공식을 대입하면 아슬아슬하다는 답이 나오지만 그 답은 우리 조건의 답이 아닙니다. 그러면 남는 방법은 하나 — 우리 조건을 그대로 담은 model을 만들어 직접 재는 것입니다. 즉 verification infrastructure가 있어야 **설계 결정을 내릴 수 있습니다.** 설계를 확정하기 전부터 검증이 필요한 것이지요.

셋째, 이 IP의 값어치 자체가 verification 가능성입니다. 여러 번 못 박았듯 이 accelerator는 classical linear scan보다 빠르지 않습니다. emulation 비용이 $O(\text{number of states}\sqrt{\text{number of states}/\text{number of solutions}})$ 이라 원리적으로 그럴 수 없습니다. 이 IP가 주는 것은 속도가 아니라 **quantum algorithm의 동작을 결정적이고 재현 가능하게 재현·검증하는 전용 hardware**라는 성질입니다. 그렇다면 "재현 가능하다"는 주장을 뒷받침하는 절차가 곧 제품 사양의 일부입니다. verification 계획이 부실하면 기능이 부실한 것과 같습니다.

18.2 3단 verification ladder

핵심 구조는 단순합니다. 참조 implementation을 **세 칸**으로 나누고, 각 칸이 잡는 버그의 종류를 다르게 둡니다. 마지막에는 board에서 확인합니다.



1단 — NumPy float64와 Qiskit Aer. 묻는 것은 "algorithm이 맞나". amplitude를 double-precision real number로 두고 Grover iteration·Born-rule sampling·BBHT shot loop를 그대로 옮깁니다. 수 표현은 사실상 무한 precision이라고 보고, 오직 논리만 봅니다. 이 칸이 잡는 버그는 predicate 판정의 방향(`<` 를 `>` 로 짠 것), diffusion operator의 sign, number of iterations를 한 칸 잘못 센 overshoot, BBHT의 sampling range 갱신 누락 같은 것들입니다. 전부 "circuit 이전"의 실수이고, 여기서 못 잡으면 뒤 칸에서는 훨씬 비싸게 잡힙니다. 우리 저장소에는 이 칸이 둘입니다 — 직접 짠 NumPy state vector와 범용 quantum simulator Qiskit Aer. 둘은 Iteration-Count LFSR의 같은 random number와 같은 measurement 규칙으로 BBHT를 돌아, 500 workloads 전부에서 결과 index·shot count·requested iterations가 **서로** 같았습니다(500/500).

이 칸의 결과를 2단과 그대로 맞대면 안 된다는 점을 짚어 둡니다. floating-point와 fixed-point는 probability distribution이 아주 조금 다르고, 같은 random number라도 prefix sum이 경계를 넘는 칸이 달라질 수 있습니다. 실제로 1 solution에서 requested iterations의 median이 1단은 149.5, 2단과 board는 163.5였습니다(shot count median은 둘 다 23). 둘 다 옳은 결과입니다 — BBHT trajectory가 같아야 하는 것은 **같은 칸끼리**입니다.

2단 — Q1.22 bit-exact reference model. 묻는 것은 "수 표현이 맞나". 1단과 논리는 같지만 real number를 전부 integer로 바꿉니다. amplitude는 23-bit integer, shift는 arithmetic right shift에 round-half-to-even, 넘치면 symmetric saturation, accumulator는 정해진 폭, random number는 RTL과 같은 생성기와 같은 소비 규약. Python으로 쓰되 **floating-point를 한 줄도 섞지 않는 것**이 규율입니다. 이 칸이 잡는 버그는 rounding 편향, 예

상 밖의 saturation, accumulator 폭 부족, shift를 rounding 앞에 둘지 뒤에 둘지 같은 것들입니다. 12.7절이 넘겨 둔 질문 — 22 fraction bits가 충분한가 — 의 답도 이 칸에서 나왔습니다. **이 model이 곧 "reference model"입니다.** RTL과 bit 단위로 맞춰 볼 상대가 이 칸이지 1단이 아닙니다. 저장소에서는 arithmetic·random number·measurement를 담은 `fixed_point_statevector.py` 와, 그 위에 BBHT 제어·checkpoint policy·enumeration을 얹은 `checkpoint_bbht_model.py` 가 이 칸입니다(software reference model 안내).

3단 — RTL. 묻는 것은 "circuit이 맞나". 이 칸이 잡는 버그는 앞의 둘과 완전히 종류가 다릅니다. FSM state 전이, address generator의 off-by-one, row·lane indexing(하위 5 bits인데 상위 5 bits를 쓴다든가), pipeline stage 사이의 latency 정렬, 같은 address를 읽으면서 쓰는 hazard, CSR handshake, reset 후 초기값. arithmetic은 이미 2단에서 검증됐으므로, 여기서 값이 어긋나면 원인은 **거의 언제나 timing이나 제어**입니다. 합격 기준은 2단과 **bit-exact** — iteration이 끝난 시점의 amplitude 16,384칸이 전수 일치하고, 같은 seed를 넣었을 때 shot별 후보 index sequence가 완전히 같아야 합니다. 2단이 만든 requested-j test vector 256 cases가 앞의 기준을, 500 workloads RTL campaign이 뒤의 기준을 봅니다.

그리고 board. verification ladder의 마지막 확인은 실물입니다. 같은 500 workloads를 board에 올려, 2단 reference model과 board가 결과 index·shot count·requested iterations에서 500/500, checkpoint의 executed iterations까지 500/500 같았습니다. RTL과 board도 BBHT trajectory는 500/500 같았습니다 — 다만 cycle 값 자체는 달라서(final configuration은 대부분의 workload가 정확히 3,279 cycles 차이), board에서는 wall-clock time을, RTL에서는 cycle을 인용합니다.

이 verification ladder에서 가장 중요한 규율은 **한 칸도 건너뛰지 않는 것**입니다. RTL을 곧바로 float64 model과 비교하면, 값이 어긋났을 때 원인이 rounding 규칙인지 FSM인지 구분할 방법이 없습니다. 두 종류의 error가 섞여 있으면 "얼마나 어긋나야 버그인가"라는 판정 기준조차 세울 수 없고, 그러면 허용 error를 임의로 넓게 잡고 넘어가는 유혹에 빠집니다. 중간 칸이 있어야 판정이 **일치 / 불일치**라는 이진값이 되고, 이진값이어야 regression test가 자동화됩니다.

18.3 bit-exact가 성립하기 위한 조건

2단과 3단이 bit 단위로 같아야 한다는 기준은 강력하지만 저절로 성립하지는 않습니다. 두 implementation이 **같은 결정을 같은 자리에서** 내려야 성립합니다. 아래 가운데 하나라도 어긋나면 reference model과 RTL은 일치하지 않고 더 나쁜 것은 그 불일치가 "미세한 error"처럼 보여서 원인을 찾는 데 며칠이 든다는 점입니다.

고정해야 할 것	우리 규약	어긋나면 어떻게 보이나
rounding 규칙	round-half-to-even, 조건 $\mathrm{guard} \wedge (\mathrm{sticky} \vee \mathrm{LSB})$ (유도는 12.4절, circuit은 13.5절)	대부분의 값은 맞고 정확히 절반인 경우에만 1 LSB 차
rounding이 적용되는 **지점**	diffusion operator의 13-bit right shift **한 곳뿐** (12.4절)	subtraction 뒤에도 rounding을 넣으면 iteration마다 어긋남이 누적
saturation 지점과 범위	되쓰기 직전, symmetric $\pm 4{,}194{,}303$ (12.5절)	saturation이 안 일어나는 조건에서는 완전히 일치하다가, solution amplitude가 1에 닿는 iteration에서만 갈라짐
shift 위치와 방향	$2\bar{a}$ 는 $\mathrm{number\ of\ qubits} - 1 = 13$ bits arithmetic right shift (13.5절)	signed 값의 logical shift는 음수에서만 틀림
accumulator 폭	partial sum 28 bits · 전체 sum 39 bits, 자르지 않음 (13.7절)	중간에서 자르면 addition 순서가 결과를 바꿈
measurement arithmetic과 순서	square 46 · row weight 51 · total weight 60 bits, 자르지 않음, 엄격한 `>`, group → row → lane (14.5절)	같은 random number에서 이웃 칸이 뽑힘
Iteration-Count LFSR	32-bit LFSR, 후보마다 7 steps 전진, rejection sampling(버린 후보도 sampling 한 번), seed 0이면 대체 constant (16.2절)	첫 shot부터 number of iterations가 달라 아무것도 비교할 수 없음
Measurement PRNG	xorshift64, 32-bit seed를 64 bits로 확장, rejection된 후보마다 64-bit block 하나, 하위 bit를 (total weight − 1)의 bit 길이만큼 (14.4절)	몇 shot은 우연히 맞다가 어긋나기 시작 — 가장 찾기 어려운 형태
checkpoint policy의 동률 규칙	비용 → 전진하는 segment 개수 → 정해진 정렬 순서 (16.5절)	BBHT trajectory는 같은데 executed iterations만 다름

초판은 이 표의 random number 두 줄을 **미정**으로 남겨 두면서, "이 둘은 미룰 수 있는 미정이 아니다 — 정해지기 전까지 verification ladder의 위 두 칸을 아예 맞춰 볼 수 없다"고 적었습니다. 그 말 그대로였습니다. 두 규약이 못 박힌 뒤에야 reference model이 board와 BBHT trajectory를 500/500 맞추게 되었고, 마지막 줄의 동률 규칙까지 옮기고 나서 executed iterations까지 500/500이 되었습니다.

실무적으로는 규약을 **문서 한 장에 못 박고 두 implementation이 그 문서만 참조**하게 만듭니다. 더 나은 방법은 constant를 한 곳에서만 만드는 것입니다. 우리 저장소에서 algorithm constant(`INIT_AMP`, sampling range 경계 ROM 28 entries, requested iterations budget, 대체 seed)는 RTL의 `grover_param.vh` 에 있고, reference model은 같은 규칙에서 같은 값을 스스로 계산합니다 — 둘이 어긋나면 첫 workload에서 BBHT trajectory가 갈라지므로 금방 드러납니다. CSR offset은 사정이 다릅니다. 예전에는 source of truth file 하나에서 C·Verilog·Python header를 생성했지만, 지금은 CSR 규격 문서와 reference model의 계약 module 두 곳을 손으로 맞춥니다. 같은 constant를 사람이 두 번 적으면 언젠가 반드시 어긋나므로, 한쪽을 고칠 때는 다른 쪽과 기계적으로 대조하십시오.

한 가지 언어별 함정을 덧붙입니다. Python의 `>>` 는 음수에 대해 $-\infty$ 방향으로 내림하는 arithmetic shift라서 Verilog의 `>>>` 와 같은 동작을 하지만, C의 right shift는 음수에서 implementation-defined이고 NumPy는 폭이 고정된 integer type에서 조용히 wraparound합니다. reference model을 Python 기본 integer로 쓰되 폭 제한은 **명시적인 mask와 saturation function으로 직접 implementation**하는 편이 안전합니다.

18.4 첫 test vector는 9.4절의 손계산

verification을 어디서부터 시작할지는 이미 정해져 있습니다. **9.4절에서 손으로 푼 4-state 예제**입니다.

uniform superposition $(\frac{1}{2}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2})$ 에서 출발해 oracle이 세 번째 칸의 sign을 뒤집으면 $(\frac{1}{2}, \frac{1}{2}, -\frac{1}{2}, \frac{1}{2})$ 이 되고, mean $\bar{a} = \frac{1}{4}$ 를 중심으로 반사하면 $(0, 0, 1, 0)$ 이 됩니다. iteration 한 번으로 probability 100%입니다. 이 예제를 Q1.22 규칙으로 따라가 보면 흥미로운 일이 생깁니다. $\texttt{INIT_AMP} = 2^{22}/2 = 2{,}097{,}152$ 는 0.5 의 정확한 표현이고, oracle 뒤 sum은 2^{22}, $2\bar{a}$ 는 $\text{number of qubits} - 1 = 1$ bit right shift라 2^{21}, 그리고 solution 칸은 $2^{21} - (-2^{21}) = 2^{22}$ — **정확히 1.0입니다.** 그런데 12.2절에서 봤듯 Q1.22는 $+1.0$ 을 담지 못합니다. 이 값은 symmetric saturation으로 $4{,}194{,}303$ 에 붙고, `amp_overflow` 가 섭니다. 가장 작은 손계산이 곧바로 **saturation 경로를 밟는 test**가 되는 셈입니다. 한 번 더 돌리면 sum이 $-4{,}194{,}303$ 이라 $2\bar{a}$ 가 $-2{,}097{,}151.5$ 로 **정확히 절반**에 걸립니다. 12.4절의 round-half-to-even 규칙이 여기서 처음 적용됩니다.

하나를 더 붙입니다. 8 states · solution $\{1, 3, 5\}$ · 3 solutions 예제입니다($\theta = 37.761^\circ$, 최적 iteration 1번). Q1.22에서 $\texttt{INIT_AMP} = \text{round}(2^{22}/\sqrt{8}) = 1{,}482{,}910$ 이고, iteration 한 번이면 세 solution 칸이 **정확히 2,224,365**, 다섯 non-solution 칸이 **정확히** $-741{,}455$ 가 됩니다. real number로 환산하면 0.530330 과 -0.176777 이니 이론값 0.53033 / -0.17678 과 맞습니다. 이 예제에서는 **두 번째 iteration**도 살펴볼 필요가 있습니다. 그 시점의 sum이 $-10{,}380{,}370$ 이라 $2\bar{a} = -10{,}380{,}370/4 = -2{,}595{,}092.5$ 로 **정확히 절반**에 걸립니다. round-half-to-even은 짝수 쪽인 $-2{,}595{,}092$ 를 고르고, arithmetic right shift를 그대로 쓰는 truncation은 $-2{,}595{,}093$ 을 고릅니다. 흔히 쓰는 단순한 implementation(truncation)과 우리 규칙이 갈리는 자리가 두 번째 iteration에서 자연히 나오는 것입니다. 18.3절 표의 첫 row를 검사하는 최소 test vector가 이렇게 손에 들어옵니다.

다만 이 두 예제는 **RTL에 그대로 걸 수 없습니다.** RTL은 14 qubits로 합성되어 state space가 늘 16,384 states이고, reference model의 hardware profile도 14 qubits(와 폐기된 16 qubits 확장)만 받습니다. 그래서 4-state · 8-state 예제는 몇 줄짜리 별도 script로 규칙을 확인하는 데 쓰고, RTL의 첫 vector는 **실제 크기에서 손으로 확인되는 경우**를 고릅니다.

그런 경우가 있습니다. 14 qubits · 1 solution · 1 iteration입니다. 초기 amplitude가 모두 $32{,}768$ 이고 solution 한 칸만 뒤집히므로 sum은 $16{,}382 \times 32{,}768 = 536{,}805{,}376$, 이것을 13 bits 밀면 $2\bar{a} = 65{,}528$ 로 **나누어떨어집니다.** 그러면 solution 칸은 $65{,}528 + 32{,}768 = 98{,}296$, 나머지는 $65{,}528 - 32{,}768 = 32{,}760$ 입니다. 이론값과 대조하면 $\sin 3\theta = 3/128 - 4/128^3$ 이라 2^{22} 을 곱하면 $98{,}304 - 8 = 98{,}296$ — **rounding 없이 정확히 같습니다.** requested-j test vector의 방향성 case `dir_000001_eq_m1_j1` 이 바로 이것이고, 기대 amplitude file에 32,760이 16,383칸, 98,296이 한 칸 들어 있으며 iteration 추적에도 sum 536,805,376과 $2\bar{a}$ 65,528이 적혀 있습니다. waveform viewer에서 cycle을 짚어 가며 대조하기에 적합한 출발점입니다. 같은 묶음이 0 · 4 · 8 · 16 · 64 iterations와 최적점 100 iterations, 범위 끝 127 iterations로 한 걸음씩 넓혀 갑니다.

state가 적은 경우를 보고 싶을 때 RTL에서 할 수 있는 것은 **`DATA_COUNT` 를 작게** 두는 것입니다. state space는 16,384 states 그대로지만 그 뒤 칸은 전부 non-solution이 되므로, 한 칸짜리 data(`dir_000021_eq_data_count1_m1_j1`)나 row 경계를 걸치는 개수(31·32·33)로 padding 문과 lane 경계를 찔러 볼 수 있습니다. 이 기본 예제를 확인한 뒤에는 predicate 판정을 아예 빼고 solution index를 constant로 못 박은 최소 oracle을 잠깐 써도 좋습니다(13.3절) — 다만 그것은 verification용 예제이지 실제 설계가 아닙니다.

18.5 predicate 4종과 경계값

기본 예제를 확인한 뒤에는 실제 predicate circuit을 봅니다. 13.3절에서 네 predicate를 모두 **strict inequality**로 고정했으므로, 경계값 test는 그 약속이 circuit에 그대로 반영됐는지를 확인하는 일입니다.

가장 먼저 볼 것은 **threshold와 정확히 같은 값**입니다. `value == thr_a` 인 칸은 `LT` 에서도 `GT` 에서도 solution이 아니어야 하고, `EQ` 에서만 solution이어야 하며, range predicate `thr_a < value < thr_b` 에서는 양 끝 모두 제외돼야 합니다. 이 네 가지가 한 data array에서 동시에 확인되도록 threshold와 같은 값을 data에 **일부러 심어 둡니다.** inequality sign을 한 칸 잘못 짠 circuit은 다른 test를 전부 통과하고 이 test에서만 걸립니다.

다음은 data word의 **범위 양 끝**입니다. data word width가 16 bits(signed)이므로 값은 $[-32768,\ 32767]$ 입니다. `thr_a = -32768` 로 두면 `LT` 의 solution이 하나도 없고(0 solutions), `thr_a = 32767` 로 두면 `GT` 의 solution이 하나도 없습니다. 이때 확인할 것은 결과만이 아니라 **subtractor의 폭**입니다. 13.3절이 sign extension한 17-bit subtraction을 쓰는 이유가 정확히 이 극단에서 드러납니다 — 16 bits로 빼면 $32767 - (-32768)$ 이 넘쳐 MSB가 뒤집히고, predicate가 정반대 답을 냅니다. range predicate에서는 폭이 좁은 경우와 넓은 경우를 넣습니다. requested-j 묶음의 방향성 case(`gt_max_m0`, `lt_min_m0`, `range_narrow`, `range_wide`)가 이 자리입니다.

그다음은 number of solutions의 극단입니다. 네 가지를 각각 시나리오로 둡니다.

- **0 solutions — solution이 하나도 없다.** BBHT는 영원히 실패하므로 16.9절의 종료 규칙에 따라 멈춰야 합니다. 확인할 것은 shot cap이나 requested iterations budget에서 FSM이 **멈춰 있지 않고 종료하는가**, 그리고 해당 status bit가 서는가입니다.
- **1 solution.** 최적 iteration이 100회인 표준 경우입니다. number of iterations별 success probability 표(100 iterations에서 100%)를 1단 model이 재현하는지, 그리고 100 iterations · 127 iterations 방향성 case가 2단·3단에서 bit 단위로 같은지 봅니다.
- $\text{number of solutions} > \text{number of states}/2$ **— 역주행.** 14.8절의 함정 2입니다. solution을 절반 넘게, 혹은 전부 심으면 (`eq_mhalf`, `eq_mall`) oracle이 사실상 $-I$ 에 가까워져 iteration이 measurement distribution을 거의 바꾸지 못합니다. 확인할 것은 "그래도 답이 나오는가"입니다 — 초반 shot은 iteration이 0 근처라 사실상 균등 무작위 추출이고, solution이 많으면 그것만으로 곧바로 맞아야 합니다.
- **enumeration에서 solution이 256개를 넘을 때 — FIFO가 찬다.** solution이 FIFO 깊이보다 많은 data를 넣고 firmware가 일부러 늦게 뽑아, hardware가 멈췄다가(`FIFO_STALL` 증가) 뽑는 대로 다시 진행하는지, 결과가 하나도 빠지거나 겹치지 않는지 확인합니다.

data distribution은 **세 종류**를 씁니다. 첫째, uniform random number — 기본 regression입니다. 둘째, 정렬된 data — solution이 index의 연속 interval에 몰립니다. 우리 indexing은 하위 5 bits가 lane이므로 연속 interval은 이웃한 row들의 lane에 골고루 퍼지고, 그 성질이 실제로 성립하는지 확인하는 것이 이 distribution의 목적입니다. 셋째, **solution이 한 row나 한 group 경계에 몰리는 인위적 data** — row weight의 prefix sum이 group 경계(32 rows마다)나 row 경계(lane 31/32)에서 넘어가는 자리입니다. Two-Level Select의 group·row 선택이 가장 까다로운 입력을 받는 경우이고, prefix sum의 off-by-one이 여기서 드러납니다(`eq_min_lane31`, `eq_max_lane32`, `d31`·`d32`·`d33` case). 중복 값이 많은 data를 넷째로 두면 `EQ` predicate에서 solution이 갑자기 많아지는 상황도 함께 볼 수 있습니다.

18.6 Born sampler를 어떻게 검증하나

measurement는 확률적입니다. 그런데 앞에서 합격 기준을 bit-exact로 잡았습니다. 모순처럼 보이지만 아닙니다 — **서로 다른 두 가지를 검사하는 것**이고, 둘을 섞으면 verification이 무너집니다.

첫째, equivalence check. seed를 고정하면 sampler는 완전히 결정적인 circuit입니다. 같은 amplitude array와 같은 seed를 reference model과 RTL에 넣으면 두 implementation은 **같은 후보 index sequence**를 내야 하고, 여기에는 허용 error가 없습니다. 이것은 distribution에 관한 시험이 아니라 두 implementation이 같은 function인지 보는 시험입니다. 18.3절의 random number 규약이 확정돼야 성립하는 것이 바로 이 검사이고, 지금은 500 workloads 전부에서 성립합니다.

둘째, distribution test. equivalence check는 두 implementation이 **똑같이 틀렸을** 가능성을 배제하지 못합니다. 그래서 distribution이 정말 $a_i^2/\text{total weight}$ 인지는 따로 봐야 합니다. 방법은 statistical test입니다 — seed를 바꿔 가며 sample을 많이 뽑고, empirical distribution과 theoretical distribution의 chi-square goodness-of-fit test를 합니다.

이때 조심할 것이 둘 있습니다. 하나는 **cell 개수**입니다. 14 qubits면 index가 16,384개라 웬만한 sample 수로는 대부분의 cell의 expected frequency가 1 미만이 되어 chi-square 근사가 성립하지 않습니다. cell을 묶어야 합니다 — 16 groups 단위로 묶으면 degrees of freedom 15가 되고, solution/non-solution 두 group으로 묶으면 success rate test가 됩니다. 다른 하나는 **판정 방식**입니다. significance level 0.05로 test하면 정상인 circuit도 스무 번에 한 번은 기각됩니다. 한 번의 $p < 0.05$ 로 실패를 선언하면 regression test가 무작위로 실패를 보고하게 됩니다. test를 여러 번 되풀이해 p 값들이 $[0, 1]$ 에 균등하게 흩어지는지 보는 편이 훨씬 안정적입니다.

또 하나 — **test 대상을 좁혀야 합니다.** 여기서 보려는 것은 "sampler가 주어진 amplitude array에서 Born distribution을 내는가"이지 "Grover가 맞는가"가 아닙니다. 그래서 amplitude array를 Grover로 만들지 말고 **알려진 값으로 직접 채워 넣고** sampler만 떼어 시험하는 것이 깔끔합니다. 극단적인 array(한 칸만 0이 아닌 경우, 전부 같은 값인 경우, 두 칸만 크고 나머지가 아주 작은 경우)을 넣어 보면 경계 처리의 실수가 빨리 드러납니다.

Two-Level Select가 distribution을 바꾸지 않는다는 것은 증명보다 강하게 확인할 수 있습니다. 14.5절에 대수적 증명이 있지만, 증명이 보장하는 것은 **의도한 절차**이지 **implementation**이 아닙니다. 다행히 우리 sampler는 전부 integer로 자르지 않고 계산하므로, 같은 random number에 대해 "group → row → lane으로 좁혀 가며 찾은 index"와 "0번부터 차례로 prefix sum을 쌓아 처음 그 random number를 넘는 index"가 **정확히 같아야** 합니다. statistical test가 아니라 random number 하나하나에 대한 일치 검사가 가능하다는 뜻입니다. group prefix sum의 경계, row 안 잔여값 계산, 엄격한 `>` — 이 셋 중 하나만 어긋나도 경계 근처의 random number에서 이웃 칸이 나오고, 이 검사가 그것을 바로 잡습니다.

마지막으로 **드문 경로도 직접 시험해야 합니다.** rejection sampling은 후보가 total weight 이상일 때만 다시 뽑는데, 받아들여질 probability가 늘 절반을 넘으므로 여러 번 연달아 rejection되는 경로는 정상 simulation에서 거의 나오지 않습니다. total weight 0(`zero_weight_error`)은 정상 동작에서 아예 나오지 않습니다. 이런 경로는 random number나 amplitude를 강제로 주입해 **직접 밟아 봐야** 합니다. 실행된 적 없는 code가 정확할 probability는 낮습니다. 같은 방식으로 $\text{number of iterations} \sim U[0, \text{sampling range})$ 의 균등성도 histogram으로 확인합니다 — sampling range가 2의 거듭제곱이 아닐 때 rejection sampling이 제대로 동작하지 않으면 distribution이 기울고, 그 기울기가 BBHT의 "success probability $\geq 1/4$" 보장을 약화시킵니다.

18.7 checkpoint를 어떻게 검증하나

checkpoint는 이 project의 headline 기여이면서 동시에 **오류가 가장 드러나기 어려운** 부분입니다. 다행히 합격 기준이 아주 선명합니다.

기본 검사는 전수 비교입니다. 16.6절이 $(DO)^{x+y} = (DO)^x (DO)^y$ 라는 associativity로 두 경로가 같은 연산열임을 보였고, 우리 arithmetic이 결정적이므로 결과는 근사적으로 같은 정도가 아니라 **bit 단위로 동일**해야 합니다. 그래서 같은 request에 대해 (a) 새로 Init하고 요청된 횟수만큼 전진한 array와 (b) checkpoint slot에 저장된 state에서 이어 차이만큼 전진한 array를 만들어, **16,384칸 전부를** 비교합니다. requested-j vector가 (a)의 expected result를 이미 갖고 있으므로, checkpoint 경로의 최종 amplitude를 그 hash와 대조하면 됩니다. 몇 칸만 sample로 뽑거나 error를 허용하는 비교는 하면 안 됩니다. 같아야 하는 근거가 수학적 항등식이므로, 한 bit라도 다르면 그것은 error가 아니라 버그입니다.

무효화 조건은 각각 하나씩 regression test를 세웁니다. 16.7절이 무효화를 hardware에 맡긴 이유가 "software에 맡기면 반드시 새는 곳이 생긴다"였는데, hardware에 맡겨도 조건 하나를 빠뜨리면 같은 일이 벌어집니다. data load, predicate 종류, threshold A·B, data 개수, enumeration의 Found Mask — **하나하나에 대해** 바꾼 직후의 search가 Init부터 시작하는지 확인합니다. 반대 방향도 세웁니다. seed·shot cap·mode bit를 바꿨을 때는 checkpoint slot이 **유효하게 남아 있어야** 합니다. 비워도 결과는 맞지만 성능 이득이 사라져 결과만으로는 문제를 알아차리기 어렵습니다.

여기서 **가장 위험한 버그의 모양**을 알아 두는 것이 중요합니다. 무효화가 빠지면 circuit은 겉으로 완벽히 정상 동작합니다. 첫 search도 맞고, 두 번째 search도 맞고, threshold를 바꾸지 않는 한 계속 맞습니다. 그러다 threshold를 바꾼 순간부터 **이전 predicate로 증폭된 amplitude array 위에서** 새 predicate의 oracle이 돌기 시작하고, 겉으로 드러나지 않게 답이 틀립니다 — solution check가 non-solution은 걸러 주지만, shot count와 requested iterations가 Restart mode와 달라지는 형태로 드러납니다. 이런 고장은 무작위 regression으로는 잡히지 않습니다. "설정을 바꾼다 → checkpoint slot이 유효한 채로 남는다 → BBHT trajectory가 달라진다"는 시나리오를 **의도적으로 만들어** test에 넣어야 합니다.

checkpoint on/off equivalence는 checkpoint가 관측량을 바꾸지 않는다는 16.6절 주장의 실험적 확인입니다. 같은 seed·같은 data·같은 predicate에서 Restart mode와 Checkpoint mode가 **동일한 결과 index**, **동일한 shot count와 requested iterations**를 내야 하고, 오직 **executed iterations와 cycle만** 달라야 합니다. 이 비교는 이미 여러 층에서 돌았습니다 — 최상단 testbench가 Restart mode/Checkpoint mode 짝을 대조하고, 500 workloads RTL campaign과 board benchmark가 모두 BBHT trajectory 500/500 일치였습니다. 여기에 policy까지 옮긴 reference model이 board와 **executed iterations까지** 500/500 맞았습니다.

끝으로 policy의 드문 경로를 잊지 마십시오. request가 checkpoint slot의 iteration과 정확히 같아 비용이 0인 경우, request가 모든 checkpoint slot보다 작아 Init부터 가는 경우, 중간 state를 다른 checkpoint slot에 저장하는 경우입니다. BBHT에서 sampling range는 커지기만 하므로 뒤의 두 경우는 드물게 나옵니다. seed를 골라 강제로 만들거나 request 수열을 직접 주입해 경로를 밟아 보십시오. 그리고 policy가 미리 풀어 둔 계획과 실제 request가 어긋난 횟수(`PLAN_FIFO_MISMATCH_COUNT`)는 언제나 0이어야 합니다. 0이 아니면 앞으로 올 request를 예측하는 circuit이 실제 Iteration-Count LFSR 출력과 어긋났다는 직접 증거입니다.

18.8 잰 수치와 그 조건

초판의 이 절은 "다시 재야 하는 수치"의 목록이었습니다. 이제 그 수치들을 쟀고, 무엇을 어떤 조건에서 쟀는지 표로 정리합니다. 값의 source of truth는 15.4절입니다.

수치	값	benchmark 조건	상세
fraction bits 충분성	모든 실행에서 amplitude·probability·success probability·norm error $< 10^{-3}$. 14 qubits는 22 fraction bits에서 probability $\ell_2 \leq 10^{-5}$	같은 arithmetic(14 qubits · Q1.22)의 reference model 671,600회 campaign과 precision 분석	12.7
saturation	BBHT 1,000회당 2.67번, 기능 결과 불일치 0	같은 campaign	12.3
mean shot count	22.7 (median 23)	1 solution, 500 workloads RTL campaign 중 100개	16.3
executed iterations	31,424 → 9,582 (−69.5%)	500 workloads, Restart mode → a Checkpoint-Capacity of 3 and 3-shot Lookahead	16.5
RTL cycle	42,308,335 → 6,890,470 (6.1401×)	같은 500 workloads, Six-Stage Ablation	15.5
board wall-clock time	425,502 µs → 55,798 µs (7.626×)	같은 500 workloads, search 호출 interval만	15.5
같은 board 순수 software 대비	116,426× (wall-clock time)	같은 chip·같은 workload·같은 seed, RISC-V core 하나. 32 lanes 병렬과 clock 두 배 때문이지 algorithm 우위가 아님	15.5
resource · timing	LUT 45,284 · BRAM 116 · DSP 132, 100 MHz WNS +0.126 ns	2026-09-07 implementation report (board 전체)	15.5
reference model ↔ board	BBHT trajectory 500/500, executed iterations 500/500	Common500 비교	18.2

아직 재지 않은 것도 적어 둡니다. software reference model은 cycle을 흉내 내지 않습니다 — policy가 결정을 내리는 latency, Plan Queue stall, 4 PEs와 Measurement Microarchitecture의 timing은 hardware의 몫이라, cycle은 언제나 RTL이나 board에서 인용합니다. iteration마다의 amplitude를 DRAM에 저장하는 두 번째 variant(DRAM Variant)는 동작 model 위의 RTL regression까지만 있고 board 수치가 없습니다.

비교 실험의 공정성 기준도 여기서 정합니다. 비교 대상은 **Qiskit AerSimulator와 NumPy state vector simulator**, 그리고 같은 board의 순수 software baseline입니다. classical linear scan과는 비교하지 않습니다 — emulation 비용이 $O(\text{number of states}\sqrt{\text{number of states}/\text{number of solutions}})$ 이라 $O(\text{number of states})$ scan보다 느린 것이 당연하고, algorithm의 $\sqrt{\text{number of states}}$ 이득은 실제 quantum computer의 이야기입니다. 이 비교로 보이려는 것은 속도 우위가 아니라, **같은 algorithm을 같은 순서로 돌렸을 때 전용 hardware가 어느 지점에서 이득을 내는가**입니다.

같게 두어야 할 것은 다음과 같습니다. number of qubits, data array(같은 seed로 생성한 같은 `value[]`), predicate와 threshold, random number seed, 그리고 **BBHT schedule 전체** — 같은 growth factor, 같은 sampling range 수열, 같은 Iteration-Count LFSR random number. shot마다 뽑히는 iteration이 다르면 전체 iteration이 달라지고, 그러면 무엇을 비교한 것인지 알 수 없게 됩니다. 성공 판정도 같은 기준(solution check 한 칸)으로 통일합니다. Common500 비교 실험이 정확히 이 조건으로, board와 같은 다섯 dataset과 seed 100쌍에 NumPy·Qiskit·fixed-point model을 걸었습니다.

재는 것은 둘로 나눕니다. **wall-clock time**은 무엇을 포함했는지를 반드시 밝힙니다. NumPy는 state vector 계산 core만, Qiskit은 Aer 실행만, board는 명령 발행부터 결과까지를 쟀습니다. 예를 들어 1 solution의 median이 NumPy 약 9 ms, Qiskit Aer 약 1.8 s, board의 final configuration 약 0.21 ms였는데, 경계가 서로 달라 이 비율은 참고값입니다. 성능 주장에 쓰는 것은 같은 경계 안의 비교 — board configuration끼리의 speedup — 입니다. 그리고 마지막으로 밝혀 둘 것 — Aer는 double-precision complex number로 amplitude 하나에 16 bytes를 쓰고 우리는 23-bit real number를 씁니

다. **같은 수 표현으로 같은 문제를 푸는 것이 아닙니다.** 그러므로 정확도를 나란히 놓고 우열을 논하지 말고, "같은 algorithm·같은 shot sequence를 도는 데 걸리는 시간"만 비교하십시오.

18.9 toolchain과 directory

마지막으로 이 verification이 실제로 어디서 도는지를 정리합니다.

이 project 환경에는 필요한 도구가 **전부 local에** 있습니다 — RTL simulator `verilator`(cycle-accurate C++ model)와 `iverilog`, commercial simulator Questa, synthesis·implementation·bitstream을 내는 Vivado, RISC-V firmware를 빌드하는 GCC, 그리고 SoC 전체를 생성하는 RVX. 화면이 없는 환경이라 waveform viewer와 Vivado GUI는 띄울 수 없고, Vivado는 batch mode로만 씁니다. waveform 대신 testbench가 값을 file로 저장하고 script가 대조합니다.

작업 순서는 verification ladder를 따릅니다. 1단과 2단은 Python이라 어디서든 돌고, 3단 regression은 verilator로 몇 초에서 몇십 분이며, 그다음이 synthesis·timing closure·resource benchmark, 마지막이 board입니다. 순서를 뒤집으면 synthesis 한 번에 수십 분을 쓰면서 simulation으로 5분이면 잡을 버그를 찾게 됩니다. 두 simulator는 역할이 다릅니다. `verilator` 는 빨라서 250·500 workloads testbench 같은 긴 regression을 맡고, Six-Stage Ablation의 2,500회 campaign은 `iverilog` 로 돌았습니다. reference model이 Python이므로 simulator harness가 결과를 file로 저장해 대조하는 형태가 자연스럽습니다.

directory는 이렇게 나뉩니다.

경로	담는 것
`hardware_bram/src/`	RTL — Bus Interface(CSR · DMA Engine)와 Accelerator Core
`hardware_bram/testbench/`	Verilog testbench와 C++ harness
`hardware_bram/sim/`	regression 진입점 `Makefile` 과 runner·보고 script
`hardware_bram/results/` · `hardware_bram/configurations/*/results/`	simulation campaign 결과 묶음 (configuration 사이 비교 · configuration별)
`software/reference_model_develop/models/`	2단 reference model (Q1.22 bit-exact · checkpoint policy)
`software/pc_grover_develop/`	1단 기준 (NumPy · Qiskit) 과 C++ implementation
`software/reference_model_data/`	2단이 만든 test vector · expected result · 비교 결과
`software/testbench_data/`	testbench · 실험이 읽는 입력 dataset
`software/reference_model_develop/` · `software/reference_model_data/`	Common500 비교 실험과 그 결과

reference model이 `hardware_bram/` 이 아니라 `software/` 에 있는 것이 의도적입니다. 같은 reference model을 BRAM Variant와 DRAM Variant가 **함께** 쓰기 때문입니다 — DRAM Variant는 같은 stimulus를 BRAM Variant에도 걸어 search의 BBHT trajectory가 같은지 대조하고, 그 기준이 되는 의미 규약은 두 variant 어디에도 속하지 않습니다. regression을 실제로 돌리는 명령과, 저장소 상황에 따라 먼저 되살려야 할 file은 toolchain 사용법에 있습니다.

18.10 이 장의 요약

- **verification이 설계의 일부인 이유 셋**: 1차 자료가 틀리고(15.6절), 이론이 답을 못 주는 자리가 있으며(fraction bits 충분성, 12.7절), 이 IP의 값어치 자체가 재현 가능성이라 verification 절차가 곧 제품 사양입니다.
- **3단 verification ladder + board**: float64·Qiskit(**algorithm**이 맞나) → Q1.22 bit-exact reference model(**수 표현**이 맞나) → RTL(**circuit**가 맞나) → board. 각 칸이 잡는 버그의 종류가 다르고, 한 칸을 건너뛰면 rounding 버그와 FSM 버그가 섞여 판정 기준을 세울 수 없습니다. **reference model은 2단**이고, BBHT trajectory가 같아야 하는 것은 같은 칸끼리입니다(1단의 float BBHT trajectory는 2단과 다를 수 있습니다).

- **bit-exact 성립 조건**: rounding 규칙 · rounding 지점 · saturation 지점과 범위 · shift · accumulator 폭 · measurement arithmetic과 순서 · **Iteration-Count LFSR 규약** · **Measurement PRNG 규약** · policy의 동률 규칙. 초판이 미정으로 남긴 random number 두 줄이 정해진 뒤 reference model과 board가 BBHT trajectory 500/500, 동률 규칙까지 옮긴 뒤 executed iterations 500/500이 되었습니다. algorithm constant는 RTL header 한 곳에, CSR offset은 손으로 맞추는 두 곳에 있습니다.
- **첫 test vector는 9.4절 손계산**: 4-state 예제는 Q1.22에서 solution amplitude가 정확히 1.0이 되어 **saturation 경로**를 밟고, 두 번째 iteration에서 절반에 걸립니다. 8 states·3 solutions 예제는 두 번째 iteration의 $2\bar{a} = -2{,}595{,}092.5$ 에서 round-half-to-even과 truncation이 갈립니다. RTL의 첫 vector는 실제 크기에서 손으로 확인되는 **14 qubits·1 solution·1 iteration** — solution 98,296, 나머지 32,760이 이론값과 rounding 없이 같습니다.
- **predicate 경계값**: threshold와 같은 값(strict inequality 확인) · 16-bit signed data word 범위 양 끝(17-bit subtractor의 존재 이유) · 좁고 넓은 범위 · 0 solutions · 1 solution · solution이 state의 절반을 넘는 역주행 · FIFO가 차는 enumeration. data distribution은 uniform random number · 정렬 · **solution을 row·group 경계에 몰아넣은 인위적 array**.
- **Born sampler는 두 시험이 다릅니다**: seed를 고정하면 결정적이라 **bit-exact equivalence check**가 되고, distribution 자체는 **chi-square test**로 따로 봅니다(cell은 group 단위로 묶고, 한 번의 $p < 0.05$ 로 실패를 선언하지 않습니다). Two-Level Select는 integer 정확 계산이라 순차 prefix sum 선택과 **random number 하나하나에서 같아야** 하고, 연속된 rejection과 total weight 0 같은 드문 경로는 강제 주입으로 직접 시험합니다.
- **checkpoint verification의 합격 기준은 전수 비교**: 이어 간 경로와 새 Init 경로의 amplitude 16,384칸이 **bit 단위로 전부 일치**해야 합니다. 무효화 조건 각각과 무효화하지 않아야 할 조건 각각에 regression을 두고, on/off가 같은 seed에서 **같은 결과·같은 shot count·같은 requested iterations**를 내고 **executed iterations와 cycle만** 다른지 확인합니다 — board와 RTL에서 500/500이었고, Plan Queue miss counter는 늘 0이어야 합니다.
- **잰 수치**: fraction bits 충분성(error $< 10^{-3}$) · saturation(1,000회당 2.67) · mean 22.7 shots · executed iterations −69.5% · RTL 6.1401× · board 7.626× · resource와 timing. 비교 대상은 Qiskit Aer / NumPy / 같은 board의 순수 software며 **classical scan과는 비교하지 않습니다.** 같게 둘 것은 number of qubits·data·predicate·seed·BBHT schedule 전체, wall-clock time은 benchmark 경계를 반드시 밝힙니다.
- **toolchain**: simulator·synthesis·firmware·SoC 생성이 전부 local이고, 작업 순서는 verification ladder를 따릅니다. reference model은 두 variant가 함께 쓰므로 `software/` 에 있습니다.

이것으로 해설서를 마칩니다. index 에서 "왜 quantum computer인가"로 시작해 9장에서 Grover를 손으로 돌려 보았고, 11장부터 그 algorithm을 memory와 arithmetic과 제어와 bus로 구현했으며 이 장에서 그 구현이 맞는지 어떤 절차로 확인했는지 살펴보았습니다. board 위에서 도는 이 circuit은, 여러분이 1장에서 처음 본 amplitude라는 숫자가 끝까지 정해진 규칙에 따라 계산된 결과입니다.

부록 A. 수학 빠른 복습 — complex number와 linear algebra

> **성격**: 참조용. 이 해설서가 *실제로 쓰는* 수학만 추렸습니다. 완전한 강의가 아니라, 본문을 읽다 막혔을 때 돌아와 확인하는 최소 사전입니다.

이 해설서를 읽으려면 두 가지 배경 지식이 필요합니다. 1장부터 9장까지의 quantum 계산은 **complex number**와 **linear algebra(vector·matrix)**를 이용하고 11장 이후의 hardware 설계는 **binary fixed-point arithmetic**과 **기초 probability**를 씁니다. 처음 배우거나 오래 잊었다면 앞쪽에서는 α^*, $|\alpha|^2$, $e^{i\theta}$, matrix multiplication에서 멈추게 되고, 뒤쪽에서는 Q1.22 같은 Q format이나 arithmetic shift, prefix sum에서 다시 멈추게 됩니다.

그래서 이 부록도 두 부분으로 나뉩니다 — **A.1~A.4가 quantum 계산 쪽, A.5~A.6이 hardware 쪽**입니다. 처음부터 순서대로 읽을 것은 아니고 막힌 기호가 있는 절만 펴 보면 됩니다. 여기 있는 것만 알면 9장까지의 손계산과 12장의 수 표현, 16장의 shot 통계를 손으로 따라갈 수 있습니다.

A.1 complex number

imaginary unit i 는 square가 -1 이 되는 수입니다: $i^2 = -1$. real number 범위에서는 square가 negative number인 수가 없으므로, 그런 수를 새로 하나 들여온 것입니다.

complex number는 real number 두 개를 붙인 $\alpha = a + bi$ 꼴입니다. a 를 real part, b 를 imaginary part라 부릅니다. complex number는 평면 위의 한 점 (a, b) 로 볼 수 있어서, "complex plane"에 화살표로 그리곤 합니다.

complex conjugate는 imaginary part의 sign만 뒤집은 것으로, 별표로 씁니다: $\alpha^* = a - bi$. 예를 들어 $(3 + 4i)^* = 3 - 4i$ 입니다. complex conjugate는 곧 배울 bra(2장)와 probability 계산에서 계속 등장합니다.

absolute value(크기)는 원점에서 그 점까지의 거리입니다:

$$|\alpha| = \sqrt{a^2 + b^2}, \qquad |\alpha|^2 = a^2 + b^2 = \alpha^* \alpha$$

마지막 등식 $|\alpha|^2 = \alpha^* \alpha$ 가 특히 중요합니다. $\alpha^* \alpha = (a - bi)(a + bi) = a^2 + b^2$ 이므로 항상 **non-negative real number**가 됩니다. 이 해설서 전체에서 **measurement probability = amplitude의 absolute value square** $= |\alpha|^2$ 이라는 규칙(4장 Born rule)이 나오는데, 여기서 probability가 언제나 0 이상의 real number가 되는 이유가 바로 이 성질입니다. 예: $|3 + 4i| = \sqrt{9 + 16} = 5$, $|3 + 4i|^2 = 25$.

Euler's formula와 phase. 크기가 1인 complex number는 전부 다음 꼴로 쓸 수 있습니다:

$$e^{i\theta} = \cos\theta + i \sin\theta$$

이것은 complex plane의 **unit circle 위**의 점이고($|e^{i\theta}| = 1$), 각도 θ 를 **phase**라 부릅니다. 몇 개만 외워두면 편합니다: $e^{i0} = 1$, $e^{i\pi/2} = i$, $e^{i\pi} = -1$, $e^{i2\pi} = 1$. 임의의 complex number는 크기와 phase로 $\alpha = r\, e^{i\theta}$ (polar form)처럼 분해됩니다. sign $-1 = e^{i\pi}$ 도 phase의 특수한 경우라, quantum에서 "solution state의 sign을 뒤집는다"는 말이 곧 "phase를 π 만큼 돌린다"는 뜻입니다(3장 Z, 7장 oracle).

정리하면, quantum computing이 real number 대신 complex number를 쓰는 이유는 이렇습니다. amplitude가 complex number라야 크기(→ probability)와 phase(→ interference)라는 **두 가지 정보**를 동시에 담을 수 있고, phase들이 서로 더해지고 상쇄되는 **interference**가 algorithm의 핵심 원리이기 때문입니다.

다만 이 해설서가 끝까지 따라가는 표준 Grover만큼은 예외입니다 — amplitude가 처음부터 끝까지 real number로만 유지되어 complex number가 필요 없고(9.8절), 이 성질이 amplitude 한 칸을 fixed-point real number **하나**로 저장하는(A.5) 근거이자 complex phase를 amplitude에 직접 곱하는 quantum phase estimation 계열을 설계에서 제외한 이유입니다.

A.2 vector

이 해설서에서 qubit state는 성분이 complex number인 **column vector** 입니다. qubit 하나는 성분 두 개짜리 vector죠:

$$|\psi\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}$$

vector의 **norm**은 성분들의 absolute value square를 더해 square root를 씌운 것입니다:

$$\big\||\psi\rangle\big\| = \sqrt{|\alpha|^2 + |\beta|^2}$$

크기가 1인 vector를 **unit vector**라 하고, 어떤 vector를 그 크기로 나눠 unit vector로 만드는 일을 **normalization**이라 합니다. 이 해설서에서 **모든 qubit state는 unit vector**입니다 — 즉 $|\alpha|^2 + |\beta|^2 = 1$ (1장). 그 이유(measurement probability의 sum이 1)는 4장에서 밝혀집니다. qubit가 여러 개로 늘면 vector의 성분 수가 $2^{\text{number of qubits}}$ 개로 늘 뿐, "state = unit vector"라는 원칙은 똑같습니다(5장).

$2^{\text{number of qubits}}$ 라는 수는 이 해설서에서 두 가지 맥락으로 쓰입니다. 하나는 방금 말한 **성분의 개수**이고, 다른 하나는 그 성분에 붙는 **번호(index)** 입니다. 성분에 0 부터 $2^{\text{number of qubits}} - 1$ 까지 번호를 매기면 번호 하나가 qubit마다 bit 하나씩인 binary number 하나와 일대일로 대응하고, 그래서 $|x\rangle$ 의 x 를 어떤 곳에서는 bit string으로, 어떤 곳에서는 integer로 부릅니다. 우리 설계는 14 qubits이므로 $2^{14} = 16{,}384$ states이고 index는 **14 bits**입니다.

binary number의 성질 하나만 더 챙기면 뒤쪽 절반이 훨씬 편해집니다. 어떤 수의 **하위 k bit는 그 수를 2^k 로 나눈 나머지**이고, 남는 위쪽 bit는 **몫**입니다. index i 라면 하위 5 bits가 $i \bmod 32$, 상위 9 bits가 $\lfloor i/32 \rfloor$ 인 것이죠. 그래서 14-bit index를 위아래로 자르기만 하면 divider 없이 두 정보가 동시에 나옵니다 — 하위 5 bits는 row 안의 몇 번째 lane인지, 상위 9 bits는 몇 번째 row인지입니다. 11.7절의 row · lane 배치는 이 한 줄에서 시작합니다.

A.3 matrix와 matrix multiplication

matrix는 숫자를 직사각형으로 배열한 것이고, 이 해설서에서 gate는 대부분 2×2 matrix입니다(3장).

matrix × vector. gate를 qubit에 "적용"하는 것이 바로 이 곱입니다. 2×2 matrix에 성분 2개짜리 vector를 곱하는 규칙은 이렇습니다:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} ax + by \\ cx + dy \end{bmatrix}$$

즉 결과의 k 번째 성분은 "matrix의 k 번째 **row**와 vector를 원소끼리 곱해 더한 값"입니다. 구체적인 예로, X gate(3장)가 amplitude를 맞바꾸는 것을 확인해 봅시다:

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} 0 \cdot \alpha + 1 \cdot \beta \\ 1 \cdot \alpha + 0 \cdot \beta \end{bmatrix} = \begin{bmatrix} \beta \\ \alpha \end{bmatrix}$$

matrix × matrix. gate를 연달아 적용하면 matrix끼리 곱하게 됩니다(3장, 6장). 곱 AB 의 (i, j) 성분은 "A 의 i 번째 row"와 "B 의 j 번째 column"을 원소끼리 곱해 더한 값입니다. 주의할 점은 matrix multiplication은 **순서를 바꾸면 값이 달라진다**는 것($AB \neq BA$ 가 보통)이고, 그래서 3장에서 "circuit의 gate 순서와 matrix multiplication의 순서가 반대"라는 함정이 생깁니다.

transpose와 conjugate transpose. matrix의 row와 column을 뒤바꾼 것을 **transpose** A^T 라 하고, transpose한 뒤 모든 성분의 complex conjugate를 취한 것을 **conjugate transpose(dagger)** $A^\dagger = (A^T)^*$ 라 합니다. 이 † 는 bra와 unitary 조건($U^\dagger U = I$)에서 핵심이 되며, 2장에서 본격적으로 다룹니다. 여기서는 "transpose + complex conjugate" 두 동작이라는 것만 기억하면 충분합니다.

A.4 이 책이 실제로 쓰는 것만 (한눈에)

기호	읽는 법	뜻	처음 나오는 곳
i	imaginary unit	$i^2 = -1$	A.1
α^*	alpha conjugate	$a + bi \to a - bi$	2장
$\|\alpha\|^2$	absolute value square	$\alpha^*\alpha = a^2 + b^2$ = probability	4장
$e^{i\theta}$	phase factor	unit circle 위의 점, $\|e^{i\theta}\| = 1$	4장
$\begin{bmatrix}\alpha \\ \beta\end{bmatrix}$	column vector	qubit state (unit vector)	1장
$U	\psi\rangle$	matrix × vector	gate 적용
AB	matrix × matrix	gate composition (순서 중요)	3장
$A^\dagger$	dagger	transpose + complex conjugate	2장

더 깊이 보려면 3Blue1Brown의 *Essence of Linear Algebra* 영상 시리즈와 complex number 기초 자료를 권합니다. 위 표는 1~9장의 손계산에서 실제로 계산에 쓰는 기호를 모은 것입니다. 다만 표가 유도를 대신하지는 않습니다 — 기호를 알아본 뒤에도 막힌다면 그 장의 계산을 한 줄씩 따라 적어 보는 편이 빠릅니다. 11장 이후 hardware 쪽에서 처음 만나는 기호는 다음 두 절에 따로 모았습니다.

A.5 fixed-point arithmetic

11장 이후의 amplitude는 complex number도 real number도 아니고 **정해진 폭의 binary integer**입니다. 그 integer를 읽고 계산하는 규약 다섯 가지를 아래에 정리했습니다. 결정의 근거는 12장에 있고, 값의 source of truth는 15.4절 설계 결정표입니다.

two's complement. 정해진 data word width 안에서 negative number까지 표현하는 표준 규약입니다. 최상위 bit만 자리값이 negative인 $-2^{\text{data word width}-1}$ 이고 나머지 bit는 평소대로 읽으면 됩니다. 표현 범위는 $[-2^{\text{data word width}-1},\ 2^{\text{data word width}-1} - 1]$ 이고, sign을 뒤집는 연산은 **모든 bit를 반전한 뒤 1을 더하는 것**입니다. 이 규약을 쓰는 이유는 addition·subtraction circuit이 unsigned integer일 때와 **똑같아서**입니다 — sign을 위한 별도 hardware가 필요 없습니다.

Q format. 저장된 integer를 $2^{-\text{fraction bits}}$ 단위로 읽기로 한 약속이 fixed-point입니다. integer v 가 실제로 뜻하는 값은 $v/2^{\text{fraction bits}}$ 이고, 소수점은 circuit 어디에도 없이 약속 속에만 있습니다. 표기는 `Q<integer bits>.<fraction bits>` 인데 문헌마다 sign bit를 integer bits에 넣기도 하고 빼기도 하므로, **이 해설서는 sign을 integer bits에 포함**시켜 일관되게 정의합니다(12.2절). 그러면 data word width는 언제나 integer bits와 fraction bits를 더한 값입니다. 우리 amplitude는 **Q1.22** — 23 bits이고 sign bit 1 + fraction bits 22, resolution은 2^{-22} 입니다. integer bits에 sign밖에 없어서 real number 1.0 에 해당하는 integer $2^{22} = 4{,}194{,}304$ 는 담기지 않고, 담을 수 있는 maximum은 $4{,}194{,}303$ 입니다.

shift. binary number를 왼쪽으로 k 칸 밀면 2^k 를 곱한 것이고, 오른쪽으로 밀면 2^k 로 나눈 것입니다. 오른쪽으로 밀 때는 반드시 **arithmetic right shift**(Verilog `>>>`)를 써야 합니다 — 빈 자리를 0이 아니라 sign bit로 채워야 negative number가 negative number로 남기 때문입니다. logical shift(`>>`)로 밀면 -4 가 거대한 positive number로 바뀝니다. multiplier 없이 $2\bar{a} - \alpha$ 를 만드는 circuit 전체가 이 성질을 이용합니다(13.5절).

rounding. right shift는 아래 bit를 **버립니다.** 버리는 방식이 셋인데, 그냥 잘라 내면(truncate) two's complement에서는 언제나 작은 쪽으로 밀리므로 mean -0.5 least significant bit만큼의 한쪽 bias가 남습니다. **half-up**은 정확히 절반인 값을 늘 위로 올리고, **round-half-to-even**은 절반일 때 결과의 least significant bit가 0이 되는 쪽으로 붙입니다. integer를 2로 나누는 1-bit shift를 예로 들면, $5/2 = 2.5$ 를 half-up은 3 으로 올리지만 half-to-even은 2 로 내리고, $7/2 = 3.5$ 는 둘 다 4 입니다. 절반인 경우가 위·아래로 반반 갈리므로 half-to-even은 bias가 정확히 0이고, 같은 rounding을 백 번 넘게 되풀이하는 loop에서는 이 차이가 error의 성장 속도를 바꿉니다(12.4절).

saturation. 결과가 표현 범위를 넘을 때 two's complement를 그대로 두면 maximum 바로 위가 minimum이 되어 **sign까지 뒤집힙니다** (wraparound). saturation은 그 대신 maximum을 넘으면 maximum에, minimum을 밑돌면 minimum에 붙입니다. 값은 틀리지만 sign과 크기 순서는 유지되고 error는 그 칸에만 남습니다(12.5절).

A.6 probability와 random number

measurement를 circuit으로 만들려면(14장) probability를 손으로 다뤄야 합니다. 필요한 것은 셋뿐입니다.

uniform random number. 어떤 interval의 값 중 어느 것이나 같은 probability로 나오는 random number를 $r \sim U[0, \text{upper limit})$ 처럼 적습니다. hardware에 진짜 random number는 없으므로 LFSR 같은 PRNG로 만드는데, 같은 seed에서 늘 같은 수열이 나온다는 점이 verification에서는 오히려 이득입니다(18.3절).

prefix sum과 inverse transform sampling. index i 마다 0 이상의 row weight가 붙어 있고 그 비율대로 index 하나를 뽑고 싶다고 합시다. prefix sum

$$\text{prefix sum}(i) = \sum_{k \le i} \text{row weight}_k, \qquad \text{total weight} = \text{prefix sum}(\text{last index})$$

을 만들어 두고 $r \sim U[0, \text{total weight})$ 를 뽑은 뒤, $\text{prefix sum}(i) > r$ 이 되는 **첫 번째** i 를 답으로 씁니다. 길이가 제각각인 막대를 한 줄로 이어 붙여 놓고 그 위에 점을 아무 데나 찍는 그림을 떠올리면 됩니다 — 점이 i 번 막대에 떨어질 probability는 그 막대가 차지한 길이의 비율 $\text{row weight}_i / \text{total weight}$ 이고, 그것이 우리가 원한 distribution입니다. probability가 되도록 row weight를 normalization하는 대신 r 의 범위를 total weight로 잡은 것은 비교의 양변에 같은 constant를 곱한 것과 같아서, divider 없이 같은 distribution을 얻습니다. 14.4절이 이 절차에 $\text{row weight}_i = a_i^2$ 를 넣은 것입니다.

expected number of shots. success probability가 p 인 shot을 성공할 때까지 독립적으로 되풀이하면, 필요한 number of shots의 mean은

$$E[\text{number of shots}] = \sum_{k \ge 1} k\,(1-p)^{k-1} p = \frac{1}{p}$$

입니다(geometric distribution). $p = 1/4$ 면 mean이 4 shots죠. 전제를 반드시 함께 기억해야 합니다 — 이 공식은 p **가 shot마다 같을 때**만 성립합니다. 16.3절의 shot loop는 초반 shot의 success probability가 거의 0이고 shot마다 값이 달라지므로 mean이 4 shots가 아니라 22.7 shots(14 qubits · 1 solution, 100 workloads)로 나오는데, 이 차이는 위 공식의 전제가 성립하지 않기 때문에 생깁니다.

방금 괄호 안에 붙인 조건이 그 22.7 shots를 어떻게 얻었는지도 말해 줍니다. probability를 식으로 풀기 어려울 때는 random number로 실제로 여러 번 돌려 보고 결과의 mean을 구하는데, 이것을 **Monte Carlo**라 합니다. 22.7 shots는 shot loop 전체를 서로 다른 100 seeds로 돌려(board benchmark와 같은 workload) mean number of shots를 잰 값입니다.

여기까지가 이 해설서를 손으로 따라가는 데 필요한 최소 사전입니다. 이제 1장으로 가서, 이 vector가 왜 하필 "qubit의 state"가 되는지 봅시다.